Frequently Asked Questions - VLSI Design

1. Explain why & how a MOSFET works.

2. Draw Vds-Ids curve for a MOSFET. Now, show how this curve changes (a) with increasing Vgs (b) with increasing transistor width (c) considering Channel Length Modulation

3. Explain the various MOSFET Capacitances & their significance

4. Draw a CMOS Inverter. Explain its transfer characteristics

5. Explain sizing of the inverter

6. How do you size NMOS and PMOS transistors to increase the threshold voltage?

7. What is Noise Margin? Explain the procedure to determine Noise Margin

8. Give the expression for CMOS switching power dissipation

9. What is Body Effect?

10. Describe the various effects of scaling

11. Give the expression for calculating Delay in CMOS circuit

12. What happens to delay if you increase load capacitance?

13. What happens to delay if we include a resistance at the output of a CMOS circuit?

14. What are the limitations in increasing the power supply to reduce delay?

15. How does Resistance of the metal lines vary with increasing thickness and increasing length?

16. You have three adjacent parallel metal lines. Two out of phase signals pass through the outer two metal lines. Draw the waveforms in the center metal line due to interference. Now, draw the signals if the signals in outer metal lines are in phase with each other

17. What happens if we increase the number of contacts or via from one metal layer to the next?

18. Draw a transistor level two input NAND gate. Explain its sizing (a) considering Vth (b) for equal rise and fall times

19. Let A & B be two inputs of the NAND gate. Say signal A arrives at the NAND gate later than signal B. To optimize delay, of the two series NMOS inputs A & B, which one would you place near the output?

20. Draw the stick diagram of a NOR gate. Optimize it

21. For CMOS logic, give the various techniques you know to minimize power consumption

22. What is Charge Sharing? Explain the Charge Sharing problem while sampling data from a Bus

23. Why do we gradually increase the size of inverters in buffer design? Why not give the output of a circuit to one large inverter?

24. In the design of a large inverter, why do we prefer to connect small transistors in parallel (thus increasing effective width) rather than lay out one transistor with large width?

25. Given a layout, draw its transistor level circuit. (I was given a 3 input AND gate and a 2 input Multiplexer. You can expect any simple 2 or 3 input gates)

26. Give the logic expression for an AOI gate. Draw its transistor level equivalent. Draw its stick diagram

27. Why don't we use just one NMOS or PMOS transistor as a transmission gate?

28. For a NMOS transistor acting as a pass transistor, say the gate is connected to VDD, give the output for a square pulse input going from 0 to VDD

29. Draw a 6-T SRAM Cell and explain the Read and Write operations

30. Draw the Differential Sense Amplifier and explain its working. Any idea how to size this circuit? (Consider Channel Length Modulation)

31. What happens if we use an Inverter instead of the Differential Sense Amplifier?

32. Draw the SRAM Write Circuitry

33. Approximately, what were the sizes of your transistors in the SRAM cell? How did you arrive at those sizes?

34. How does the size of PMOS Pull Up transistors (for bit & bit- lines) affect SRAM's performance?

35. What's the critical path in a SRAM?

36. Draw the timing diagram for a SRAM Read. What happens if we delay the enabling of Clock signal?

37. Give a big picture of the entire SRAM Layout showing your placements of SRAM Cells, Row Decoders, Column Decoders, Read Circuit, Write Circuit and Buffers

38. In a SRAM layout, which metal layers would you prefer for Word Lines and Bit Lines? Why?

39. How can you model a SRAM at RTL Level?

40. What's the difference between Testing & Verification?

41. For an AND-OR implementation of a two input Mux, how do you test for Stuck-At-0 and Stuck-At-1 faults at the internal nodes? (You can expect a circuit with some redundant logic)

42. What is Latch Up? Explain Latch Up with cross section of a CMOS Inverter. How do you avoid Latch Up?

•

FPGA is suited for timing circuit becauce they have more registers , but CPLD is suitedfor control circuit because they have more combinational circuit. At the same time, If yousynthesis the same code for FPGA for many times, you will find out that each timingreport is different. But it is different in CPLD synthesis, you can get the same result.As CPLDs and FPGAs become more advanced the differences between the two device types willcontinue to blur. While this trend may appear to make the two types more difficult to keep apart,the architectural advantage of CPLDs combining low cost, non-volatile configuration, and macrocells with predictable timing characteristics will likely be sufficient to maintain a productdifferentiation for the foreseeable future.

[illegible]

•

This question is very popular in VLSI fresher interviews. It looks simple but a deeper insight into the subject reveals the fact that there are lot of thinks to be understood !! Sohere is the answer.

FPGA vs. ASIC

•

Difference between ASICs and FPGAs mainly depends on costs, tool availability,performance and design flexibility. They have their own pros and cons but it is designersresponsibility to find the advantages of the each and use either FPGA or ASIC for theproduct. However, recent developments in the FPGA domain are narrowing down thebenefits of the ASICs.

FPGA

•

Field Programable Gate Arrays

FPGA Design Advantages

•

Faster time-to-market:

No layout, masks or other manufacturing steps are needed for FPGA design. Readymade FPGA is available and burn your HDL code to FPGA ! Done !!

•

No NRE (Non Recurring Expenses):

This cost is typically associated with an ASICdesign. For FPGA this is not there. FPGA tools are cheap. (sometimes its free ! You needto buy FPGA.... thats all !). ASIC youpay huge NRE and tools are expensive. I would say"very expensive"...Its in crores....!!

•

Simpler design cycle:

This is due to software that handles much of the routing,placement, and timing. Manual intervention is less.The FPGA design flow eliminates thecomplex and time-consuming floorplanning, place and route, timing analysis.

•

More predictable project cycle:

The FPGA design flow eliminates potential re-spins,wafer capacities, etc of the project since the design logic is already synthesized andverified in FPGA device.

•

Field Reprogramability:

A new bitstream (i.e. your program) can be uploaded remotely,instantly. FPGA can be reprogrammed in a snap while an ASIC can take $50,000 andmore than 4-6 weeks to make the same changes. FPGA costs start from a couple of dollars to several hundreds or more depending on the hardware features.

Reusability:

Reusability of FPGA is the main advantage. Prototype of the design can beimplemented on FPGA which could be verified for almost accurate results so that it canbe implemented on an ASIC. Ifdesign has faults change the HDL code, generate bitstream, program to FPGA and test again.Modern FPGAs are reconfigurable both partiallyand dynamically.

•

FPGAs are good for prototyping and limited production.If you are going to make 100-200boards it isn't worth to make an ASIC.

•

Generally FPGAs are used for lower speed, lower complexity and lower volumedesigns.But today's FPGAs even run at 500 MHz with superior performance. Withunprecedented logic density increases and a host of other features, such as embeddedprocessors, DSP blocks, clocking, and high-speed serial at ever lower price, FPGAs aresuitable for almost any type of design.

•

Unlike ASICs, FPGA's have special hardwares such as Block-RAM, DCM modules,MACs, memories and highspeed I/O, embedded CPU etc inbuilt, which can be used toget better performace. Modern FPGAs are packed with features. Advanced FPGAsusually come with phase-locked loops, low-voltage differential signal, clock datarecovery, more internal routing, high speed, hardware multipliers for DSPs,memory,programmable I/O, IP cores and microprocessor cores. Remember Power PC(hardcore) and Microblaze (softcore) in Xilinx and ARM (hardcore) and Nios(softcore) inAltera. There are FPGAs available now with built in ADC ! Using all these featuresdesigners can build a system on a chip. Now, dou yo really need an ASIC ?

•

FPGA sythesis is much more easier than ASIC.

•

In FPGA you need not do floor-planning, tool can do it efficiently. In ASIC you have do it.

FPGA Design Disadvantages

•

Powe consumption in FPGA is more. You don't have any control over the power optimization. This is where ASIC wins the race !

•

You have to use the resources available in the FPGA. Thus FPGA limits the design size.

•

Good for low quantity production. As quantity increases cost per product increasescompared to the ASIC implementation.

ASIC

Application Specific Intergrated Circiut

ASIC Design Advantages

Cost....cost....cost....Lower unit costs:

For very high volume designs costs comes out to bevery less. Larger volumes of ASIC design proves to be cheaper than implementingdesign using FPGA.

•

Speed...speed...speed....ASICs are faster than FPGA:

ASIC gives design flexibility.This gives enoromous opportunity for speed optimizations.

How you will choose an FPGA?

□

H

□

ow clock is routed through out FPGA?What are difference between

PLL and DLL ?What is soft processor?

What is hard processor?

Verilog Coding Guidelines- Part 5

5. FILE STRUCTURE

5.1 One file, one module

Create separate files for each modules. Name the file .v. The only exceptions for this file naming convention shall be the technology-dependent modules (top module or macro wrapper modules). These files shall be appropriately named like design_name_fpga.v, design_name_tsmc.v, or design_name_virtex.v.

5.2 File header

Each source file should contain a header at the top of the file in the following format:

```
////////////////////////////////////////////////////////////////
//////////
//(c) Copyright 2008 Verilog Course Team/Company Name. All rights reserved
//
// File:
// Project:
// Purpose:
// Author: <author's>
//
// $Id: index.html,v 1.1 2008/0773/23 01:55:57 VCT $
//
// Detailed description of the module included in the file.
//Include relevant part of the spec
// Logical hierarchy tree
// Block diagrams
// Timing diagrams etc.
//
```

//

The above example is for verilog. Change the comment characters appropriately for other source types. Example: "#" in Tcl, Perl and CSH. The presence of variable Id in the header will capture the filename, user, version information every time the file is checked-in/committed.

5.3 Modification history

Each file should contain a log section at the bottom of the file in the following format:

```
//////////////////////////////////////////////////////////////////////
////
//
// Modification History:
//
// $Log$
//
//////////////////////////////////////////////////////////////////////
```

Listing the modification history at the top of the file can be annoying as one has to scroll down to reach the code every time the file is opened for reading. The variable Log will cause RCS/CVS to capture the user-comments entered during each check-in/commit as comments in footer section.

5.4 Include Files

Keep the `define statements and Parameters for a design in a single separate file and name the file DesignName_params.v</author's>
Posted by . at 6:24 AM 0 comments
Labels: Coding Guidelines

Verilog Coding Guidelines - Part 4

4. DO'S AND DONT'S

4.1Use non-blocking assignments in sequential blocks

All registers assignments are concurrent. No combinatorial logic is allowed in sequential blocks. Always use non-blocking statements here.

```
module two_stage(Q, D, CLK);
input D, CLK;
output Q;

reg Q, P;

always @ (posedge CLK)
  begin
    P <= D;
    Q< = P;
end
```

4.2 Use blocking assignments in combinational blocks

Concurrency is not needed here. Often the combinatorial logic is implemented in multiple steps. Always use blocking statements for combinatorial blocks.

```
module two_stage(Q, D, data);
input D, data;
output Q;

reg Q, P;

always @ (data)
  begin
    Q = P;
    P = D;
 end
```

4.3 Ensure that there are no unused signals

Unused signals in the designs are often clear indication of incomplete or erroneous design. Check to make sure that design does not contain such signals.

4.3 Ensure that there are no un-driven signals

Un-driven signals in the designs are mostly clear indication of design errors. Check to make sure that design does not contain such signals.
Posted by . at 6:20 AM 0 comments
Labels: Coding Guidelines

Verilgo Coding Guidelines -Part 3

3. COMMENT

3.1 Comment blocks vs scattered comments

Describe a group of logic at the beginning of the file (in the header) or at the top of a block or group of blocks. Avoid scattering the comment for a related logic. Typically the reader would like to go through the comment and then understand the code itself. Scattered comment can make this exercise more tedious.

Example:

//File:
//purpose:
//Project:
//Author:

3.2 Meaningful comments

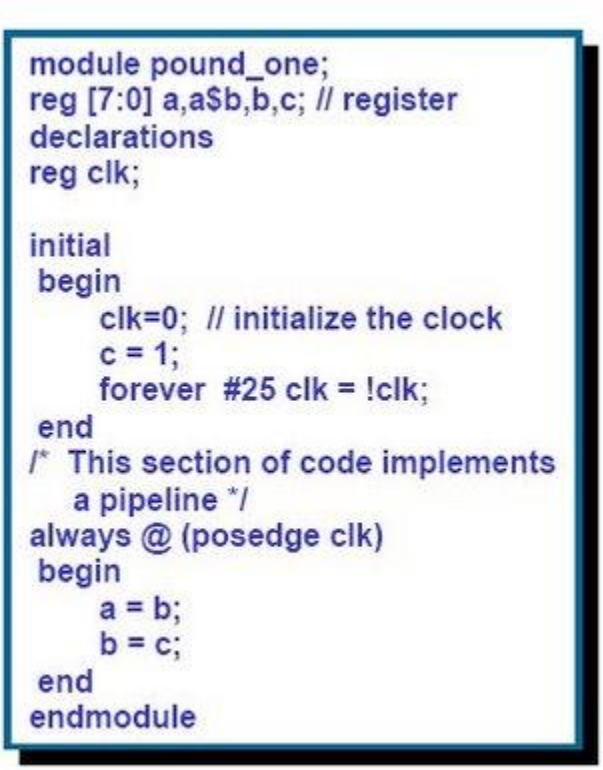
Do not include what is obvious in the code in your comments. The comment should typically cover what is not expressed through the code itself.

Example:

History of a particular implementation, why a particular signal is used, any algorithm being implemented etc.

3.3 Single line comments

Use single line comments where ever possible. i.e. Use comments starting with '//' rather than '/* .. */' style. This makes it easy to cut-paste or move around the code and comments. It is also easy to follow the indentation with single line comments which makes the code more readable.



```
module pound_one;
reg [7:0] a,a$b,b,c; // register
declarations
reg clk;

initial
 begin
      clk=0;  // initialize the clock
      c = 1;
      forever  #25 clk = !clk;
 end
/*  This section of code implements
    a pipeline */
always @ (posedge clk)
 begin
      a = b;
      b = c;
 end
endmodule
```

Verilog Coding Guidelines -Part 2

2. STYLE

2.1 Page width: 75 characters

Considering the limited page width supported in many terminals and printers, restrict the maximum line length to 75 characters. For reuse macros reduce this number to 72 to comply with RMM.

2.2 No tabs

Do not use tabs for indentation. Tab settings are different in different environments and hence can spoil the indentation in some setup.

2.3 Port ordering

Arrange the port list and declarations in a cause and effect order. Group the list/declaration on the basis of functionality rather than port direction etc. Specify the reset and clock signals at the top of the list.

2.4 One statement per line

Limit the number of HDL statements per line to one. Do not include multiple statements, separated by semicolon, in the same line. This will improve readability and will make it is easy to process the code using scripts and utilities.

2.5 One declaration per line

Limit the number of port, wire or reg declaration per line to one. Do not include multiple declarations, separated by commas, in the same line. This will make it easy to comment, add, or delete the declared objects.

Example:

Wrong way:
input trdy_n, stop_n;

Right way:
input trdy_n;
input stop_n;

1) Write a verilog code to swap contents of two registers with and without a temporary register?

With temp reg ;

```
always @ (posedge clock)
begin
temp=b;
b=a;
a=temp;
end
```

Without temp reg;

```
always @ (posedge clock)
begin
a <= b;
b <= a;
end
```

Click to view more

2) Difference between blocking and non-blocking?(Verilog interview questions that is most commonly asked)

The Verilog language has two forms of the procedural assignment statement: blocking and non-blocking. The two are distinguished by the = and <= assignment operators. The blocking assignment statement (= operator) acts much like in traditional programming languages. The whole statement is done before control passes on to the next statement. The non-blocking (<= operator) evaluates all the right-hand sides for the current time unit and assigns the left-hand sides at the end of the time unit. For example, the following Verilog program

```
// testing blocking and non-blocking assignment

module blocking;
reg [0:7] A, B;
initial begin: init1
A = 3;
#1 A = A + 1; // blocking procedural assignment
B = A + 1;

$display("Blocking: A= %b B= %b", A, B ); A = 3;
#1 A <= A + 1; // non-blocking procedural assignment
B <= A + 1;
#1 $display("Non-blocking: A= %b B= %b", A, B );
end
endmodule
```

produces the following output:
Blocking: A= 00000100 B= 00000101
Non-blocking: A= 00000100 B= 00000100

The effect is for all the non-blocking assignments to use the old values of the variables at the beginning of the current time unit and to assign the registers new values at the end of the current time unit. This reflects how register transfers occur in some hardware systems.
blocking procedural assignment is used for combinational logic and non-blocking procedural assignment for sequential

Click to view more

Tell me about verilog file I/O?

OPEN A FILE

```
integer file;
file = $fopenr("filename");
file = $fopenw("filename");
file = $fopena("filename");
```

The function $fopenr opens an existing file for reading. $fopenw opens a new file for writing, and $fopena opens a new file for writing where any data will be appended to the end of the file. The file name can be either a quoted string or a reg holding the file name. If the file was successfully opened, it returns an integer containing the file number (1..MAX_FILES) or NULL (0) if there was an error. Note that these functions are not the same as the built-in system function $fopen which opens a file for writing by $fdisplay. The files are opened in C with 'rb', 'wb', and 'ab' which allows reading and writing binary data on the PC. The 'b' is ignored on Unix.

CLOSE A FILE

```
integer file, r;
r = $fcloser(file);
r = $fclosew(file);
```

The function $fcloser closes a file for input. $fclosew closes a file for output. It returns EOF if there was an error, otherwise 0. Note that these are not the same as $fclose which closes files for writing.

Click to view more

3) Difference between task and function?

Function:
A function is unable to enable a task however functions can enable other functions.
A function will carry out its required duty in zero simulation time. (The program time will not be incremented during the function routine)
Within a function, no event, delay or timing control statements are permitted
In the invocation of a function their must be at least one argument to be passed.
Functions will only return a single value and can not use either output or inout statements.

Tasks:
Tasks are capable of enabling a function as well as enabling other versions of a Task
Tasks also run with a zero simulation however they can if required be executed in a non zero simulation time.
Tasks are allowed to contain any of these statements.
A task is allowed to use zero or more arguments which are of type output, input or inout.
A Task is unable to return a value but has the facility to pass multiple values via the output and

inout statements .

4) Difference between inter statement and intra statement delay?

```
//define register variables
reg a, b, c;

//intra assignment delays
initial
begin
a = 0; c = 0;
b = #5 a + c; //Take value of a and c at the time=0, evaluate
//a + c and then wait 5 time units to assign value
//to b.
end

//Equivalent method with temporary variables and regular delay control
initial
begin
a = 0; c = 0;
temp_ac = a + c;
#5 b = temp_ac; //Take value of a + c at the current time and
//store it in a temporary variable. Even though a and c
//might change between 0 and 5,
//the value assigned to b at time 5 is unaffected.
end
```

5) What is delta simulation time?

6) Difference between $monitor,$display & $strobe?

These commands have the same syntax, and display text on the screen during simulation. They are much less convenient than waveform display tools like cwaves?. $display and $strobe display once every time they are executed, whereas $monitor displays every time one of its parameters changes.
The difference between $display and $strobe is that $strobe displays the parameters at the very end of the current simulation time unit rather than exactly where it is executed. The format string is like that in C/C++, and may contain format characters. Format characters include %d (decimal), %h (hexadecimal), %b (binary), %c (character), %s (string) and %t (time), %m (hierarchy level). %5d, %5b etc. would give exactly 5 spaces for the number instead of the space needed. Append b, h, o to the task name to change default format to binary, octal or hexadecimal.
Syntax:

```
$display (“format_string”, par_1, par_2, ... );
$strobe (“format_string”, par_1, par_2, ... );
$monitor (“format_string”, par_1, par_2, ... );
```

7) What is difference between Verilog full case and parallel case?

A "full" case statement is a case statement in which all possible case-expression binary patterns can be matched to a case item or to a case default. If a case statement does not include a case default and if it is possible to find a binary case expression that does not match any of the defined case items, the case statement is not "full."
A "parallel" case statement is a case statement in which it is only possible to match a case expression to one and only one case item. If it is possible to find a case expression that would match more than one case item, the matching case items are called "overlapping" case items and the case statement is not "parallel."

8) What is meant by inferring latches,how to avoid it?

Consider the following :

```
always @(s1 or s0 or i0 or i1 or i2 or i3)
case ({s1, s0})
2'd0 : out = i0;
2'd1 : out = i1;
2'd2 : out = i2;
endcase
```

in a case statement if all the possible combinations are not compared and default is also not specified like in example above a latch will be inferred ,a latch is inferred because to reproduce the previous value when unknown branch is specified.
For example in above case if {s1,s0}=3 , the previous stored value is reproduced for this storing a latch is inferred.
The same may be observed in IF statement in case an ELSE IF is not specified.
To avoid inferring latches make sure that all the cases are mentioned if not default condition is provided.

9) Tell me how blocking and non blocking statements get executed?

Execution of blocking assignments can be viewed as a one-step process:
1. Evaluate the RHS (right-hand side equation) and update the LHS (left-hand side expression) of the blocking assignment without interruption from any other Verilog statement. A blocking assignment "blocks" trailing assignments in the same always block from occurring until after the current assignment has been completed

Execution of nonblocking assignments can be viewed as a two-step process:
1. Evaluate the RHS of nonblocking statements at the beginning of the time step. 2. Update the LHS of nonblocking statements at the end of the time step.

10) Variable and signal which will be Updated first?

Signals

11) What is sensitivity list?

The sensitivity list indicates that when a change occurs to any one of elements in the list change, begin…end statement inside that always block will get executed.

12) In a pure combinational circuit is it necessary to mention all the inputs in sensitivity disk? if yes, why?

Yes in a pure combinational circuit is it necessary to mention all the inputs in sensitivity disk other wise it will result in pre and post synthesis mismatch.

13) Tell me structure of Verilog code you follow?

A good template for your Verilog file is shown below.

```
// timescale directive tells the simulator the base units and precision of the simulation
`timescale 1 ns / 10 ps
module name (input and outputs);
// parameter declarations
parameter parameter_name = parameter value;
// Input output declarations
input in1;
input in2; // single bit inputs
output [msb:lsb] out; // a bus output
// internal signal register type declaration - register types (only assigned within always statements). reg register variable 1;
reg [msb:lsb] register variable 2;
// internal signal. net type declaration - (only assigned outside always statements) wire net variable 1;
// hierarchy - instantiating another module
reference name instance name (
.pin1 (net1),
.pin2 (net2),
.
.pinn (netn)
);
// synchronous procedures
always @ (posedge clock)
begin
.
end
// combinatinal procedures
always @ (signal1 or signal2 or signal3)
begin
```

.
end
assign net variable = combinational logic;
endmodule

14) Difference between Verilog and vhdl?

Compilation
VHDL. Multiple design-units (entity/architecture pairs), that reside in the same system file, may be separately compiled if so desired. However, it is good design practice to keep each design unit in it's own system file in which case separate compilation should not be an issue.

Verilog. The Verilog language is still rooted in it's native interpretative mode. Compilation is a means of speeding up simulation, but has not changed the original nature of the language. As a result care must be taken with both the compilation order of code written in a single file and the compilation order of multiple files. Simulation results can change by simply changing the order of compilation.

Data types
VHDL. A multitude of language or user defined data types can be used. This may mean dedicated conversion functions are needed to convert objects from one type to another. The choice of which data types to use should be considered wisely, especially enumerated (abstract) data types. This will make models easier to write, clearer to read and avoid unnecessary conversion functions that can clutter the code. VHDL may be preferred because it allows a multitude of language or user defined data types to be used.

Verilog. Compared to VHDL, Verilog data types a re very simple, easy to use and very much geared towards modeling hardware structure as opposed to abstract hardware modeling. Unlike VHDL, all data types used in a Verilog model are defined by the Verilog language and not by the user. There are net data types, for example wire, and a register data type called reg. A model with a signal whose type is one of the net data types has a corresponding electrical wire in the implied modeled circuit. Objects, that is signals, of type reg hold their value over simulation delta cycles and should not be confused with the modeling of a hardware register. Verilog may be preferred because of it's simplicity.

Design reusability
VHDL. Procedures and functions may be placed in a package so that they are avail able to any design-unit that wishes to use them.

Verilog. There is no concept of packages in Verilog. Functions and procedures used within a model must be defined in the module. To make functions and procedures generally accessible from different module statements the functions and procedures must be placed in a separate system file and included using the `include compiler directive.

15) What are different styles of Verilog coding I mean gate-level,continuous level and others explain in detail?

16) Can you tell me some of system tasks and their purpose?

$display, $displayb, $displayh, $displayo, $write, $writeb, $writeh, $writeo.
The most useful of these is $display.This can be used for displaying strings, expression or values of variables.
Here are some examples of usage.
$display("Hello oni");
--- output: Hello oni
$display($time) // current simulation time.
--- output: 460
counter = 4'b10;
$display(" The count is %b", counter);
--- output: The count is 0010
$reset resets the simulation back to time 0; $stop halts the simulator and puts it in interactive mode where the
user can enter commands; $finish exits the simulator back to the operating system

17) Can you list out some of enhancements in Verilog 2001?

In earlier version of Verilog ,we use 'or' to specify more than one element in sensitivity list . In Verilog 2001, we can use comma as shown in the example below.
// Verilog 2k example for usage of comma
always @ (i1,i2,i3,i4)

Verilog 2001 allows us to use star in sensitive list instead of listing all the variables in RHS of combo logics . This removes typo mistakes and thus avoids simulation and synthesis mismatches,
Verilog 2001 allows port direction and data type in the port list of modules as shown in the example below
module memory (
input r,
input wr,
input [7:0] data_in,
input [3:0] addr,
output [7:0] data_out
);

18)Write a Verilog code for synchronous and asynchronous reset?

Synchronous reset, synchronous means clock dependent so reset must not be present in sensitivity disk eg:

always @ (posedge clk)

begin if (reset)
. . . end
Asynchronous means clock independent so reset must be present in sensitivity list.
Eg
Always @(posedge clock or posedge reset)
begin
if (reset)
. . . end

19) What is pli?why is it used?

Programming Language Interface (PLI) of Verilog HDL is a mechanism to interface Verilog programs with programs written in C language. It also provides mechanism to access internal databases of the simulator from the C program.
PLI is used for implementing system calls which would have been hard to do otherwise (or impossible) using Verilog syntax. Or, in other words, you can take advantage of both the paradigms - parallel and hardware related features of Verilog and sequential flow of C - using PLI.

20) There is a triangle and on it there are 3 ants one on each corner and are free to move along sides of triangle what is probability that they will collide?

Ants can move only along edges of triangle in either of direction, let's say one is represented by 1 and another by 0, since there are 3 sides eight combinations are possible, when all ants are going in same direction they won't collide that is 111 or 000 so probability of not collision is 2/8=1/4 or collision probability is 6/8=3/4

How to write FSM is verilog?

there r mainly 4 ways 2 write fsm code
1) using 1 process where all input decoder, present state, and output decoder r combine in one process.
2) using 2 process where all comb ckt and sequential ckt separated in different process
3) using 2 process where input decoder and persent state r combine and output decoder seperated in other process
4) using 3 process where all three, input decoder, present state and output decoder r separated in 3 process.

Click to view more

(Also refer to Tutorial section for more)

Verilog interview Questions

21)What is difference between freeze deposit and force?

$deposit(variable, value);
This system task sets a Verilog register or net to the specified value. variable is the register or net to be changed; value is the new value for the register or net. The value remains until there is a subsequent driver transaction or another $deposit task for the same register or net. This system task operates identically to the ModelSim force -deposit command.

The force command has -freeze, -drive, and -deposit options. When none of these is specified, then -freeze is assumed for unresolved signals and -drive is assumed for resolved signals. This is designed to provide compatibility with force files. But if you prefer -freeze as the default for both resolved and unresolved signals.

Verilog interview Questions

22)Will case infer priority register if yes how give an example?

yes case can infer priority register depending on coding style

```
reg r;
// Priority encoded mux,
always @ (a or b or c or select2)
begin
r = c;
case (select2)
2'b00: r = a;
2'b01: r = b;
endcase
end
```

Verilog interview Questions

23)Casex,z difference,which is preferable,why?

CASEZ :
Special version of the case statement which uses a Z logic value to represent don't-care bits.
CASEX :
Special version of the case statement which uses Z or X logic values to represent don't-care bits.

CASEZ should be used for case statements with wildcard don’t cares, otherwise use of CASE is required; CASEX should never be used.
This is because:
Don’t cares are not allowed in the "case" statement. Therefore casex or casez are required. Casex will automatically match any x or z with anything in the case statement. Casez will only match z’s -- x’s require an absolute match.

Verilog interview Questions

24)Given the following Verilog code, what value of "a" is displayed?

```
always @(clk) begin
a = 0;
a <= 1;
$display(a);
end
```

This is a tricky one! Verilog scheduling semantics basically imply a
four-level deep queue for the current simulation time:
1: Active Events (blocking statements)
2: Inactive Events (#0 delays, etc)
3: Non-Blocking Assign Updates (non-blocking statements)
4: Monitor Events ($display, $monitor, etc).
Since the "a = 0" is an active event, it is scheduled into the 1st "queue".
The "a <= 1" is a non-blocking event, so it's placed into the 3rd queue.
Finally, the display statement is placed into the 4th queue. Only events in the active queue are completed this sim cycle, so the "a = 0" happens, and then the display shows a = 0. If we were to look at the value of a in the next sim cycle, it would show 1.

25) What is the difference between the following two lines of Verilog code?

```
#5 a = b;
a = #5 b;
```

#5 a = b; Wait five time units before doing the action for "a = b;".
a = #5 b; The value of b is calculated and stored in an internal temp register,After five time units, assign this stored value to a.

26)What is the difference between:

c = foo ? a : b;
and
if (foo) c = a;
else c = b;

The ? merges answers if the condition is "x", so for instance if foo = 1'bx, a = 'b10, and b = 'b11, you'd get c = 'b1x. On the other hand, if treats Xs or Zs as FALSE, so you'd always get c = b.

27)What are Intertial and Transport Delays ??

28)What does `timescale 1 ns/ 1 ps signify in a verilog code?

'timescale directive is a compiler directive.It is used to measure simulation time or delay time.
Usage : `timescale / reference_time_unit : Specifies the unit of measurement for times and

delays. time_precision: specifies the precision to which the delays are rounded off.

29) What is the difference between === and == ?

output of "==" can be 1, 0 or X.
output of "===" can only be 0 or 1.
When you are comparing 2 nos using "==" and if one/both the numbers have one or more bits as "x" then the output would be "X" . But if use "===" outpout would be 0 or 1.
e.g A = 3'b1x0
B = 3'b10x
A == B will give X as output.
A === B will give 0 as output.
"==" is used for comparison of only 1's and 0's .It can't compare Xs. If any bit of the input is X output will be X
"===" is used for comparison of X also.

30)How to generate sine wav using verilog coding style?

A: The easiest and efficient way to generate sine wave is using CORDIC Algorithm.

31) What is the difference between wire and reg?

Net types: (wire,tri)Physical connection between structural elements. Value assigned by a continuous assignment or a gate output. Register type: (reg, integer, time, real, real time) represents abstract data storage element. Assigned values only within an always statement or an initial statement. The main difference between wire and reg is wire cannot hold (store) the value when there no connection between a and b like a->b, if there is no connection in a and b, wire loose value. But reg can hold the value even if there in no connection. Default values:wire is Z,reg is x.

32)How do you implement the bi-directional ports in Verilog HDL?

```
module bidirec (oe, clk, inp, outp, bidir);

// Port Declaration
input oe;
input clk;
input [7:0] inp;
output [7:0] outp;
inout [7:0] bidir;
reg [7:0] a;
reg [7:0] b;
assign bidir = oe ? a : 8'bZ ;
assign outp = b;
// Always Construct
always @ (posedge clk)
```

```
begin
b <= bidir;
a <= inp;
end
endmodule
```

34)what is verilog case (1) ?

```
wire [3:0] x;
always @(...) begin
case (1'b1)
x[0]: SOMETHING1;
x[1]: SOMETHING2;
x[2]: SOMETHING3;
x[3]: SOMETHING4;
endcase
end
```

The case statement walks down the list of cases and executes the first one that matches. So here, if the lowest 1-bit of x is bit 2, then something3 is the statement that will get executed (or selected by the logic).

35) Why is it that "if (2'b01 & 2'b10)..." doesn't run the true case?

This is a popular coding error. You used the bit wise AND operator (&) where you meant to use the logical AND operator (&&).

36)What are Different types of Verilog Simulators ?

There are mainly two types of simulators available.

Event Driven
Cycle Based

Event-based Simulator:

This Digital Logic Simulation method sacrifices performance for rich functionality: every active signal is calculated for every device it propagates through during a clock cycle. Full Event-based simulators support 4-28 states; simulation of Behavioral HDL, RTL HDL, gate, and transistor representations; full timing calculations for all devices; and the full HDL standard. Event-based simulators are like a Swiss Army knife with many different features but none are particularly fast.

Cycle Based Simulator:

This is a Digital Logic Simulation method that eliminates unnecessary calculations to achieve huge performance gains in verifying Boolean logic:

1.) Results are only examined at the end of every clock cycle; and
2.) The digital logic is the only part of the design simulated (no timing calculations). By limiting the calculations, Cycle based Simulators can provide huge increases in performance over conventional Event-based simulators.
Cycle based simulators are more like a high speed electric carving knife in comparison because they focus on a subset of the biggest problem: logic verification.
Cycle based simulators are almost invariably used along with Static Timing verifier to compensate for the lost timing information coverage.

37)What is Constrained-Random Verification ?

Introduction

As ASIC and system-on-chip (SoC) designs continue to increase in size and complexity, there is an equal or greater increase in the size of the verification effort required to achieve functional coverage goals. This has created a trend in RTL verification techniques to employ constrained-random verification, which shifts the emphasis from hand-authored tests to utilization of compute resources. With the corresponding emergence of faster, more complex bus standards to handle the massive volume of data traffic there has also been a renewed significance for verification IP to speed the time taken to develop advanced testbench environments that include randomization of bus traffic.

Directed-Test Methodology

Building a directed verification environment with a comprehensive set of directed tests is extremely time-consuming and difficult. Since directed tests only cover conditions that have been anticipated by the verification team, they do a poor job of covering corner cases. This can lead to costly re-spins or, worse still, missed market windows.

Traditionally verification IP works in a directed-test environment by acting on specific testbench commands such as read, write or burst to generate transactions for whichever protocol is being tested. This directed traffic is used to verify that an interface behaves as expected in response to valid transactions and error conditions. The drawback is that, in this directed methodology, the task of writing the command code and checking the responses across the full breadth of a protocol is an overwhelming task. The verification team frequently runs out of time before a mandated tape-out date, leading to poorly tested interfaces. However, the bigger issue is that directed tests only test for predicted behavior and it is typically the unforeseen that trips up design teams and leads to extremely costly bugs found in silicon.

Constrained-Random Verification Methodology

The advent of constrained-random verification gives verification engineers an effective method to achieve coverage goals faster and also help find corner-case problems. It shifts the emphasis

from writing an enormous number of directed tests to writing a smaller set of constrained-random scenarios that let the compute resources do the work. Coverage goals are achieved not by the sheer weight of manual labor required to hand-write directed tests but by the number of processors that can be utilized to run random seeds. This significantly reduces the time required to achieve the coverage goals.

Scoreboards are used to verify that data has successfully reached its destination, while monitors snoop the interfaces to provide coverage information. New or revised constraints focus verification on the uncovered parts of the design under test. As verification progresses, the simulation tool identifies the best seeds, which are then retained as regression tests to create a set of scenarios, constraints, and seeds that provide high coverage of the design.

What are the differences between blocking and nonblocking assignments?

While both blocking and nonblocking assignments are procedural assignments, they differ in behaviour with respect to simulation and logic
synthesis as follows:

Table 1-3. Differences between blocking and nonblocking assignments

Blocking assignments	**Nonblocking assignments**
In a blocking assignment, the evaluation of the expression on the RHS is updated to the LHS variable autonomously based on the delay value (either 0 if no delay specified, or scheduled as a future event if a non-0 value is specified)	Nonblocking assignment to LHS is *scheduled* to occur when the next evaluation cycle occurs in simulation and not immediately. Updates are not available immediately within the same time unit
When multiple blocking assignments are present in a process, the trailing assignments are blocked from occurring until the current assignment is completed	Multiple nonblocking assignments can be scheduled to occur concurrently on the next evaluation cycle in simulation
There is a possibility of race conditions on the variables of blocking assignments if assignments happen to it from two processes concurrently	The race conditions are avoided as the updated value is assigned after evaluation
Recommended to use within combinatorial **always** blocks	Recommended to use within the sequential **always** blocks
Can be used in procedural assignments like ***initial***, ***always*** and continuous assignments to nets like ***assign*** statements	Can be used only in the procedural blocks like ***initial*** and ***always***; Continuous assignment to nets like the ***assign*** statement is not permitted
Represented by "=" operator sign between LHS and RHS	Represented by "<=" operator sign between LHS and RHS

How can I model a bi-directional net with assignments influencing both source and destination?

The assign statement constitutes a continuous assignment. The changes on the RHS of the statement immediately reflect on the LHS net. However, any changes on the LHS don't get reflected on the RHS. For example, in the following statement, changes to the rhs net will update the lhs net, but not vice versa.

System Verilog has introduced a keyword alias, which can be used only on nets to have a two-way assignment. For example, in the following code, any changes to the rhs is reflected to the lh s , and vice versa.

```
wire rhs , lhs
assign lhs=rhs;
```

System Verilog has introduced a keyword alias, which can be used only on nets to have a two-way assignment. For example, in the following code, any changes to the rhs is reflected to the lh s , and vice versa.

```
module test ();
wire rhs,lhs;


alias lhs=rhs;
```

In the above example, any change to either side of the net gets reflected on the other side.

Are tasks and functions re-entrant, and how are they different from static task and function calls?

In Verilog-95, tasks and functions were not re-entrant. From Verilog version 2001 onwards, the tasks and functions are reentrant. The reentrant tasks have a keyword automatic between the keyword task and the name of the task. The presence of the keyword automatic replicates and allocates the variables within a task dynamically for each task entry during concurrent task calls, i.e., the values don’t get overwritten for each task call. Without the keyword, the variables are allocated statically, which means these variables are shared across different task calls, and can hence get overwritten by each task call.

Reentrant task	Static task
Has the keyword ***automatic*** between the ***task*** keyword and identifier	*Doesn't* have the keyword ***autom*** between the ***task*** keyword and the identifier
Variables declared within the task are allocated dynamically for each concurrent task call	Variable declarations within the ta are allocated statically
All variables will be replicated in each concurrent call to store state specific to that invocation	Each concurrent call to the task w OVERWRITE the statically allocated local variables of the tas from all other concurrent calls to task
Variables declared are de-allocated at the end of task invocation	Variables retain their values betw invocations
Task items cannot be accessed by hierarchical inferences	Task items can be accessed by hierarchical inferences
Task items shall be allocated new across all uses of the task executing concurrently	Task items can be shared across a uses of the task executing concurrently

How can I override variables in an automatic task?

By default, all variables in a module are static, i.e., these variables will be replicated for all instances of a

module. However, in the case of task and function, either the task/function itself or the variables within them can be defined as static or automatic. The following explains the inferences through different combinations of the task/function and/or its variables, declared either as static or automatic:

No automatic definition of task/function or its variables This is the Verilog-1995 format, wherein the task/function and its variables were implicitly static. The variables are allocated only once. Without the mention of the automatic keyword, multiple calls to task/function will override their variables.

static task/function definition

System Verilog introduced the keyword static. When a task/function is explicitly defined as static, then its variables are allocated only once, and can be overridden. This scenario is exactly the same scenario as before.

automatic task/function definition

From Verilog-2001 onwards, and included within SystemVerilog, when the task/function is declared as automatic, its variables are also implicitly automatic. Hence, during multiple calls of the task/function, the variables are allocated each time and replicated without any overwrites.

static task/function and automatic variables

SystemVerilog also allows the use of automatic variables in a static task/function. Those without any changes to automatic variables will remain implicitly static. This will be useful in scenarios wherein the implicit static variables need to be initialised before the task call, and the automatic variables can be allocated each time.

automatic task/function and static variables

SystemVerilog also allows the use of static variables in an automatic task/function. Those without any changes to static variables will remain implicitly automatic. This will be useful in scenarios wherein the static variables need to be updated for each call, whereas the rest can be allocated each time.

What are the rules governing usage of a Verilog function?

The following rules govern the usage of a Verilog function construct:

A function cannot advance simulation-time, using constructs like #, @. etc.
A function shall not have nonblocking assignments.
A function without a range defaults to a one bit reg for the return value.
It is illegal to declare another object with the same name as the function in the scope where the function is declared.

How do I prevent selected parameters of a module from being overridden during instantiation?

If a particular parameter within a module should be prevented from being overridden, then it should be declared using the localparam construct, rather than the parameter construct. The localparam construct has been introduced from Verilog-2001. Note that a localparam variable is fully identical to being defined as a parameter, too. In the following example, the localparam construct is used to specify num_bits, and hence trying to override it directly gives an error message.

```
module localparam_list (addr, data);
parameter width = 32;
parameter depth = 64;
localparam num_bits = width * depth;
input  [width-1 : 0] addr;
input  [depth-1 : 0] data;
...
endmodule
```

Note, however, that, since the width and depth are specified using the parameter construct, they can be overridden during instantiation or using defparam, and hence will indirectly override the num_bits values. In general, localparam constructs are useful in defining new and localized identifiers whose values are derived from regular parameters.

What are the pros and cons of specifying the parameters using the defparam construct vs. specifying during instantiation?

The advantages of specifying parameters during instantiation method are:

All the values to all the parameters don't need to be specified. Only those parameters that are assigned the new values need to be specified. The unspecified parameters will retain their default values

specified within its module definition.

The order of specifying the parameter is not relevant anymore, since the parameters are directly specified and linked by their name.

The disadvantage of specifying parameter during instantiation are:

This has a lower precedence when compared to assigning using defparam.

The advantages of specifying parameter assignments using defparam are:

This method always has precedence over specifying parameters during instantiation.

All the parameter value override assignments can be grouped inside one module and together in one place, typically in the top-level testbench itself.

When multiple defparams for a single parameter are specified, the parameter takes the value of the last defparam statement encountered in the source if, and only if, the multiple defparam's are in the same file. If there are defparam's in different files that override the same parameter, the final value of the parameter is indeterminate.

The disadvantages of specifying parameter assignments using defparam are:

The parameter is typically specified by the scope of the hierarchies underneath which it exists. If a particular module gets ungrouped in its hierarchy, [sometimes necessary during synthesis], then the scope to specify the parameter is lost, and is unspecified. B

For example, if a module is instantiated in a simulation testbench, and its internal parameters are then overridden using hierarchical defparam constructs (For example, defparam U1.U_fifo.width = 32;). Later, when this module is synthesized, the internal hierarchy within U1 may no longer exist in the gate-level netlist, depending upon the synthesis strategy chosen. Therefore post-synthesis simulation will fail on the hierarchical defparam override.

Can there be full or partial no-connects to a multi-bit port of a module during its instantiation?

No. There cannot be full or partial no-connects to a multi-bit port of a module during instantiation

What happens to the logic after synthesis, that is driving an unconnected output port that is left open (, that is, noconnect) during its module instantiation?

An unconnected output port in simulation will drive a value, but this value does not propagate to any other logic. In synthesis, the cone of any combinatorial logic that drives the unconnected output will get

optimized away during boundary optimisation, that is, optimization by synthesis tools across hierarchical boundaries.

How is the connectivity established in Verilog when connecting wires of different widths?

When connecting wires or ports of different widths, the connections are right-justified, that is, the rightmost bit on the RHS gets connected to the rightmost bit of the LHS and so on, until the MSB of either of the net is reached.

Can I use a Verilog function to define the width of a multi-bit port, wire, or reg type?

The width elements of ports, wire or reg declarations require a constant in both MSB and LSB. Before Verilog 2001, it is a syntax error to specify a function call to evaluate the value of these widths. For example, the following code is erroneous before Verilog 2001 version.

reg [port1(val1:vla2) : port2 (val3:val4)] reg1;

In the above example, get_high and get_low are both function calls of evaluating a constant result for MSB and LSB respectively. However, Verilog-2001 allows the use of a function call to evaluate the MSB or LSB of a width declaration

What is the implication of a combinatorial feedback loops in design testability?

The presence of feedback loops should be avoided at any stage of the design, by periodically checking for it, using the lint or synthesis tools. The presence of the feedback loop causes races and hazards in the design, and 104 RTL Design
leads to unpredictable logic behavior. Since the loops are delay-dependent, they cannot be tested with any ATPG algorithm. Hence, combinatorial loops should be avoided in the logic.

What are the various methods to contain power during RTL coding?

Any switching activity in a CMOS circuit creates a momentary current flow from VDD to GND during logic transition, when both N and P type transistors are ON, and, hence, increases power consumption.
The most common storage element in the designs being the synchronous FF, its output can change whenever its data input toggles, and the clock triggers. Hence, if these two elements can be asserted in a controlled fashion, so that the data is presented to the D input of the FF only when required, and the clock is also triggered only when required, then it will reduce the switching activity, and, automatically the power.

The following bullets summarize a few mechanisms to reduce the power consumption:

- Reduce switching of the data input to the Flip-Flops.
- Reduce the clock switching of the Flip-Flops.

- Have area reduction techniques within the chip, since the number of gates/Flip-Flops that toggle can be reduced.

How do I model Analog and Mixed-Signal blocks in Verilog?

First, this is a big area.Analog and Mixed-Signal designers use tools like Spice to fully characterize and model their designs.My only involvement with Mixed-Signal blocks has been to utilize behavioral models of things like PLLs, A/Ds, D/As within a larger SoC.There are some specific Verilog tricks to this which is what this FAQ is about (I do not wish to trivialize true Mixed-Signal methodology, but us chip-level folks need to know this trick).

A mixed-signal behavioral model might model the digital and analog input/output behavior of, for example, a D/A (Digital to Analog Converter).So, digital input in and analog voltage out.Things to model might be the timing (say, the D/A utilizes an internal Success Approximation algorithm), output range based on power supply voltages, voltage biases, etc.A behavioral model may not have any knowledge of the physical layout and therefore may not offer any fidelity whatsoever in terms of noise, interface, cross-talk, etc.A model **might** be parameterized given a specific characterization for a block.Be very careful about the assumptions and limitations of the model!

Issue #1; how do we model analog voltages in Verilog.Answer: use the Verilog real data type, declare “analog wires” as wire[63:0] in order to use a 64-bit floating-type represenation, and use the built-in PLI functions:

$rtoi converts reals to integers w/truncation e.g. 123.45 -> 123

$itor converts integers to reals e.g. 123 -> 123.0

$realtobits converts reals to 64-bit vector

$bitstoreal converts bit pattern to real

That was a lot.This is a trick to be used in vanilla Verilog.The 64-bit wire is simply a ways to actually interface to the ports of the mixed-signal block.In other words, our example D/A module may have an output called AOUT which is a voltage.Verilog does not allow us to declare an output port of type REAL.So, instead declare AOUT like this:

module dtoa (clk, reset..... aout.....);

....

wire [63:0]aout;// Analog output

....

We use 64 bits because we can use floating-point numbers to represent out voltage output (e.g. 1.22x10-3 for 1.22 millivolts).The floating-point value is relevant only to Verilog and your workstation and processor, and the IEEE floating-point format has NOTHING to do with the D/A implementation.Note the disconnect in terms of the netlist itself.The physical "netlist" that you might see in GDS may have a single metal interconnect that is AOUT, and obviously NOT 64 metal wires.Again, this is a trick.The 64-bit bus is only for wiring.You may have to do some quick netlist substitutions when you hand off a netlist.

In Verilog, the **real** data type is basically a floating-point number (e.g. like double in C).If you want to model an analog value either within the mixed-signal behavorial model, or externally in the system testbench (e.g. the sensor or actuator), use the real data type.You can convert back and forth between real and your wire [63:0] using the PLI functions listed above.A trivial D/A model could simply take the digital input value, convert it to real, scale it according to some #defines, and output the value on AOUT as the 64-bit "psuedo-analog" value.Your testbench can then do the reverse and print out the value, or whatever.More sophisticated models can model the Successive Approximation algorithm, employ look-ups, equations, etc. etc.

That's it.If you are getting a mixed-signal block from a vendor, then you may also receive (or you should ask for) the behavioral Verilog models for the IP.

How do I synthesize Verilog into gates with Synopsys?

The answer can, of course, occupy several lifetimes to completely answer.. BUT.. a straight-forward Verilog module can be very easily synthesized using Design Compiler (e.g. dc_shell). Most ASIC projects will create very elaborate synthesis scripts, CSH scripts, Makefiles, etc. This is all important in order automate the process and generalize the synthesis methodology for an ASIC project or an organization. BUT don't let this stop you from creating your own simple dc_shell experiments!

Let's say you create a Verilog module named foo.v that has a single clock input named 'clk'. You want to synthesize it so that you know it is synthesizable, know how big it is, how fast it is, etc. etc. Try this:

```
target_library = { CORELIB.db } <--- This part you need to get from your vendor...

read -format verilog foo.v

create_clock -name clk -period 37.0

set_clock_skew -uncertainty 0.4 clk

set_input_delay 1.0 -clock clk all_inputs() - clk - reset
```

```
set_output_delay 1.0 -clock clk all_outputs()

compile

report_area

report_timing

write -format db -hierarchy -output foo.db

write -format verilog -hierarchy -output foo.vg

quit
```

You can enter all this in interactively, or put it into a file called 'synth_foo.scr' and then enter:

```
dc_shell -f synth_foo.scr
```

You can spend your life learning more and more Synopsys and synthesis-related commands and techniques, but don't be afraid to begin using these simple commands.

How can I pass parameters to my simulation?

A testbench and simulation will likely need many different parameters and settings for different sorts of tests and conditions. It is definitely a good idea to concentrate on a single testbench file that is parameterized, rather than create a dozen seperate, yet nearly identical, testbenches. Here are 3 common techniques:

- Use a **define**. This is almost exactly the same approach as the #define and -D compiler arg that C programs use. In your Verilog code, use a `define to define the variable condition and then use the Verilog preprocessor directives like `ifdef. Use the '+define+' Verilog command line option. For example:

... to run the simulation ..

```
verilog testbench.v cpu.v +define+USEWCSDF

... in your code ...
```

`ifdef USEWCSDF

initial $sdf_annotate (testbench.cpu, "cpuwc.sdf");

`endif

The +define+ can also be filled in from your Makefile invocation, which in turn, can be finally filled in the your UNIX promp command line.

Defines are a blunt weapon because they are very global and you can only do so much with them since they are a pre-processor trick. Consider the next approach before resorting to defines.

- Use **parameters** and parameter definition modules. Parameters are not preprocessor definitions and they have *scope* (e.g. parameters are associated with specific modules). Parameters are therefore more clean, and if you are in the habit of using a lot of defines; consider switching to parameters. As an example, lets say we have a test (e.g. test12) which needs many parameters to have particular settings. In your code, you might have this sort of stuff:

  module testbench_uart1 (....)

  parameter BAUDRATE = 9600;

  ...

  if (BAUDRATE > 9600) begin

  ... E.g. use the parameter in your code like you might any general variable

  ... BAUDRATE is completely local to this module and this instance. You might

  ... have the same parameters in 3 other UART instances and they'd all be different

  ... values...

  Now, your test12 has all kinds of settings required for it. Let's define a special module

  called **testparams** which specifies all these settings. It will itself be a module instantiated

under the testbench:

```
module testparams;
defparam testbench.cpu.uart1.BAUDRATE = 19200;
defparam testbench.cpu.uart2.BAUDRATE = 9600;
defparam testbench.cpu.uart3.BAUDRATE = 9600;
defparam testbench.clockrate CLOCKRATE = 200; // Period in ns.
... etc ...
endmodule
```

The above module always has the same module name, but you would have many different filenames; one for each test. So, the above would be kept in test12_params.v. Your Makefile includes the appropriate params file given the desired make target. (BTW: You may run across this sort of technique by ASIC vendors who might have a module containing parameters for a memory model, or you might see this used to collect together a large number of system calls that turn off timing or warnings on particular troublesome nets, etc. etc.)

- Use **memory blocks**. Not as common a technique, but something to consider. Since Verilog has a very convenient syntax for declaring and loading memories, you can store your input data in a hex file and use $readmemh to read all the data in at once.

In your testbench:

```
module testbench;
...
```

```
reg [31:0] control[0:1023];

...

initial $readmemh ("control.hex", control);

...

endmodule
```

You could vary the filename using the previous techniques. The control.hex file is just a file of hex values for the parameters. Luckily, $readmemh allows embedded comments, so you can keep the file very readable:

```
A000 // Starting address to put boot code in

10 // Activate all ten input pulse sources

... etc...
```

Obviously, you are limitied to actual hex values with this approach. Note, of course, that you are free to mix and match all of these techniques!

Verilog gate level expected questions.

1) Tell something about why we do gate level simulations?

a. Since scan and other test structures are added during and after synthesis, they are not checked by the rtl simulations and therefore need to be verified by gate level simulation.
b. Static timing analysis tools do not check asynchronous interfaces, so gate level simulation is required to look at the timing of these interfaces.
c. Careless wildcards in the static timing constraints set false path or mutlicycle path constraints where they don't belong.

d. Design changes, typos, or misunderstanding of the design can lead to incorrect false paths or multicycle paths in the static timing constraints.

e. Using create_clock instead of create_generated_clock leads to incorrect static timing between clock domains.

f. Gate level simulation can be used to collect switching factor data for power estimation.

g. X's in RTL simulation can be optimistic or pessimistic. The best way to verify that the design does not have any unintended dependence on initial conditions is to run gate level simulation.

f. It's a nice "warm fuzzy" that the design has been implemented correctly.

2) Say if I perform Formal Verification say Logical Equivalence across Gatelevel netlists(Synthesis and post routed netlist). Do you still see a reason behind GLS.?

If we have verified the Synthesized netlist functionality is correct when compared to RTL and when we compare the Synthesized netlist versus Post route netlist logical Equivalence then I think we may not require GLS after P & R. But how do we ensure on Timing . To my knowledge Formal Verification Logical Equivalence Check does not perform Timing checks and dont ensure that the design will work on the operating frequency , so still I would go for GLS after post route database.

3)An AND gate and OR gate are given inputs X & 1 , what is expected output?

AND Gate output will be X

OR Gate output will be 1.

4) What is difference between NMOS & RNMOS?

RNMOS is resistive nmos that is in simulation strength will decrease by one unit , please refer to below Diagram.

Input Strength	Output Strength
supply	pull
strong	pull
pull	weak
weak	medium
large	medium
medium	small
small	small
highz	highz

4) Tell something about modeling delays in verilog?

Verilog can model delay types within its specification for gates and buffers. Parameters that can be modelled are T_rise, T_fall and T_turnoff. To add further detail, each of the three values can have minimum, typical and maximum values

T_rise, t_fall and t_off

Delay modelling syntax follows a specific discipline;
gate_type #(t_rise, t_fall, t_off) gate_name (paramters);
When specifiying the delays it is not necessary to have all of the delay values specified. However, certain rules are followed.

and #(3) gate1 (out1, in1, in2);
When only 1 delay is specified, the value is used to represent all of the delay types, i.e. in this example, t_rise = t_fall = t_off = 3.

or #(2,3) gate2 (out2, in3, in4);
When two delays are specified, the first value represents the rise time, the second value represents the fall time. Turn off time is presumed to be 0.

buf #(1,2,3) gate3 (out3, enable, in5);
When three delays are specified, the first value represents t_rise, the second value represents t_fall and the last value the turn off time.

Min, typ and max values

The general syntax for min, typ and max delay modelling is;
gate_type #(t_rise_min:t_ris_typ:t_rise_max, t_fall_min:t_fall_typ:t_fall_max, t_off_min:t_off_typ:t_off_max) gate_name (paramteters);

Similar rules apply for th especifying order as above. If only one t_rise value is specified then this value is applied to min, typ and max. If specifying more than one number, then all 3 MUST be scpecified. It is incorrect to specify two values as the compiler does not know which of the parameters the value represents.

An example of specifying two delays;
and #(1:2:3, 4:5:6) gate1 (out1, in1, in2);
This shows all values necessary for rise and fall times and gives values for min, typ and max for both delay types.

Another acceptable alternative would be;
or #(6:3:9, 5) gate2 (out2, in3, in4);
Here, 5 represents min, typ and max for the fall time.

N.B. T_off is only applicable to tri-state logic devices, it does not apply to primitive logic gates because they cannot be turned off.

5) With a specify block how to defining pin-to-pin delays for the module ?

```
module A( q, a, b, c, d )
        input a, b, c, d;
        output q;
        wire e, f;
        // specify block containing delay statements
        specify
                ( a => q ) = 6;   // delay from a to q
                ( b => q ) = 7;   // delay from b to q
                ( c => q ) = 7;   // delay form c to q
                ( d => q ) = 6;   // delay from d to q
        endspecify
        // module definition
        or o1( e, a, b );
        or o2( f, c, d );
        exor ex1( q, e, f );
endmodule
```

```
module A( q, a, b, c, d )
        input a, b, c, d;
        output q;
        wire e, f;
        // specify block containing full connection statements
        specify
                ( a, d *> q ) = 6;      // delay from a and d to q
```

```
        ( b, c *> q ) = 7;     // delay from b and c to q
    endspecify
    // module definition
    or o1( e, a, b );
    or o2( f, c, d );
    exor ex1( q, e, f );
endmodule
```

6) What are conditional path delays?

Conditional path delays, sometimes called *state dependent path delays*, are used to model delays which are dependent on the values of the signals in the circuit. This type of delay is expressed with an **if** conditional statement. The operands can be scalar or vector module input or inout ports, locally defined registers or nets, compile time constants (constant numbers or specify block parameters), or any bit-select or part-select of these. The conditional statement can contain any bitwise, logical, concatenation, conditional, or reduction operator. The **else** construct cannot be used.

```
//Conditional path delays
Module A( q, a, b, c, d );
    output q;
    input a, b, c, d;
    wire e, f;
    // specify block with conditional timing statements
    specify
        // different timing set by level of input a
        if (a) ( a => q ) = 12;
        if ~(a) ( a => q ) = 14;
        // delay conditional on b and c
        // if b & c is true then delay is 7 else delay is 9
        if ( b & c ) ( b => q ) = 7;
        if ( ~( b & c )) ( b => q ) = 9;
        // using the concatenation operator and full connections
        if ( {c, d} = 2'b10 ) ( c, d *> q ) = 15;
        if ( {c, d} != 2'b10 ) ( c, d *> q ) = 12;
    endspecify
    or o1( e, a, b );
    or o2( f, c, d );
    exor ex1( q, e, f );
endmodule
```

6) Tell something about Rise, fall, and turn-off delays?

Timing delays between pins can be expressed in greater detail by specifying rise, fall, and turn-off delay values. One, two, three, six, or twelve delay values can be specified for any path. The order in which the delay values are specified must be strictly followed.

```
// One delay used for all transitions
specparam delay = 15;
( a => q ) = delay;
// Two delays gives rise and fall times
specparam rise = 10, fall = 11;
( a => q ) = ( rise, fall );
// Three delays gives rise, fall and turn-off
// rise is used for 0-1, and z-1, fall for 1-0, and z-0, and turn-off for 0-z, and 1-z.
specparam rise = 10, fall = 11, toff = 8;
( a => q ) = ( rise, fall, toff );
// Six delays specifies transitions 0-1, 1-0, 0-z, z-1, 1-z, z-0
// strictly in that order
specparam t01 = 8, t10 = 9, t0z = 10, tz1 = 11, t1z = 12, tz0 = 13;
( a => q ) = ( t01, t10, t0z, tz1, t1z, tz0 );
// Twelve delays specifies transitions:
// 0-1, 1-0, 0-z, z-1, 1-z, z-0, 0-x, x-1, 1-x, x-0, x-z, z-x
// again strictly in that order
specparam t01 = 8, t10 = 9, t0z = 10, tz1 = 11, t1z = 12, tz0 = 13;
specparam t0x = 11, tx1 = 14, t1x = 12, tx0 = 10, txz = 8, tzx = 9;
( a => q ) = ( t01, t10, t0z, tz1, t1z, tz0, t0x, tx1, t1x, tx0, txz, tzx );
```

7)Tell me about In verilog delay modeling?

Distributed Delay

Distributed delay is delay assigned to each gate in a module. An example circuit is shown below.

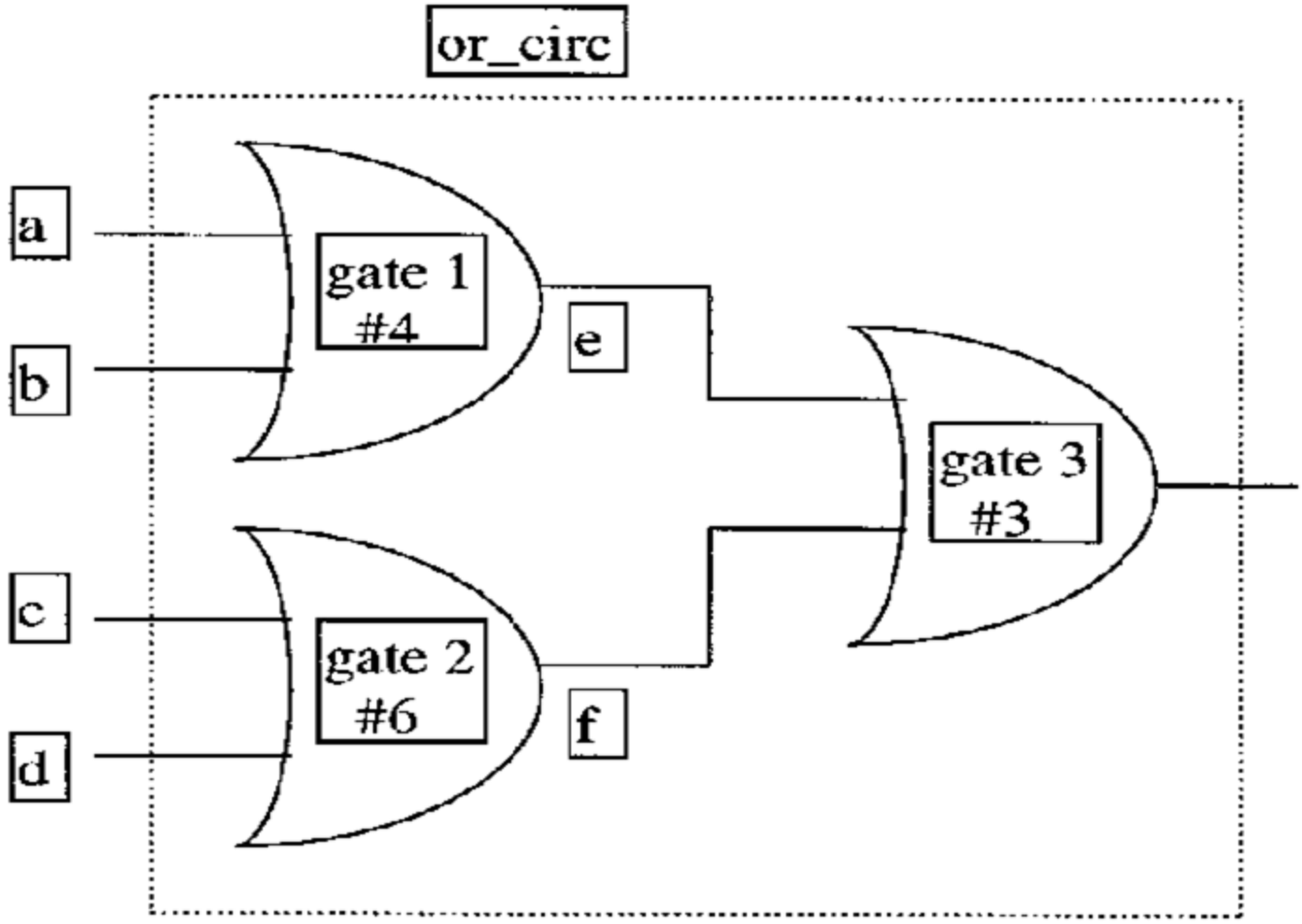


Figure 1: Distributed delay

As can be seen from Figure 1, each of the or-gates in the circuit above has a delay assigned to it:

- gate 1 has a delay of 4
- gate 2 has a delay of 6
- gate 3 has a delay of 3

When the input of any gate change, the output of the gate changes after the delay value specified.

The gate function and delay, for example for gate 1, can be described in the following manner:

or #4 a1 (e, a, b);

A delay of 4 is assigned to the or-gate. This means that the output of the gate, e, is delayed by 4 from the inputs a and b.

The module explaining Figure 1 can be of two forms:

```
1)
Module or_circ (out, a, b, c, d);
output out;
input a, b, c, d;
wire e, f;
        //Delay distributed to each gate
        or #4 a1 (e, a, b);
        or #6 a2 (f, c, d);
        or #3 a3 (out, e, f);
endmodule
2)
Module or_circ (out, a, b, c, d);
output out;
input a, b, c, d;
wire e, f;
        //Delay distributed to each expression
        assign #4 e = a & b;
        assign #6 e = c & d;
        assign #3 e = e & f;
endmodule
```

Version 1 models the circuit by assigning delay values to individual gates, while version 2 use delay values in individual assign statements. (An assign statement allows us to describe a combinational logic function without regard to its actual structural implementation. This means that the assign statement does not contain any modules with port connections.)

The above or_circ modules results in delays of (4+3) = 7 and (6+3) = 9 for the 4 connections part from the input to the output of the circuit.

Lumped Delay

Lumped delay is delay assigned as a single delay in each module, mostly to the output gate of the module. The cumulative delay of all paths is lumped at one location. The figure below is an example of lumped delay. This figure is similar as the figure of the

distributed delay, but with the sum delay of the longest path assigned to the output gate: (delay of gate 2 + delay of gate 3) = 9.

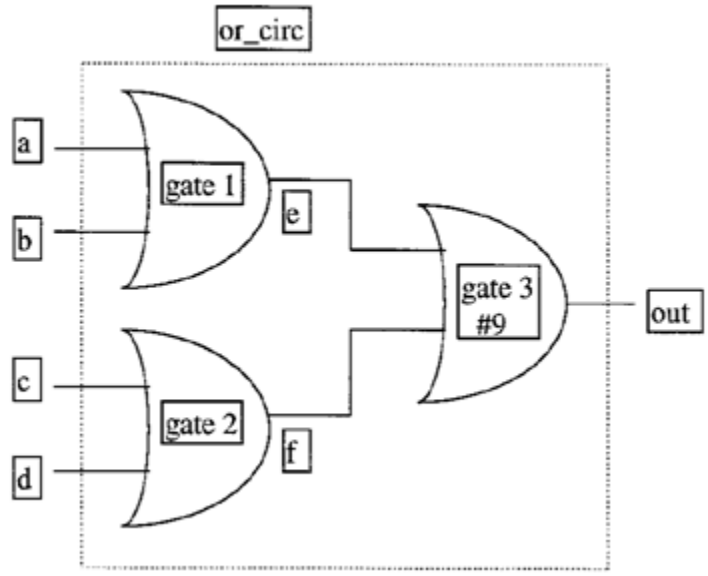


Figure 2: Lumped delay

As can be seen from Figure 2, gate 3 has got a delay of 9. When the input of this gate changes, the output of the gate changes after the delay value specified.

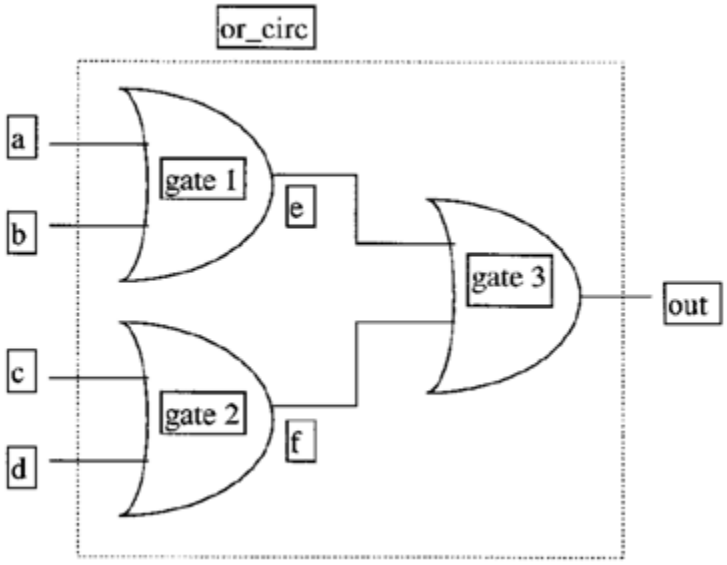
The program corresponding to Figure 2, is very similar to the one for distributed delay. The difference is that only or - gate 3 has got a delay assigned to it:

```
1)
Module or_circ (out, a, b, c, d);
output out;
input a, b, c, d;
wire e, f;
        or a1 (e, a, b);
        or a2 (f, c, d);
        or #9 a3 (out, e, f); //delay only on the output gate
endmodule
```

This model can be used if delay between different inputs is not required.

Pin - to Pin Delay

Pin - to - Pin delay, also called path delay, is delay assigned to paths from each input to each output. An example circuit is shown below.



path a - e - out, delay = 7
path b - e - out, delay = 7
path c - f - out, delay = 9
path d - f - out, delay = 9

Figure 3: Pin - to Pin delay

The total delay from each input to each output is given. The same example circuit as for the distributed and lumped delay model is used. This means that the sum delay from each input to each output is the same.

The module for the above circuit is shown beneath:

```
Module or_circ (out, a, b, c, d);
output out;
input a, b, c, d;
wire e, f;
//Blocks specified with path delay
specify
        (a => out) = 7;
        (b => out) = 7;
        (c => out) = 9;
        (d => out) = 9;
endspecify
//gate calculations
```

or a1(e, a, b);
or a2(f, c, d);
or a3(out, e, f);
endmodule

Path delays of a module are specified incide a specify block, as seen from the example above. An example of delay from the input, a, to the output, out, is written as (a => out) = delay, where delay sets the delay between the two ports. The gate calculations are done after the path delays are defined.

For larger circuits, the pin - to - pin delay can be easier to model than distributed delay. This is because the designer writing delay models, needs to know only the input / output pins of the module, rather than the internals of the module. The path delays for digital circuits can be found through different simulation programs, for instance SPICE. Pin - to - Pin delays for standard parts can be found from data books. By using the path delay model, the program speed will increase.

8) Tell something about delay modeling timing checks?

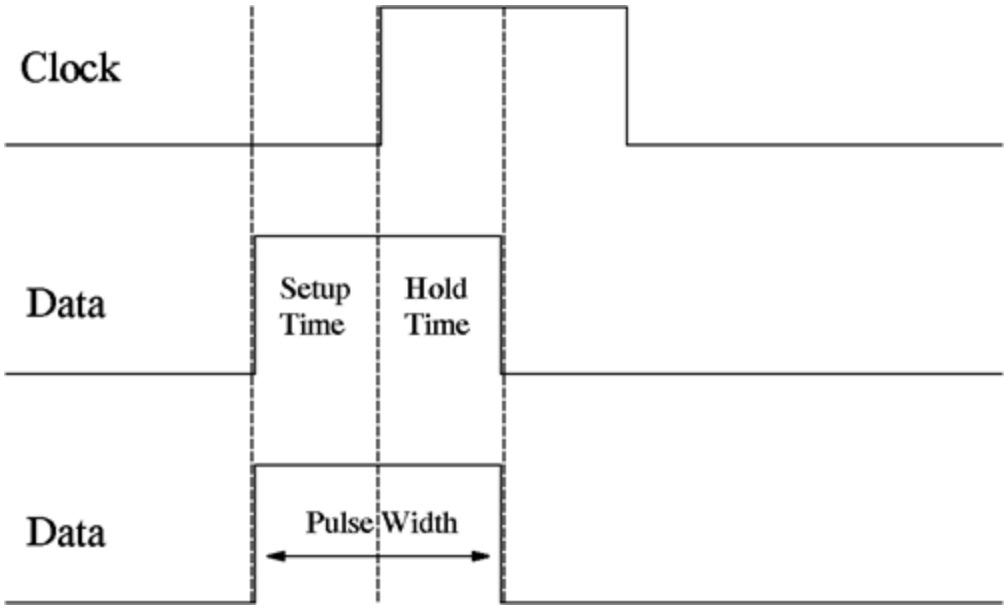
Delay Modeling: Timing Checks.

Keywords: $setup, $hold, $width

This section, the final part of the delay modeling chapter, discusses some of the various system tasks that exist for the purposes of timing checks. Verilog contains many timing-check system tasks, but only the three most common tasks are discussed here: **$setup**, **$hold** and **$width**. Timing checks are used to verify that timing constraints are upheld, and are especially important in the simulation of high-speed sequential circuits such as microprocessors. All timing checks must be contained within **specify** blocks as shown in the example below.

The **$setup** and **$hold** tasks are used to monitor the *setup* and *hold* constraints during the simulation of a sequential circuit element. In the example, the setup time is the minimum allowed time between a change in the input *d* and a positive clock edge. Similarly, the hold time is the minimum allowed time between a positive clock edge and a change in the input *d*.

The **$width** task is used to check the minimum width of a positive or negative-going pulse. In the example, this is the time between a negative transition and the transition back to 1.



Syntax:

NB: *data_change*, *reference* and *reference1* must be declared *wire*s.

$setup(*data_change*, *reference*, *time_limit*);

data_change: signal that is checked against the *reference*

reference: signal used as reference

time_limit: minimum time required between the two events.

Violation if: Treference - Tdata_change < time_limit.

$hold(*reference*, *data_change*, *time_limit*);

reference: signal used as reference

data_change: signal that is checked against the *reference*

time_limit: minimum time required between the two events.

Violation if: Tdata_change - Treference < time_limit

$width(*reference1*, *time_limit*);

reference1: first transition of signal

time_limit: minimum time required between *transition1* and *transition2*.

Violation if: Treference2 - Treference1 < time_limit

Example:

```
module d_type(q, clk, d);
  output q;
  input  clk, d;

  reg    q;

  always @(posedge clk)
    q = d;

endmodule // d_type

module stimulus;

  reg  clk, d;
  wire q, clk2, d2;

  d_type dt_test(q, clk, d);

  assign d2=d;
  assign clk2=clk;

  initial
    begin
      $display ("\t\t     clock d q");
      $display ($time,"   %b   %b %b", clk, d, q);
      clk=0;
      d=1;
      #7 d=0;
      #7 d=1; // causes setup violation
      #3 d=0;
      #5 d=1; // causes hold violation
      #2 d=0;
      #1 d=1; // causes width violation
    end // initial begin

  initial
```

```
    #26 $finish;

  always
    #3 clk = ~clk;

  always
    #1 $display ($time,"   %b   %b %b", clk, d, q);

  specify
    $setup(d2, posedge clk2, 2);
    $hold(posedge clk2, d2, 2);
    $width(negedge d2, 2);
  endspecify
endmodule // stimulus
```

Output:

```
                clock d q
               0   x   x x
               1   0   1 x
               2   0   1 x
               3   1   1 x
               4   1   1 1
               5   1   1 1
               6   0   1 1
               7   0   0 1
               8   0   0 1
               9   1   0 1
              10   1   0 0
              11   1   0 0
              12   0   0 0
              13   0   0 0
              14   0   1 0
              15   1   1 0

"timechecks.v", 46: Timing violation in stimulus
   $setup( d2:14, posedge clk2:15, 2 );

              16   1   1 1
              17   1   0 1
              18   0   0 1
              19   0   0 1
              20   0   0 1
              21   1   0 1
              22   1   1 0
```

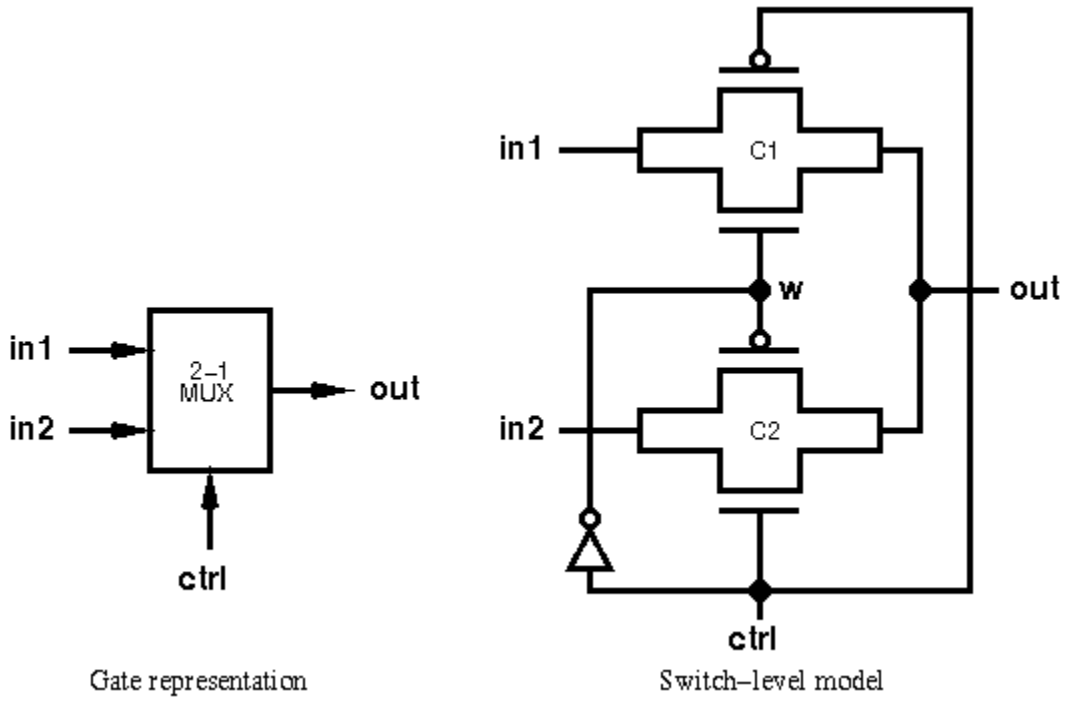
"timechecks.v", 47: Timing violation in stimulus
    $hold(posedge clk2:21, d2:22, 2);

                23   1   1 0
                24   0   0 0
                25   0   1 0

"timechecks.v", 48: Timing violation in stimulus
    $width(negedge d2:24, : 25, 2);

9) Draw a 2:1 mux using switches and verilog code for it?

1-bit 2-1 Multiplexer



This circuit assigns the output **out** to either inputs **in1** or **in2** depending on the low or high values of **ctrl** respectively.

// Switch-level description of a 1-bit 2-1 multiplexer
// ctrl=0, out=in1; ctrl=1, out=in2

module mux21_sw (out, ctrl, in1, in2);

```
  output out;                  // mux output
  input  ctrl, in1, in2;       // mux inputs
  wire       w;                // internal wire

  inv_sw I1 (w, ctrl);         // instantiate inverter module

  cmos C1 (out, in1, w, ctrl);  // instantiate cmos switches
  cmos C2 (out, in2, ctrl, w);
```

endmodule

An inverter is required in the multiplexer circuit, which is instantiated from the previously defined module.

Two transmission gates, of instance names C1 and C2, are implemented with the *cmos* statement, in the format cmos [instancename]([output],[input],[nmosgate],[pmosgate]). Again, the instance name is optional.

10)What are the synthesizable gate level constructs?

The <GATETYPE> Keywords				
and	buf	nmos	tran	pullup
nand	not	pmos	tranif0	pulldown
nor	bufif0	cmos	tranif1	
or	bufif1	rnmos	rtran	
xor	notif0	rpmos	rtranif0	
xnor	notif1	rcmos	rtranif1	

The above table gives all the gate level constructs of only the constructs in first two columns are synthesizable.

Verilog Coding Guidelines -Part 1

1. Naming Conventions

1.1 Character set

Use only the characters [a-z][A-Z][0-9] $ and "_" in the identifiers used for naming module, ports, wires,

regs, blocks etc.

Do not use escaped identifiers to include special characters in identifiers. Do not use the character "_" as the first or last character of an identifier. Do not use numerals as first character.

Do not use capital letters for identifier except Parameter and define

Example:conventions.v

```
module conventions;
reg [7:0] a,a$b,b,c;
reg clk;

initial
 begin
      clk=0;
      c = 1;
      forever  #25 clk = !clk;
 end


always @ (posedge clk)
 begin
      a = b;
      b = c;
 end
endmodule
```

1.2 Case sensitive

Use lower case letters for all identifiers leaving the upper case letters for macros and parameters. Do not use the mixed case style. Also, ensure that all the identifiers in the design are unique even in a case insensitive environment.

Example:

module // keyword Module // unique identifier but not keyword

MODULE // unique identifier but not keyword Identifier

Name: fifoReadPointer. Use: fifo_read_pointer- instead.

1.3 No keywords

Do not use Verilog keywords as identifiers.
Avoid keywords from both the HDLs as RTL code of a re-usable design may have to be made available in both languages.

Example:

input –keyword

output –keyword

1.4 Use meaningful Names

Create identifiers by concatenating meaningful words or commonly used acronyms separated by character "_".

Example:

Use en_state_transition instead of est or en_st_trn.

1.5 Identifier length, and number of parameters

Do not to use very long identifiers. This is especially true for parameters. Design unit names of parameterized modules are created by concatenating instance names, values and parameter names during design elaboration. Limit the maximum number of characters in an identifier to 25.

1.6 Parameter/Define naming convention

Parameter and Define must be declared in Capital Letter.

Example:

Parameter DATA_WIDTH=3'b111 ; `define EXAMPLE

1.7 Module names

Name the top level module of the design as _top. Module name & file name must be identical This is typically the module containing IO buffers and other technology- dependent components in addition to module _core. Module _core should contain only technology independent portion of the design. Name the modules created as macro wrappers _wrap.

Example:

module test (port1,port2,…);
……
……

.....
endmodule

The file should be saved as test.v

1.8 Instance names

If the module has single instance in that scope use inst_ as instance name. If there are more than one instance, then add meaningful suffixes to uniquify the names. Remember that the instance name in gate level netlist is a concatenation of RTL instance name and all the parameter ids and values in the instantiated module.

- A module may be instantiated within another module
- There may be multiple instances of the same module
- Ports are either by order or by name
- Use by order unless there are lots of ports
- Can not mix the two syntax's in one instantiation
- Always use module name as instance name.

Example:

memory memory_instance

syntax for instantiation with port order:
module_name instance_name (signal, signal...);

syntax for instantiation with port name:
module_name instance_name (.port_name(signal), .port_name (signal)…);

1.9 Blocks names

Label all the always blocks in the RTL code with meaningful names. This will be very useful for grouping/ungrouping of the design in synthesis tool and will result in better error/info messages. It is a standard practice to append the block labels with "_comb" or "_seq" depending on whether it is combinatorial or sequential.

Example:

```
module block_names;
reg [7:0] a,b,c; // register declarations
reg clk;

always@(b or c)
 begin:comb
      c<=a;
       b<=a;
 end
always @ (posedge clk):
 begin:seq
       a <= b;
       b <= c;
 end
endmodule
```

1.10 Global signals

Keep same names for global signals (rst, clk etc.) in all the hierarchies of the design.
This should be true for any signal which are used in multiple design hierarchies. The actuals and formals in instantiation port maps should be the same IDs.

1.11 Clock signals

Name the clock signal as clk if there is only one clock in the design. In case of multiple clocks, use _clk as suffix.

Example:

pci_clk, vci_clk.

Never include the clock frequecy in clock signal name (40MHz_clk) since clock frequencies often change in the middle of the design cycle.

1.12 Reset signals

Name the reset signal as rst if there is only one reset in the design. In case of multiple resets, use _rst as suffix.

Example:

pci_rst, vci_rst.

1.13 Active low signals

All signals are lowercase alpha, numeric and underscore only. Use _n as suffix.

Example:

intr_n, rst_n, irdy_n.

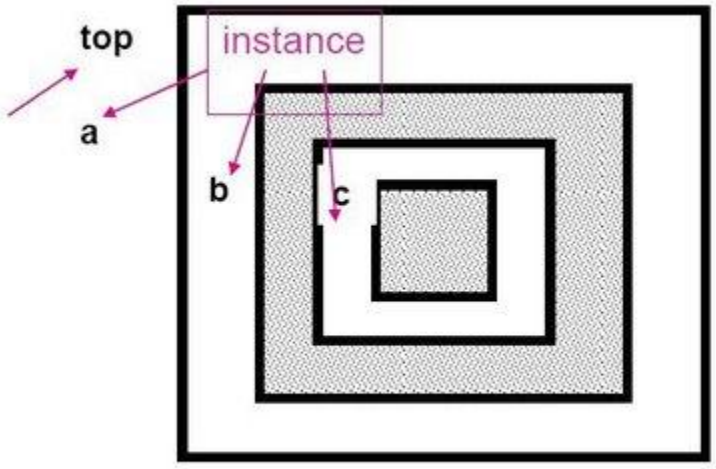
Avoid using characters ’#’ or ’N’ as suffixes even in documents.

1.14 Module Hierarchy

A hierarchical path in Verilog is in form of:

module_name.instance_name.instance_name

top.a.b.c is the path for the hierarchy below.



1.15 Use of Macros

Macros are required to be used for any non-trivial constants, and for all bit-ranges. This rule is essential both for readability and maintainability of code. Having two inter-connected modules, each of which defines a bus as '17:0' is a recipe for disaster. Busses are preferably defined with a scheme such as the following:

`define BUS_MSB 17

`define BUS_LSB 0

`define BUS_SIZE (`BUS_MSB-`BUS_LSB+1)

`define BUS_RANGE `BUS_MSB:`BUS_LSB

This will minimize the number of places that have to be changed if the bus size must be changed.

1.16 MEMORY DECLARTION

Memories are declared as two-dimensional arrays of registers.

syntax: reg [msb:lsb] identifier [first_addr:last_addr] ;

where msb:lsb determine the width (word size) of the memory first_addr:last_addr determine the depth (address range) of the memory



1.17 Abbreviation

Use consistent abbreviation as shown:

Signal Naming Abbreviation

Signal/CellType	Abbreviation	Signal/Cell	Type Abbreviatio
address	addr	shift	shift
IP address	ipaddr	chip select	cs
data	data	output enable	oe
control	ctrl	clock	clk
master	mst	comparator	comp
port	port	bank	bank
pipe	pip	adder/addition	add
packet	pack	subtractor/subtract	sub
counter	cnt	buffer	buff
count enable	cnten	fifo	fifo
load enable	lden	input	in
ready	rdy	output	out
early	erly	bidirectional	bidir
multiplexor	mux	main memory	mm
multiplexor select	muxsel	interface	if
register	reg (more stuff here...)		

Posted by . at 4:12 AM 0 comments

Labels: Coding Guidelines

Tuesday, July 15, 2008

Verilog 8

1. How to generate random number in Verilog.

2.Is this code is synthesizable?
always@(negedge clk or rst)

3.What is a code coverage and list the types.

4. How to swap 2 variables A and B without using 3 variable.

5. Write Verilog Code to generate 80 MHZ clock with 50% duty cycle.

Posted by . at 11:46 AM 0 comments

Labels: Verilog

Thursday, July 10, 2008

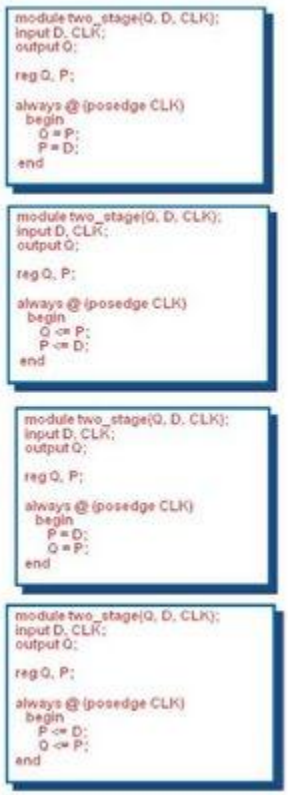
Verilog 7

Consider the following code,
always@(posedge clk)
begin
a=b;
b=c;
c=a;
end
What logic does the code implies.

Posted by . at 3:38 AM 1 comments

Labels: Verilog

Synthesis 1



What is FPGA ?

A field-programmable gate array is a semiconductor device containing programmable logic components called "logic blocks", and programmable interconnects. Logic blocks can be programmed to perform the function of basic logic gates such as AND, and XOR, or more complex combinational functions such as decoders or mathematical functions. In most FPGAs, the logic blocks also include memory elements, which may be simple flip-flops or more complete blocks of memory. A hierarchy of programmable interconnects allows logic blocks to be interconnected as needed by the system designer, somewhat like a one-chip programmable breadboard. Logic blocks and interconnects can be programmed by the customer or designer, after the FPGA is manufactured, to implement any logical function—hence the name "field-programmable". FPGAs are usually slower than their application-specific integrated circuit (ASIC) counterparts, cannot handle as complex a design, and draw more power (for any given semiconductor process). But their advantages include a shorter time to market, ability to re-program in the field to fix bugs, and lower non-recurring engineering costs. Vendors can sell cheaper, less flexible versions of their FPGAs which cannot be modified after the design is committed. The designs are developed on regular FPGAs and then migrated into a fixed version that more resembles an ASIC.

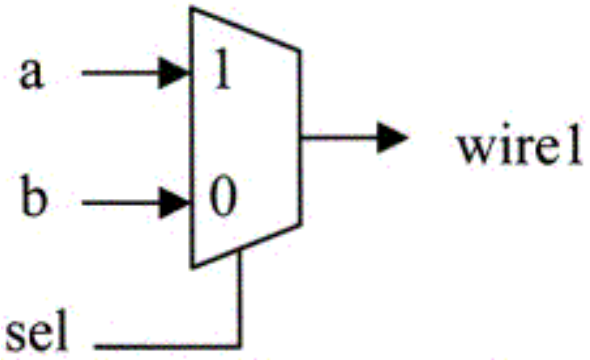
What logic is inferred when there are multiple assign statements targeting the same wire?

It is illegal to specify multiple assign statements to the same wire in a synthesizable code that will become an output port of the module. The synthesis tools give a syntax error that a net is being driven by more than one source.
However, it is legal to drive a three-state wire by multiple assign statements.

What do conditional assignments get inferred into?

Conditionals in a continuous assignment are specified through the “?:” operator. Conditionals get inferred into a multiplexor. For example, the following is the code for a simple multiplexor

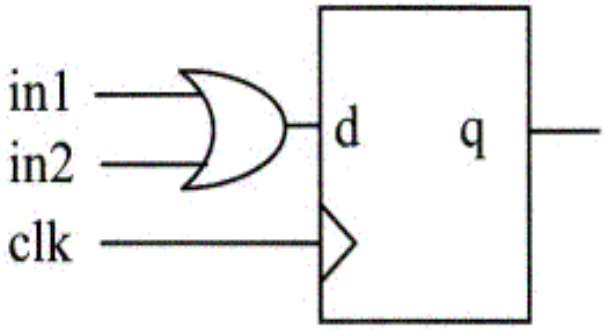
```
assign wire1 = (sel==1'b1) ? a : b;
```



What value is inferred when multiple procedural assignments made to the same reg variable in an always block?

When there are multiple nonblocking assignments made to the same reg variable in a sequential always block, then the last assignment is picked up for logic synthesis. For example

```
always @ (posedge clk) begin
out <= in1^in2;
out <= in1 &in2;
out <= in1|in2;
```



In the example just shown, it is the OR logic that is the last assignment. Hence, the logic synthesized was indeed the OR gate. Had the last assignment been the "&" operator, it would have synthesized an AND gate.

1) What is minimum and maximum frequency of dcm in spartan-3 series fpga?

Spartan series dcm's have a minimum frequency of 24 MHZ and a maximum of 248

2)Tell me some of constraints you used and their purpose during your design?

There are lot of constraints and will vary for tool to tool ,I am listing some of Xilinx constraints
a) Translate on and Translate off: the Verilog code between Translate on and Translate off is ignored for synthesis.
b) CLOCK_SIGNAL: is a synthesis constraint. In the case where a clock signal goes through combinatorial logic before being connected to the clock input of a flip-flop, XST cannot identify what input pin or internal net is the real clock signal. This constraint allows you to define the clock net.
c) XOR_COLLAPSE: is synthesis constraint. It controls whether cascaded XORs should be collapsed into a single XOR.
For more constraints detailed description refer to constraint guide.

3) Suppose for a piece of code equivalent gate count is 600 and for another code equivalent gate count is 50,000 will the size of bitmap change?in other words will size of bitmap change it gate count change?

The size of bitmap is irrespective of resource utilization, it is always the same,for Spartan xc3s5000 it is 1.56MB and will never change.

4) What are different types of FPGA programming modes?what are you currently using ?how to change from one to another?

Before powering on the FPGA, configuration data is stored externally in a PROM or some other nonvolatile medium either on or off the board. After applying power, the configuration data is written to the FPGA using any of five different modes: Master Parallel, Slave Parallel, Master Serial, Slave Serial, and Boundary Scan (JTAG). The Master and Slave Parallel modes
Mode selecting pins can be set to select the mode, refer data sheet for further details.

5) Tell me some of features of FPGA you are currently using?

I am taking example of xc3s5000 to answering the question .

Very low cost, high-performance logic solution for
high-volume, consumer-oriented applications
- Densities as high as 74,880 logic cells
- Up to 784 I/O pins
- 622 Mb/s data transfer rate per I/O
- 18 single-ended signal standards
- 6 differential I/O standards including LVDS, RSDS
- Termination by Digitally Controlled Impedance
- Signal swing ranging from 1.14V to 3.45V
- Double Data Rate (DDR) support
• Logic resources
- Abundant logic cells with shift register capability
- Wide multiplexers
- Fast look-ahead carry logic
- Dedicated 18 x 18 multipliers
- Up to 1,872 Kbits of total block RAM
- Up to 520 Kbits of total distributed RAM
• Digital Clock Manager (up to four DCMs)
- Clock skew elimination
• Eight global clock lines and abundant routing

6) What is gate count of your project?

Well mine was 3.2 million, I don't know yours.!

7) Can you list out some of synthesizable and non synthesizable constructs?

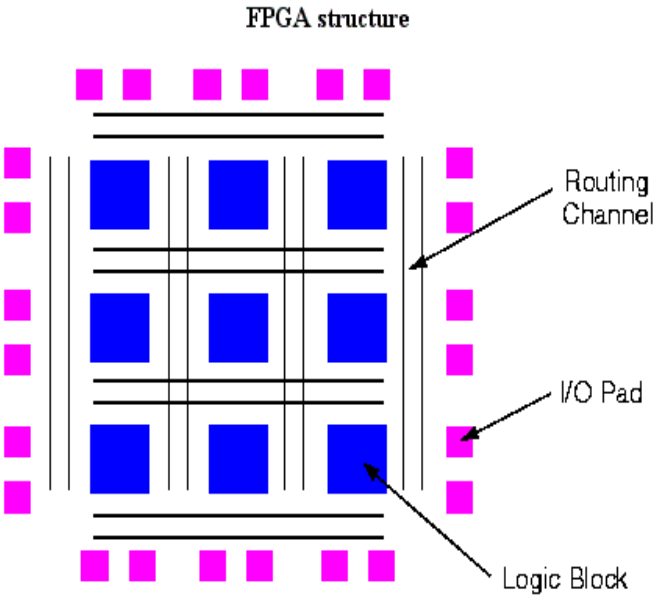
not synthesizable->>>>
initial
ignored for synthesis.
delays
ignored for synthesis.
events
not supported.
real
Real data type not supported.
time
Time data type not supported.
force and release
Force and release of data types not supported.
fork join
Use nonblocking assignments to get same effect.
user defined primitives
Only gate level primitives are supported.

synthesizable constructs->>
assign,for loop,Gate Level Primitives,repeat with constant value...

8)Can you explain what struck at zero means?

These stuck-at problems will appear in ASIC. Some times, the nodes will permanently tie to 1 or 0 because of some fault. To avoid that, we need to provide testability in RTL. If it is permanently 1 it is called stuck-at-1 If it is permanently 0 it is called stuck-at-0.

9) Can you draw general structure of fpga?



10) Difference between FPGA and CPLD?

FPGA:
a)SRAM based technology.
b)Segmented connection between elements.
c)Usually used for complex logic circuits.
d)Must be reprogrammed once the power is off.
e)Costly

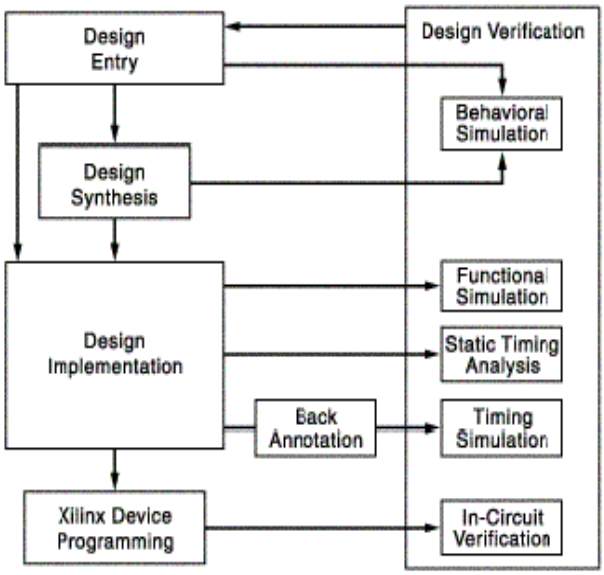
CPLD:
a)Flash or EPROM based technology.
b)Continuous connection between elements.
c)Usually used for simpler or moderately complex logic circuits.
d)Need not be reprogrammed once the power is off.
e)Cheaper

11) What are dcm's?why they are used?

Digital clock manager (DCM) is a fully digital control system that
uses feedback to maintain clock signal characteristics with a
high degree of precision despite normal variations in operating
temperature and voltage.
That is clock output of DCM is stable over wide range of temperature and voltage , and also skew

associated with DCM is minimal and all phases of input clock can be obtained . The output of DCM coming form global buffer can handle more load.

12) FPGA design flow?



Also,Please refer to presentation section synthesis ppt on this site.

13)what is slice,clb,lut?

I am taking example of xc3s500 to answer this question

The Configurable Logic Blocks (CLBs) constitute the main logic resource for implementing synchronous as well as combinatorial circuits.
CLB are configurable logic blocks and can be configured to combo,ram or rom depending on coding style CLB consist of 4 slices and each slice consist of two 4-input LUT (look up table) F-LUT and G-LUT.

14) Can a clb configured as ram?

YES.

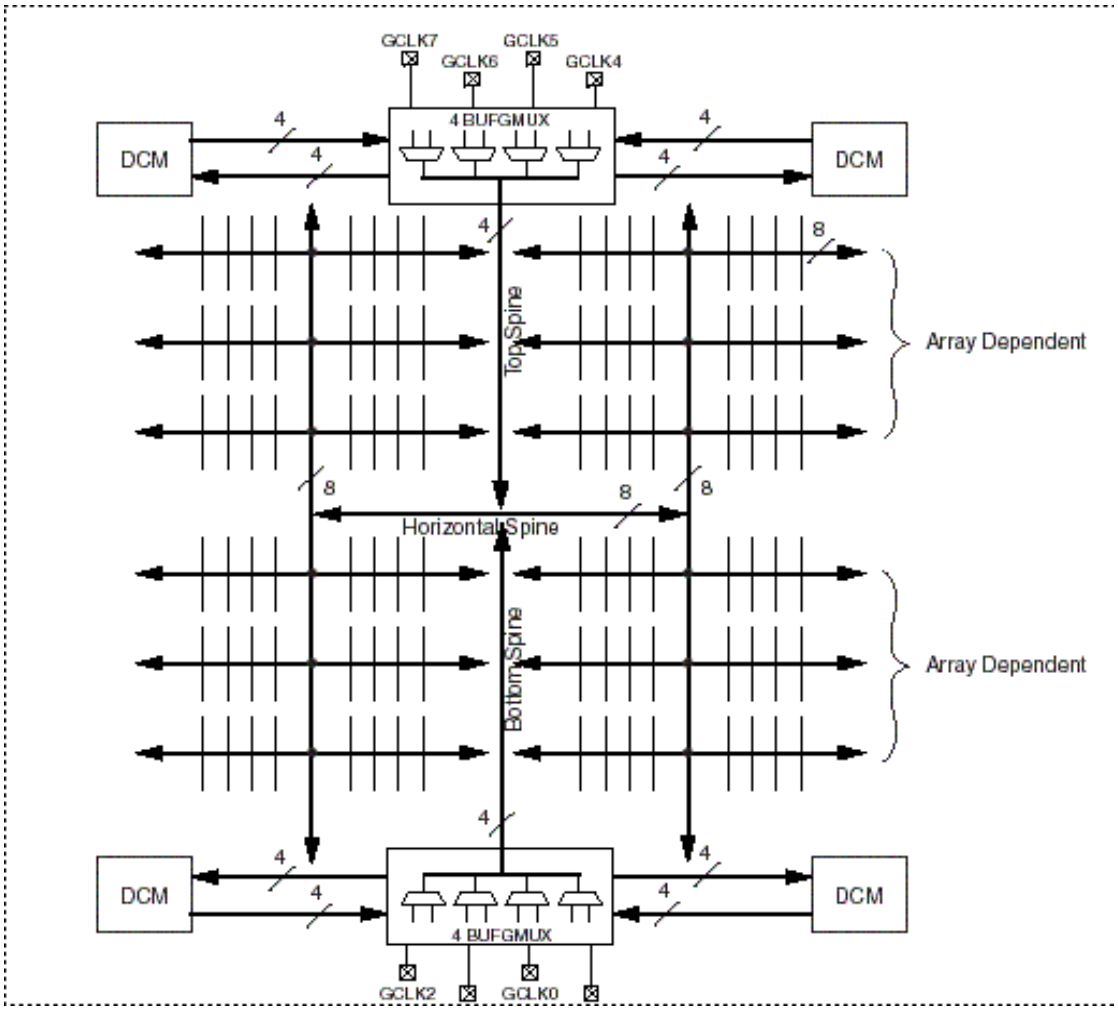
The memory assignment is a clocked behavioral assignment, Reads from the memory are asynchronous, And all the address lines are shared by the read and write statements.

15)What is purpose of a constraint file what is its extension?

The UCF file is an ASCII file specifying constraints on the logical design. You create this file and enter your constraints in the file with a text editor. You can also use the Xilinx Constraints Editor to create constraints within a UCF(extention) file. These constraints affect how the logical design is implemented in the target device. You can use the file to override constraints specified during design entry.

16) What is FPGA you are currently using and some of main reasons for choosing it?

17) Draw a rough diagram of how clock is routed through out FPGA?



18) How many global buffers are there in your current fpga,what is their significance?

There are 8 of them in xc3s5000
An external clock source enters the FPGA using a Global Clock Input Buffer (IBUFG), which directly accesses the global clock network or an Input Buffer (IBUF). Clock signals within the FPGA drive a global clock net using a Global Clock Multiplexer Buffer (BUFGMUX). The global clock net connects directly to the CLKIN input.

19) What is frequency of operation and equivalent gate count of u r project?

20)Tell me some of timing constraints you have used?

21)Why is map-timing option used?

Timing-driven packing and placement is recommended to improve design performance, timing, and packing for highly utilized designs.

22)What are different types of timing verifications?

Dynamic timing:
a. The design is simulated in full timing mode.
b. Not all possibilities tested as it is dependent on the input test vectors.
c. Simulations in full timing mode are slow and require a lot of memory.
d. Best method to check asynchronous interfaces or interfaces between different timing domains.
Static timing:
a. The delays over all paths are added up.
b. All possibilities, including false paths, verified without the need for test vectors.
c. Much faster than simulations, hours as opposed to days.
d. Not good with asynchronous interfaces or interfaces between different timing domains.

23) Compare PLL & DLL ?

PLL:
PLLs have disadvantages that make their use in high-speed designs problematic, particularly when both high performance and high reliability are required.
The PLL voltage-controlled oscillator (VCO) is the greatest source of problems. Variations in temperature, supply voltage, and manufacturing process affect the stability and operating performance of PLLs.

DLLs, however, are immune to these problems. A DLL in its simplest form inserts a variable delay line between the external clock and the internal clock. The clock tree distributes the clock to all registers and then back to the feedback pin of the DLL.
The control circuit of the DLL adjusts the delays so that the rising edges of the feedback clock align with the input clock. Once the edges of the clocks are aligned, the DLL is locked, and both the input buffer delay and the clock skew are reduced to zero.
Advantages:
· precision
· stability
· power management

· noise sensitivity
· jitter performance.

24) Given two ASICs. one has setup violation and the other has hold violation. how can they be made to work together without modifying the design?

Slow the clock down on the one with setup violations..
And add redundant logic in the path where you have hold violations.

25)Suggest some ways to increase clock frequency?

· Check critical path and optimize it.
· Add more timing constraints (over constrain).
· pipeline the architecture to the max possible extent keeping in mind latency req's.

26)What is the purpose of DRC?

DRC is used to check whether the particular schematic and corresponding layout(especially the mask sets involved) cater to a pre-defined rule set depending on the technology used to design. They are parameters set aside by the concerned semiconductor manufacturer with respect to how the masks should be placed , connected , routed keeping in mind that variations in the fab process does not effect normal functionality. It usually denotes the minimum allowable configuration.

27)What is LVs and why do we do that. What is the difference between LVS and DRC?

The layout must be drawn according to certain strict design rules. DRC helps in layout of the designs by checking if the layout is abide by those rules.
After the layout is complete we extract the netlist. LVS compares the netlist extracted from the layout with the schematic to ensure that the layout is an identical match to the cell schematic.

28)What is DFT ?

DFT means design for testability. 'Design for Test or Testability' - a methodology that ensures a design works properly after manufacturing, which later facilitates the failure analysis and false product/piece detection
Other than the functional logic,you need to add some DFT logic in your design.This will help you in testing the chip for manufacturing defects after it come from fab. Scan,MBIST,LBIST,IDDQ testing etc are all part of this. (this is a hot field and with lots of opportunities)

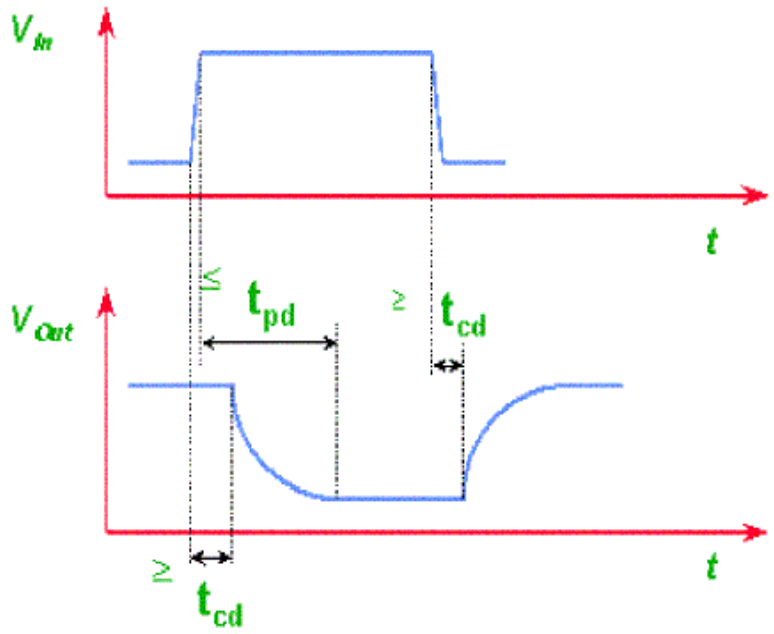
29) There are two major FPGA companies: Xilinx and Altera. Xilinx tends to promote its hard processor cores and Altera tends to promote its soft processor cores. What is the difference between a hard

processor core and a soft processor core?

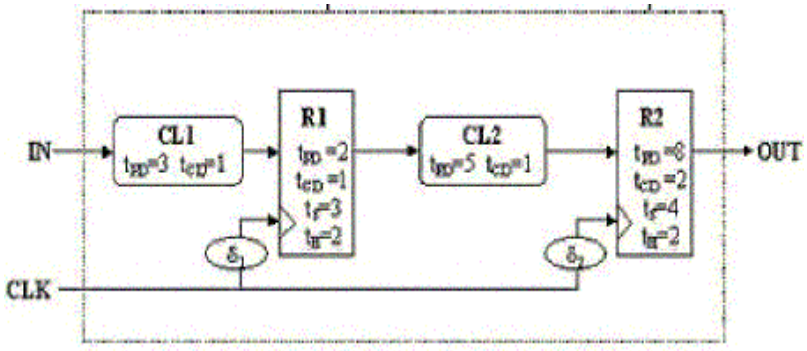
A hard processor core is a pre-designed block that is embedded onto the device. In the Xilinx Virtex II-Pro, some of the logic blocks have been removed, and the space that was used for these logic blocks is used to implement a processor. The Altera Nios, on the other hand, is a design that can be compiled to the normal FPGA logic.

30)What is the significance of contamination delay in sequential circuit timing?

Look at the figure below. tcd is the contamination delay.



Contamination delay tells you if you meet the hold time of a flip flop. To understand this better please look at the sequential circuit below.



The contamination delay of the data path in a sequential circuit is critical for the hold time at the flip

flop where it is exiting, in this case R2.
mathematically, th(R2) <= tcd(R1) + tcd(CL2)
Contamination delay is also called tmin and Propagation delay is also called tmax in many data sheets.

31)When are DFT and Formal verification used?

DFT:
· manufacturing defects like stuck at "0" or "1".
· test for set of rules followed during the initial design stage.

Formal verification:
· Verification of the operation of the design, i.e, to see if the design follows spec.
· gate netlist == RTL ?
· using mathematics and statistical analysis to check for equivalence.

32)What is Synthesis?

Synthesis is the stage in the design flow which is concerned with translating your Verilog code into gates - and that's putting it very simply! First of all, the Verilog must be written in a particular way for the synthesis tool that you are using. Of course, a synthesis tool doesn't actually produce gates - it will output a netlist of the design that you have synthesised that represents the chip which can be fabricated through an ASIC or FPGA vendor.

33)We need to sample an input or output something at different rates, but I need to vary the rate? What's a clean way to do this?

Many, many problems have this sort of variable rate requirement, yet we are usually constrained with a constant clock frequency. One trick is to implement a digital NCO (Numerically Controlled Oscillator). An NCO is actually very simple and, while it is most naturally understood as hardware, it also can be constructed in software. The NCO, quite simply, is an accumulator where you keep adding a fixed value on every clock (e.g. at a constant clock frequency). When the NCO "wraps", you sample your input or do your action. By adjusting the value added to the accumulator each clock, you finely tune the AVERAGE frequency of that wrap event. Now - you may have realized that the wrapping event may have lots of jitter on it. True, but you may use the wrap to increment yet another counter where each additional Divide-by-2 bit reduces this jitter. The DDS is a related technique. I have two examples showing both an NCOs and a DDS in my File Archive. This is tricky to grasp at first, but tremendously powerful once you have it in your bag of tricks. NCOs also relate to digital PLLs, Timing Recovery, TDMA and other "variable rate" phenomena

Improving the Quality of Results

The quality of the synthesized design can be improved using the following techniques:

Module partitioning

_ Adding structure

_ Horizontal partitioning

_ Adding hierarchy (vertical partitioning)

_ Performing operations in parallel

_ Use multiplexers for logic implementation

Module Partitioning

Where possible, register module outputs and keep the critical path in one block.

Keep as much of the critical path in one module or block as possible. This enables DesignCompiler™ to optimize the critical path while it is compiling a single module or block without having to iterate between several different modules. Placing the registers on module outputs also simplifies the compilation process because timing budgets for registered module outputs are not needed. Registering module inputs does not yield much improvement since the input arrival times can be computed and output using the characterize and write_script commands. The size of a module should be based on a logical partitioning as opposed to an arbitrary gate count. Instantiating a set of pre-compiled basic building blocks can reduce the complexity of the design and the associated compile effort even for larger modules. In this case, a large percentage of the gate count ends up in the instantiated or inferred modules. The last point is to keep most of the logic in the leaf modules. This simplifies the compilation

process because the top-level modules will need little or no compilation and constraints can more easily be propagated down the hierarchy. The following design examples were compiled using Synopsys Design Compiler version 2.2b, the LSI LCA200k library, and the “B3X3” wire load model with WCCOM operating

conditions. The constraints were:

```
set_load 5 * load_of (IV/A) all_outputs()

set_drive drive_of (IV/Z) all_inputs()
```

Adding Structure

One goal of logic synthesis is to produce an optimal netlist that is independent of the original structure. Until this goal is achieved, controlling the structure of a logic description is one of the best ways to ensure an optimal implementation. Controlling the structure by using separate assignment statements and through the use of parentheses really has very little effect on the generated logic. The only case where parentheses have a significant effect is when resources are used. Resources and function invocations are assigned and preserved when the HDL code is read and have a significant impact on the generated logic.

Note: In v3.0 there is tree-height minimization of expressions. The following

behavioral code specifies a 32-bit arithmetic shift right operation:

```
assign shifta = {{31{a[31]}}, a} >> shiftCnt;
```

An iteration value of 31 is the largest value that is required. For smaller shift amounts the extra bits will be truncated when assigned to the variable shift a, which is 32 bits wide. This expression produces a design of 742 gates that is almost 50% slower than the structural logic design. The shift right arithmetic function can also be described without using any extra bits:

```
// 16.63 ns, 1431 gates

assign shifta = (a[31] ? ((a >> shiftCnt) |
```

```
(((32'b1 << shiftCnt) - 1) << (32-shiftCnt))) : a >> shiftCnt);
```

This arithmetic shift operation shifts the 32 bits right by the shift count, and replaces

the vacated positions with bit 31 of variable a. The expression `(a >> shiftCnt)` shifts "a" by the shift count but doesn't replace the vacated positions. The expression `((32'b1 <<shiftCnt) - 1)` produces a string of 1s equal in length to the value of the shift count, which is equal to the number of vacated bit positions. This string of 1s needs to occupy the vacated bit positions starting with bit 31. The expression `(32-shiftCnt)` is the number of bit positions that the string of 1s needs to be left shifted. The final result is the logical OR of the shifted string of 1s and the logical right shift value `(a>>shiftCnt`). While this expression

is equivalent, it is much too complex to be practical. When synthesized and optimized,

this specification produces a design with 1,431 gates that is three times slower and

over twice the area of the structural logic design (see Figure 1):

```
// structured shift right arithmetic design
// 6.87 ns, 613 gates, fastest, optimal version
module shiftRightAr(a, shiftCnt, z);
input [31:0] a;
input [4:0] shiftCnt;
output [31:0] z;
wire [31:0] d0, d1, d2, d3, notA;
assign notA = ~a;
mux2to1L32 m21_0 (notA,{notA[31], notA[31:1]}, shiftCnt[0], d0);
mux2to1L32 m21_1 (d0,{{ 2{a[31]}}, d0[31:2]}, shiftCnt[1], d1);
mux2to1L32 m21_2 (d1,{{ 4{notA[31]}}, d1[31:4]}, shiftCnt[2],
d2);
mux2to1L32 m21_3 (d2,{{ 8{a[31]}}, d2[31:8]}, shiftCnt[3], d3);
```

```
mux2to1L32 m21_4 (d3,{{16{notA[31]}}, d3[31:16]},shiftCnt[4],
z);
endmodule
module mux2to1L32 (a, b, s, z);
input [31:0] a, b;
input s;
output [31:0] z;
assign z = ~(s ? b : a);
endmodule
```

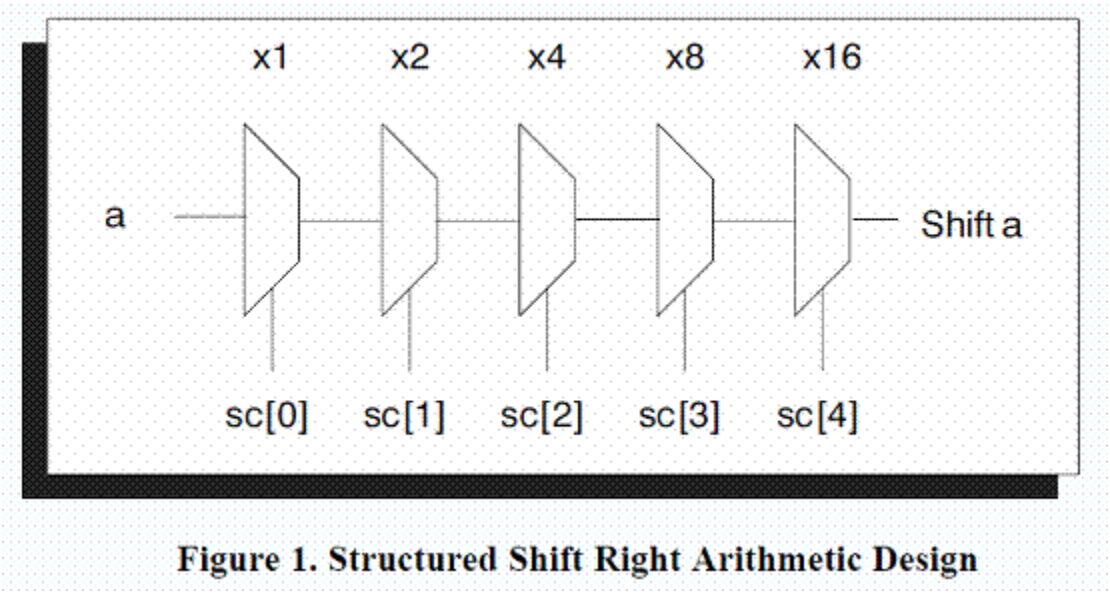


Figure 1. Structured Shift Right Arithmetic Design

The structural logic design produces a design of 613 gates that is three times faster. While each of the 32-bit multiplexers is instantiated, the module mux2to1L32 is defined without using gate-level instantiation. The shifter may also be specified without the mux instantiations:

```
// 8.00 ns, 518 gates, with no structuring

// 8.55 ns, 598 gates, with structuring

// smallest design but 20% slower than optimal

module shiftRightAr(a, shiftCnt, z);

input [31:0] a;

input [4:0] shiftCnt;

output [31:0] z;

wire [31:0] d0, d1, d2, d3, notA;

assign notA = ~a;

assign

d0 = (shiftCnt[0] ? {a[31], a} >> 1 : a),

d1 = ~(shiftCnt[1] ? {{2{a[31]}}, d0} >> 2 : d0),

d2 = ~(shiftCnt[2] ? {{4{notA[31]}}, d1} >> 4 : d1),

d3 = ~(shiftCnt[3] ? {{8{a[31]}}, d2} >> 8 : d2),

z = ~(shiftCnt[4] ? {{16{notA[31]}}, d3} >> 16 : d3);

endmodule
```

Now the logic synthesis tool is free to optimize various pieces of each multiplexer. This specification, when compiled without structuring, produces a design that is only 518 gates, but is also 20% slower. With the default structuring enabled the resultant design is actually bigger and slower, a case where no structuring is a big win. Furthermore, due to the optimizations, the symmetry is lost, making default structuring unattractive for use in a data path design.

Horizontal Partitioning

A combinational circuit can be expressed as a sum of products that can be implemented by a circuit with two levels of logic. However this may result in gates with a maximum fan-in equal to the number of inputs. By building up the logic in levels, a circuit with a regular structure can be designed. In addition, the circuit can broken into horizontal slices to minimize the maximum fan-in of a logic gate. A carry lookahead adder is a classic example. A 32-bit adder is broken into eight 4-bit blocks. Each adder block generates a `group propagate` and `group generate` signal. A carry-lookahead block takes as input the `groupgenerate` and `propagate` signals and generates a carry-in to each block and a carry-out for the entire adder.

A 32-bit priority encoder is another example where bit-slicing can yield significant results. The first priority encoder, `priorityEncode32b`, compiles to produce a design of 205 gates. The critical path consists of seven levels of logic. The second module, `priorityEncode32`, is restructured using four 8-bit blocks (see Figure 2). The restructured priority encoder compiles to 182 gates and four levels of logic. The worst-case delay is reduced by 26%, while the gate count is reduced by 23. This restructuring reduces the scope and complexity of the problem from 32 bits to 8 bits, which allows the HDL compiler to produce a more optimal design.

```
// 7.14 ns, 205 gates (original design)

module priorityEncode32b (bitVector, adr, valid);

input [31:0] bitVector;

output [4:0] adr;

output valid;

function [4:0] encode;

input [31:0] in;

integer i;

begin: _encode

encode = 5'd0;

for ( i=31; i>=0; i=i-1 )
```

```
if ( !in[i] ) begin
encode = i;
disable _encode;
end
end
endfunction
assign adr = encode(bitVector);
assign valid = ~(&bitVector);
endmodule
// 5.31 ns, 182 gates (smaller/faster design)
module priorityEncode32 (bitVector, adr, valid);
input [31:0] bitVector;
output [4:0] adr;
output valid;
// synopsys dc_script_begin,
// dont_touch -cell {pe0 pe1 pe2 pe3}
// synopsys dc_script_end
wire [3:0] anyValid;
wire [2:0] adr0, adr1, adr2, adr3, adr4, afAdr;
wire [2:0] msgAdr, lsgAdr;
wire msaf = |anyValid[3:2];
// partition into 8-bit groups for optimal speed/gate-count
priorityEncode8 pe0 (bitVector[7:0], adr0, anyValid[0]);
priorityEncode8 pe1 (bitVector[15:8], adr1, anyValid[1]);
priorityEncode8 pe2 (bitVector[23:16], adr2, anyValid[2]);
```

```
priorityEncode8 pe3 (bitVector[31:24], adr3, anyValid[3]);
// select most significant group using valid bits
assign msgAdr = anyValid[3] ? adr3 : adr2;
assign lsgAdr = anyValid[1] ? adr1 : adr0;
assign afAdr = msaf ? msgAdr : lsgAdr;
assign adr = {msaf, msaf ? anyValid[3] : anyValid[1], afAdr};
assign valid = |anyValid;
endmodule

module priorityEncode8 (in, out, anyValid);
input [7:0] in;
output [2:0] out;
output anyValid;
function [2:0] encode;
input [7:0] in;
integer i;
begin : _encode
encode = 3'd0;
for ( i=7; i>=0; i=i-1 )
if ( !in[i] ) begin
encode = i;
disable _encode;
end
end
endfunction
```

```
assign out = encode(in);

assign anyValid = !(&in);

endmodule
```

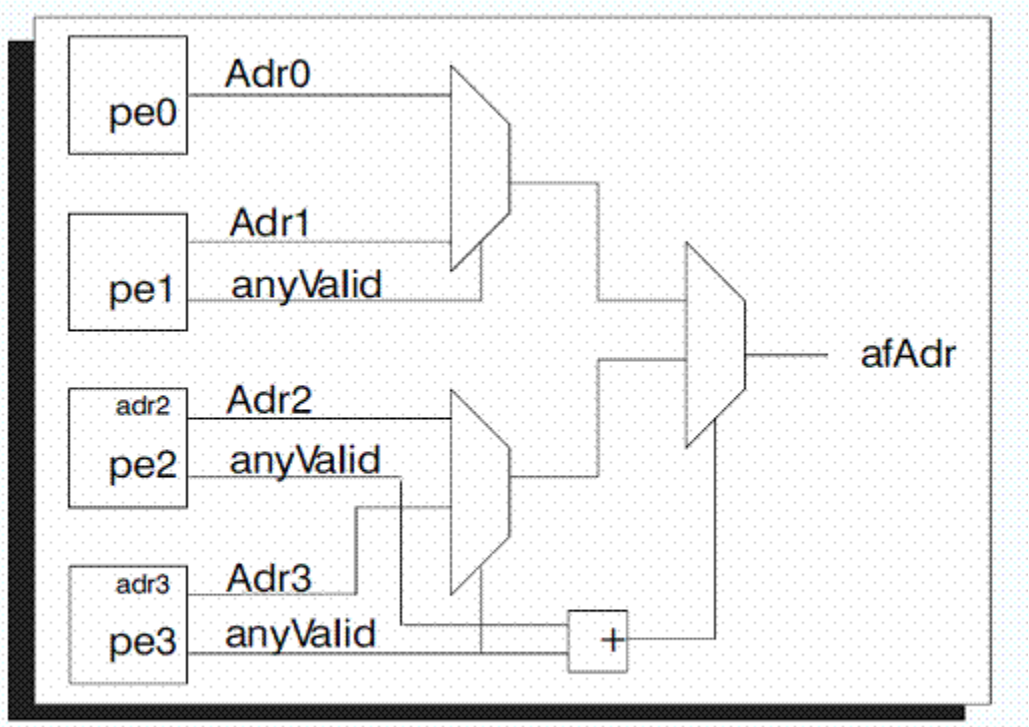


Figure 2. 32-Bit Fast Priority Encoder

Adding Hierarchy

Collapsing the hierarchy results in more efficient synthesis in some cases. In other cases, adding hierarchy can improve the design. This was shown in the case of the shift right arithmetic function described earlier. In the case of the priority encoder, additional hierarchy achieved significant improvements in both speed and area. Another case where additional hierarchy achieves significant results is in the balanced tree decoder described in the following example. Adding the hierarchy in these cases helps to define the final implementation and preserves the structure that yields an optimal design. A 32-bit decoder with negative asserting output can be coded as:

```
// decoder using variable array index

module decoder32V1(adr, decode);

input [4:0] adr;

output [31:0] decode;

reg [31:0] decode; // note: pseudo_reg

always @(adr) begin

decode = 32'hffffffff;

decode[adr] = 1'b0;

end

endmodule
```

This design turns out to be the least-efficient implementation of several alternative designs. It compiles to 125 gates. A more concise representation of a decoder is given as

```
// decoder using shift operator

module decoder32V2(adr, decode);

input [4:0] adr;

output [31:0] decode;

assign decode = ~(1'b1 << adr);

endmodule
```

This produces a slightly faster design of 94 gates. More dramatic results can be obtained by using a balanced tree decoder. By adding a second level of hierarchy, a balanced tree decoder can be specified (see Figure 3).

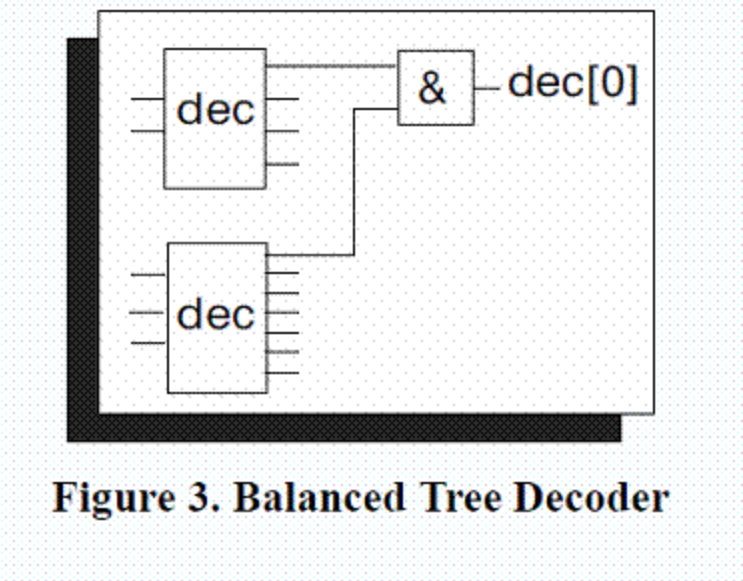


Figure 3. Balanced Tree Decoder

```
// balanced tree decoder: smaller and faster
//
module decoder32BT (adr, dec);
input [4:0] adr;
output [31:0] dec;
wire [3:0] da = 1'b1 << adr[1:0]; // 2 to 4 decoder
wire [7:0] db = 1'b1 << adr[4:2]; // 3 to 8 decoder
decode32L2 d32l2 (da, db, dec);
endmodule
module decode32L2(da, db, dec);
input [3:0] da;
input [7:0] db;
output [31:0] dec;
wire [31:0] dbVec =
{{4{db[7]}}, {4{db[6]}}, {4{db[5]}}, {4{db[4]}},
```

```
{4{db[3]}}, {4{db[2]}}, {4{db[1]}}, {4{db[0]}}};

wire [31:0] daVec = {8{da}};

assign dec = ~(dbVec & daVec);

endmodule
```

This design compiles to 68 gates, which is about 50% smaller than the `decoder32V1` module, and it is the fastest of the three modules.

Performing Operations in Parallel

This is the classic technique of using more resources to achieve a decrease in speed. In this

example, an array of four 6-bit counters are compared. The output is the index of the smallest counter. Various search strategies are used to find the smallest element: linear search, binary search, and parallel search. The first implementation uses a `task` and a `for` loop to compare all the values. This results in a serial compare. First `cntr[0]` is compared to `cntr[1]`, the smallest `cntr` is selected and then compared to `cntr[2]`. The smallest result from the second comparator is selected and compared to `cntr[3]`. This process involves three comparators and two multiplexers in series for a total delay of 22.41 ns and 18 levels of logic. The total area is 527 gates.

```
// Linear Search - 22.41 ns, 527 gates

module arrayCmpV1(clk, reset, inc, index, min);

input clk, reset, inc;

input [1:0] index;

output [1:0] min;

reg [5:0] cntr[0:4];

reg [1:0] min; // pseudo register

integer i;

// compare each array element to mincount
```

```
task sel;

output [1:0] sel;

reg [5:0] mincount;

begin : _sc

mincount = cntr[0];

sel = 2'd0;

for ( i = 1; i <= 3; i=i+1 )

if ( cntr[i] < mincount ) begin

mincount = cntr[i];

sel = i;

end

end

endtask

always @(cntr[0] or cntr[1] or cntr[2] or cntr[3])

sel(min);

always @(posedge clk)

if (reset)

for( i=0; i<=3; i=i+1 )

cntr[i] <= 6'd0;

else if (inc)

cntr[index] <= cntr[index] + 1'b1;

endmodule
```

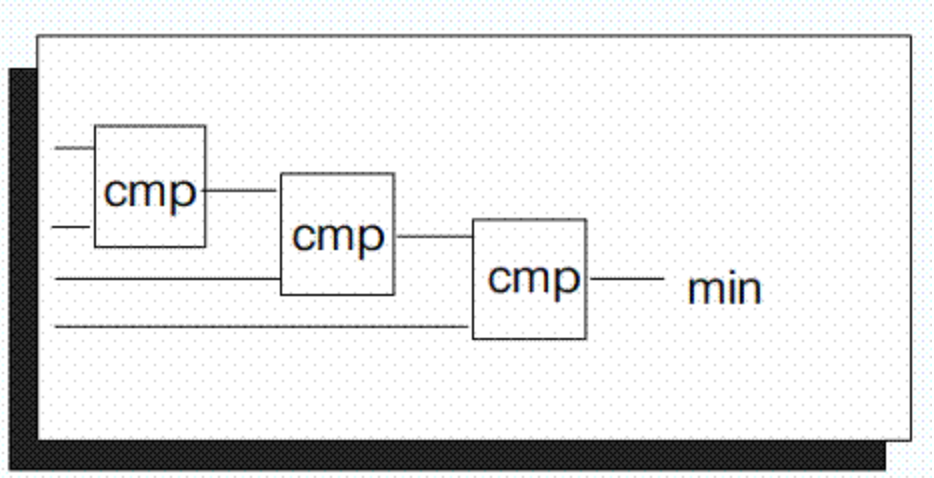


Figure 4. Linear Search

A second version of this design needs two comparators in series and takes 14.9 ns with eleven levels of logic and a total area of 512 gates. This design is both smaller and faster than the first version.

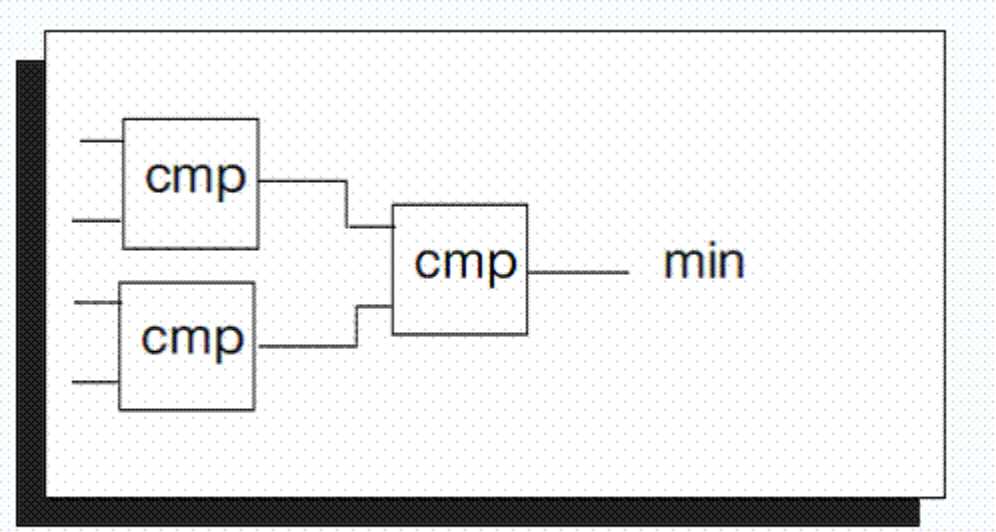


Figure 5. Binary Search

```
// Binary Search - 14.9 ns, 512 gates (smallest area)
module arrayCmpV2(clk, reset, inc, index, min);
input clk, reset, inc;
input [1:0] index;
output [1:0] min;
reg [5:0] cntr[0:4];
integer i;
// binary tree comparison
wire c3lt2 = cntr[3] < cntr[2];
wire c1lt0 = cntr[1] < cntr[0];
wire [5:0] cntr32 = c3lt2 ? cntr[3] : cntr[2];
wire [5:0] cntr10 = c1lt0 ? cntr[1] : cntr[0];
wire c32lt10 = cntr32 < cntr10;
// select the smallest value
assign min = {c32lt10, c32lt10 ? c3lt2: c1lt0};
always @(posedge clk)
if (reset)
for( i=0; i<=3; i=i+1 )
cntr[i] <= 6'd0;
else if (inc)
cntr[index] <= cntr[index] + 1;
endmodule
```

A third implementation performs all the comparisons in parallel. The same path now takes 11.4 ns with eight levels of logic and has a total area of 612 gates. This version is about 20% faster than the second version, with a 20% increase in area.

Exploit parallelism to improve speed.

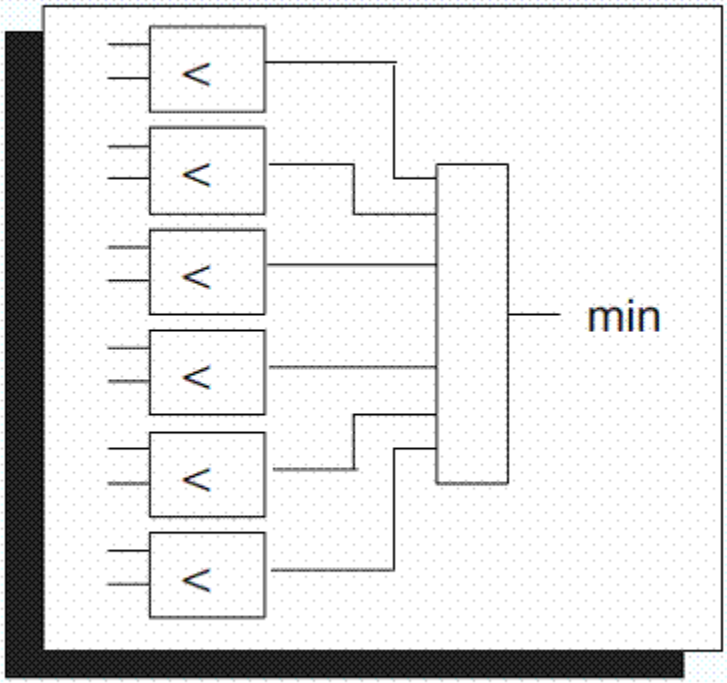


Figure 6. Parallel Search

```
// Parallel Search - 11.4 ns, 612 gates (fastest design)

module arrayCmpV3(clk, reset, inc, index, min);

input clk, reset, inc;

input [1:0] index;

output [1:0] min;

reg [5:0] cntr[0:4];
```

```
integer i;

// compare all counters to each other

wire l32 = cntr[3] < cntr[2];

wire l31 = cntr[3] < cntr[1];

wire l30 = cntr[3] < cntr[0];

wire l21 = cntr[2] < cntr[1];

wire l20 = cntr[2] < cntr[0];

wire l10 = cntr[1] < cntr[0];

// select the smallest value

assign min = {l31&l30 | l21&l20, l32&l30 | l10&~l21};

always @(posedge clk)

if (reset)

for( i=0; i<=3; i=i+1 )

cntr[i] <= 6'd0;

else if (inc)

cntr[index] <= cntr[index] + 1;

endmodule
```

Use Multiplexers for Logic Implementation

When using CMOS technology, there is a significant speed advantage in using pass-gate multiplexers.

The following code takes advantage of some symmetry in the specification and uses the branch condition as the index for an 8-to-1 multiplexer. This turns out to be the most optimal design in terms of both area and speed.

```
// Mux version - 5.17 ns, 30 gates
module condcodeV2 (cc, bc, valid);
input [3:0] cc;
input [3:0] bc;
output valid;
wire n, z, v, c;
wire [7:0] ccdec;
assign {n, z, v, c} = cc;
assign ccdec = {v, n, c, c | z, n ^ v, z | (n ^ v), z, 1'b0};
assign valid = bc[3] ^ ccdec[bc[2:0]];
endmodule
```

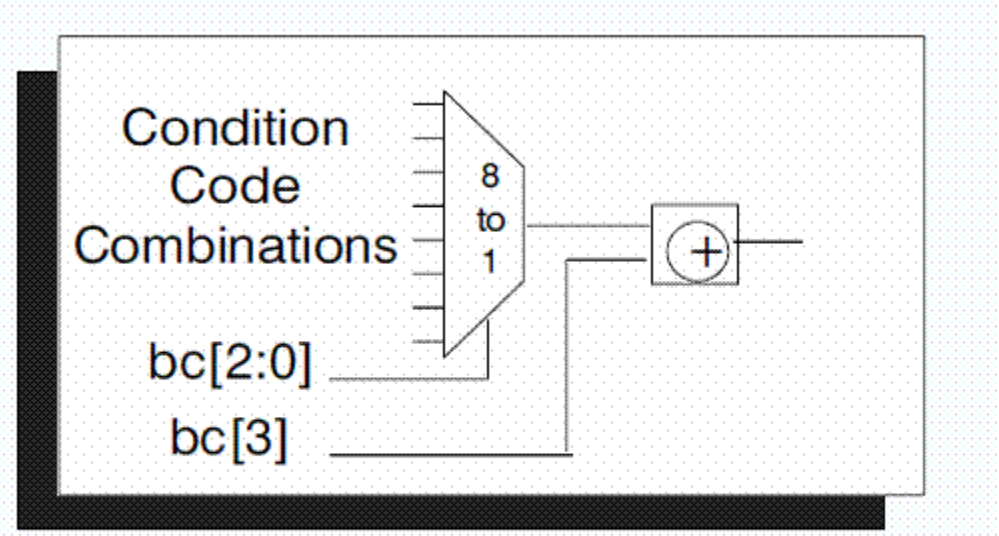


Figure 7. Condition Code Logic Using Mux

Vendor-Independent HDL

Technology-transparent design makes it easy to target a design to a specific vendor. This provides flexibility in choosing a primary vendor and in selecting an alternative source, if one is desired. Since it is unlikely that two vendors will have exactly the same macro library or process, the design will have to be resynthesized, instead of just translated using the new library. The main way to achieve a vendor-independent design is to avoid instantiating vendor macrocells in the design. Another technique is to create a small set of user defined macrocells and define these in terms of the vendor library. One set of these user-defined macros need to be created for each vendor. Using module definitions for simple components often achieves the same result without instantiating vendor cells:

```
module mux4to1x2 (in0, in1, in2, in3, sel, out);

input [1:0] in0, in1, in2, in3;

input [1:0] sel;

output [1:0] out;

mux4to1 m4_0 (in0[0], in1[0], in2[0], in3[0], sel, out[0]);

mux4to1 m4_1 (in0[1], in1[1], in2[1], in3[1], sel, out[1]);

endmodule

// Module mux4to1 takes the place of vendor cell

module mux4to1 (in0, in1, in2, in3, sel, out);

input in0, in1, in2, in3;

input [1:0] sel;

output out;

wire [3:0] vec = {in3, in2, in1, in0};

assign out = vec[sel];

endmodule
```

In Design Compiler v3.0 you may now use vendor-independent cells from the GTECH (for generic technology) library. There is a 4-to-1 mux in the library, called the GTECH_MUX4, that can replace the mux4to1 in the previous example. The command `set_map_only` can be used to instruct the compiler to select and preserve an equivalent cell from the technology library. This is the only way to insure the equivalent cell will be selected from the target library. Currently there is no Verilog simulation library for the GTECH components but a library can easily be created by changing the names in an existing library.

```
// GTECH version of mux4to1x2

// use of GTECH components makes this vendor independent

// set_map_only command locks in equivalent cells from target

lib

module mux4to1x2 (in0, in1, in2, in3, sel, out);

input [1:0] in0, in1, in2, in3;

input [1:0] sel;

output [1:0] out;

GTECH_MUX4 m4_0 (in0[0], in1[0], in2[0], in3[0],

sel[0], sel[1], out[0]);

GTECH_MUX4 m4_1 (in3[1], in2[1], in1[1], in0[1],

sel[0], sel[1], out[1]);

endmodule
```

Using Verilog Constructs

Don’t-Care Conditions

A design must be flattened or compiled using Boolean optimization in order to use

*dont_cares.*Don't-care conditions for Synopsys can be specified by assigning an output to 'bx. Dont_cares are usually found inside a case statement where some input combinations should never occur, or where outputs are dont_cares. While the meaning of the don't-care specification is consistent between the simulation and synthesis tools, it is treated differently. When an output is assigned a value of 'bx in Verilog, the output becomes unknown. Synopsys treats the assignment of 'bx as a don't-care specification. It will use this don't-care specification to minimize the synthesized logic. The gate-level design will produce an output whose value, while not undefined, is dependent on the particular logic generated. If the default clause in a

case statement specifies don't-care conditions that should never occur, the Verilog model will produce an unknown output when the default statement is executed. If this output is used, unknowns will propagate to other parts of the design and the error will be detected. If the default clause of a case statement specifies a known value and there are no overlapping case items (when parallel case is used), then the RTL version will match the synthesized gate-level version. The proper use of dont_cares should not produce any significant simulation discrepency. In Synopsys v3.0 or earlier, in order to utilize the don't-care conditions in the optimization phase, the design must either be flattened, compiled using Boolean optimization, or the state table must be extracted before compilation.

Put don't care assignments in a lower-level block so flattening can work.

For this reason the designer may want to partition the design such that don't-care assignments are in a lower-level block, so flattening can work. This is necessary for the FSM extract command.

Procedural Assignment

Use the non-blocking construct for procedural assignments to state regs.

With the blocking assignment operator, statements are executed sequentially. In the PC chain example below, all the PC registers would get the value of fetchAdr.

```
always @(posedge clk) if (~hold) begin // pc chain

fetchPC = fetchAdr;

decPC = fetchPC;

execPC = decPC;

writePC = execPC;

end
```

This problem can be avoided by using fork and join in addition to an intra assignment delay. When using Synopsys, the fork and join must be bypassed using compiler directives.

```
always @(posedge clk) if (~hold) // pc chain

/* synopsys translate_off */ fork /* synopsys translate_on */

fetchPC = #d2q fetchAdr;

decPC = #d2q fetchPC;

execPC = #d2q decPC;

writePC = #d2q execPC;

/* synopsys translate_off */ join /* synopsys translate_on */
```

The non-blocking procedural assignment "<=" allows multiple procedural assignments to be specified within the same `always` block. Furthermore, it will simulate without the need for an intra-assignment delay.

```
reg [31:0] fetchPC, decPC, execPC, writePC;

always @(posedge clk) if (~hold) begin // pc chain

fetchPC <= fetchAdr;
```

```
decPC <= fetchPC;

execPC <= decPC

writePC <= execPC;

end
```

The pc-chain can also be expressed as a single shift-register-like statement:

```
always @(posedge clk) if (~hold) // pc chain

{fetchPC,decPC,execPC,writePC}<={fetchAdr,fetchPC,decPC,execPC};
```

Don't mix blocking and non-blocking assignments in the same block.

Mixing blocking and non-blocking assignments in the same block is not allowed and will result in a Synopsys Verilog HDL Compiler™ read error.

Using Functions with Component Implication

Function invocations can be used instead of module instantiations. This is really a question of style, not a recommendation. The function invocation can also be mapped to a specific implementation using the map_to_module compiler directive. The compiler directives `map_to_module` and `return_port_name` map a function or a task to a module. For the purposes of simulation, the contents of the function or task are used, but for synthesis the module is used in place of the function or task. This allows component instantiation to be used in a design for optimization purposes without altering the behavioral model. When multiple outputs are required, use a task instead of a function. When using component implication, the RTL-level model and the gate-level model may be different. Therefore the design can not be fully verified until simulation is run on the gate-level design. For this reason, component implication should be used with caution or not at all. The following code illustrates

the use of component implication:

```
function mux8; // 8to1 mux

// synopsys map_to_module mux8to1

// synopsys return_port_name out

input [7:0] vec;

input [2:0] sel;

mux8 = vec[sel];

endfunction

function mux32; // 32to1 mux

// synopsys map_to_module mux32to1

// synopsys return_port_name out

input [31:0] vec;

input [4:0] sel;

mux32 = vec[sel];

endfunction

wire valid = ~(a[5] ? mux8(v8, a[4:2]) : mux32(v32, a[4:0]));
```

Register Inference

Using the Synopsys Verilog HDL compiler, flip-flops can be inferred by declaring a variable of type `reg` and by using this variable inside an `always` block that uses the `posedge` or `negedge clk` construct. The following example will generate an 8-bit register that recirculates the data when the `loadEnable` signal is not active:

```
reg [7:0] Q;

always @(posedge clk)

if (loadEnable)

Q <= D;
```

All variables that are assigned in the body of an `always` block must be declared to be of type `reg` even though a flip-flop will not be inferred. These "`reg`s" are really just wires, and for the sake of clarity should be commented as such. Please note that this is a Verilog language feature and not a Synopsys limitation. Register inference allows designs to be technology-independent. Much of the design functionality

can be specified using the more easily understood procedural assignment. There is

no need to specify default assignments to prevent latch inference. If a design is free of component instantiations, then a gate-level simulation library is not required. Therefore, the design can be targeted to any vendor library without the need to recode macro libraries. The instance names of `reg`s are "`name_reg`", which allows easy identification for layout. Scanstring stitching can key off the inferred register instance names. The design of state machines is made easier because state assignments can be made using parameters and the enum compiler directive. The state machine compiler can easily make trade-offs between fully decoded one-hot state assignments and fully encoded states. Refer to the *Synopsys HDL Compiler for Verilog Reference Manual* for more details about using the state machine compiler.

The Reset Problem

For sync resets, set the v3.0 variable `compile_preserve_sync_resets =`

`true.`

The use of a synchronous reset results in the creation of discrete logic to perform the reset function, which under some circumstances will not initialize the flip-flop during logic simulation. This problem can be solved in one of two ways. The first and preferred solution is to set the v3.0 compile variable: `compile_preserve_sync_resets = true`. This places the reset logic next to the flip-flop and in some cases a flip-flop with a synchronous reset is generated. The other alternative is to use an asynchronous reset.

```
reg [7:0] Q;

// synchronous reset

always @(posedge clk)

if (reset)

Q <= 0;

else if (loadEnable)

Q <= D;

// asynchronous reset

always @(posedge clk or posedge reset)

if (reset)

Q <= 0;

else if (loadEnable)

Q <= D;
```

Latch Inference

Within an always block, fully specify assignments or use default conditions.

A variable of type `reg` which is assigned a value inside an `always` block without the `posedge` or `negedge` clock construct generates a latch if the variable is not assigned a value under all conditions. If there is a set of input conditions for which the variable is not assigned, the variable will retain its current value. The example below is for a round-robin priority selector. `grant` designates the device that currently has the bus. `request` designates those devices that are requesting use of the bus. `grant_next` is the next device to be granted the bus. Priority is always given to the device one greater than the current one. Using a default for the case statement will not prevent the creation of a latch. If no requests are active, then `grant_next` is not assigned so a latch will be created. The solution is to specify the default conditions at the beginning of the `always` block before the case statement.

```
module rrpriocase(request,grant,grant_next);

input [3:0] request, grant;

output [3:0] grant_next;

always @(request or grant) begin

grant_next = grant; // put default here to avoid latch inference

case(1) // synopsys parallel_case full_case

grant[0]:

if (request[1]) grant_next = 4'b0010;

else if (request[2]) grant_next = 4'b0100;

else if (request[3]) grant_next = 4'b1000;

else if (request[0]) grant_next = 4'b0001;

grant[1]:

if (request[2]) grant_next = 4'b0100;

else if (request[3]) grant_next = 4'b1000;

else if (request[0]) grant_next = 4'b0001;

else if (request[1]) grant_next = 4'b0010;

grant[2]:

if (request[3]) grant_next = 4'b1000;

else if (request[0]) grant_next = 4'b0001;

else if (request[1]) grant_next = 4'b0010;

else if (request[2]) grant_next = 4'b0100;

grant[3]:

if (request[0]) grant_next = 4'b0001;
```

```
else if (request[1]) grant_next = 4'b0010;

else if (request[2]) grant_next = 4'b0100;

else if (request[3]) grant_next = 4'b1000;

endcase

end

endmodule
```

Another solution is to add the following else clause at the end of each case item:

```
else grant_next = grant;
```

Using Arrays of Multiplexers

Instantiate multiplexers that output vectors.

When using v3.0 or earlier it is a good idea to instantiate multiplexers that output vectors. (For time-critical signals it may also be necessary to instantiate multiplexers that only output a single bit.) This involves creating a library of multiplexer modules that can be instantiated in the design. In the following example, there is a single input vector to simplify parameter passing through multiple levels of hierarchy. Use parameterized modules to build different versions of a design.

```
// may want to pass an array as a parameter

reg [1:0] in [0:3];

// convert array to vector

wire [2*4:1] inVec = {in[0], in[1], in[2], in[3]};
```

```
// multiplexer instantiation:
mux4to1x4(inVec, sel, out);
// multiplexer module
module mux4to1x4 (inVec, sel, out);
input [4*4:1] inVec;
input [1:0] sel;
output [3:0] out;
wire [3:0] in0, in1, in2, in3;
assign {in3, in2, in1, in0} = inVec;
// synopsys dc_script_begin
// dont_touch {m4_0 m4_1 m4_2 m4_3}
// synopsys dc_script_end
// note: instance names must be unique
mux4to1 m4_0 ({in3[0], in2[0], in1[0], in0[0]}, sel, out[0]);
mux4to1 m4_1 ({in3[1], in2[1], in1[1], in0[1]}, sel, out[1]);
mux4to1 m4_2 ({in3[2], in2[2], in1[2], in0[2]}, sel, out[2]);
mux4to1 m4_3 ({in3[3], in2[3], in1[3], in0[3]}, sel, out[3]);
endmodule
// Module mux4to1 should map to vendor MUX4 cell
module mux4to1 (vec, sel, out);
input [3:0] vec;
input [1:0] sel;
output out;
assign out = vec[sel];
endmodule
```

Using Arrays

The Synopsys Verilog HDL Compiler supports memory arrays. For a group of registers that are accessed using an index, the memory array construct provides a more concise specification. The Verilog HDL models memories as an array of register variables. Each register in the array is addressed by a single array index.

The following example declares a memory array called `cntr` that is used to implement a bank of eight counters. A counter index variable can directly select the register to be incremented:

```
reg [5:0] cntr[0:7];

always @(posedge clk or posedge reset)

if (reset)

for (i=0; i<8; i=i+1)

cntr[i] <= 6'd0;

else if (inc)

cntr[index] <= cntr[index] + 1'b1;
```

Without the use of arrays, this description requires eight incrementors instead of one and many more lines of code.

```
reg [5:0] cntr7, cntr6, cntr5, cntr4;

reg [5:0] cntr3, cntr2, cntr1, cntr0;

always @(posedge clk or posedge reset)

if (reset)

{cntr7, cntr6, cntr5, cntr4} <= {6'd0, 6'd0, 6'd0, 6'd0};
```

```
{cntr3, cntr2, cntr1, cntr0} <= {6'd0, 6'd0, 6'd0,

6'd0};

else if (inc)

case(index) // parallel_case full_case

3'd0: cntr0 <= cntr0 + 1;

3'd1: cntr1 <= cntr1 + 1;

3'd2: cntr2 <= cntr2 + 1;

3'd3: cntr3 <= cntr3 + 1;

3'd4: cntr4 <= cntr4 + 1;

3'd5: cntr5 <= cntr5 + 1;

3'd6: cntr6 <= cntr6 + 1;

3'd7: cntr7 <= cntr7 + 1;

endcase
```

The eight incrementors can be reduced to one by rewriting the increment section of the code and adding even more lines of code:

```
reg [5:0] result; // pseudo reg

always @(index or cntr7 or cntr6 or cntr5 or cntr0

or cntr3 or cntr2 or cntr1 or cntr0)

case(index) // parallel_case

3'd0: result = cntr0;

3'd1: result = cntr1;

3'd2: result = cntr2;

3'd3: result = cntr3;
```

```
3’d4: result = cntr4;
3’d5: result = cntr5;
3’d6: result = cntr6;
3’d7: result = cntr7;
default: result = ‘bx;
endcase
wire [5:0] inc = result + 1’b1;
always @(posedge clk or posedge reset)
if (reset)
{cntr7, cntr6, cntr5, cntr4} <= {6’d0, 6’d0, 6’d0,
6’d0};
{cntr3, cntr2, cntr1, cntr0} <= {6’d0, 6’d0, 6’d0,
6’d0};
else if (inc)
case(index) // parallel_case full_case
3’d0: cntr0 <= inc;
3’d1: cntr1 <= inc;
3’d2: cntr2 <= inc;
3’d3: cntr3 <= inc;
3’d4: cntr4 <= inc;
3’d5: cntr5 <= inc;
3’d6: cntr6 <= inc;
3’d7: cntr7 <= inc;
endcase
```

While arrays can be used to specify the function more concisely, the resultant logic is not very optimal in terms of area and speed. Instead of using the index operator [], the index can be generated using a decoder. This decoded index can then be used for both fetching the selected counter and assigning a new value to it. This technique of using a decoded value as the select control produces a more optimal design. The use of a for-loop in the example below is just a more succinct way of writing the resultant description.

```
wire [7:0] incDec = inc << index;

always @(posedge clk or posedge reset)

if (reset)

for( i=0; i<=7; i=i+1 )

cntr[i] <= 6'd0;

else for( i=0; i<=7; i=i+1 )

if (incDec[i])

cntr[i] <= cntr[i] + 1'b1;
```

A decoder with enable can be inferred through the use of a function call:

```
wire [7:0] incDec = decode8en(index, inc);
```

Register File Example

The following example declares a memory array called `rf` which consists of thirty-two 32- bit registers.

```
reg [31:0] rf[0:31];
```

While a multi-ported register file would most likely be implemented as a megacell, it can also be generated by the design compiler using a memory array. The following example contains the specification for a 32-location register file that would be contained in a typical 32-bit microprocessor.

```
module regfile(clk, weA, weB, dinA, dinB, destA, destB,

srcA, srcB, doutA, doutB);

input clk, weA, weB;

input [31:0] dinA, dinB;

input [4:0] destA, destB;

input [4:0] srcA, srcB;

output [31:0] doutA, doutB;

reg [31:0] rf [0:31];

assign doutA = srcA==0 ? 0 : rf[srcA];

assign doutB = srcB==0 ? 0 : rf[srcB];

always @ (posedge clk) begin

if ( weA )

rf[destA] <= dinA;

if ( weB )

rf[destB] <= dinB;

end

endmodule
```

The logic generated for this example is not very optimal in terms of speed or area. A more optimal implementation of a register file can be generated by using a decoded version of the source and destination addresses. A selector operator is used to multiplex between dinB and dinA. Priority is automatically given to `destB` without the use of additional logic. For this version, the generated logic for each element is identical to the specification in the for-loop, with

the “?” operator mapping to a 2-to-1 multiplexer. This version is more efficient because there are fewer indexing operations and because the decode logic is explicitly specified.

```
module regfile(clk, weA, weB, dinA, dinB, destA, destB,

srcA, srcB, doutA, doutB);

input clk, weA, weB;

input [31:0] dinA, dinB;

input [4:0] destA, destB;

input [4:0] srcA, srcB;

output [31:0] doutA, doutB;

reg [31:0] rf [0:31];

integer i;

assign doutA = srcA==0 ? 0 : rf[srcA];

assign doutB = srcB==0 ? 0 : rf[srcB];

wire [31:0] weDecA = (weA << destA); // additional detail

wire [31:0] weDecB = (weB << destB); // additional detail

wire [31:0] weDec = weDecA | weDecB; // additional detail

always @ (posedge clk) begin

for ( i=0; i<=31; i=i+1 ) // for-loop replaces random access

rf[i] <= weDec[i] ? (weDecB[i] ? dinB : dinA) : rf[i];

end

endmodule
```

Array of Counters

The following example describes an array of counters. On each clock, one counter can be conditionally incremented and one counter conditionally decremented. If an attempt is made to increment and decrement the same counter in the same clock, the old value should be preserved. The signal inhibit is used to achieve this function.

```
reg [5:0] cntr[0:7]; // 2-D array declaration

wire inhibit = incEnable & decEnable & (incAdr == decAdr);

wire inc = incEnable & ~inhibit;

wire dec = decEnable & ~inhibit;

always @(posedge clk) begin

cntr[incAdr] <= inc ? cntr[incAdr]+1 : cntr[incAdr];

cntr[decAdr] <= dec ? cntr[decAdr]-1 : cntr[decAdr];

end
```

When inc&~dec&(incAdr==decAdr), the example above will not simulate correctly. The second statement will take priority over the first statement since it is the last one in the block and it will overwrite the incremented value assigned in the first statement. Furthermore the selected `cntr` value to increment is computed twice instead of once. Here is the corrected example:

```
always @(posedge clk) begin

if (inc)

cntr[incAdr] <= cntr[incAdr]+ 1'b1;

if (dec)

cntr[decAdr] <= cntr[decAdr]- 1'b1;

end
```

The quality of the synthesized circuit can still be dramatically improved by using a decoded version of the address.

```
reg [5:0] cntr[0:7];
wire [7:0] inc = incEnable << incAdr;
wire [7:0] dec = decEnable << decAdr;
always @(posedge clk) begin
for (i = 0; i <= 7; i = i + 1)
cntr[i] <= inc[i] ^ dec[i] ?
(inc[i] ? cntr[i] + 1 : cntr[i] - 1) : cntr[i];
end
```

This code still has a problem in that eight incrementors and eight decrementors will be created. The following example fixes this last problem.

```
wire [5:0] cntrPlus1 = cntr[incAdr] + 1'b1;
wire [5:0] cntrMinus1 = cntr[decAdr] - 1'b1;
always @(posedge clk) begin
for (i = 0; i <= 7; i = i + 1)
cntr[i] <= inc[i] ^ dec[i] ?
(inc[i] ? cntrPlus1 : cntrMinus1) : cntr[i];
end
```

Multiple Assignments to the Same Variable

Avoid multiple assignments to the same variable except for arrays and vectors.

Assignments to the same variable in separate statements should be avoided except when used with 2-D arrays or vectors, where different elements can be updated at the same time without contention. When the same element is being written, the later assignment is dominant. In the case of multiple assignments to the same variable in different synchronous blocks, Synopsys infers two separate flip-flops which are ANDed together to produce a single output.

```
module test(clk, load1, a1, load2, a2, q);

input clk, load1, load2, a1, a2;

output q;

reg q;

always @ (posedge clk) begin

if (load1)

q <= a1;

end

always @ (posedge clk) begin

if (load2)

q <= a2;

end

endmodule
```

Putting both assignments in the same block avoids this problem. In this case, `load2` is dominant since it occurs later in the block. The logic to load variable `a1` is `load1` and `~load2`. If both inputs are mutually exclusive, then use the case `always` instead.

```
always @ (posedge clk) begin

if (load1)

q <= a1;
```

```
if (load2)

q <= a2;

end
```

In the case of vectors and 2-D arrays, different locations in the array or vector can be updated without contention. For example:

```
reg[31:0] flag;

always @(posedge clk) begin

(set_flag)

flag[set_index] <= 1'b1;

if (reset_flag)

flag[reset_index] <= 1'b0;

end
```

If `set_index` and `reset_index` specify the same location, the selected bit will be reset since that function corresponds to the last assignment in the block.

The following statement invokes the functions `dec32L` and `dec32`, which map to a userdefined decode module or DesignWare in order to create an optimized version of the previous example.

```
// Version using function invocation & map_to_module (not shown)

flag <= flag & dec32L(reset_index) | dec32(set_index);

// Version using DesignWare & module instantiation

wire [31:0] reset_dec, set_dec;

DW01_decode #(5) dec32_0 (reset_index, reset_dec);

DW01_decode #(5) dec32_1 (set_index, set_dec);
```

```
flag <= flag & ~reset_dec | set_dec;
```

Using `case` Statements

The following case statement implements a selector function:

```
always @(sela or selb or selc or seld or a or b or c or d)

case ({sela, selb, selc, seld})

4'b1000: dout = a;

4'b0100: dout = b;

4'b0010: dout = c;

4'b0001: dout = d;

endcase
```

If multiple select lines are active, then none of the case items will be selected and the variable `dout` will be unchanged from it's previous value. This results in the creation of a latch for dout in order conform with the Verilog language specification. Latch inference can be avoided either with the use of the Synopsys `full_case` compiler directive or with the use of the default clause. The `full_case` directive tells the tool that all valid states are represented. During simulation, if the case expression evaluates to a value that is not covered by the case-items, the Verilog simulation and the gate-level simulation will not compare. The `default` statement can also prevent latch inference but its meaning is different from the `full_case` directive. The `default` statement is used for ambiguity handling. It specifies

the output for any input values not defined. In v3.0, Design Compiler will automatically use full and/or parallel case when appropriate, provided all the case item expressions are constants. Assigning the output to 'bx in the default statement allows unknown propagation for Verilog simulation and specifies don't-care conditions for synthesis. This usually generates fewer gates. For these reasons, the default statement is preferred over the `full_case` directive.

```
default: dout = 3'bx; // for ambiguity handling
```

```
Given that the select signals are mutually exclusive, a more optimal

selector design can be implemented using the casez statement.

always @(sela or selb or selc or seld or a or b or c or d)

casez ({sela, selb, selc, seld}) // synopsys parallel_case

4'b1???: dout = a;

4'b?1??: dout = b;

4'b??1?: dout = c;

4'b???1: dout = d;

default: dout = 'bx;

endcase
```

The Verilog `case` statement is evaluated by comparing the expression following the `case` keyword to the case-item expressions in the exact order they are given. The statement corresponding to the first case item that matches the case expression is executed. If all the comparisons fail, the default statement is executed. This may not be what the designer intended.

The `parallel_case` directive instructs the Design Compiler to evaluate all case items in parallel and for all case items that match the case expression, to execute the corresponding statements. Without this directive, the logic generated implements a type of priority encode logic for each case item. If more than one case item evaluates true, the generated gate-level design will not match the behavior of the original Verilog source. Without the `parallel_case` directive, the selector function using the `casez` would actually be equivalent to the following selector description:

```
always @(sela or selb or selc or seld or a or b or c or d)

casez ({sela, selb, selc, seld}) // equivalent design

4'b1???: dout = a;

4'b01??: dout = b;
```

```
4’b001?: dout = c;

4’b0001: dout = d;

default: dout = ‘bx;

endcase
```

Use the case `always` *to implement selector type functions.*

The selector function can be more concisely specified as:

```
// note: inputs must be mutually exclusive

always @(sela or selb or selc or seld or a or b or c or d)

case(1’b1) // synopsys parallel_case

sela: dout = a;

selb: dout = b;

selc: dout = c;

seld: dout = d;

default: dout = ‘bx;

endcase
```

This construct is best used whenever the inputs are mutually exclusive. The following example shows the execute unit of a V7 SPARC integer unit.

```
wire // instruction decode

AND = op==9’h81, ANDN = op==9’h85,

OR = op== 9’h82, ORN = op==9’h86,

XOR = op==9’h83, XNOR = op==9’h87,

ANDCC = op==9’h91, ANDNCC = op==9’h95,
```

```
ORCC = op==9'h92, ORNCC = op==9'h96,
XORCC = op==9'h93, XNORCC = op==9'h97; // etc...
always @(operand1 or operand2 or Y or PSR or WIM or TBR or
AND or ANDN or ANDCC or ANDNCC OR or ORN or ORCC or ORNCC or
XOR or XNOR or XORCC or XNORCC or WRY or WRPSR or WRWIM or WRTBR or
MULSCC or SLL or SRL or SRA or RDY or RDPSR or RDWIM or RDTBR)
case(1) // synopsys parallel_case
AND, ANDN, ANDCC, ANDNCC:
result = operand1 & operand2;
OR, ORN, ORCC, ORNCC:
result = operand1 | operand2;
XOR, XNOR, XORCC, XNORCC, WRY, WRPSR, WRWIM, WRTBR:
result = operand1 ^ operand2;
MULSCC: result = Y[0] ? sum : operand1;
SLL: result = operand1 << operand2[4:0];
SRL: result = operand1 >> operand2[4:0];
SRA: result = {{31{operand1[31]}}, operand1} >> operand2[
4:0];
RDY: result = Y;
RDPSR: result = PSR;
RDWIM: result = WIM;
RDTBR: result = {TBR, 4'd0};
default: result = sum; // for all other instructions
endcase
```

Compiler Directives

Imbed dont_touch directives in the Verilog source code.

Compiler commands can be placed in the Verilog source code using the directives

`dc_script_begin` and `dc_script_end`. The Synopsys Design Compiler will automatically compile submodules unless the `dont_touch` attribute is placed on the cell instances. For modules that are instantiated more than once in a design and not uniquified the `dont_touch` attribute is required. When required, `dont_touch` directives should be placed in the Verilog source code.

```
// synopsys dc_script_begin

// dont_touch {cx cl ap sc ll rb rt pb}

// synopsys dc_script_end
```

Parameterized Designs

Verilog provides the capability to build parameterized designs by changing parameter values in any module instance. The method supported by HDL Compiler is to use the module instance parameter value assignment instead of the `defparam` statement. In the following design, Design Compiler builds a 32-bit version of the 2-to-1 mux and uses this instance in the shiftLeft design. In v3.0 analyze and elaborate replace the use of templates.

```
module shiftLeft(a, shiftCnt, shifta);

input [31:0] a;

input [4:0] shiftCnt;

output [31:0] shifta;

wire [31:0] d0, d1, d2, d3;
```

```
// assign shifta = a << shiftCnt;

mux2to1 #(32) m21_0 (a, {a[30:0], 1'b0}, shiftCnt[0], d0);

mux2to1 #(32) m21_1 (d0, {d0[29:0], 2'b0}, shiftCnt[1], d1);

mux2to1 #(32) m21_2 (d1, {d1[27:0], 4'b0}, shiftCnt[2], d2);

mux2to1 #(32) m21_3 (d2, {d2[23:0], 8'b0}, shiftCnt[3], d3);

mux2to1 #(32) m21_4 (d3, {d3[15:0], 16'b0}, shiftCnt[4], shifta);

endmodule

module mux2to1 (a, b, s, z);

parameter width = 2;

input [width-1:0] a, b;

input s;

output [width-1:0] z;

// synopsys template

assign z = s ? b : a;

endmodule
```

Tasks

For the purposes of synthesis, task statements are similar to functions in Verilog except they can have more than one output or no outputs, and they can have inout ports. Only `reg` variables can receive the output from a task, unlike functions.The tasks logic becomes part of the module from which it is invoked, so it is not necessary to input all the variables explicitly.

```
always task_invocation (in1, in2, reg1, reg2);
```

Although perhaps not as readable, a function can also return multiple values by concatenating the results together and then using an assign statement or procedural assignment to separate the values in the calling module.

```
// continuous assignment

assign {wire1, wire2, wire3} = function_invocation(in1, in2);

always @(posedge clk ) // procedural assignment

{reg1, reg2, reg3} <= function_invocation(in1, in2);
```

State Machine Design

A state machine can be specified using a number of different formats: Verilog, Synopsys State Table Design Format, or the PLA Design Format. The Synopsys Verilog HDL Compiler can "extract" the state table from a Verilog description if the `state_vector` and `enum` directives are used. The `state_vector` directive requires the use of inferred flop-flops within the same module as the state machine specification. However, an extracted state table is not necessary in order to compile a state machine. The use of an extracted state table does provide the following benefits:

_ It provides good documentation of the state machine behavior

_ State minimization can be performed

_ Don't-care conditions are utilized since FSM extraction includes flattening

_ Tradeoffs between different encoding styles can easily be made

_ Don't-care state codes are automatically derived

state machine compiler is especially effective when using a one-hot encoding style. After using the state machine compiler, Design Compiler can also be used for further optimization. (When using an encoded state vector with few don't-care states, there is not much benefit in using the state machine compiler.) When a state table is extracted, Design Compiler enumerates all state

transitions. For example, a reset function generates an explicit transition from every possible state to the reset state. This can potentially cause the state table to explode in size. Even though invalid input combinations or mutually exclusive inputs can be specified using the `casez` construct or the `parallel_case` directive, they cannot be concisely

represented in the state table format. However, the PLA format provides a way to

specify these cases concisely, potentially resulting in a more optimal design. The PLA format can also specify the various don't-care conditions and have these utilized without flattening the design. (The PLA format is already a flat, two-level sum of products.)

In a Mealy machine, the outputs depend on the current inputs. In a Moore machine the outputs are either registered or depend only on the current state. An "incompletely specified state machine" means that the transition behavior is not specified for all possible input conditions and there exists a next-state don't-care set. The next-state assignments can be performed inside a sequential block or in a combinational block. The combinational block can either be an `always` block or a function. If all the outputs are registered, the output assignments can be included in a sequential block; otherwise they must be in a combinational block. State machine outputs that drive asynchronous logic or preset and clear inputs of flipflops must not glitch. These outputs must be registered or else gray-code state encoding must be used for the corresponding state transitions.

The example below describes a simple state machine with an inferred state register.

```
// moore machine

module fsm_example(clk, reset, c, valid);

input clk, reset, c;

output valid;

parameter [2:0] // synopsys enum fsm_states

idle = 3'd0,

one_str = 3'd1,

zero_str = 3'd2,

valid_str = 3'd4,

invalid_str = 3'd3;
```

```
reg [2:0] /* synopsys enum fsm_states */ state;

// synopsys state_vector state

// next state assignments in sequential block

always @(posedge clk)

if (reset)

state <= idle;

else case(state)

idle: state <= c ? one_str : invalid_str;

one_str: state <= c ? one_str : zero_str;

zero_str: state <= c ? valid_str : zero_str;

valid_str: state <= c ? valid_str : invalid_str;

invalid_str: state <= invalid_str;

default: state <= 3'bx; // dont_care conditions

endcase

// note: valid becomes msb of state register

assign valid = state == valid_str;

assign first_one = (state == idle) & c;

endmodule
```

The next-state assignments can also be done in a combinational block with the next-state-tocurrent- state assignment done in a sequential block. Using this technique, unregistered outputs can be assigned along with the next state. These techniques are illustrated in the following example:

```
// mealy machine

module fsm_example(clk, reset, c, valid);
```

```
input clk, reset, c;

output valid;

parameter [3:0] // synopsys enum fsm_states

idle = 3'd0,

one_str = 3'd1,

zero_str = 3'd2,

valid_str = 3'd4,

invalid_str = 3'd3;

reg valid, first_one; // a wire

reg [2:0] /* synopsys enum fsm_states */ state;

// synopsys state_vector state

// next state assignments in combinational block

always @(c or state or reset)

if (reset) begin

nxt_state = idle;

valid = 0;

first_one = 0;

end

else begin

valid = 0; // put defaults here

first_one = 0; // put defaults here

case(state)

idle:

if (c) begin

nxt_state = one_str;
```

```
first_one = 1;
end else
next_state = idle;
one_str:
if (c) nxt_state = one_str;
else nxt_state = zero_str;
zero_str:
if (c) nxt_state = valid_str;
else nxt_state = zero_str;
valid_str: begin
if (c) nxt_state = valid_str;
else nxt_state = invalid_str;
valid = 1;
end
invalid_str: begin
nxt_state = invalid_str;
valid = 0;
end
default: nxt_state = 3'bx; // dont_care conditions
endcase
end
// an additional sequential block is needed
always @(posedge clk)
state <= next_state;
endmodule
```

Outputs that depend only on the current state can be decoded using a `case` statement with case variable "`state`". Outputs that depend on the state transition (which implies a dependency on both the current state and the inputs) can be conditionally asserted in a particular state or they can be decoded from the variable "`nxt_state`".

Conclusion

By judicious partitioning of the design, using various combinations of horizontal and vertical partitioning as well as the addition of hierarchy, a designer can control the synthesis process. Horizontal partitioning breaks up a design into smaller slices that are more easily synthesized. Vertical partitioning tries to keep an entire critical path in one module and tries to register outputs of modules. The addition of hierarchy preserves a user-specified structure. It is possible to achieve good results and still have a technology-transparent design by using userdefined macrocell libraries, the GTECH library, and DesignWare. These hand-crafted libraries become the building blocks for success. Another guideline is to start simple and refine the structure as needed to meet speed and area objectives. This enables timing analysis and subsequent design changes to be made early in the design process. Verilog constructs

should be used in a manner that prevents simulation mismatch between RTL and gate-level versions of the design. You should have complete sensitivity lists, use non-blocking assignments, and don't mix blocking and non-blocking assignments. When using `always` blocks, be sure to either specify all the default assignments at the front of the block or check to insure all variables are assigned a value regardless of the input combinations. For `case` statements, the use of a default clause may not be sufficient. The use of procedural assignments within sequential blocks can simplify the code because default assignments are not needed. However, only registered outputs can be used in a sequential block; nonregistered outputs must be specified either with a continuous assignment or in a combinational `always` block.

11) What are different ways to synchronize between two clock domains?

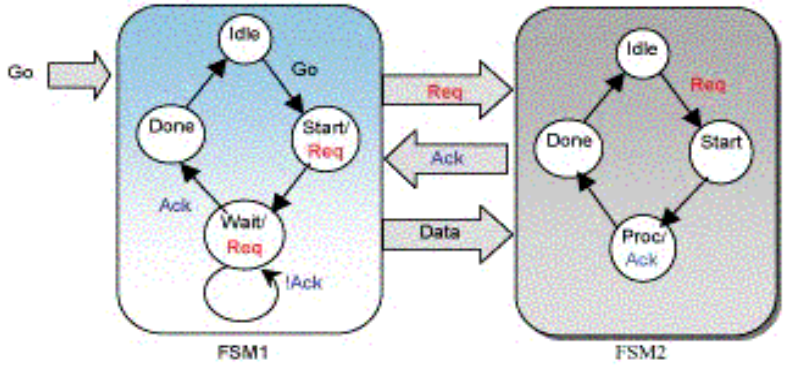
Clock Domain Crossing. . .

The following section explains clock domain interfacing

One of the biggest challenges of system-on-chip (SOC) designs is that different blocks operate on independent clocks. Integrating these blocks via the processor bus, memory ports, peripheral busses, and other interfaces can be troublesome because unpredictable behavior can result when the

asynchronous interfaces are not properly synchronized

A very common and robust method for synchronizing multiple data signals is a handshake technique as shown in diagram below This is popular because the handshake technique can easily manage changes in clock frequencies, while minimizing latency at the crossing. However, handshake logic is significantly more complex than standard synchronization structures.



FSM1(Transmitter) asserts the req (request) signal, asking the receiver to accept the data on the data bus. FSM2(Receiver) generally a slow module asserts the ack (acknowledge) signal, signifying that it has accepted the data.

it has loop holes: when system Receiver samples the systems Transmitter req line and Transmitter samples system Receiver ack line, they have done it with respect to their internal clock, so there will be setup and hold time violation. To avoid this we go for double or triple stage synchronizers, which increase the MTBF and thus are immune to metastability to a good extent. The figure below shows how this is done.

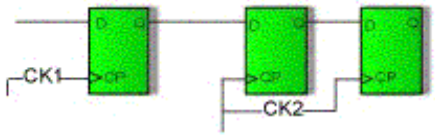


Figure 1a — Single-bit metastability sync

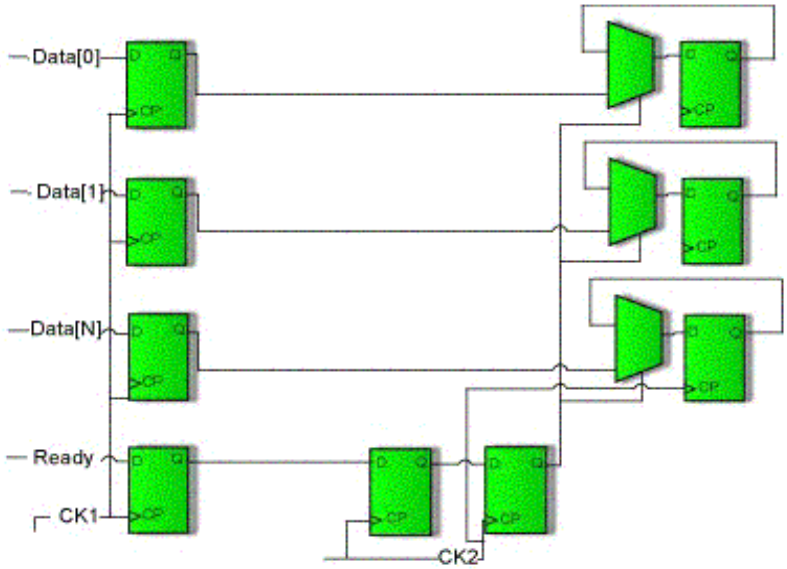


Figure 1b — Multi-bit sync

Blocking vs Non-Blocking. . .

self triggering blocks -

```
module osc2 (clk);
output clk;
reg clk;
initial #10 clk = 0;
always @(clk) #10 clk <= ~clk;
endmodule
```

After the first @(clk) trigger, the RHS expression of the nonblocking assignment is evaluated and the LHS value scheduled into the nonblocking assign updates event queue.
Before the nonblocking assign updates event queue is "activated," the @(clk) trigger statement is encountered and the always block again becomes sensitive to changes on the clk signal. When the nonblocking LHS value is updated later in the same time step, the @(clk) is again triggered.

```
module osc1 (clk);
output clk;
reg clk;
initial #10 clk = 0;
always @(clk) #10 clk = ~clk;
endmodule
```

Blocking assignments evaluate their RHS expression and update their LHS value without interruption. The blocking assignment must complete before the @(clk) edge-trigger event can be scheduled. By the time the trigger event has been scheduled, the blocking clk assignment has completed; therefore, there is no trigger event from within the always block to trigger the @(clk) trigger.

Bad modeling: - (using blocking for seq. logic)

```
always @(posedge clk) begin
q1 = d;
q2 = q1;
q3 = q2;
end
```

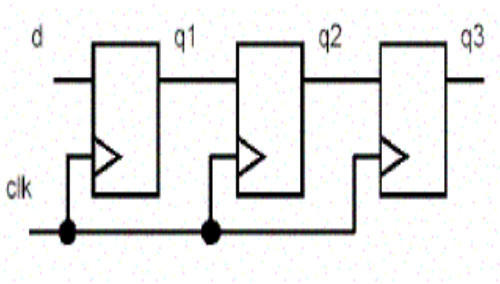
Race Condition

```
always @(posedge clk) q1=d;
always @(posedge clk) q2=q1;
always @(posedge clk) q3=q2;

always @(posedge clk) q2=q1;
always @(posedge clk) q3=q2;
always @(posedge clk) q1=d;

always @(posedge clk) begin
q3 = q2;
q2 = q1;
q1 = d;
end
```

Bad style but still works



Good modeling: -

```
always @(posedge clk) begin
q1 <= d;
q2 <= q1;
q3 <= q2;
end

always @(posedge clk) begin
q3 <= q2;
q2 <= q1;
q1 <= d;
end
```

No matter of sequence for Nonblocking

```
always @(posedge clk) q1<=d;
always @(posedge clk) q2<=q1;
always @(posedge clk) q3<=q2;

always @(posedge clk) q2<=q1;
always @(posedge clk) q3<=q2;
always @(posedge clk) q1<=d;
```

Good Combinational logic :- (Blocking)

```
always @(a or b or c or d) begin
tmp1 = a & b;
tmp2 = c & d;
y = tmp1 | tmp2;
end
```

Bad Combinational logic :- (Nonblocking)

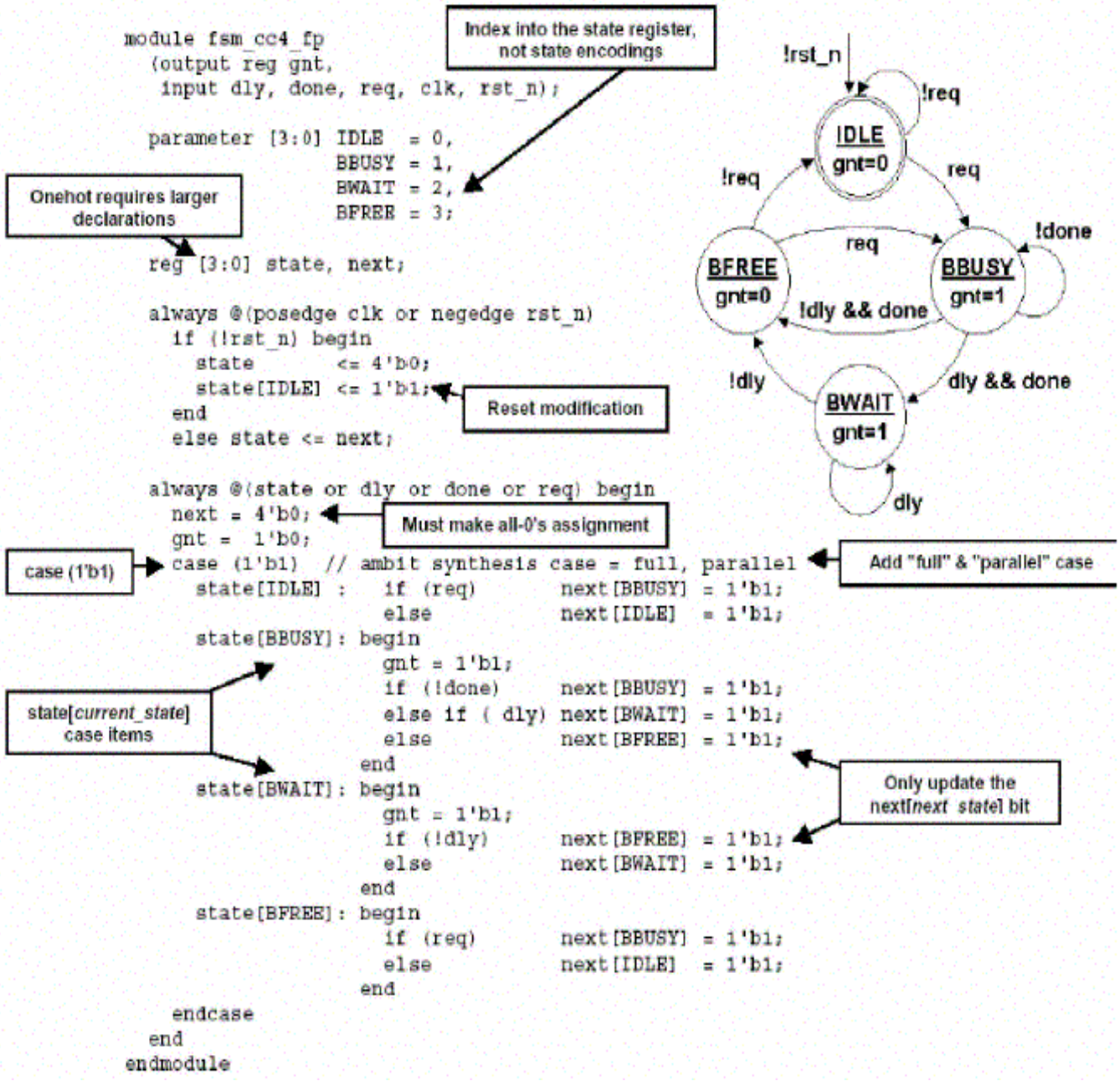
always @(a or b or c or d) begin will simulate incorrectly…

```
tmp1 <= a & b; need tmp1, tmp2 insensitivity
tmp2 <= c & d;
y <= tmp1 | tmp2;
end
```

Mixed design: -

Use Nonblocking assignment.
In case on multiple non-blocking assignments last one will win.

Verilog FSM




```
module fsm_cc4_fp
  (output reg gnt,
   input dly, done, req, clk, rst_n);

  parameter [3:0] IDLE  = 0,
                  BBUSY = 1,
                  BWAIT = 2,
                  BFREE = 3;

  reg [3:0] state, next;

  always @(posedge clk or negedge rst_n)
    if (!rst_n) begin
      state       <= 4'b0;
      state[IDLE] <= 1'b1;
    end
    else state <= next;

  always @(state or dly or done or req) begin
    next = 4'b0;
    gnt =  1'b0;
    case (1'b1)  // ambit synthesis case = full, parallel
      state[IDLE] :   if (req)       next[BBUSY] = 1'b1;
                      else           next[IDLE]  = 1'b1;
      state[BBUSY]: begin
                      gnt = 1'b1;
                      if (!done)     next[BBUSY] = 1'b1;
                      else if ( dly) next[BWAIT] = 1'b1;
                      else           next[BFREE] = 1'b1;
                    end
      state[BWAIT]: begin
                      gnt = 1'b1;
                      if (!dly)      next[BFREE] = 1'b1;
                      else           next[BWAIT] = 1'b1;
                    end
      state[BFREE]: begin
                      if (req)       next[BBUSY] = 1'b1;
                      else           next[IDLE]  = 1'b1;
                    end
    endcase
  end
endmodule
```

1) Explain about setup time and hold time, what will happen if there is setup time and hold tine violation, how to overcome this?

Set up time is the amount of time before the clock edge that the input signal needs to be stable to guarantee it is accepted properly on the clock edge.
Hold time is the amount of time after the clock edge that same input signal has to be held before

changing it to make sure it is sensed properly at the clock edge.
Whenever there are setup and hold time violations in any flip-flop, it enters a state where its output is unpredictable: this state is known as metastable state (quasi stable state); at the end of metastable state, the flip-flop settles down to either '1' or '0'. This whole process is known as metastability

2) What is skew, what are problems associated with it and how to minimize it?

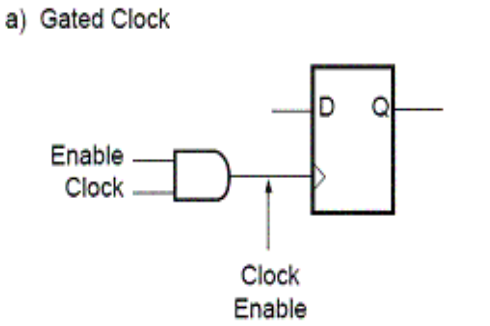
In circuit design, clock skew is a phenomenon in synchronous circuits in which the clock signal (sent from the clock circuit) arrives at different components at different times.
This is typically due to two causes. The first is a material flaw, which causes a signal to travel faster or slower than expected. The second is distance: if the signal has to travel the entire length of a circuit, it will likely (depending on the circuit's size) arrive at different parts of the circuit at different times. Clock skew can cause harm in two ways. Suppose that a logic path travels through combinational logic from a source flip-flop to a destination flip-flop. If the destination flip-flop receives the clock tick later than the source flip-flop, and if the logic path delay is short enough, then the data signal might arrive at the destination flip-flop before the clock tick, destroying there the previous data that should have been clocked through. This is called a hold violation because the previous data is not held long enough at the destination flip-flop to be properly clocked through. If the destination flip-flop receives the clock tick earlier than the source flip-flop, then the data signal has that much less time to reach the destination flip-flop before the next clock tick. If it fails to do so, a setup violation occurs, so-called because the new data was not set up and stable before the next clock tick arrived. A hold violation is more serious than a setup violation because it cannot be fixed by increasing the clock period.
Clock skew, if done right, can also benefit a circuit. It can be intentionally introduced to decrease the clock period at which the circuit will operate correctly, and/or to increase the setup or hold safety margins. The optimal set of clock delays is determined by a linear program, in which a setup and a hold constraint appears for each logic path. In this linear program, zero clock skew is merely a feasible point.
Clock skew can be minimized by proper routing of clock signal (clock distribution tree) or putting variable delay buffer so that all clock inputs arrive at the same time

3) What is slack?

'Slack' is the amount of time you have that is measured from when an event 'actually happens' and when it 'must happen'.. The term 'actually happens' can also be taken as being a predicted time for when the event will 'actually happen'.
When something 'must happen' can also be called a 'deadline' so another definition of slack would be the time from when something 'actually happens' (call this Tact) until the deadline (call this Tdead).
Slack = Tdead - Tact.
Negative slack implies that the 'actually happen' time is later than the 'deadline' time...in other words it's too late and a timing violation....you have a timing problem that needs some attention.

4) What is glitch? What causes it (explain with waveform)? How to overcome it?

The following figure shows a gated clock. The gated clock's corresponding timing diagram shows that this implementation can lead to clock glitches, which can cause the flip-flop to clock at the wrong time.



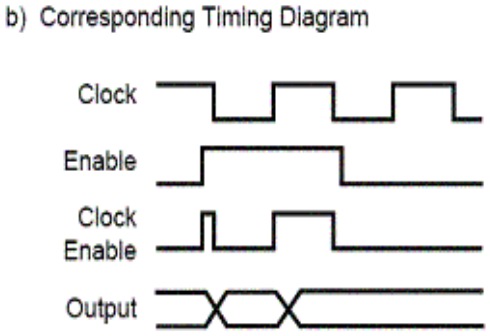


Figure 2-12: **Gated Clock**

The following figure shows a synchronous alternative to the gated clock using a data path. The flip-flop is clocked at every clock cycle and the data path is controlled by an enable. When the enable is Low, the multiplexer feeds the output of the register back on itself. When the enable is High, new data is fed to the flip-flop and the register changes its state

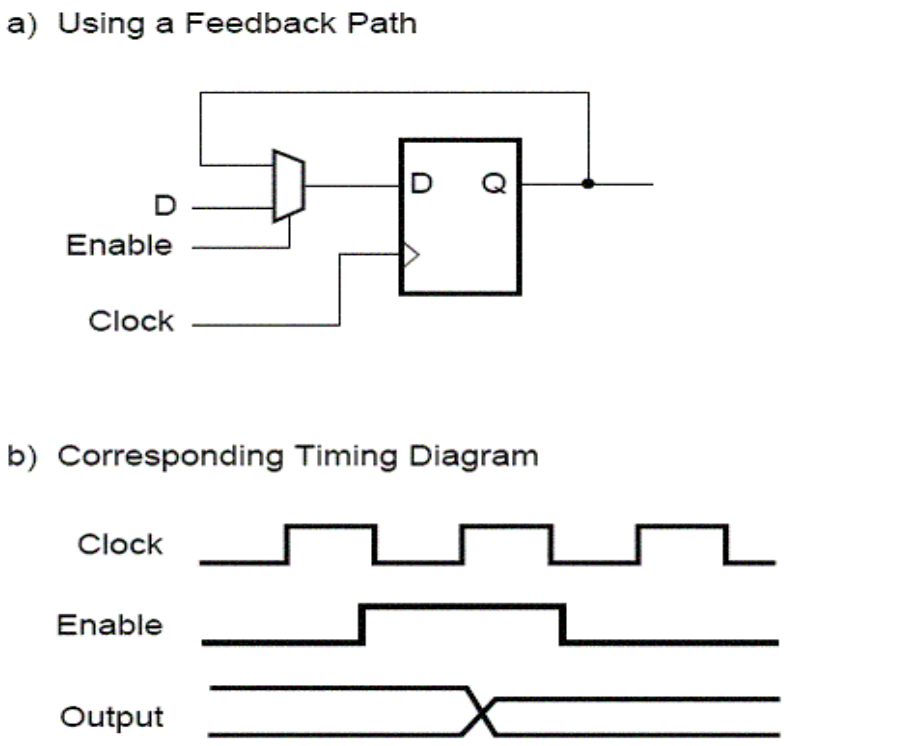


Figure 2-13: **Synchronous Design Using Data Feedback**

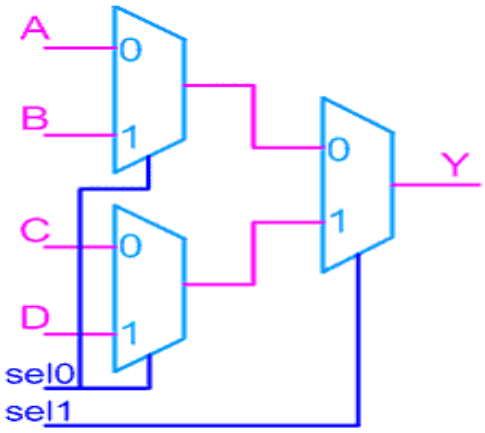
5) Given only two xor gates one must function as buffer and another as inverter?

Tie one of xor gates input to 1 it will act as inverter.
Tie one of xor gates input to 0 it will act as buffer.

6) What is difference between latch and flipflop?

The main difference between latch and FF is that latches are level sensitive while FF are edge sensitive. They both require the use of clock signal and are used in sequential logic. For a latch, the output tracks the input when the clock signal is high, so as long as the clock is logic 1, the output can change if the input also changes. FF on the other hand, will store the input only when there is a rising/falling edge of the clock.

7) Build a 4:1 mux using only 2:1 mux?



Difference between heap and stack?

The Stack is more or less responsible for keeping track of what's executing in our code (or what's been "called"). The Heap is more or less responsible for keeping track of our objects (our data, well... most of it - we'll get to that later.).
Think of the Stack as a series of boxes stacked one on top of the next. We keep track of what's going on in our application by stacking another box on top every time we call a method (called a Frame). We can only use what's in the top box on the stack. When we're done with the top box (the method is done executing) we throw it away and proceed to use the stuff in the previous box on the top of the stack. The Heap is similar except that its purpose is to hold information (not keep track of execution most of the time) so anything in our Heap can be accessed at any time. With the Heap, there are no constraints as to what can be accessed like in the stack. The Heap is like the heap of clean laundry on our bed that we have not taken the time to put away yet - we can grab what we need quickly. The Stack is like the stack of shoe boxes in the closet where we have to take off the top one to get to the one underneath it.

9) Difference between mealy and moore state machine?

A) Mealy and Moore models are the basic models of state machines. A state machine which uses only Entry Actions, so that its output depends on the state, is called a Moore model. A state machine which uses only Input Actions, so that the output depends on the state and also on inputs, is called a Mealy model. The models selected will influence a design but there are no general indications as to which model is better. Choice of a model depends on the application, execution means (for instance, hardware systems are usually best realized as Moore models) and personal preferences of a designer or programmer

B) Mealy machine has outputs that depend on the state and input (thus, the FSM has the output written on edges)
Moore machine has outputs that depend on state only (thus, the FSM has the output written in the

state itself.

Adv and Disadv
In Mealy as the output variable is a function both input and state, changes of state of the state variables will be delayed with respect to changes of signal level in the input variables, there are possibilities of glitches appearing in the output variables. Moore overcomes glitches as output dependent on only states and not the input signal level.
All of the concepts can be applied to Moore-model state machines because any Moore state machine can be implemented as a Mealy state machine, although the converse is not true.
Moore machine: the outputs are properties of states themselves... which means that you get the output after the machine reaches a particular state, or to get some output your machine has to be taken to a state which provides you the output.The outputs are held until you go to some other state Mealy machine:
Mealy machines give you outputs instantly, that is immediately upon receiving input, but the output is not held after that clock cycle.

10) Difference between onehot and binary encoding?

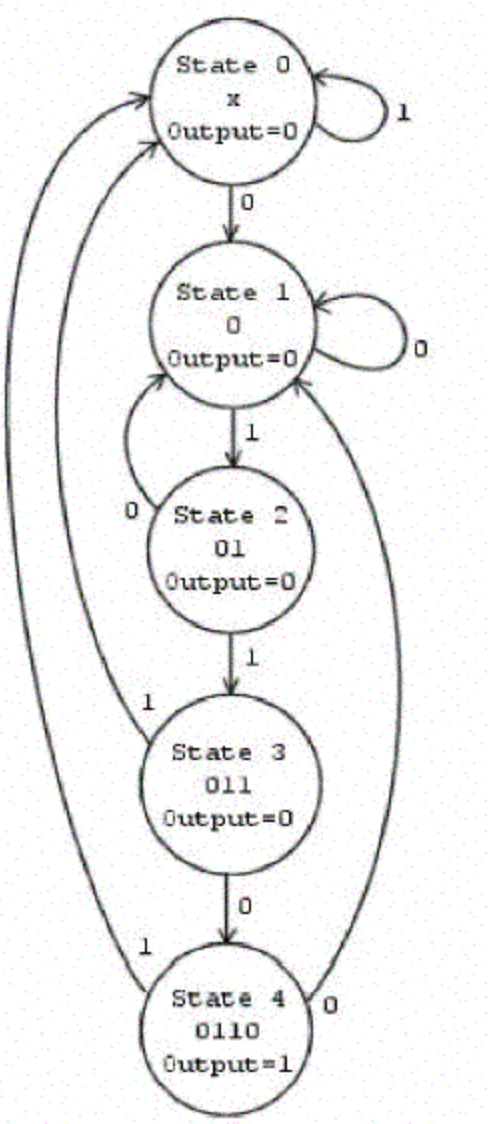
Common classifications used to describe the state encoding of an FSM are Binary (or highly encoded) and One hot.
A binary-encoded FSM design only requires as many flip-flops as are needed to uniquely encode the number of states in the state machine. The actual number of flip-flops required is equal to the ceiling of the log-base-2 of the number of states in the FSM.
A onehot FSM design requires a flip-flop for each state in the design and only one flip-flop (the flip-flop representing the current or "hot" state) is set at a time in a one hot FSM design. For a state machine with 9- 16 states, a binary FSM only requires 4 flip-flops while a onehot FSM requires a flip-flop for each state in the design
FPGA vendors frequently recommend using a onehot state encoding style because flip-flops are plentiful in an FPGA and the combinational logic required to implement a onehot FSM design is typically smaller than most binary encoding styles. Since FPGA performance is typically related to the combinational logic size of the FPGA design, onehot FSMs typically run faster than a binary encoded FSM with larger combinational logic blocks

12) How to calculate maximum operating frequency?

13) How to find out longest path?

You can find answer to this in timing.ppt of presentations section on this site

14) Draw the state diagram to output a "1" for one cycle if the sequence "0110" shows up (the leading 0s cannot be used in more than one sequence)?



15) How to achieve 180 degree exact phase shift?

Never tell using inverter
a) dcm's an inbuilt resource in most of fpga can be configured to get 180 degree phase shift.
b) Bufgds that is differential signaling buffers which are also inbuilt resource of most of FPGA can be used.

16) What is significance of ras and cas in SDRAM?

SDRAM receives its address command in two address words.
It uses a multiplex scheme to save input pins. The first address word is latched into the DRAM

chip with the row address strobe (RAS).
Following the RAS command is the column address strobe (CAS) for latching the second address word.
Shortly after the RAS and CAS strobes, the stored data is valid for reading.

17) Tell some of applications of buffer?

a)They are used to introduce small delays
b)They are used to eliminate cross talk caused due to inter electrode capacitance due to close routing.
c)They are used to support high fanout,eg:bufg

18) Implement an AND gate using mux?

This is the basic question that many interviewers ask. for and gate, give one input as select line,incase if u r giving b as select line, connect one input to logic '0' and other input to a.

19) What will happen if contents of register are shifter left, right?

It is well known that in left shift all bits will be shifted left and LSB will be appended with 0 and in right shift all bits will be shifted right and MSB will be appended with 0 this is a straightforward answer

What is expected is in a left shift value gets Multiplied by 2 eg:consider 0000_1110=14 a left shift will make it 0001_110=28, it the same fashion right shift will Divide the value by 2.

20)Given the following FIFO and rules, how deep does the FIFO need to be to prevent underflow or overflow?

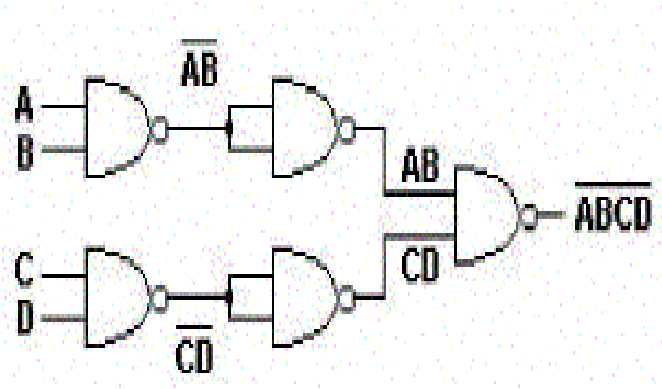
RULES:
1) frequency(clk_A) = frequency(clk_B) / 4
2) period(en_B) = period(clk_A) * 100
3) duty_cycle(en_B) = 25%

Assume clk_B = 100MHz (10ns)
From (1), clk_A = 25MHz (40ns)
From (2), period(en_B) = 40ns * 400 = 4000ns, but we only output for

1000ns,due to (3), so 3000ns of the enable we are doing no output work. Therefore, FIFO size = 3000ns/40ns = 75 entries.

21) Design a four-input NAND gate using only two-input NAND gates ?

A:Basically, you can tie the inputs of a NAND gate together to get an inverter, so...



22)Difference between Synchronous and Asynchronous reset.?

Synchronous reset logic will synthesize to smaller flip-flops, particularly if the reset is gated with the logic generating the d-input. But in such a case, the combinational logic gate count grows, so the overall gate count savings may not be that significant.
The clock works as a filter for small reset glitches; however, if these glitches occur near the active clock edge, the Flip-flop could go metastable. In some designs, the reset must be generated by a set of internal conditions. A synchronous reset is recommended for these types of designs because it will filter the logic equation glitches between clock.

Disadvantages of synchronous reset:
Problem with synchronous resets is that the synthesis tool cannot easily distinguish the reset signal from any other data signal.
Synchronous resets may need a pulse stretcher to guarantee a reset pulse width wide enough to ensure reset is present during an active edge of the clock[if you have a gated clock to save power, the clock may be disabled coincident with the assertion of reset. Only an asynchronous reset will work in this situation, as the reset might be removed prior to the resumption of the clock.
Designs that are pushing the limit for data path timing, can not afford to have added gates and

additional net delays in the data path due to logic inserted to handle synchronous resets.
Asynchronous reset :
The biggest problem with asynchronous resets is the reset release, also called reset removal. Using an asynchronous reset, the designer is guaranteed not to have the reset added to the data path. Another advantage favoring asynchronous resets is that the circuit can be reset with or without a clock present. Disadvantages of asynchronous reset: ensure that the release of the reset can occur within one clock period. if the release of the reset occurred on or near a clock edge such that the flip-flops went metastable.

23) Why are most interrupts active low?

This answers why most signals are active low
If you consider the transistor level of a module, active low means the capacitor in the output terminal gets charged or discharged based on low to high and high to low transition respectively. when it goes from high to low it depends on the pull down resistor that pulls it down and it is relatively easy for the output capacitance to discharge rather than charging. hence people prefer using active low signals.

24)Give two ways of converting a two input NAND gate to an inverter?

(a) short the 2 inputs of the nand gate and apply the single input to it.
(b) Connect the output to one of the input and the other to the input signal.

25) What are set up time & hold time constraints? What do they signify? Which one is critical for estimating maximum clock frequency of a circuit?

set up time: - the amount of time the data should be stable before the application of the clock signal, where as the hold time is the amount of time the data should be stable after the application of the clock. Setup time signifies maximum delay constraints; hold time is for minimum delay constraints. Setup time is critical for establishing the maximum clock frequency.

26) Differences between D-Latch and D flip-flop?

D-latch is level sensitive where as flip-flop is edge sensitive. Flip-flops are made up of latches.

27) What is a multiplexer?

Is a combinational circuit that selects binary information from one of many input lines and directs it to a single output line. (2n =>n).

28)How can you convert an SR Flip-flop to a JK Flip-flop?

By giving the feed back we can convert, i.e !Q=>S and Q=>R.Hence the S and R inputs will act as J and K

respectively.

29)How can you convert the JK Flip-flop to a D Flip-flop?

By connecting the J input to the K through the inverter.

30)What is Race-around problem?How can you rectify it?

The clock pulse that remains in the 1 state while both J and K are equal to 1 will cause the output to complement again and repeat complementing until the pulse goes back to 0, this is called the race around problem.To avoid this undesirable operation, the clock pulse must have a time duration that is shorter than the propagation delay time of the F-F, this is restrictive so the alternative is master-slave or edge-triggered construction.

31)How do you detect if two 8-bit signals are same?

XOR each bits of A with B (for e.g. A[0] xor B[0]) and so on.the o/p of 8 xor gates are then given as i/p to an 8-i/p nor gate. if o/p is 1 then A=B.

32)7 bit ring counter's initial state is 0100010. After how many clock cycles will it return to the initial state?

6 cycles

33) Convert D-FF into divide by 2. (not latch) What is the max clock frequency the circuit can handle, given the following information?

T_setup= 6nS T_hold = 2nS T_propagation = 10nS

Circuit: Connect Qbar to D and apply the clk at clk of DFF and take the O/P at Q. It gives freq/2. Max. Freq of operation: 1/ (propagation delay+setup time) = 1/16ns = 62.5 MHz

34)Guys this is the basic question asked most frequently. Design all the basic gates(NOT,AND,OR,NAND,NOR,XOR,XNOR) using 2:1 Multiplexer?

Using 2:1 Mux, (2 inputs, 1 output and a select line)
(a) NOT
Give the input at the select line and connect I0 to 1 & I1 to 0. So if A is 1, we will get I1 that is 0 at the O/P.
(b) AND
Give input A at the select line and 0 to I0 and B to I1. O/p is A & B
(c) OR

Give input A at the select line and 1 to I1 and B to I0. O/p will be A | B
(d) NAND
AND + NOT implementations together
(e) NOR
OR + NOT implementations together
(f) XOR
A at the select line B at I0 and ~B at I1. ~B can be obtained from (a) (g) XNOR
A at the select line B at I1 and ~B at I0

35)N number of XNOR gates are connected in series such that the N inputs (A0,A1,A2.....) are given in the following way: A0 & A1 to first XNOR gate and A2 & O/P of First XNOR to second XNOR gate and so on..... Nth XNOR gates output is final output. How does this circuit work? Explain in detail?

If N=Odd, the circuit acts as even parity detector, ie the output will 1 if there are even number of 1's in the N input...This could also be called as odd parity generator since with this additional 1 as output the total number of 1's will be ODD.
If N=Even, just the opposite, it will be Odd parity detector or Even Parity Generator.

36)An assembly line has 3 fail safe sensors and one emergency shutdown switch.The line should keep moving unless any of the following conditions arise:
(i) If the emergency switch is pressed
(ii) If the senor1 and sensor2 are activated at the same time.
(iii) If sensor 2 and sensor3 are activated at the same time.
(iv) If all the sensors are activated at the same time
Suppose a combinational circuit for above case is to be implemented only with NAND Gates. How many minimum number of 2 input NAND gates are required?

No of 2-input NAND Gates required = 6 You can try the whole implementation.

37)Design a circuit that calculates the square of a number? It should not use any multiplier circuits. It should use Multiplexers and other logic?

This is interesting....
1^2=0+1=1
2^2=1+3=4
3^2=4+5=9
4^2=9+7=16
5^2=16+9=25
and so on
See a pattern yet?To get the next square, all you have to do is add the next odd number to the previous square that you found.See how 1,3,5,7 and finally 9 are added.Wouldn't this be a possible solution to your question since it only will use a counter,multiplexer and a couple of adders?It seems it would take

n clock cycles to calculate square of n.

38) How will you implement a Full subtractor from a Full adder?

all the bits of subtrahend should be connected to the xor gate. Other input to the xor being one.The input carry bit to the full adder should be made 1. Then the full adder works like a full subtractor

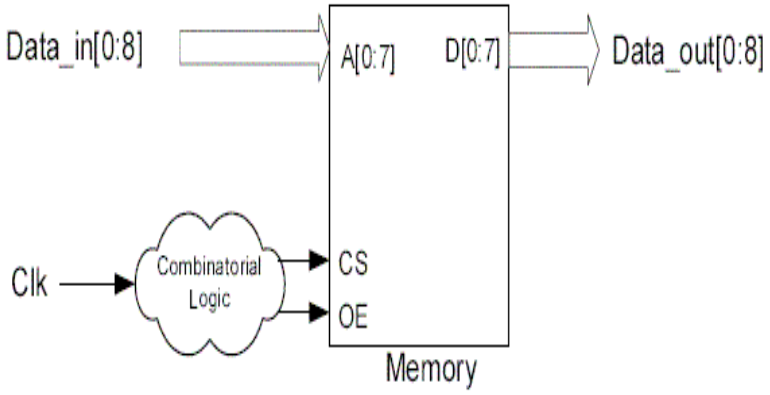
39)A very good interview question... What is difference between setup and hold time. The interviewer was looking for one specific reason , and its really a good answer too..The hint is hold time doesn't depend on clock, why is it so...?

Setup violations are related to two edges of clock, i mean you can vary the clock frequency to correct setup violation. But for hold time, you are only concerned with one edge and does not basically depend on clock frequency.

40)In a 3-bit Johnson's counter what are the unused states?

2(power n)-2n is the one used to find the unused states in johnson counter.
So for a 3-bit counter it is 8-6=2.Unused states=2. the two unused states are 010 and 101

41)The question is to design minimal hardware system, which encrypts 8-bit parallel data. A synchronized clock is provided to this system as well. The output encrypted data should be at the same rate as the input data but no necessarily with the same phase?



The encryption system is centered around a memory device that perform a LUT (Look-Up Table) conversion. This memory functionality can be achieved by using a PROM, EPROM, FLASH and etc. The device contains an encryption code, which may be burned into the device with an external programmer. In encryption operation, the data_in is an address pointer into a memory cell and the combinatorial logic generates the control signals. This creates a read access from the memory. Then the memory device goes to the appropriate address and outputs the

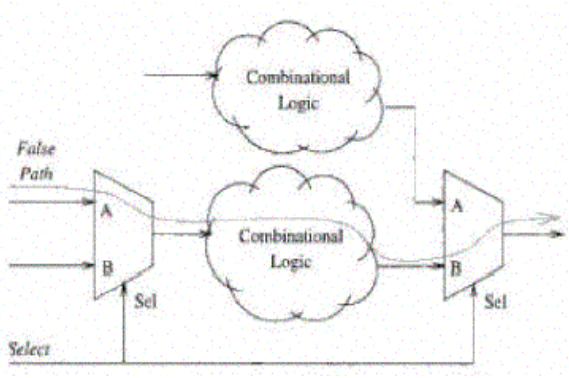
associate data. This data represent the data_in after encryption.

41) What is an LFSR .List a few of its industry applications.?

LFSR is a linear feedback shift register where the input bit is driven by a linear function of the overall shift register value. coming to industrial applications, as far as I know, it is used for encryption and decryption and in BIST(built-in-self-test) based applications..

42)what is false path?how it determine in ckt? what the effect of false path in ckt?

By timing all the paths in the circuit the timing analyzer can determine all the critical paths in the circuit. However, the circuit may have false paths, which are the paths in the circuit which are never exercised during normal circuit operation for any set of inputs.
An example of a false path is shown in figure below. The path going from the input A of the first MUX through the combinational logic out through the B input of the second MUS is a false path. This path can never be activated since if the A input of the first MUX is activated, then Sel line will also select the A input of the second MUX.
STA (Static Timing Analysis) tools are able to identify simple false paths; however they are not able to identify all the false paths and sometimes report false paths as critical paths. Removal of false paths makes circuit testable and its timing performance predictable (sometimes faster)



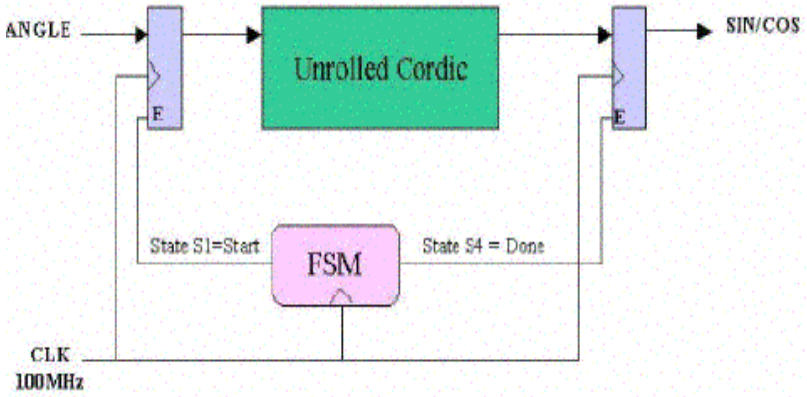
43)Consider two similar processors, one with a clock skew of 100ps and other with a clock skew of 50ps. Which one is likely to have more power? Why?

Clock skew of 50ps is more likely to have clock power. This is because it is likely that low-skew processor has better designed clock tree with more powerful and number of buffers and overheads to make skew better.

44)What are multi-cycle paths?

Multi-cycle paths are paths between registers that take more than one clock cycle to become stable.
For ex. Analyzing the design shown in fig below shows that the output SIN/COS requires 4 clock-cycles after the input ANGLE is latched in. This means that the combinatorial block (the Unrolled Cordic) can take up to 4 clock periods (25MHz) to propagate its result. Place and Route

tools are capable of fixing multi-cycle paths problem.



45)You have two counters counting upto 16, built from negedge DFF , First circuit is synchronous and second is "ripple" (cascading), Which circuit has a less propagation delay? Why?

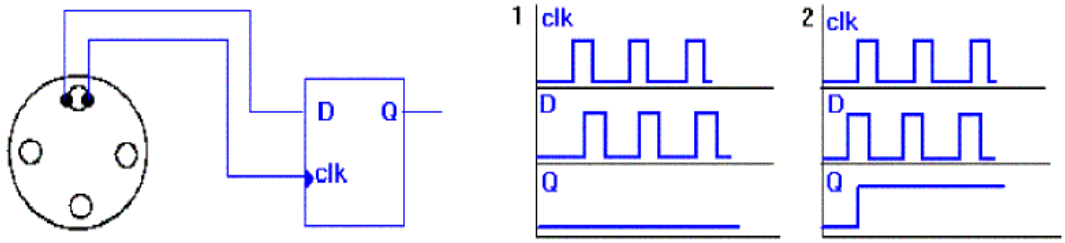
The synchronous counter will have lesser delay as the input to each flop is readily available before the clock edge. Whereas the cascade counter will take long time as the output of one flop is used as clock to the other. So the delay will be propagating. For Eg: 16 state counter = 4 bit counter = 4 Flip flops Let 10ns be the delay of each flop The worst case delay of ripple counter = 10 * 4 = 40ns The delay of synchronous counter = 10ns only.(Delay of 1 flop)

46) what is difference between RAM and FIFO?

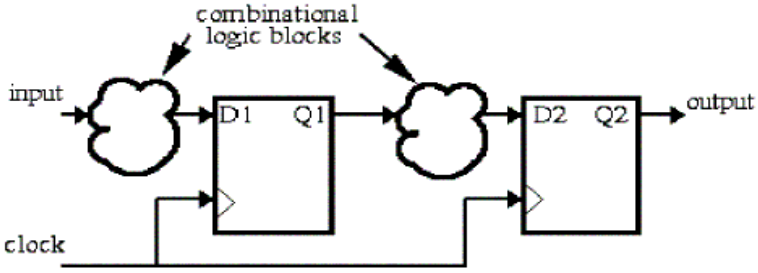
FIFO does not have address lines
Ram is used for storage purpose where as fifo is used for synchronization purpose i.e. when two peripherals are working in different clock domains then we will go for fifo.

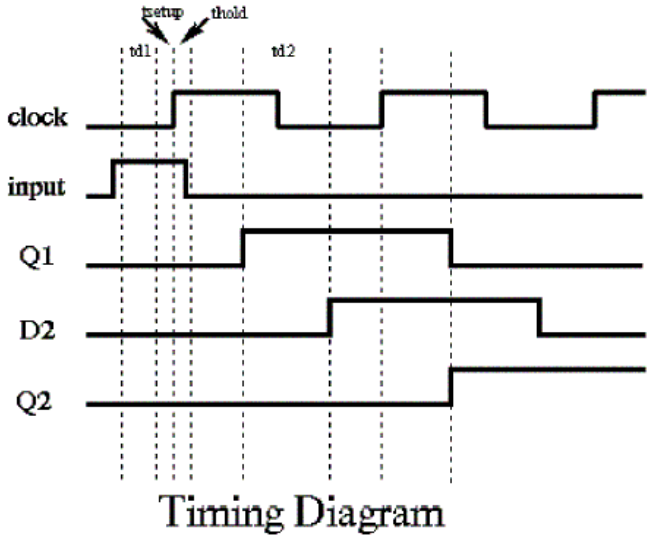
47)The circle can rotate clockwise and back. Use minimum hardware to build a circuit to indicate the direction of rotating.?

2 sensors are required to find out the direction of rotating. They are placed like at the drawing. One of them is connected to the data input of D flip-flop,and a second one - to the clock input. If the circle rotates the way clock sensor sees the light first while D input (second sensor) is zero - the output of the flip-flop equals zero, and if D input sensor "fires" first - the output of the flip-flop becomes high.



48) Draw timing diagrams for following circuit.?





Timing Diagram

49)Implement the following circuits:
(a) 3 input NAND gate using min no of 2 input NAND Gates
(b) 3 input NOR gate using min no of 2 inpur NOR Gates
(c) 3 input XNOR gate using min no of 2 inpur XNOR Gates
Assuming 3 inputs A,B,C?

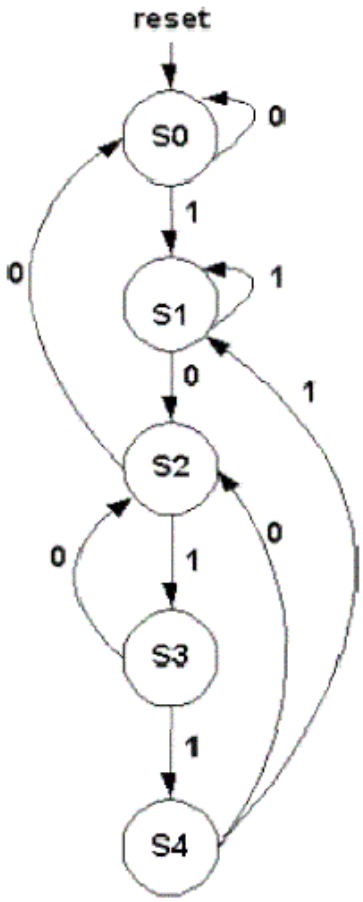
3 input NAND:
Connect :
a) A and B to the first NAND gate
b) Output of first Nand gate is given to the two inputs of the second NAND gate (this basically realizes the inverter functionality)
c) Output of second NAND gate is given to the input of the third NAND gate, whose other input is C
((A NAND B) NAND (A NAND B)) NAND C Thus, can be implemented using '3' 2-input NAND gates. I guess this is the minimum number of gates that need to be used.
3 input NOR:

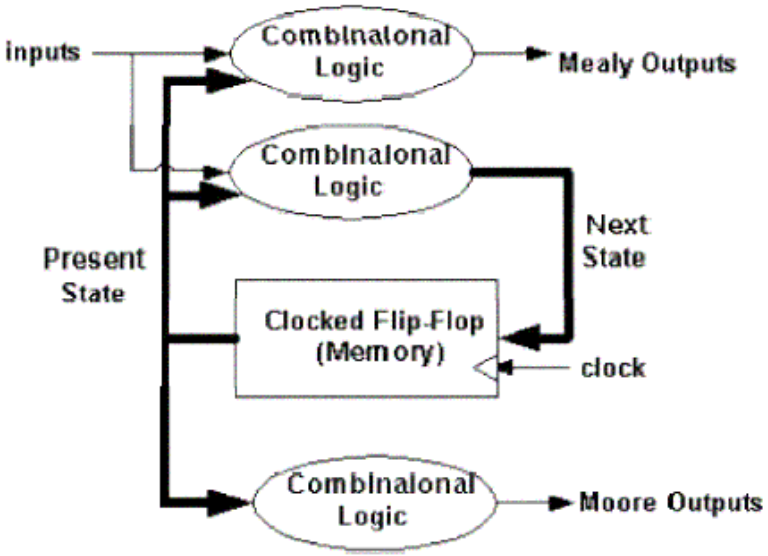
Same as above just interchange NAND with NOR ((A NOR B) NOR (A NOR B)) NOR C
3 input XNOR:
Same as above except the inputs for the second XNOR gate, Output of the first XNOR gate is one of the inputs and connect the second input to ground or logical '0'
((A XNOR B) XNOR 0)) XNOR C

50) Is it possible to reduce clock skew to zero? Explain your answer ?

Even though there are clock layout strategies (H-tree) that can in theory reduce clock skew to zero by having the same path length from each flip-flop from the pll, process variations in R and C across the chip will cause clock skew as well as a pure H-Tree scheme is not practical (consumes too much area).

51)Design a FSM (Finite State Machine) to detect a sequence 10110?



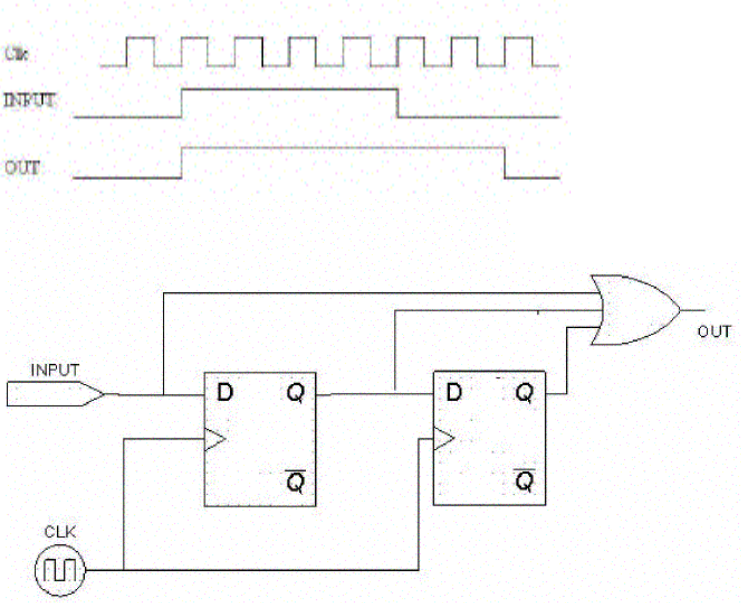


52)Convert D-FF into divide by 2. (not latch)? What is the max clock frequency of the circuit , given the following information?
T_setup= 6nS
T_hold = 2nS

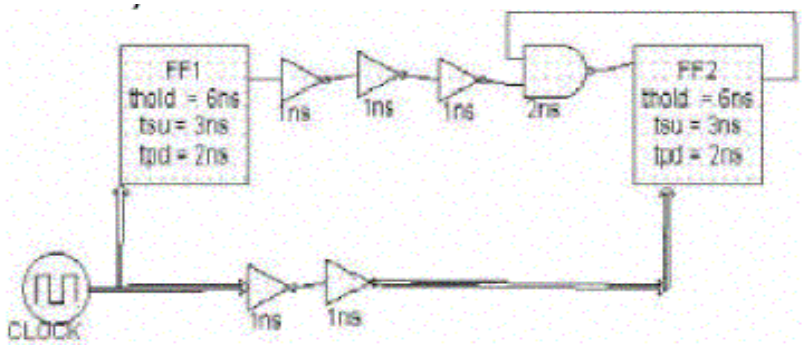
T_propagation = 10nS

Circuit:
Connect Qbar to D and apply the clk at clk of DFF and take the O/P at Q. It gives freq/2.
Max. Freq of operation:
1/ (propagation delay+setup time) = 1/16ns = 62.5 MHz

53)Give the circuit to extend the falling edge of the input by 2 clock pulses?The waveforms are shown in the following figure.

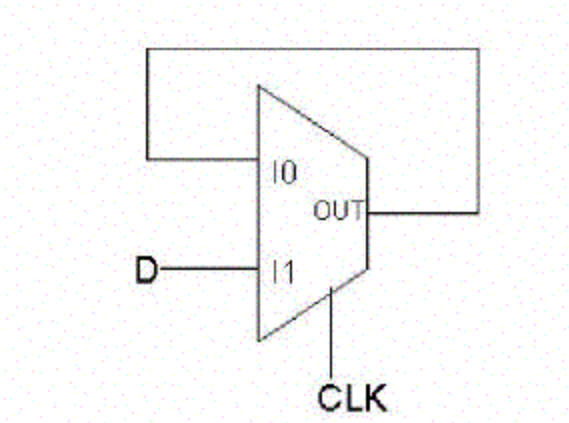


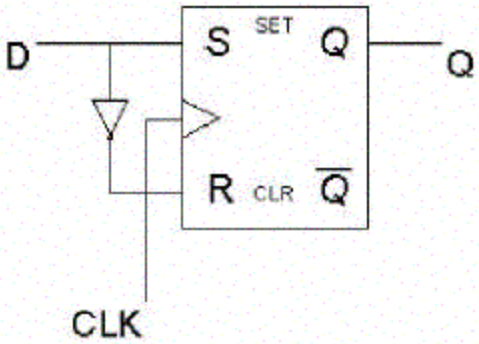
54) For the Circuit Shown below, What is the Maximum Frequency of Operation?Are there any hold time violations for FF2? If yes, how do you modify the circuit to avoid them?



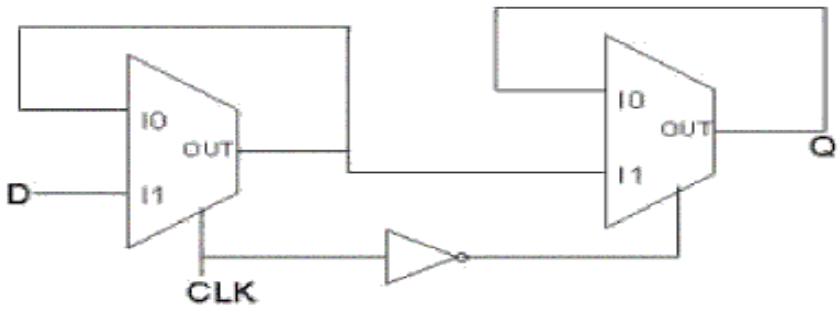
The minumum time period = 3+2+(1+1+1) = 8ns Maximum Frequency = 1/8n= 125MHz.
And there is a hold time violation in the circuit,because of feedback, if you observe, tcq2+AND gate delay is less than thold2,To avoid this we need to use even number of inverters(buffers). Here we need to use 2 inverters each with a delay of 1ns. then the hold time value exactly meets.

55)Design a D-latch using (a) using 2:1 Mux (b) from S-R Latch ?





56)How to implement a Master Slave flip flop using a 2 to 1 mux?

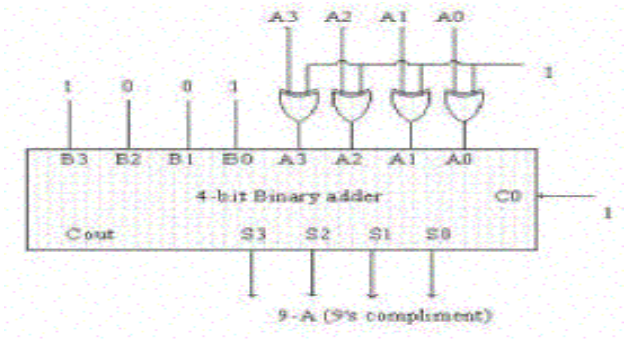


57)how many 2 input xor's are needed to inplement 16 input parity generator ?

It is always n-1 Where n is number of inputs.So 16 input parity generator will require 15 two input xor's .

58)Design a circuit for finding the 9's compliment of a BCD number using 4-bit binary adder and some external logic gates?

9's compliment is nothing but subracting the given no from 9.So using a 4 bit binary adder we can just subract the given binary no from 1001(i.e. 9).Here we can use the 2's compliment method addition.



59) what is Difference between writeback and write through cache?

A caching method in which modifications to data in the cache aren't copied to the cache source until absolutely necessary. Write-back caching is available on many microprocessors , including all Intel processors since the 80486. With these microprocessors, data modifications to data stored in the L1 cache aren't copied to main memory until absolutely necessary. In contrast, a write-through cache performs all write operations in parallel -- data is written to main memory and the L1 cache simultaneously. Write-back caching yields somewhat better performance than write-through caching because it reduces the number of write operations to main memory. With this performance improvement comes a slight risk that data may be lost if the system crashes. A write-back cache is also called a copy-back cache.

60)Difference between Synchronous,Asynchronous & Isynchronous communication?

Sending data encoded into your signal requires that the sender and receiver are both using the same enconding/decoding method, and know where to look in the signal to find data. Asynchronous systems do not send separate information to indicate the encoding or clocking information. The receiver must decide the clocking of the signal on it's own. This means that the receiver must decide where to look in the signal stream to find ones and zeroes, and decide for itself where each individual bit stops and starts. This information is not in the data in the signal sent from transmitting unit.

Synchronous systems negotiate the connection at the data-link level before communication begins. Basic synchronous systems will synchronize two clocks before transmission, and reset their numeric counters for errors etc. More advanced systems may negotiate things like error correction and compression.

Time-dependent. it refers to processes where data must be delivered within certain time constraints. For example, Multimedia stream require an isochronous transport mechanism to

ensure that data is delivered as fast as it is displayed and to ensure that the audio is synchronized with the video.

61) What are different ways Multiply & Divide?

Binary Division by Repeated Subtraction

- Set quotient to zero
- Repeat while dividend is greater than or equal to divisor
    - Subtract divisor from dividend
    - Add 1 to quotient
- End of repeat block
- quotient is correct, dividend is remainder
- STOP

Binary Division by Shift and Subtract

Basically the reverse of the mutliply by shift and add.

- **Set** quotient to 0
- Align leftmost digits in dividend and divisor
- **Repeat**
    - **If** that portion of the dividend above the divisor is greater than or equal to the divisor
        - **Then** subtract divisor from that portion of the dividend and
        - Concatentate 1 to the right hand end of the quotient
        - **Else** concatentate 0 to the right hand end of

the quotient

- Shift the divisor one place right

- **Until** dividend is less than the divisor
- quotient is correct, dividend is remainder

- **STOP**

Binary Multiply - Repeated Shift and Add

Repeated shift and add - starting with a result of 0, shift the second multiplicand to correspond with each 1 in the first multiplicand and add to the result. Shifting each position left is equivalent to multiplying by 2, just as in decimal representation a shift left is equivalent to multiplying by 10.

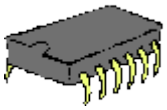
- Set result to 0

- Repeat

    - Shift 2nd multiplicand left until rightmost digit is lined up with leftmost 1 in first multiplicand
    - Add 2nd multiplicand in that position to result
    - Remove that 1 from 1st multiplicand

- Until 1st multiplicand is zero

- Result is correct
- STOP

TIMING Interview Questions 

Timing, an important parameter associated with Sequential Circuit design will be discussed in this tutorial. We will begin with the general concepts associated with timing and then will proceed with examples to better understand their application to digital design. This tutorial consists of three sections.

PART 1 Introduction and terminology

PART 2 Equations

PART 3 Example problems

PART 1: Introduction and terminology

==========

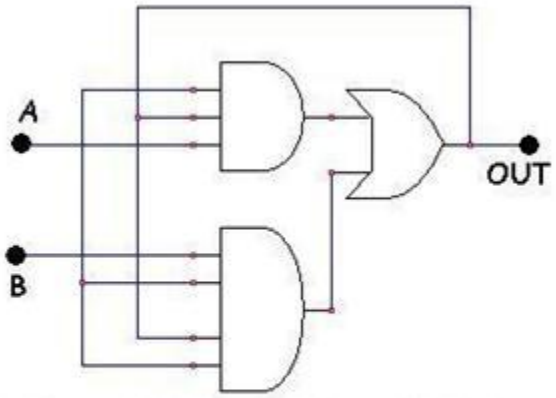
A Digital System Design circuit can be characterized as a 'Combinational circuit' or a 'Sequential Circuit' and while calculating for Timing we will have to first identify what type of circuit is involved.

***Q1**.How do we know, if given a circuit, whether it is a Combinational Circuit or a Sequential Circuit?*

***[Ans]** If a circuit has only combinational devices (e.g.. gates like AND, OR etc and MUX(s))and no Memory elements then it is a Combinational circuit. If the circuit has memory elements such as Flip Flops, Registers, Counters, or other state devices then it is a Sequential Circuit. Synchronous sequential circuits will also have a clearly labeled clock input.*

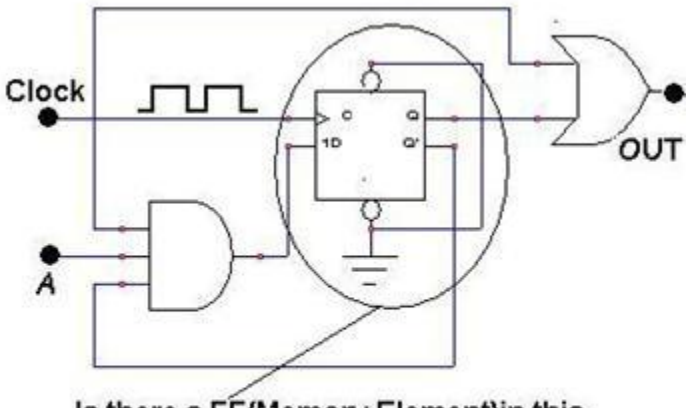
Q2. *Are the following circuits combinational or sequential?*

[Ans]



Is there a FF(Memory Element)in this Circuit ?

No!! so this is purely 'Combinational' Circuit



Is there a FF(Memory Element)in this Circuit ?

Yes!! So this is a 'Sequential' Circuit

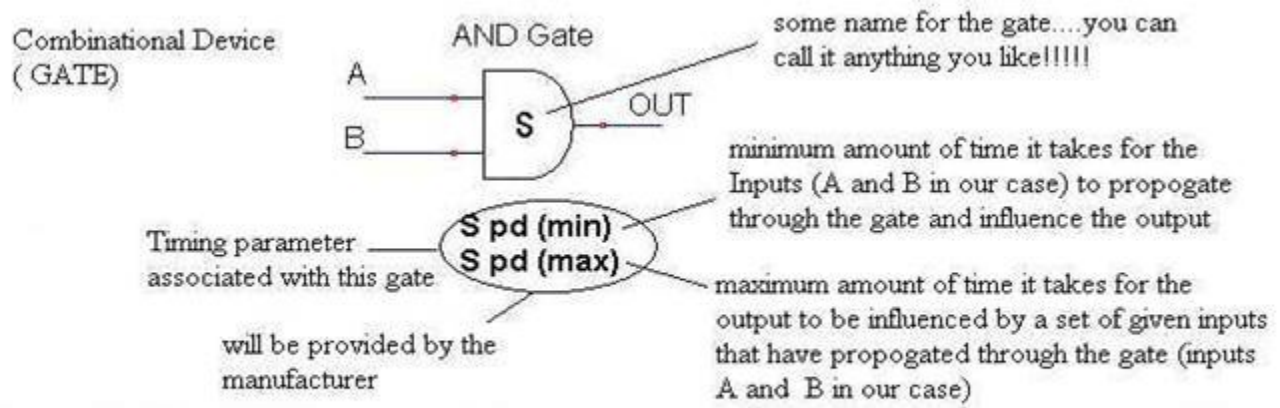
Q3. *Why do we have to identify the type of circuit? Does it really matter?*

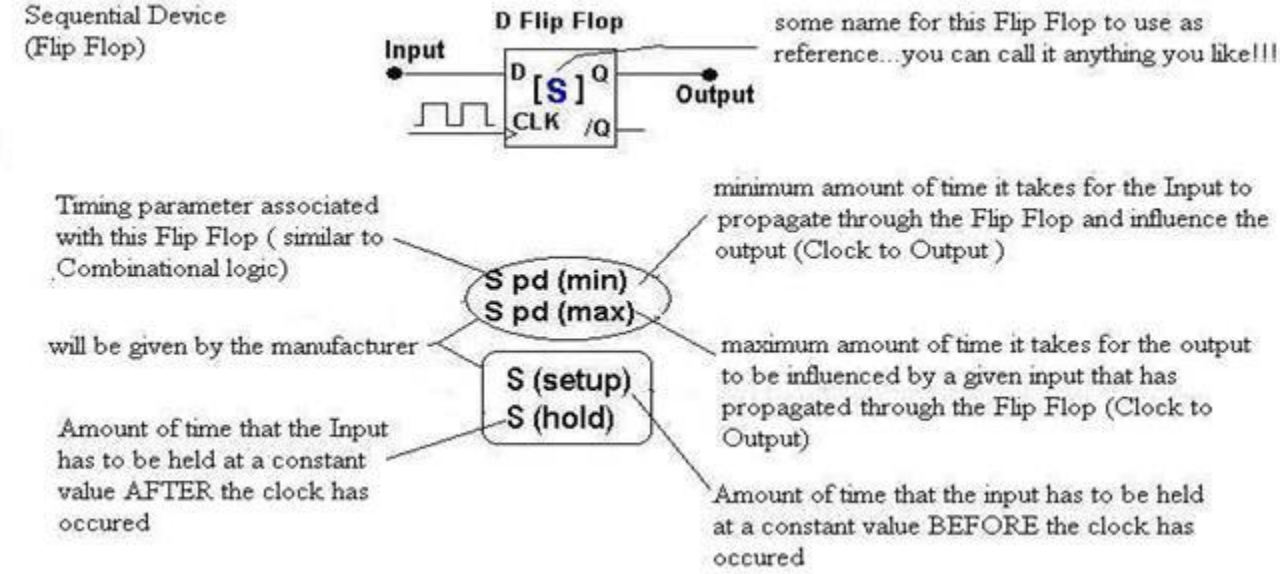
[Ans] *It is important to identify the type of circuit because our timing calculation approach differs accordingly. Combinational circuits timing analysis deals primarily with propagation delay issues. Sequential circuits have additional specific timing characteristics that must be satisfied in order to prevent metastability, including setup time, hold time, and minimum clock period. Designers of sequential devices must specify these important timing characteristics in order to allow the device to be used without error.*

Q4. *Do all Digital Devices like gates and Flip Flops have timing parameters?*

[Ans] *Yes, all digital devices have timing parameters. In the real environment (not Ideal as in our lab) there will be a real (non zero) value associated with every digital device. Observe the examples below*

Example 1 and 2:





Q5*.Phew!!! So many things all at the same time.....what is propagation delay?*

[Ans] *All devices have some delay associated with transferring an input change to the output. These changes are not immediate in a real environment. This delay that is due to the signal propagation through the device is called the propagation delay.*

Q6. *What is Setup time?*

[Ans] *Setup time is a timing parameter associated with Sequential Devices (for simplicity henceforth I will be only referring to the Flip Flop). The Setup time is used to meet the minimum pulse width requirement for the first (Master) latch makes up a flip flop is. More simply, the setup time is the amount of time that an input signal (to the device) must be stable (unchanging) before the clock ticks in order to guarantee minimum pulse width and thus avoid possible metastability.*

Q7. *What is Hold time?*

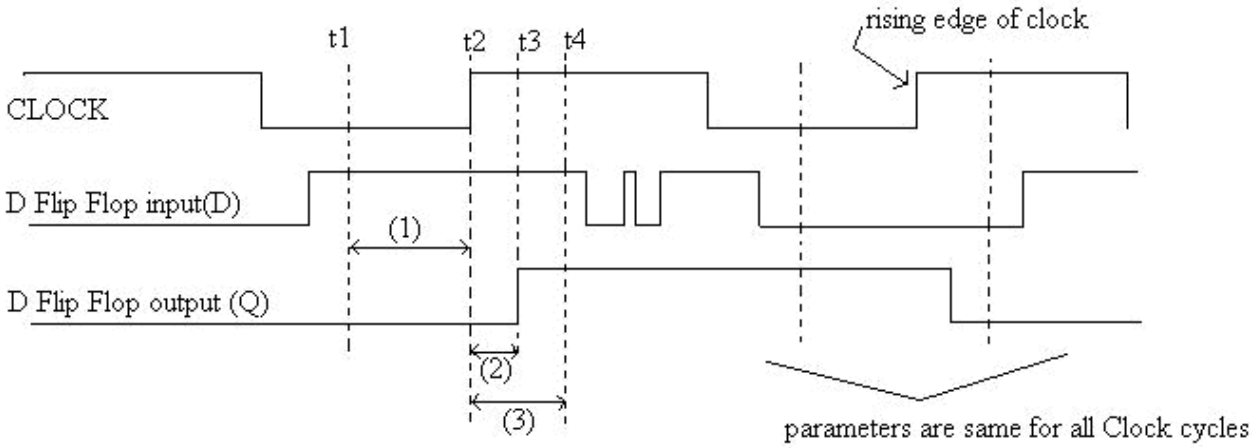
[Ans] *Hold time is also a timing parameter associated with Flip Flops and all other sequential devices. The Hold time is used to further satisfy the minimum pulse width requirement for the first (Master) latch that makes up a flip flop. The input must not change until enough time has passed after the clock tick to guarantee the master latch is fully disabled. More simply, hold time is the amount of time that an input signal (to a sequential device) must be stable (unchanging) after the clock tick in order to guarantee minimum pulse width and thus avoid possible metastability.*

Q8. *Can you give an example that can help me better understand the Setup and Hold time concept?*

[Ans] *Lets consider the situation where-in I am the Flip Flop and I am to receive an Input (a photo of an old friend whom I have to recognize) now the amount of time it would take to setup the photo in the right position so that it is visible to me from where I am sitting (since I am lazy to walk over) can be considered as the "Setup time". Now once shown the photo the amount of time that I keep staring at it till I feel comfortable enough to start relating it to known faces can be considered as the "Hold time".*

Q9*. What is a timing diagram? Can we use it to better understand Setup and Hold time?*

[Ans] *Timing diagram is a complete description of a digital machine. We can use the timing diagram (waveform) to illustrate Setup and Hold time. Observe the waveform given below:*



From the timing diagram we observe that we have three signals: the Clock, the Flip Flop Input (D) and the Flip Flop output (Q). We have four timing instances and three time periods. The inferences from this waveform will help us understand the concept of propagation delay Setup and Hold time.

(1) i.e. [t2 - t1] is the Setup Time: the minimum amount of time Input must be held constant BEFORE the clock tick. Note that D is actually held constant for somewhat longer than the minimum amount. The extra "constant" time is sometimes called the setup margin.

(2) i.e. [t3 - t2] is the Propagation delay of the Flip Flop: the minimum/maximum time for the input to propagate and influence the output.

(3) i.e. [t4 - t2] is the Hold time: the minimum amount of time the Input is held constant AFTER the clock tick. Note that Q is actually held constant for somewhat longer than the minimum amount. The extra "constant" time is sometimes called the hold margin.

(The above timing diagram has 2 clock cycles; the timing parameters for the second cycle will also be similar to that of the first cycle)

PART 2: Equations

==========

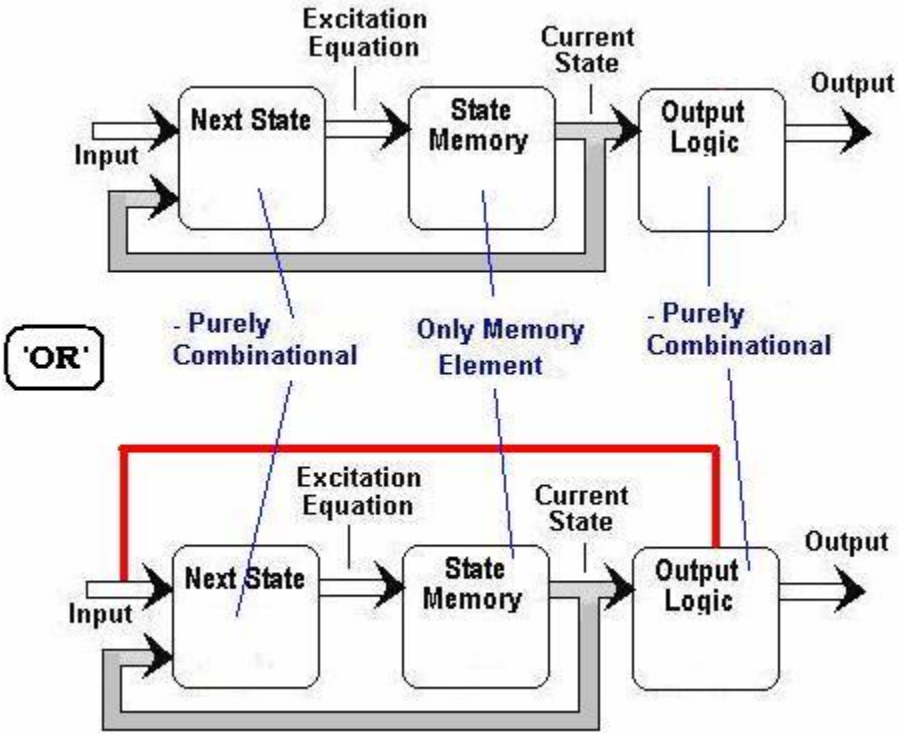
This part of the tutorial introduces us to the various different timing calculations associated with this course. We may be given a sequential circuit and asked to solve for the timing parameters. Let us discuss in detail how we should approach such problems.

Q11. *What is the first thing to do if given a sequential circuit and asked to analyze its timing?*

[Ans] *Given a sequential circuit it is often advisable to first divide the circuit in to three distinct parts i.e. Input Logic, State Memory and the Output Logic. Such division will also help with identifying whether the given circuit is Mealy or Moore. The input logic (Next State Logic) and the output logic blocks constitute of only combinational logic components like gates, muxes etc. The state memory block is made of <u>only</u> sequential components like Flip Flops etc.*

Q12. *Can you explain the answer to Q11 more elaborately?*

[Ans] *Let me explain using block diagrams. A given sequential circuit can be represented in either of the two ways as shown below.*

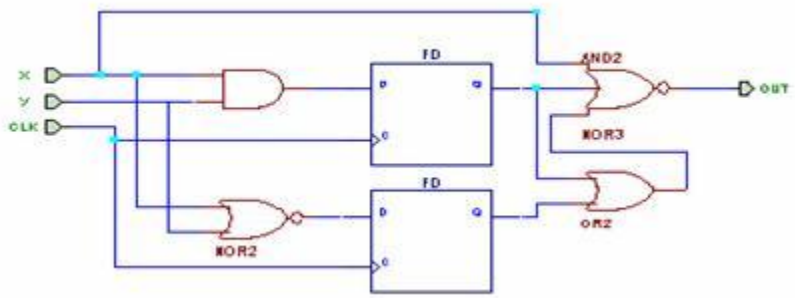


The first representation shows the sequential circuit where the input(s) have to pass through the State memory to affect the output. Such machines are called Moore machines.

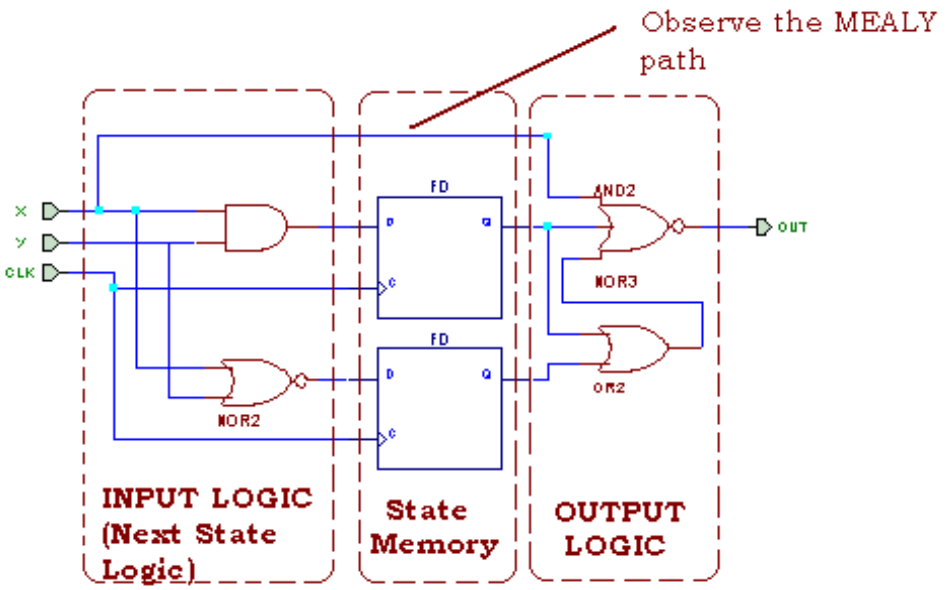
The second representation shows the 'red bypass' which signifies that the output can be directly affected by the inputs without having to pass through the state memory device(s). Such devices are called Mealy machines.

Q13. *Can you explain this with an example?*

[Ans] *Ok, consider the sequential circuit shown below*



Let us now identify the three distinct parts in this given sequential circuit. Observe the division on the circuit below.



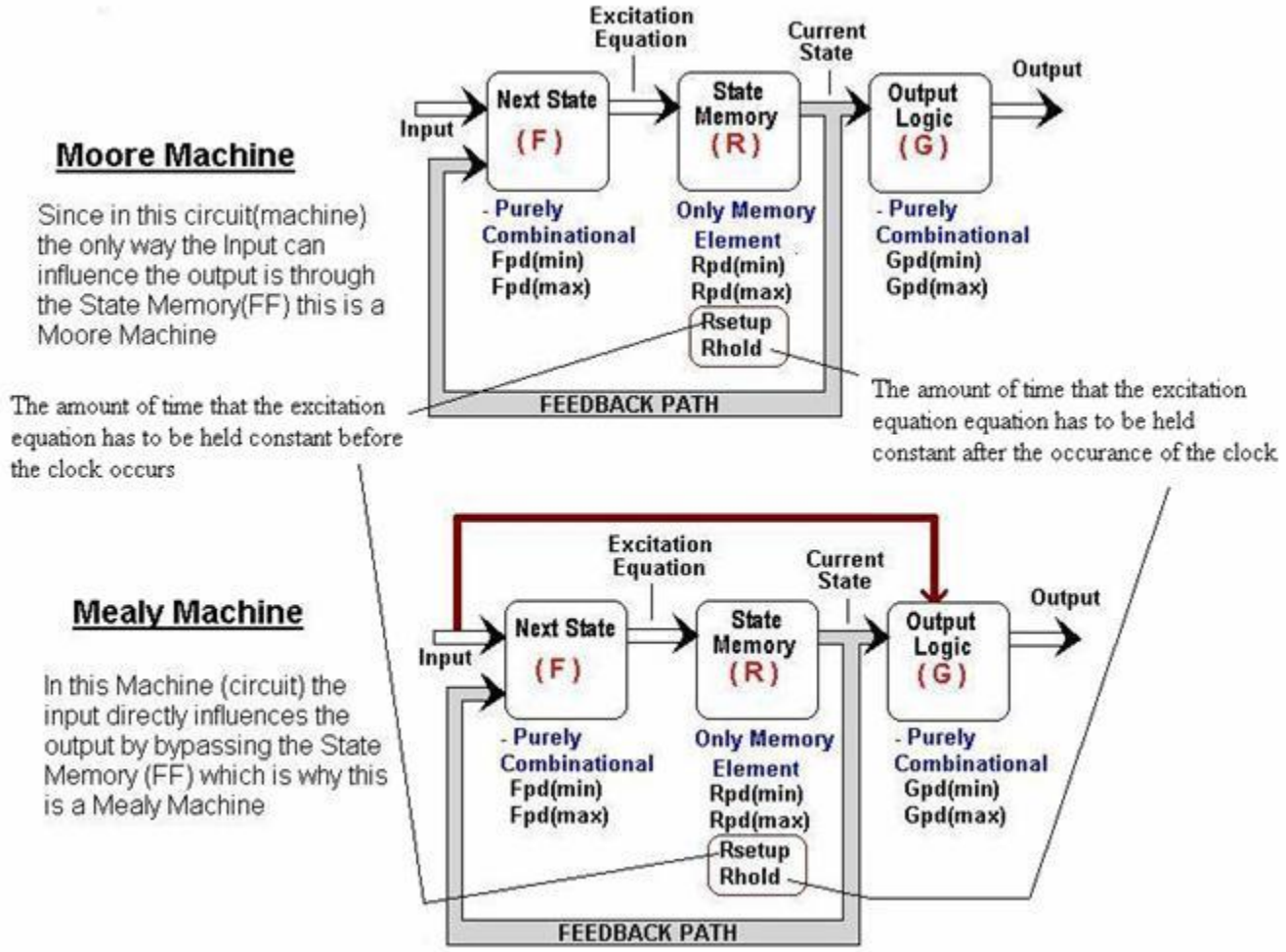
.

Observation: This given circuit is a MEALY machine.

Q14. *Now that we have divided the circuit into more distinct parts how do we proceed with calculating the timing parameters?*

[Ans] *Remember from our discussion in Part 1 of this tutorial we know that combinational devices and sequential devices have different timing parameters. Now that we have separated them both into separate blocks we can define them more clearly. To relate them to the blocks let us follow some convention (already discussed in part 1). Let us refer to the timing parameters for the input logic (also referred to as the next state logic) and output logic with the letter 'F' and*

'G' respectively. Similarly, let us refer to all timing parameters associated with the State memory block with the letter 'R'.



Q15. *What timing parameters are commonly used?*

[Ans] *The list of the timing parameters that you may be asked to calculate for a given sequential circuit is*

1. *Propagation delay, Clock to Output (minimum)*
2. *Propagation delay, Clock to Output (maximum)*
3. *Propagation delay, Input to Output (minimum)*
4. *Propagation delay, Input to Output (maximum)*

5. *Setup Time (Data input before clock)*
6. *Hold Time (Data input after clock)*
7. *Maximum Clock rate (or its reciprocal, minimum clock period)*

Q16. *How do we find the Propagation delay, Clock to Output?*

[Ans] *Propagation delay (PD) for the circuit can be calculated as the summation of all delays encountered from where the clock occurs to the output. In short, the delays of the State memory and the output logic.*

$$PD_{Clock\text{-}Output}(min) = R_{pd\ (min)} + G_{pd\ (min)}$$

$$PD_{Clock\text{-}Output}(max) = R_{pd\ (max)} + G_{pd\ (max)}$$

Q17. *How do we find the Propagation delay, Input to Output?*

[Ans] *This is a property associated with Mealy machines only. In other words, for a Moore machine the value for this timing parameter is infinity (∞). The calculation (for mealy machines) is the summation of all propagation delays encountered between the input (that influences the output by bypassing the state memory) and the output.*

For MOORE machines:

$$PD_{Input\text{-}Output}(min) = infinity\ (\infty)$$

$$PD_{Input\text{-}Output}(max) = infinity\ (\infty)$$

For MEALY Machines

$$PD_{Input\text{-}Output}(min) = G_{pd\ (min)}$$

$$PD_{Input\text{-}Output}(max) = G_{pd\ (max)}$$

Q18. *How do we calculate Setup time?*

[Ans] *The calculation for setup time is the sum of the setup time for the concerned flip flop and the maximum delay from the input logic.*

$$T_{SETUP} = R_{SETUP} + F_{pd\ (MAX)}$$

Q19. *How do we get the value for the Hold time?*

[Ans] *The value for the Hold time can be obtained by the following formulae*

$$T_{HOLD} = R_{HOLD} - F_{pd\ (MIN)}$$

The concern here is how soon (minimum time) an erroneous input can propagate in from the Input logic while the Flip Flop is attempting to hold on to a stable value. The negative sign can be associated with 'after the clock occurs' to ease in remembering this formulae.

Q20. *How do we calculate the Maximum Clock rate (MCLK)?*

[Ans] *Maximum clock rate is calculated using the formula*

$$MCLK = 1/ T_{MIN}$$

So we will have to calculate T_{MIN} first. T_{MIN} here refers to the minimum time period for correct operation of the circuit, so it is calculated using all worst cases (maximum delays).

$$T_{MIN} = F_{pd\ (MAX)} + R_{SETUP} + R_{pd\ (MAX)}$$

So having found the minimum clock period let us now calculate for the MCLK

$$MCLK = 1/ T_{MIN} = (Fpd_{\ (MAX)} + R_{SETUP} + R_{pd\ (MAX)})^{-1}$$

Q21. *Please summarize.*

[Ans] *Ok, here is everything we discussed so far in Part 2*

1. $PD_{Clock\text{-}Output}(min) = R_{pd\ (min)} + G_{pd\ (min)}$
2. $PD_{Clock\text{-}Output}(max) = R_{pd\ (max)} + G_{pd\ (max)}$
3. $PD_{Input\text{-}Output}(min) = infinity\ (\infty)$ *(For MOORE machines)*
4. $PD_{Input\text{-}Output}(max) = infinity\ (\infty)$ *(For MOORE machines)*
5. $PD_{Input\text{-}Output}(min) = G_{pd\ (min)}$ *(For MEALY machines)*
6. $PD_{Input\text{-}Output}(max) = G_{pd\ (max)}$ *(For MEALY machines)*
7. $T_{SETUP} = R_{SETUP} + F_{pd\ (MAX)}$
8. $T_{HOLD} = R_{HOLD} - F_{pd\ (MIN)}$
9. $MCLK = 1/T_{MIN} = (Fpd_{(MAX)} + R_{SETUP} + R_{pd\ (MAX)})^{-1}$

PART 3: Examples

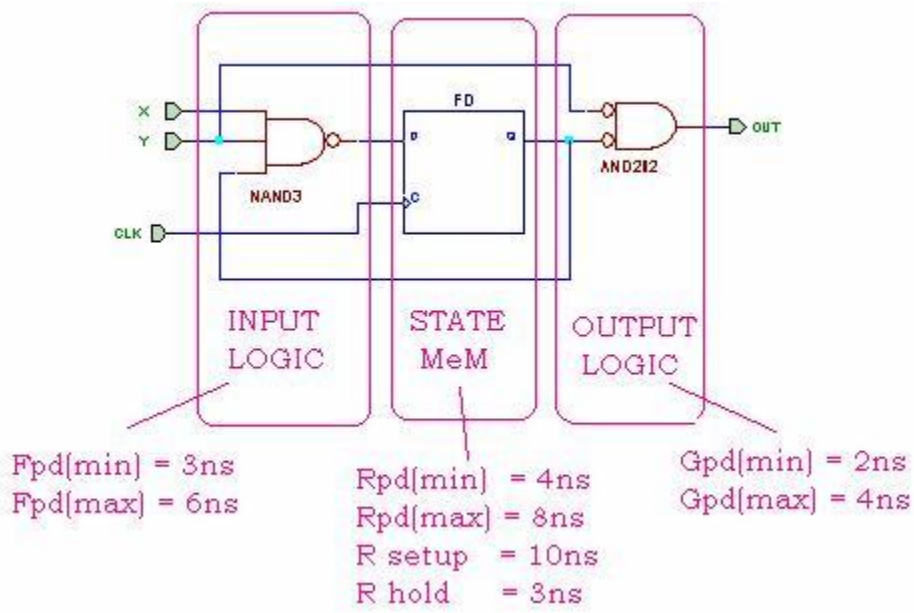
==========

Q23. *Can we go through a timing example (solved problem) so that we can have a better understanding of the concepts dealt so far?*

[Ans] *Sure, here is a simple example to begin with, you are given a sequential circuit as shown below and asked to calculate all the timing parameters discussed in Part 2 of this tutorial. The information provided to you with the question is the individual timing parameters of the components listed in the table below.*

Device	Propagation Delay (Minimum)	Propagation Delay (Maximum)	Setup Time	Hold Time
D Flip Flop	4 ns	8 ns	10 ns	3 ns
NAND Gate	3 ns	6 ns	X	X
Bubbled AND Gate	2 ns	4 ns	X	X

With this information we can approach the problem as discussed in Part 2 of this tutorial i.e. we shall first divide the given circuit into three distinct parts and then solve for timing. With practice, we can afford to skip this step of dividing the circuit into distinct parts (thereby saving time) and directly solve for timing. Since this is the first example I shall religiously follow the steps discussed in Part 2.



Observation: This is a MEALY Machine.

Now let us calculate for all the timing parameters.

1. $PD_{Clock\text{-}Output}(min) = R_{pd\ (min)} + G_{pd\ (min)}$ *= 4ns + 2ns = 6ns*
2. $PD_{Clock\text{-}Output}(max) = R_{pd\ (max)} + G_{pd\ (max)}$ *= 8ns + 4ns = 12ns*
3. $PD_{Input\text{-}Output}(min) = G_{pd\ (min)}$ *= 2ns*
4. $PD_{Input\text{-}Output}(max) = G_{pd\ (max)}$ *= 4ns*

5. $T_{SETUP} = R_{SETUP} + F_{pd\ (MAX)}$ *= 10ns + 6ns = 16ns*

6. $T_{HOLD} = R_{HOLD} - F_{pd\ (MIN)}$ *= 3ns – 3ns = 0ns.*

7. $T_{MIN} = Fpd_{(MAX)} + R_{SETUP} + R_{pd\ (MAX)}$ *= 6ns +10ns + 8ns = 24ns*

8. $MCLK = 1/T_{MIN} = (Fpd_{(MAX)} + R_{SETUP} + R_{pd\ (MAX)})^{-1}$ *= 1/24ns.*

***Q24**. Can we go through another timing example (solved problem) using more than one Flip Flop?*

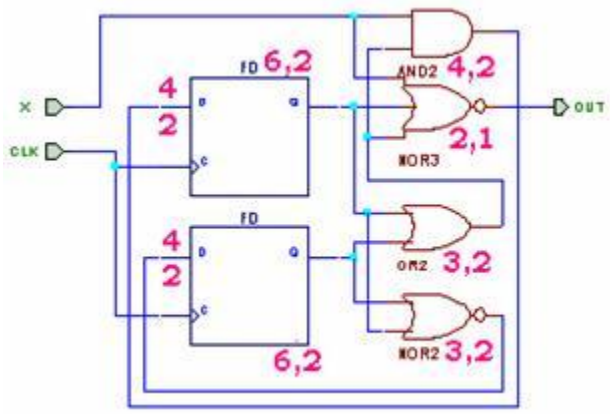
***[Ans]** Ok, here is an example (notice how I write down the corresponding timing values for simplicity in understanding)*

Given with the question is the individual timing parameter for all the components used in the Circuit. Observe the table given below.

Device	Propagation Delay (minimum)	Propagation Delay (maximum)	Setup Time	Hold Time
D Flip Flop	2ns	6ns	4ns	2ns
AND Gate	2ns	4ns	X	X
2 i/p NOR Gate	2ns	3ns	X	X
OR Gate	2ns	3ns	X	X
3 i/p NOR Gate	1ns	2ns	X	X

Writing the timing parameters next to the components (for ease in solving)

So with the timing parameters next to the components the circuit now looks like this



Dividing the circuit into distinct parts is left to the reader (will give the reader some hands-on practice)

Now let us calculate for all the timing parameters.

1. $PD_{Clock\text{-}Output}(min) = R_{pd\ (min)} + G_{pd\ (min)}$ = 2ns + 1ns = 3ns
2. $PD_{Clock\text{-}Output}(max) = R_{pd\ (max)} + G_{pd\ (max)}$ = 6ns + 3ns + 2ns = 11ns
3. $PD_{Input\text{-}Output}(min) = G_{pd\ (min)\ (For\ MEALY\ machines)}$ = 1ns
4. $PD_{Input\text{-}Output}(max) = G_{pd\ (max)\ (For\ MEALY\ machines)}$ = 2ns
5. $T_{SETUP} = R_{SETUP} + F_{pd\ (MAX)}$ = 4ns + 4ns = 8ns

6. $T_{HOLD} = R_{HOLD} - F_{pd\ (MIN)}$ *= 2ns – 2ns = 0ns.*

7. $T_{MIN} = Fpd_{(MAX)} + R_{SETUP} + R_{pd\ (MAX)}$ *= 3ns + 4ns + 4ns + 6ns = 17ns*

8. $MCLK = 1/T_{MIN} = (Fpd_{(MAX)} + R_{SETUP} + R_{pd\ (MAX)})^{-1}$ *= 1/17ns.*

Q25. *Are these two solved examples enough to introduce us to the timing concepts necessary for this course?*

[Ans] *Absolutely, the two examples together cover almost all the concepts necessary to get you started with understanding timing problems (the intent of this tutorial). More examples would result in spoon-feeding and would not be recommended. Interested students can now read the text and attempt to solve other timing related questions for practice.*

ASIC interview questions

What is Body effect ?

The threshold voltage of a MOSFET is affected by the voltage which is applied to the back contact. The voltage difference between the source and the bulk, VBS changes the width of the depletion layer and therefore also the voltage across the oxide due to the change of the charge in the depletion region. This results in a difference in threshold voltage which equals the difference in charge in the depletion region divided by the oxide capacitance, yielding:.

Click hear to view more

What are standard Cell's?

In semiconductor design, standard cell methodology is a method of designing Application Specific Integrated Circuits (ASICs) with mostly digital-logic features. Standard cell methodology is an example of design abstraction, whereby a low-level VLSI-layout is encapsulated into an abstract logic representation (such as a NAND gate). Cell-based methodology (the general class that standard-cell belongs to) makes it possible for one designer to focus on the high-level (logical function) aspect of digital-design, while another designer

focused on the implementation (physical) aspect. Along with semiconductor manufacturing advances, standard cell methodology was responsible for allowing designers to scale ASICs from comparatively simple single-function ICs (of several thousand gates), to complex multi-million gate devices (SoC).

Click hear to view more

What are Design Rule Check (DRC) and Layout Vs Schematic (LVS) ?

Design Rule Check (DRC) and Layout Vs Schematic (LVS) are verification processes. Reliable device fabrication at modern deep submicrometre (0.13 µm and below) requires strict observance of transistor spacing, metal layer thickness, and power density rules. DRC exhaustively compares the physical netlist against a set of "foundry design rules" (from the foundry operator), then flags any observed violations. LVS is a process that confirms that the layout has the same structure as the associated schematic; this is typically the final step in the layout process. The LVS tool takes as an input a schematic diagram and the extracted view from a layout. It then generates a netlist from each one and compares them. Nodes, ports, and device sizing are all compared. If they are the same, LVS passes and the designer can continue. Note: LVS tends to consider transistor fingers to be the same as an extra-wide transistor. For example, 4 transistors in parallel (each 1 um wide), a 4-finger 1 um transistor, and a 4 um transistor are all seen as the same by the LVS tool. Functionality of .lib files will be taken from spice models and added as an attribute to the .lib file.

What is Antenna effect ?

The antenna effect, more formally plasma induced gate oxide damage, is an efffect that can potentially cause yield and reliability problems during the manufacture of MOS integrated circuits. Fabs normally supply antenna rules, which are rules that must be obeyed to avoid this problem. A violation of such rules is called an antenna violation. The word antenna is somewhat of a misnomer in this context—the problem is really the collection of charge, not the normal meaning of antenna, which is a device for converting electromagnetic fields to/from electrical currents. Occasionally the phrase antenna effect is used this context[6] but this is less common since there are many effects[7] and the phrase does not make clear which is meant.

What are steps involved in Semiconductor device fabrication ?

This is a list of processing techniques that are employed numerous times in a modern electronic device and do not necessarily imply a specific order.

Wafer processing
Wet cleans
Photolithography
Ion implantation (in which dopants are embedded in the wafer creating regions of increased (or decreased) conductivity)
Dry etching
Wet etching
Plasma ashing
Thermal treatments
Rapid thermal anneal
Furnace anneals
Thermal oxidation
Chemical vapor deposition (CVD)
Physical vapor deposition (PVD)
Molecular beam epitaxy (MBE)
Electrochemical Deposition (ECD). See Electroplating
Chemical-mechanical planarization (CMP)
Wafer testing (where the electrical performance is verified)
Wafer backgrinding (to reduce the thickness of the wafer so the resulting chip can be put into a thin device like a smartcard or PCMCIA card.)
Die preparation
Wafer mounting
Die cutting
IC packaging
Die attachment
IC Bonding
Wire bonding
Flip chip
Tab bonding
IC encapsulation
Baking
Plating
Lasermarking
Trim and form
IC testing

What is Clock distribution network ?

In a synchronous digital system, the clock signal is used to define a time reference for the movement of data within that system. The clock distribution network distributes the clock signal(s) from a common point to all the elements that need it. Since this function is vital to the operation of a synchronous system, much attention has been given to the characteristics of these clock signals and the electrical networks used in their distribution. Clock signals are often regarded as simple control signals; however, these signals have some very special characteristics and attributes.
Clock signals are typically loaded with the greatest fanout, travel over the greatest distances, and operate at the highest speeds of any signal, either control or data, within the entire synchronous system. Since the data signals are provided with a temporal reference by the clock signals, the clock waveforms must be particularly clean and sharp. Furthermore, these clock signals are particularly affected by technology scaling (see Moore's law), in that long global interconnect lines become significantly more resistive as line dimensions are decreased. This increased line resistance is one of the primary reasons for the increasing significance of clock distribution on synchronous performance. Finally, the control of any differences and uncertainty in the arrival times of the clock signals can severely limit the maximum performance of the entire system and create catastrophic race conditions in which an incorrect data signal may latch within a register. The clock distribution network often takes a significant fraction of the power consumed by a chip. Furthermore, significant power can be wasted in transitions within blocks, even when their output is not needed. These observations have lead to a power saving technique called clock gating, which involves adding logic gates to the clock distribution tree, so portions of the tree can be turned off when not needed.

What is Clock Gating ?

Clock gating is one of the power-saving techniques used on many synchronous circuits including the Pentium 4 processor. To save power, clock gating refers to adding additional logic to a circuit to prune the clock tree, thus disabling portions of the circuitry where flip flops do not change state. Although asynchronous circuits by definition do not have a "clock", the term "perfect clock gating" is used to illustrate how various clock gating techniques are simply approximations of the data-dependent behavior exhibited by asynchronous circuitry, and that as the granularity on which you gate the clock of a synchronous circuit approaches zero, the power consumption of that circuit approaches that of an asynchronous circuit.

What is Netlist ?

Netlists are connectivity information and provide nothing more than instances, nets, and perhaps

some attributes. If they express much more than this, they are usually considered to be a hardware description language such as Verilog, VHDL, or any one of several specific languages designed for input to simulators.
Most netlists either contain or refer to descriptions of the parts or devices used. Each time a part is used in a netlist, this is called an "instance." Thus, each instance has a "master", or "definition". These definitions will usually list the connections that can be made to that kind of device, and some basic properties of that device. These connection points are called "ports" or "pins", among several other names.
An "instance" could be anything from a vacuum cleaner, microwave oven, or light bulb, to a resistor, capacitor, or integrated circuit chip.
Instances have "ports". In the case of a vacuum cleaner, these ports would be the three metal prongs in the plug. Each port has a name, and in continuing the vacuum cleaner example, they might be "Neutral", "Live" and "Ground". Usually, each instance will have a unique name, so that if you have two instances of vacuum cleaners, one might be "vac1" and the other "vac2". Besides their names, they might otherwise be identical.
Nets are the "wires" that connect things together in the circuit. There may or may not be any special attributes associated with the nets in a design, depending on the particular language the netlist is written in, and that language's features.
Instance based netlists usually provide a list of the instances used in a design. Along with each instance, either an ordered list of net names are provided, or a list of pairs provided, of an instance port name, along with the net name to which that port is connected. In this kind of description, the list of nets can be gathered from the connection lists, and there is no place to associate particular attributes with the nets themselves. SPICE is perhaps the most famous of instance-based netlists.
Net-based netlists usually describe all the instances and their attributes, then describe each net, and say which port they are connected on each instance. This allows for attributes to be associated with nets. EDIF is probably the most famous of the net-based netlists.

What Physical timing closure ?

Physical timing closure is the process by which an FPGA or a VLSI design with a physical representation is modified to meet its timing requirements. Most of the modifications are handled by EDA tools based on directives given by a designer. The term is also sometimes used as a characteristic, which is ascribed to an EDA tool, when it provides most of the features required in this process. Physical timing closure became more important with submicrometre technologies, as more and more steps of the design flow had to be made timing-aware. Previously only logic synthesis had to satisfy timing requirements. With present deep submicrometre technologies it is unthinkable to perform any of the design steps of placement, clock-tree synthesis and routing without timing constraints. Logic synthesis with these technologies is becoming less important. It is still required, as it provides the initial netlist of

gates for the placement step, but the timing requirements do not need to be strictly satisfied any more. When a physical representation of the circuit is available, the modifications required to achieve timing closure are carried out by using more accurate estimations of the delays.

What Physical verification ?

Physical verification of the design, involves DRC(Design rule check), LVS(Layout versus schematic) Check, XOR Checks, ERC (Electrical Rule Check) and Antenna Checks.
XOR Check

This step involves comparing two layout databases/GDS by XOR operation of the layout geometries. This check results a database which has all the mismatching geometries in both the layouts. This check is typically run after a metal spin, where in the re-spin database/GDS is compared with the previously taped out database/GDS.
Antenna Check

Antenna checks are used to limit the damage of the thin gate oxide during the manufacturing process due to charge accumulation on the interconnect layers (metal, polysilicon) during certain fabrication steps like Plasma etching, which creates highly ionized matter to etch. The antenna basically is a metal interconnect, i.e., a conductor like polysilicon or metal, that is not electrically connected to silicon or grounded, during the processing steps of the wafer. If the connection to silicon does not exist, charges may build up on the interconnect to the point at which rapid discharge does take place and permanent physical damage results to thin transistor gate oxide. This rapid and destructive phenomenon is known as the antenna effect. The Antenna ratio is defined as the ratio between the physical area of the conductors making up the antenna to the total gate oxide area to which the antenna is electrically connected.
ERC (Electrical rule check)

ERC (Electrical rule check) involves checking a design for all well and substrate areas for proper contacts and spacings thereby ensuring correct power and ground connections. ERC steps can also involve checks for unconnected inputs or shorted outputs.

What is Stuck-at fault ?

A Stuck-at fault is a particular fault model used by fault simulators and Automatic test pattern generation (ATPG) tools to mimic a manufacturing defect within an integrated circuit. Individual signals and pins are assumed to be stuck at Logical '1', '0' and 'X'. For example, an output is tied to a logical 1 state during test generation to assure that a manufacturing defect with that type of behavior can be found with a specific test pattern. Likewise the output could be tied to a logical 0 to model the behavior of a defective circuit that cannot switch its output pin.

What is Different Logic family ?

Listed here in rough chronological order of introduction along with their usual abbreviations of Logic family
* Diode logic (DL)
* Direct-coupled transistor logic (DCTL)
* Complementary transistor logic (CTL)
* Resistor-transistor logic (RTL)
* Resistor-capacitor transistor logic (RCTL)
* Diode-transistor logic (DTL)
* Emitter coupled logic (ECL) also known as Current-mode logic (CML)
* Transistor-transistor logic (TTL) and variants
* P-type Metal Oxide Semiconductor logic (PMOS)
* N-type Metal Oxide Semiconductor logic (NMOS)
* Complementary Metal-Oxide Semiconductor logic (CMOS)
* Bipolar Complementary Metal-Oxide Semiconductor logic (BiCMOS)
* Integrated Injection Logic (I2L)

What is Different Types of IC packaging ?

IC are packaged in many types they are: * BGA1
* BGA2
* Ball grid array
* CPGA
* Ceramic ball grid array
* Cerquad
* DIP-8
* Die attachment
* Dual Flat No Lead
* Dual in-line package
* Flat pack
* Flip chip
* Flip-chip pin grid array
* HVQFN
* LQFP
* Land grid array
* Leadless chip carrier
* Low insertion force
* Micro FCBGA

* Micro Leadframe Package
* MicroLeadFrame
* Mini-Cartridge
* Multi-Chip Module
* OPGA
* PQFP
* Package on package
* Pin grid array
* Plastic leaded chip carrier
* QFN
* QFP
* Quadruple in-line package
* ROM cartridge
* Shrink Small-Outline Package
* Single in-line package
* Small-Outline Integrated Circuit
* Staggered Pin Grid Array
* Surface-mount technology
* TO220
* TO3
* TO92
* TQFP
* TSSOP
* Thin small-outline package
* Through-hole technology
* UICC
* Zig-zag in-line package

What is Substrate coupling ?

In an integrated circuit, a signal can couple from one node to another via the substrate. This phenomenon is referred to as substrate coupling or substrate noise coupling.
The push for reduced cost, more compact circuit boards, and added customer features has provided incentives for the inclusion of analog functions on primarily digital MOS integrated circuits (ICs) forming mixed-signal ICs. In these systems, the speed of digital circuits is constantly increasing, chips are becoming more densely packed, interconnect layers are added, and analog resolution is increased. In addition, recent increase in wireless applications and its growing market are introducing a new set of aggressive design goals for realizing mixed-signal systems. Here, the designer integrates radio frequency (RF) analog and base band digital circuitry on a single chip. The goal is to make single-chip radio frequency integrated circuits

(RFICs) on silicon, where all the blocks are fabricated on the same chip. One of the advantages of this integration is low power dissipation for portability due to a reduction in the number of package pins and associated bond wire capacitance. Another reason that an integrated solution offers lower power consumption is that routing high-frequency signals off-chip often requires a 50O impedance match, which can result in higher power dissipation. Other advantages include improved high-frequency performance due to reduced package interconnect parasitics, higher system reliability, smaller package count, smaller package interconnect parasitics, and higher integration of RF components with VLSI-compatible digital circuits. In fact, the single-chip transceiver is now a reality.

What is Latchup ?

A latchup is the inadvertent creation of a low-impedance path between the power supply rails of an electronic component, triggering a parasitic structure, which then acts as a short circuit, disrupting proper functioning of the part and possibly even leading to its destruction due to overcurrent. A power cycle is required to correct this situation. The parasitic structure is usually equivalent to a thyristor (or SCR), a PNPN structure which acts as a PNP and an NPN transistor stacked next to each other. During a latchup when one of the transistors is conducting, the other one begins conducting too. They both keep each other in saturation for as long as the structure is forward-biased and some current flows through it - which usually means until a power-down. The SCR parasitic structure is formed as a part of the totem-pole PMOS and NMOS transistor pair on the output drivers of the gates.

CMOS interview questions.

1) What is latch up?

Latch-up pertains to a failure mechanism wherein a parasitic thyristor (such as a parasitic silicon controlled rectifier, or SCR) is inadvertently created within a circuit, causing a high amount of current to continuously flow through it once it is accidentally triggered or turned on. Depending on the circuits involved, the amount of current flow produced by this mechanism can be large enough to result in permanent destruction of the device due to electrical overstress (EOS) .

2)Why is NAND gate preferred over NOR gate for fabrication?

NAND is a better gate for design than NOR because at the transistor level the mobility of electrons is normally three times that of holes compared to NOR and thus the NAND is a faster gate.
Additionally, the gate-leakage in NAND structures is much lower. If you consider t_phl and t_plh delays you will find that it is more symmetric in case of NAND (the delay profile), but for NOR, one delay is much higher than the other(obviously t_plh is higher since the higher resistance p mos's are in series connection which again increases the resistance).

3)What is Noise Margin? Explain the procedure to determine Noise Margin

The minimum amount of noise that can be allowed on the input stage for which the output will not be effected.

4)Explain sizing of the inverter?

In order to drive the desired load capacitance we have to increase the size (width) of the inverters to get an optimized performance.

5) How do you size NMOS and PMOS transistors to increase the threshold voltage?

6) What is Noise Margin? Explain the procedure to determine Noise Margin?

The minimum amount of noise that can be allowed on the input stage for which the output will not be effected.

7) What happens to delay if you increase load capacitance?

delay increases.

8)What happens to delay if we include a resistance at the output of a CMOS circuit?

Increases. (RC delay)

9)What are the limitations in increasing the power supply to reduce delay?

The delay can be reduced by increasing the power supply but if we do so the heating effect comes because of excessive power, to compensate this we have to increase the die size which is not practical.

10)How does Resistance of the metal lines vary with increasing thickness and increasing length?

R = (*l) / A.

11)For CMOS logic, give the various techniques you know to minimize power consumption?

Power dissipation=CV2f ,from this minimize the load capacitance, dc voltage and the operating frequency.

12) What is Charge Sharing? Explain the Charge Sharing problem while sampling data from a Bus?

In the serially connected NMOS logic the input capacitance of each gate shares the charge with the load capacitance by which the logical levels drastically mismatched than that of the desired once. To eliminate this load capacitance must be very high compared to the input capacitance of the gates (approximately 10 times).

13)Why do we gradually increase the size of inverters in buffer design? Why not give the output of a circuit to one large inverter?

Because it can not drive the output load straight away, so we gradually increase the size to get an optimized performance.

14)What is Latch Up? Explain Latch Up with cross section of a CMOS Inverter. How do you avoid Latch Up?

Latch-up is a condition in which the parasitic components give rise to the Establishment of low resistance conducting path between VDD and VSS with Disastrous results.

15) Give the expression for CMOS switching power dissipation?

CV2

16) What is Body Effect?

In general multiple MOS devices are made on a common substrate. As a result, the substrate voltage of all devices is normally equal. However while connecting the devices serially this may result in an increase in source-to-substrate voltage as we proceed vertically along the series chain (Vsb1=0, Vsb2 0).Which results Vth2>Vth1.

17) Why is the substrate in NMOS connected to Ground and in PMOS to VDD?

we try to reverse bias not the channel and the substrate but we try to maintain the drain,source junctions reverse biased with respect to the substrate so that we dont loose our current into the substrate.

18) What is the fundamental difference between a MOSFET and BJT ?

In MOSFET, current flow is either due to electrons(n-channel MOS) or due to holes(p-channel MOS) - In BJT, we see current due to both the carriers.. electrons and holes. BJT is a current controlled device and MOSFET is a voltage controlled device.

19)Which transistor has higher gain. BJT or MOS and why?

BJT has higher gain because it has higher transconductance.This is because the current in BJT is exponentially dependent on input where as in MOSFET it is square law.

20)Why do we gradually increase the size of inverters in buffer design when trying to drive a high capacitive load? Why not give the output of a circuit to one large inverter?

We cannot use a big inverter to drive a large output capacitance because, who will drive the big inverter? The signal that has to drive the output cap will now see a larger gate capacitance of the BIG inverter.So this results in slow raise or fall times .A unit inverter can drive approximately an inverter thats 4 times bigger in size. So say we need to drive a cap of 64 unit inverter then we try to keep the sizing like say 1,4,16,64 so that each inverter sees a same ratio of output to input cap. This is the prime reason behind going for progressive sizing.

21)In CMOS technology, in digital design, why do we design the size of pmos to be higher than the nmos.What determines the size of pmos wrt nmos. Though this is a simple question try to list all the reasons possible?

In PMOS the carriers are holes whose mobility is less[aprrox half] than the electrons, the carriers in NMOS. That means PMOS is slower than an NMOS. In CMOS technology, nmos helps in pulling down the output to ground ann PMOS helps in pulling up the output to Vdd. If the sizes of PMOS and NMOS are the same, then PMOS takes long time to charge up the output node. If we have a larger PMOS than there will be more carriers to charge the node quickly and overcome the slow nature of PMOS . Basically we do all this to get equal rise and fall times for the output node.

22)Why PMOS and NMOS are sized equally in a Transmission Gates?

In Transmission Gate, PMOS and NMOS aid each other rather competing with each other. That's the reason why we need not size them like in CMOS. In CMOS design we have NMOS and PMOS competing which is the reason we try to size them proportional to their mobility.

23)All of us know how an inverter works. What happens when the PMOS and NMOS are interchanged with one another in an inverter?

I have seen similar Qs in some of the discussions. If the source & drain also connected properly...it acts as a buffer. But suppose input is logic 1 O/P will be degraded 1 Similarly degraded 0;

24)A good question on Layouts. Give 5 important Design techniques you would follow when doing a Layout for Digital Circuits?

a)In digital design, decide the height of standard cells you want to layout.It depends upon how big your transistors will be.Have reasonable width for VDD and GND metal paths.Maintaining

uniform Height for all the cell is very important since this will help you use place route tool easily and also incase you want to do manual connection of all the blocks it saves on lot of area.
b)Use one metal in one direction only, This does not apply for metal 1. Say you are using metal 2 to do horizontal connections, then use metal 3 for vertical connections, metal4 for horizontal, metal 5 vertical etc...
c)Place as many substrate contact as possible in the empty spaces of the layout.
d)Do not use poly over long distances as it has huge resistances unless you have no other choice.
e)Use fingered transistors as and when you feel necessary.
f)Try maintaining symmetry in your design. Try to get the design in BIT Sliced manner.

25)What is metastability? When/why it will occur?Different ways to avoid this?

Metastable state: A un-known state in between the two logical known states.This will happen if the O/P cap is not allowed to charge/discharge fully to the required logical levels.
One of the cases is: If there is a setup time violation, metastability will occur,To avoid this, a series of FFs is used (normally 2 or 3) which will remove the intermediate states.

26)Let A and B be two inputs of the NAND gate. Say signal A arrives at the NAND gate later than signal B. To optimize delay of the two series NMOS inputs A and B which one would you place near to the output?

The late coming signals are to be placed closer to the output node ie A should go to the nmos that is closer to the output.

1)Explain zener breakdown and avalanche breakdown?

A thermally generated carrier (part of reverse saturation current) falls down the junction barrier and acquires energy from the applied potential. This carriers collides with a crystal ion and imparts sufficient energy to disrupt a covalent bond.In addition to the original carrier, a new electron-hole pair has been generated. These carriers may also pick up sufficient energy and creates still another electron-hole pair. This cumulative process is called the Avalanche breakdown.
A reverse electric field at the junction causes a strong force to be applied on a bounded electron by the field to tear it out of its covalent bond. The new hole-electron pair which is created increases the reverse current, called zener breakdown.

2)What is Instrumentation Amplifier(IA) and what are all the advantages?

An instrumentation amplifier is a differential op-amp circuit providing high input impedances with ease of gain adjustment by varying a single resistor

3) What is the fundamental difference between a MOSFET and BJT ?

In MOSFET,current flow is either due to electrons(n-channel MOS) or due to holes(p-channel MOS)
- In BJT, we see current due to both the carriers.. electrons and holes. BJT is a current controlled device and MOSFET is a voltage controlled device.

4) What is the basic difference between Analog and Digital Design?

Digital design is distinct from analog design. In analog circuits we deal with physical signals which are continuous in amplitude and time. Ex: biological data, sesimic signals, sensor output, audio, video etc.

Analog design is quite challenging than digital design as analog circuits are sensitive to noise, operating voltages, loading conditions and other conditions which has severe effects on performance. Even process technology poses certain topological limitations on the circuit. Analog designer has to deal with real time continuous signals and even manipulate them effectively even in harsh environment and in brutal operating conditions.
Digital design on the other hand is easier to process and has great immunity to noise. No room for automation in analog design as every application requires a different design. Where as digital design can be automated. Analog circuits generally deal with instantaneous value of voltage and current(real time). Can take any value within the domain of specifications for the device.consists of passive elements which contribute to the noise(thermal) of the circuit . They are usually more sensitive to external noise more so because for a particular function a analog design
uses lot less transistors providing design challenges over process corners and temperature ranges. deals with a lot of device level physics and the state of the transistor plays a very important role
Digital Circuits on the other hand deal with only two logic levels 0 and 1(Is it true that according to quantum mechanics there is a third logic level?) deal with lot more transistors for a particular logic, easier to design complex designs, flexible logic synthesis and greater speed although at the cost of greater power. Less sensitive to noise. design and analysis of such circuits is dependant on the clock. challenge lies in negating the timing and load delays and ensuring there is no set up or hold violation.

5)What is ring oscillator? And derive the freq of operation?

Ring oscillator circuit is a coupled inverter chain with the output being connected to the input as feedback. The number of stages(inverters) is always odd to ensure that there is no single stable state(output value). sometimes one of the stages consists of a logic gate which is used to initialise and control the circuit. The total time period of operation is the product of 2*number of gates and gate(inverter) delay. And frequency of operation will be inverse of time period.
Application: used as prototype circuits for modeling and designing new semiconductor processes due to simplicity in design and ease of use. Also forms a part of clock recovery circuit.

6)What are RTL, Gate, Metal and FIB fixes? What is a "sewing kits"?

There are several ways to fix an ASIC-based design. >From easiest to most extreme:

RTL Fix -> Gate Fix -> Metal Fix -> FIB Fix

First, let's review fundementals. A standard-cell ASIC consists of at least 2 dozen manufactured layers/masks. Lower layers conists of materialsmaking up the actual CMOS transistors and gates of the design. The upper 3-6 layers are metal layers used ti connect everything together. ASICs, of course, are not intended to be flexible like an FPGA, however, important "fixes" can be made during the manufacturing process. The progression of possible fixes in the manufacturing life cycle is as listed above.

An RTL fix means you change the Verilog/VHDL code and you resynthesize. This usually implies a new Plance&Route. RTL fixes would also imply new masks, etc. etc. In other words - start from scratch.

A Gate Fix means that a select number of gates and their interconections may be added or subtracted from the design (e.g. the netlist). This avoids resynthesis. Gate fixes preserve the previous synthesis effort and involve manually editing a gate-level netlist - adding gates, removing gates, etc. Gate level fixes affect ALL layers of the chip and all masks.

A Metal Fix means that only the upper metal interconnect layers are affected. Connections may be broken or made, but new cells may not be added. A Sewing Kit is a means of adding a new gate into the design while only affecting the metal layers. Sewing Kits are typically added into the initial design either at the RTL level or during synthesis by the customer and are part of the netlist. A Metal Fix affects only the top layers of the wafers and does not affect the "base" layers.

Sewing Kits are modules that contain an unused mix of gates, flip-flops or any other cells considered potentially useful for an unforseen metal fix. A Sewing Kit may be specified in RTL by instantiating the literal cells from the vendor library. The cells in the kit are usually connected such that each cell's output is unconnected and the inputs are tied to ground. Clocks and resets may be wired into the larger design's signals, or not.

A FIB Fix (Focussed Ion Beam) Fix is only performed on a completed chip. FIB is a somewhat exotic technology where a particle beam is able to make and break connections on a completed die. FIB fixes are done on individual chips and would only be done as a last resort to repair an otherwise defective prototype chip. Masks are not affected since it is the final chip that is intrusively repaired.

Clearly, these sorts of fixes are tricky and risky. They are available to the ASIC developer, but must be negotiated and coordinated with the foundry. ASIC designers who have been through enough of these fixes appreciate the value of adding test and fault-tolerant design features into the RTL code so that Software Fixes can correct mior silicon problems!

What are the steps required to solve setup and Hold violations in VLSI Explain?

There are few steps that has to be performed to solved setup and hold violations in VLSI. The steps are as follows:
the optimization and restructuring of the logic between the flops are carried way. This way logics are combined and it helps in the solving this problem.
There is way to modify the flip-flops that offer lesser setup delay and provide faster services to setup a device.
Modifying the launch-flop to have a better hold on the clock pin, which provides CK->Q that makes the launch-flop to be fast and helps in fixing setup violations.
The network of the clock can be modified to reduce the delay or slowing down of the clock that captures the action of the flip-flop.
There can be added delay/buffer that allows less delay to function that is used.

What are the different ways in which antenna violation can be prevented Explain?

Antenna violation occurs during process of plasma etching in which charges generating from one metal strip to another gets accumlated at a single place. The longer the strip the more the charges gets accumulated. The prevention can be done by following method:
Creating a jogging the metal line, that consists of atleas one metal above the protected layer. There is a requirement to jog the metal that is above the metal getting the etching effect. This is due to fact that if a metal gets the etching then the other metal gets disconnected if the prevention measures are not taken care.
There is a way to prevent it by adding the reverse Diodes at the gates that are used in circuits.

What is funciton of tie-high and tie-low cells?

Tie-high and tie-low are used to connect transistors of the gate by using either the power or the ground. The gates are connected using the power or ground then it can be turned off and on due to power bounce from the ground. The cells are used to stop the bouncing and easy from of the current from one cell to another. The cells are required Vdd that connects to the tie-high cell as there is a power supply that is high and tie-low gets connected to Vss. This connection gets established and the transistors function properly without the need of any ground bounce occuring in any cell.

What is the main function of metastability in VSDL Explain?

Metastability is an unknown state that is given as neither one or zero. It is used in designing system that violates the setup or hole time requirements. The setup time requirement need data to be stable before the clock-edge and the hold time requires data to be stable after the clock edge has passed. There are potential violation that can lead to setup and hold violations as well. The data that is produced in this is totally asynchronous and clocked synchronous. This provide a way to setup the state through which it can be known that violations that are occuring in system and a proper design can be provided by use of several other functions.

What are steps involved in preventing the metastability?

Metastability is the unknown state and it prevents theof violations using the following steps: proper synchronizers are used that can be two stage or three stage whenever the data comes from the asynchronous domain. Thus helping in recovering the metastable state event.
The synchronizers are used in between cross-clocking domains. This reduces the metastability by removing the delay that is caused by data element that are coming and taking time to get removed from the surface of metal.
Use of faster flip-flops that allow the transaction to be more faster and it removes the delay time between one component to another component. It uses a narrower metastable window that makes the delay happen but faster flip-flops help in making the process faster and reduce the time delay as well.

1. What does chip utilization depend on? Standard cells and macros

2. Which cells are placed in Soft blockages? Only buffers and inverters

3. What does Prerouting mean? Routing of PG nets

4. Metal layer 1 has Maximum resistance?

5. What is the purpose of CTS(clock tree synthesis)? Minimum Skew

6. Which cells would you place in the critical path for a better timing? LVT

7. Leakage power is inversely proportional tothreshold voltage.

8. Why do you use Search and Repair? To reduce DRC

9. Utilisation of the chip after placement optimisation will increase.

What are the steps required to solve setup and Hold violations in VLSI?

There are few steps that has to be performed to solved the setup and hold violations in VLSI. The steps are as follows:
the optimization and restructuring of the logic between the flops are carried way. This way the logics are combined and it helps in solving this problem.
There is way to modify the flip-flops that offer lesser setup delay and provide faster services to setup a device.
Modifying the launch-flop to have a better hold on the clock pin, which provides CK->Q that makes the launch-flop to be fast and helps in fixing the setup violations.
The network of the clock can be modified to reduce the delay or slowing down of the clock that captures the action of the flip-flop.
There can be added delay/buffer that allows less delay to the function that is used.

What are the different ways in which antenna violation can be prevented?

Antenna violation occurs during the process of plasma etching in which the charges generating from one metal strip to another gets accumlated at a single place. The longer the strip the more the charges gets accumulated. The prevention can be done by following method:
Creating a jogging the metal line, that consists of atleas one metal above the protected layer.
There is a requirement to jog the metal that is above the metal getting the etching effect. This is due to the fact that if a metal gets the etching then the other metal gets disconnected if the prevention measures are not taken.
There is a way to prevent it by adding the reverse Diodes at the gates that are used in the circuits.

What is the funciton of tie-high and tie-low cells?

Tie-high and tie-low are used to connect the transistors of the gate by using either the power or the ground. The gates are connected using the power or ground then it can be turned off and on due to the power bounce from the ground. The cells are used to stop the bouncing and easy from of the current from one cell to another. These cells are required Vdd that connects to the tie-high cell as there is a power supply that is high and tie-low gets connected to Vss. This connection gets established and the transistors function properly without the need of any ground bounce occuring in any cell.

What is the main function of metastability in VSDL(very high speed digital subscriber line)?

Metastability is an unknown state that is given as neither one or zero. It is used in designing the system that violates the setup or hole time requirements. The setup time requirement need the data to be stable before the clock-edge and the hold time requires the data to be stable after the clock edge has passed. There are potential violation that can lead to setup and hold violations as well. The data that is produced in this is totally asynchronous and clocked synchronous. This provide a way to setup the state through which it can be known that the violations that are occuring in the system and a proper design can be provided by the use of several other functions.

What are the steps involved in preventing the metastability?

Metastability is the unknown state and it prevents the violations using the following steps:
proper synchronizers are used that can be two stage or three stage whenever the data comes from the asynchronous domain. This helps in recovering the metastable state event.

The synchronizers are used in between cross-clocking domains. This reduces the metastability by removing the delay that is caused by the data element that are coming and taking time to get removed from the surface of metal.
Use of faster flip-flops that allow the transaction to be more faster and it removes the delay time between the one component to another component. It uses a narrower metastable window that makes the delay happen but faster flip-flops help in making the process faster and reduce the time delay as well.

ASIC's provide the path to creating miniature devices that can do a lot of diverse functions. But with the impending boom in this kind of technology, what we need is a large number of people who can design these IC's. This is where we realise that we cross the threshold between a chip designer and a systems designer at a higher level. Does a person designing a chip really need to know every minute detail of the IC manufacturing process? Can there be tools that allow a designer to simply create design specifications that get translated into hardware specifications?

The solution to this is rather simple - hardware compilers or silicon compilers as they are called. We know by now, that there exist languages like Verilog which can be used to specify the design of a chip. What if we had a compiler that converts a high level language into a Verilog specification? The potential of this technology is tremendous - in simple manner, we can convert all the software programmers into hardware designers!

WHAT SORTS OF JOBS DOES AN VLSI or ASIC ENGINEER DO?

1. Design Engineer:
Takes specifications, defines architecture, does circuit design, runs simulations, supervises layout, tapes out the chip to the foundry, evaluates the prototype once the chip comes back from the fab.

2. Product Engineer:
Gets involved in the project during the design phase, ensures manufacturability, develops characterization plan, assembly guidelines, develops quality and reliability plan, evaluates the chip with the design engineer, evaluates the chip through characterization, reliability qualification and manufacturing yield point of view (statistical data analysis). He is responsible for production release and is therefore regarded as a team leader on the project. Post production, he is responsible for customer returns, failure analysis, and corrective actions including design changes.

3. Test Engineer:
Develops test plan for the chip based on specifications and data sheet, creates characterization and production program for the bench test or the ATE (Automatic Test Equipment), designs test board hardware, correlates ATE results with the bench results to validate silicon to compare with simulation results. He works closely with the product engineer to ensure smooth release to production and post release support.

4. Applications Engineer:
Defines new products from system point of view at the customerâ€™s end, based on marketing input. His mission is to ensure the chip works in the system designed or used by the customers, and complies with appropriate standards (such as Ethernet, SONET, WiFi etc.). He is responsible for all customer technical support, firmware development, evaluation boards, data sheets and all product documentation such as application notes, trade shows, magazine articles, evaluation reports, software drives and so on.

5. Process Engineer:
This is a highly specialized function which involves new wafer process development, device modeling, and lots of research and development projects. There are no quick rewards on this job! If you are R&D oriented, highly trained in semiconductor device physics area, do not mind wearing bunny suits (the clean room uniforms used in all fabs), willing to experiment, this job is for you.

6. Packaging Engineer:
This is another highly specialized job function. He develops precision packaging technology, new package designs for the chips, does the characterization of new packages, and does electrical modeling of the new designs.

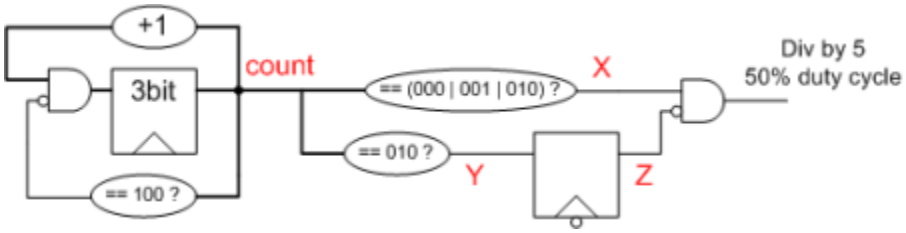
7. CAD Engineer:
This is an engineering function that supports the design engineering function. He is responsible for acquiring, maintaining or developing all CAD tools used by a design engineer. Most companies buy commercially available CAD tools for schematic capture, simulation, synthesis, test vector generation, layout, parametric extraction, power estimation, and timing closure; but in several cases, these tools need some type of customization. A CAD engineer needs to be highly skilled in the use of these tools, be able to write software routines to automate as many functions as possible and have a clear understanding of the entire design flow.

Real World Examples #5 – Clock Divider by 5

August 26, 2009

Here is a neat little circuit that was used in an actual project a long, long time ago (in a galaxy far, far away…).

The requirement was to build a divide by 5 circuit for the clock with 50% duty cycle. The initial (on reset) behavior was not important – i.e. the circuit could wake up in an undefined state, but should have settled after a given time. The engineer produced the circuit below:



Basically, the circuit is made out of a 3-bit counter, that counts from 000 to 100 and then resets. Signal **'X'** goes high when the value of the counter is either 000, 001 or 010. Signal **'Y'** goes high when the counter equals its 'middle' state 010. **'Z'** is a sample on the *falling* edge of **'Y'** in order to generate the 50% duty cycle.

So far so good. The general thinking was OK, but there was a major problem with the circuit, can you discover what it was? How would you fix it in RTL? and more important, how would you fix it in an ECO (as it was eventually done)?

No extra flops are allowed!

Posted in Real World Examples | 21 Comments »

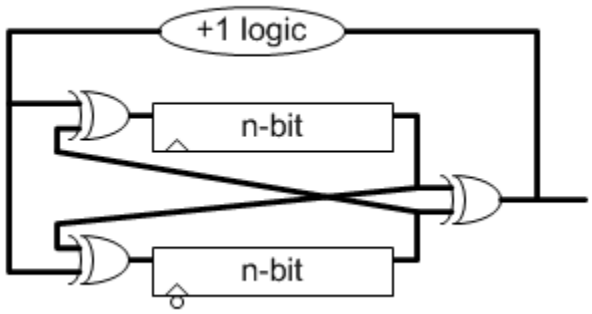
Dual Edge Binary Counters + Puzzle

June 24, 2009

I lately came across the need to use a dual edge counter, by this I mean a counter which is counting both on the rising and on the falling edge of the clock.
The limitation is that one has to use only normal single edge sensitive flops, the kind you find in each library.

There are several ways to do this, some easier than others. I would like to show you a specific design which is based on the dual edge flop I described in a previous post. This design is just used here to illustrate a point, I do not recommend you use it – there are far better ways. Please refer to the end of the post for more on that.

The figure below depicts the counter:



The counter is made of 2 n-bit arrays of flops. The one operates on the rising edge, the other on the falling edge. The "+1″ logic is calculated from the final XOR output, which is the real output of the counter! The value in each of the n-bit arrays does **not** represent the true counting value, but is **used** to calculate the final counter value. Do not make the mistake and use the value directly from either set of flops.

This leads to a small puzzle – given the conditions above, can this counter be done with less flops?

Posted in Cool Circuits, Puzzles | 5 Comments »

Reordering Nets for Low Power

May 10, 2009

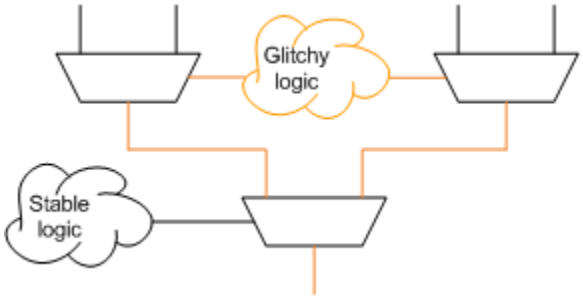
As posts accumulate, you can see that low power design aspects is a big topic on this site. I try to bring more subtle design examples for lower power design that you can control and implement (i.e. in RTL and the micro architectural stage).

Identifying "glitchy" nets is not always easy. Some good candidates are wide parity or CRC calculations (deep and wide XOR trees), complicated arithmetic paths and basically most logic that originates in very wide buses and converges to a single output controlling a specific path (e.g. as a select pin of a MUX for a wide data path).

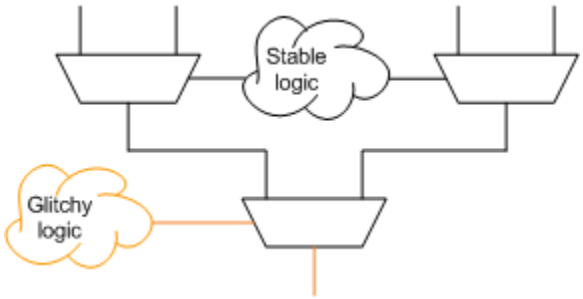
If you happen to identify a good candidate, it is advisable (when possible) that you feed the "glitchy" nets as late as possible in the calculation path. This way the total amount of toggling in the logic is reduced.

Sounds easy enough? Well the crux of the problem is identifying those opportunities – and it is far from easy. I hope this post at least makes you more aware of that possibility.

To sum up, here are two figures that illustrate the issue visually. The figure below depicts the situation before the transformation.. The nets which are highlighted represent high activity nets.



After the transformation – pushing the glitchy net calculation late in the path. The transformation is logically equivalent (otherwise there is no point…) and we see less high activity nets.



Posted in Architecture, Low Power | 2 Comments »

Parametrized Reset Values

April 19, 2009

For some odd reason some designers refuse to use parametrized blocks. I have no idea what are the reasons for such an opinion, but here is a good example why one would want to decide for the usage of parameters.

Imagine you need to design a block, which will be used several times throughout the design. The problem is, that each instance might need to have different reset values to some of its internal flops.
One (wrong) possibility is to define an extra input, which will in turn be connected as the reset value – but this is not something you'd like to do (why??)

The better option is to send the reset value as a parameter, which if it wasn't clear by now, is the way to go.

Posted in Uncategorized | 4 Comments »

Puzzle #14 – Multipliers

April 14, 2009

Here is an interview question that was circulating some of the message boards lately.
Can you create a 4×4 multiplier with only 2×2 multipliers at hand?

post your answers as a comment to this post.

Posted in Uncategorized | 5 Comments »

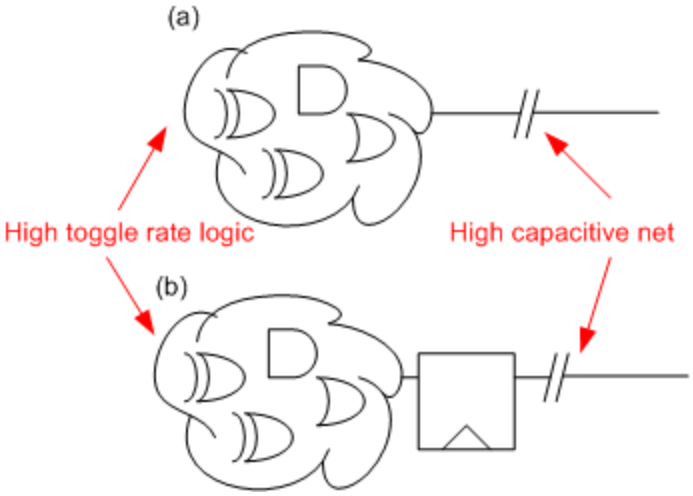
Reducing Power Through Retiming

February 23, 2009

Here is an interesting and almost trivial technique for (potential) power reduction, which I never used myself, nor seen used in others' designs. Well… maybe I am doing the wrong designs… but I thought it is well worth mentioning. So, if any of my readers use this, please do post a short comment on how exactly did you implement it and if it really resulted in some significant savings.

We usually have many high activity nets in the design. They are in many cases toggling during calculation more than once per cycle. Even worse, they often drive long and high capacitive nets. Since, in a usual synchronous design (which 99% of us do), we only need the stable result once per cycle – when the calculation is done – we can just put a register to drive the high capacitive net. The register effectively blocks all toggling on that net (where it hurts) and allows it to change maximum one time per cycle.

The image below tells the whole story. (a) is before the insertion of the flop, (b) right after.



This is all nice, but just remember that in real life it can be quite hard to identify those special nets and those special high toggling logic clouds. Moreover, most of the time we cannot afford the flop for latency reasons. But if you happen to be in the early design phase and you know more or less your floor plan, think about moving some of those flops so they will reduce the toggling on those high capacitive nets.

Posted in Layout, Low Power | 8 Comments »

Transparent Pipelining

February 15, 2009

The nice thing about posting in such a site, is that one learns quite a bit with time. During the long pause I took, I tried to read quite a bit and look for some interesting papers. Yes, I am aware that most of my readers are not really interested in reading technical papers for fun, but the bunch that I collected are IMHO quite important and teach a lot.

The one for today is about a novel clocking scheme for latch based pipelines. I found it really interesting and important. I am sure that sometime I am going to implement this for a future design. The paper could have had more examples, and I bet that a few animations would only do good for that topic – but you can't really ask for stuff like that in a technical paper, can you?

OK, enough of my words – you can find the paper here.

Posted in Latch-based Design | 3 Comments »

New Updates Coming Soon

February 4, 2009

I know it has been a while since I added new posts. There has been a lot going on here lately – new addition to the family, new job and some more smaller things, which keep me relatively busy lately.

Don't give up on me just yet. I promise to keep the interesting posts coming, although maybe not on a weekly basis as I tried doing before.

Hope you guys understand…

Posted in General | 3 Comments »

Real World Examples #4 – More on "Thinking Hardware"

January 20, 2009

I was reviewing some code not so long ago, and noticed together with the owner of the code, that we had some timing problems.
Part of the code looked something like that (Verilog):

```
wire [127:0] a;
wire [127:0] b;
wire [127:0] c;
assign c = select_register ? a : b;
```

For those not familiar with Verilog syntax, the code describes a MUX construct using the ternary operator. The two data inputs for the MUX are "a" and "b" and the select is "select_register".

So why was this code translated into a relatively slow design? The answer is in the width of the signals. The code actually synthesizes to 128 parallel MUX structures. The "select_register" has actually 128 loads.
When a construct like this is hidden within a large code, our tendency is to just neglect it by saying it is "only" 2:1 MUX deep, but we have to look more carefully than that – and always remember to consider the load.

Solving this problem is relatively easy by replication. Just creating more versions of the "select_register" helped significantly.

Posted in Coding Style, Real World Examples, Synthesis | 8 Comments »

A Message for the New Year

January 10, 2009

Holiday season is gone, the new year is just starting and I am into preaching mood.

I get many, many emails from people asking me to help them with their designs, interview questions or just give advice. Sometimes, if I am not fast enough in replying, I even get complains and emails urging me to supply the answer "ASAP". This is all OK and nice, but I would like you the reader to stop for a second and think on how much **YOU** are contributing to our community?

Not everyone can or likes to write a technical blog, but there are other options one can utilize – one of my favorites is posting on a forum. Even if you are a beginner in the field, post your questions, this is already a big help for many. I personally post from time to time on EDA board. Just go through that forum and have a quick look, some questions are very interesting while others can be extremely stupid (sorry) – who cares! What matters in my eyes, is that the forum is building a database of questions and answers that can help you and others.

I assume that most of my readers are on the passive side of things (just a hunch). I hope this post will make you open an account on one of the forums and start posting.

p.s. please use the comments section to recommend your favorite design related forums or groups.

Posted in General | 5 Comments »



Interview Question – BCD Digit, Multiplied by 5

December 21, 2008

A while back, someone sent me the interview question I am about to describe, asking for help. I think it serves a very good example of observing patterns and not rushing into conclusions.
I will immediately post the answer after describing the problem. However, I urge you to try and solve it on your own and see what you came up with. On we go with the question…

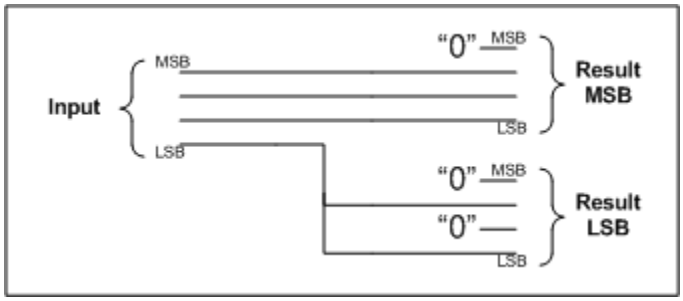
Design a circuit **with minimum logic** that receives a single digit, coded BCD (4 wires) and as an output gives you the result multiplied by 5 – also BCD coded (8 wires).

So, I hope you got a solution ready at hand and you didn't cheat .

Let's first make some order and present the input and required outputs in a table (always a good habit).

input		output	
dec	BCD	dec	BCD
0	0000	00	0000 0000
1	0001	05	0000 0101
2	0010	10	0001 0000
3	0011	15	0001 0101
4	0100	20	0010 0000
5	0101	25	0010 0101
6	0110	30	0011 0000
7	0111	35	0011 0101
8	1000	40	0100 0000
9	1001	45	0100 0101

Looking for some patterns we can see that we actually don't need any logic at all to solve this problem!!



You will be amazed how many people get stuck with a certain solution and believe it is the minimal one. Especially when the outcome is one or two single gates. When you tell them it can be done with less, they will easily find the solution. IMHO there is nothing really clever or sophisticated about this problem, but it demonstrates beautifully how it is sometimes hard for us to escape our initial ideas and approaches about a problem.

Coming to think of it, this post was more about psychology and problem solving than digital design – please forgive…

Posted in General, Interview Questions, Send Your Problem | 13 Comments »

A Coding Tip for Multi Clock Domain Designs

December 13, 2008

Multi clock domain designs are always interesting, but almost always hide some synchronization problems, which are not that trivial. There are tools on the market that identify all(??) clock domain crossings within a design. I personally had no experience with them, so I can't give an opinion (although I heard some unflattering remarks from fellow engineers).

Seems like each company has its own ways of handling this problem. One of the oldest, easiest and IMHO one of the most efficient ways, is to keep strict naming guidelines for your signals, *whether combinatorial or sequential !!*

The most common way is to add a prefix to each signal which describes its driver clock e.g. *clk_800_mux_32to1_out* or *clk_666_redge_acknowledge*.

If you don't use this simple technique, you won't believe how useful it is. Many of the related problems of synchronization are actually discovered during the coding process itself. Moreover, it even makes life easier when doing the code review.

If you have more tips on naming convention guidelines for signals in RTL – post them as a comment!

Posted in Coding Style | 14 Comments »



Another Reason to Add Hierarchies to Your Designs

November 30, 2008

We are usually very annoyed when the team leader insists on code restructuring and hierarchical design.
I also know this very well from the other side as well. Trying to convince designers to better restructure their own design which they know so very well already.

Well, here is another small, yet important reason why you might want to do this more often.
Assume your design is more or less mature, you ran some simulation, went through some synthesis runs and see that you don't meet timing.
You analyze the timing report just to find a huge timing path optimized by the tool and made of tons of NANDs, NORs, XORs and what not. Well you see the starting point and the end point very clearly, but you find yourself asking if this is the path that goes through the MUX or through the adder maybe?

Most logic designs are extremely complicated and the circuit is not just something you can draw immediately on paper. Moreover, looking at a timing report of optimized logic, it is very hard to interpret the exact path taken through the higher level structured – or in other words, what part of the code I am really looking at here??!! Adding an hierarchy will also add its name to the optimized structures in the timing report and you could then easily pin point your problems.

I even saw an engineer that uses this technique as a debugging tool. If he has a very deep logic cloud, he will intentionally build an hierarchy around say a simple 2:1 MUX in the design and look for it in the timing report. This enables him to "feel" how the synthesis tool optimizes the path and where manual optimization needs to be implemented .

Use this on your bigger blocks, it saves a lot of time and effort in the long run.

Posted in Coding Style | Tagged coding technique, optimization | 6 Comments »

Challenge #3 – Counting the Number of "1"s

November 13, 2008

Time for a new challenge! The last two had some great responses and solutions. If you read through the comments you'd see there were some disagreements on what is the best approach. Some claimed a hand crafted approach is the best, while others said it was more of a theoretical problem and we should use a synthesis tool to solve it.
Both have pros and cons, although for those specific challenges I personally tend to go with the hand crafted approach – you, of course, don't have to agree with me.

For this time we got a very practical problem that pops up again and again: counting the number of "1"s in a vector.
Use the metrics given in challenge #1 and find the minimal delay circuit for a combo cloud that counts the number of "1"s in an 8-bit vector. You get 8 bits in and supply 4 output bits which give a binary representation of the amount of "1"s in the 8-bit vector.

Oh and don't forget to mention how your method scales when counting 16-bit and 32-bit vectors.

Ready, set, go!

Posted in Puzzles | Tagged challenge | 22 Comments »

Closing the Gap Between ASIC and Custom

November 8, 2008

I don't know why I did *not* came across this wonderful, wonderful (maybe I should add another "wonderful"…) book before.

First here is a link to the book's site and an amazon link – and for those who are interested in a short overview, this short summery from DAC should give a hint what it is all about.

The book is mostly about increasing performance of your circuits. It surveys many techniques, problems and ideas (some are not fully supported by major EDA tools). It doesn't matter really if you use these techniques or not – you will learn a lot about "closing the gap" (at least I did).

This gets my full recommendation and endorsement (if anybody cares about my opinion …)

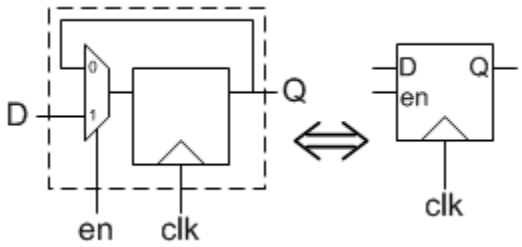
Posted in General | Tagged book | 3 Comments »

Fun With Enable Flip-Flops

October 27, 2008

Almost each library has *enable flip-flops* included. Unfortunately, they are not always used to their full potential. We will explore some of their potential in this post.

An enable flop is nothing but a regular flop which only registers new data if the enable signal is high, otherwise it keeps the old value. We normally implement this using a MUX and a feedback from the flop's output as depicted below.



So what is the big deal about it? The nice thing is that the enable flop is already implemented by the guys who built the library in a very optimized way. Usually implementing this with a MUX before the flop will eat away from the cycle time you could otherwise use for your logic. However, a short glance at your library will prove that this MUX comes almost for free when you use an *enable flop* (for my current library the cost is 20ps).

So how can we use this to our advantage?

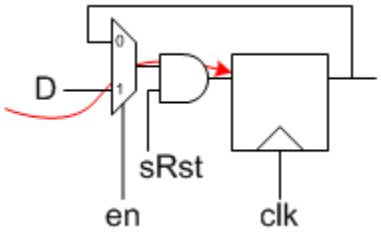
Example #1 – Soft reset coding

In many applications soft reset is a necessity. It is a signal usually driven by a register that will (soft) reset all flip flops given that a clock is running. Many times an enable "if" is also used in conjunction.
This is usually coded in this way (I use Verilog pseudo syntax and ask the forgiveness of you VHDL people):

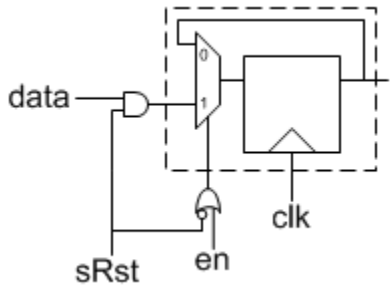
```
always @(posedge clk or negedge hard_rst)
if (!hard_rst)
ff <= 1'b0;
else if (!soft_rst)
ff <= 1'b0;
else if (en)
ff <= D;
```

The above code usually results in the construction given in the picture below. The red arrow represents the critical timing path through a MUX and the AND gate that was generated for the

soft reset.



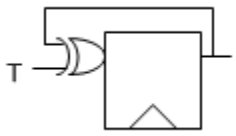
Now, if we could only exchange the order of the last two "if" commands this would put the MUX in front of the AND gate and then we could use an enable flop… well, if we do that, it will not be logically equivalent anymore. Thinking about it a bit harder, we could use a trick – let's exchange the MUX and the AND gate but during soft reset we could force the select pin of the MUX to be "1″, and thus transferring a "0″ to the flop! Here's the result in a picture form.



We can now use an enable flop and we basically got the MUX delay almost for free. This may look a bit petty to you, but this trick can save you a few extra precious tens or hundreds of pico-seconds.

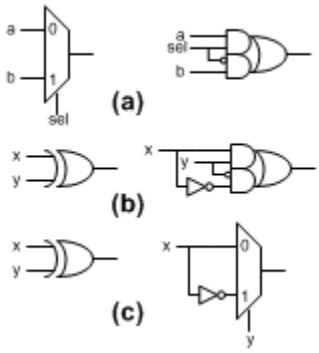
Example #2 – Toggle Flip Flops

Toggle flops are really neat, and there are many cool ways to use them. The normal implementation requires an XOR gate combining the T input and a feedback of the flop itself.

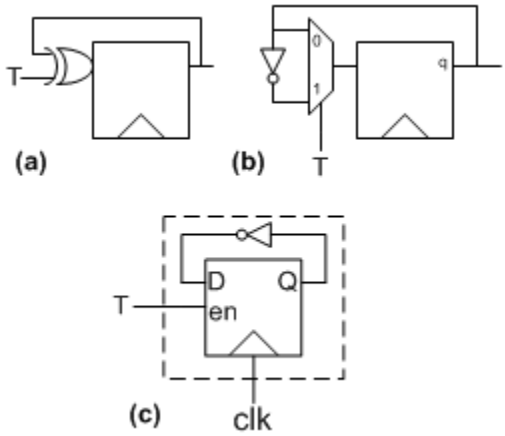


Let's have a closer look at the logical implementation of an XOR gate and how it is related to a MUX implementation: (a) is a MUX gate equivalent implementation (b) is an XOR gate

equivalent implementation and (c) is an XOR implemented from a MUX.



Now, let's try making a T flop using an enable flop. We saw already how to change the MUX into an XOR gate – all that is left, is to put everything together.



Posted in Coding Style | 5 Comments »

Challenge #2 – One Hot Detection

October 20, 2008

The last challenge was a big success with many people sending their solutions via email or just posting them as a comments.
Many of you said they were waiting for the next challenge. So, before returning to the usual set of posts about different aspects of digital design, let's look at another one.

Imagine you have a vector of 8 bits, The vector is supposed to be one hot coded (only a single logic "1″ is allowed in the set). Your task if you choose to accept it 😀, is to design a combo block to detect if the vector is indeed one-hot encoded.

We are again looking for the block with the shortest delay. As for the solution metrics for this challenge please refer to the previous challenge.

Also try to think how your design scales when the input vector is 16 bits wide, 32 bits wide and the general case of n bits wide.

Good luck!

Posted in Puzzles | Tagged challenge | 20 Comments »

Challenge #1 – DBI Detect

October 8, 2008

It has been a while since we had a challenge question on the site (last one was the divide by 3 question), and I would like to have more of those in the future. I will basically pose a problem and ask you to solve it under certain conditions – e.g. least hardware or latency, lowest power etc.

This time the challenge is related to a real problem I encountered recently. I reached a certain solution, which I do not claim to be optimal, actually I have the feeling it can be done better – I am therefore very interested in your own way of solving the problem.

Your challenge is to design a combo-block with 8 inputs and 1 output. You receive an 8-bit vector, If the vector contains 4 '1's or more, the output should be high, otherwise low (This kind of calculation is commonly used for data bus inversion detection).

What is the best way to design it with respect to minimizing latency (in term of delay units), meaning the lowest logic depth possible.

Just so we could compare solutions, let's agree on some metrics. I am aware that your own library might have different delay ratios between the different elements, but we gotta have something to work with.

- Inverter – 1 delay unit
- NOR, NAND – 2 delay units
- AND, OR – 3 delay units
- 3 or 4 input NOR, NAND – 4 delay units (2 for first stage + 2 for second stage)
- 3 or 4 input OR, AND – 6 delay units (2 for first stage + 2 for second stage)
- XOR, MUX – 7 delay units (2 AND/OR + 1 Inverter)

Please either post a comment with a detailed solution, or send me an email.

Take it from here guys…

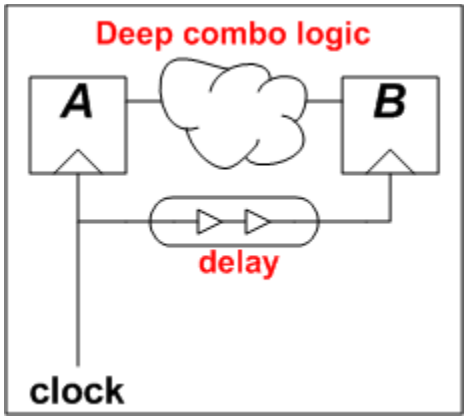
Posted in Challenges, Puzzles | Tagged arithmetic, challenge, optimization | 27 Comments »

Who Said Clock Skew is Only Bad?

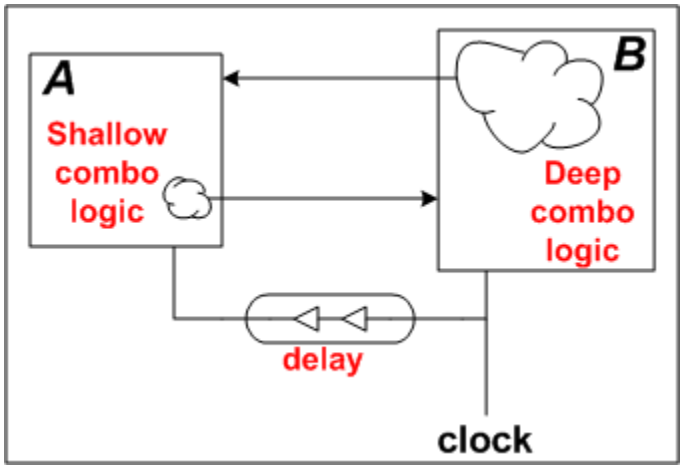
October 2, 2008

We always have this fear of adding clock skew. Well, seems like this is one of the holy cows of digital design, but sometimes clock skew can be advantageous.

Take a look at the example below. The capturing flop would normally violate setup requirements due to the deep logic cloud. By intentionally adding delay we could help make the clock arrive later and thus meet the setup condition. Nothing comes for free though, if we have another register just after the capturing one, the timing budget there will be cut.



This technique can also be implemented on the block level as well. Assume we have two blocks A and B. B's signals, which are headed towards A, are generated by a deep logic cloud. On the other hand A's signals, which arrive at B, are generated by a rather small logic cloud. Skewing the clock in the direction of A now, will give more timing budget for the B to A signals but will eat away the budget from A to B's signals.



Inserting skew is very much disliked by physical implementation guys although a lot of the modern tools know how to handle it very nicely and even account for the clock re-convergence pessimism (more on this in another post). I have the feeling this dislike is more of a relic of the past, but as we push designs to be more complex, faster, less power hungry etc. we have to consider such techniques.

Posted in Architecture, Synthesis, Timing | Tagged Clock Skew, Timing | 3 Comments »

K-Maps Walks and Gray Codes

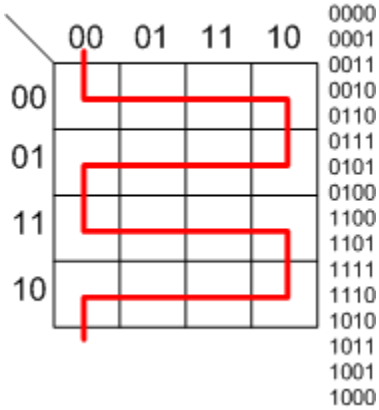
September 26, 2008

It is this time of year maybe, but I just feel I have to write another post on Gray codes.
We all remember our good friend the K-map (give yourself a point if you knew how to spell the full name, I'm getting it all wrong each time).

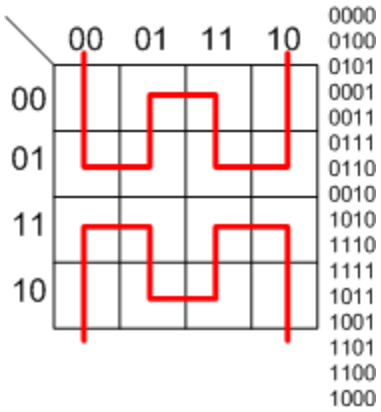
By nature of its construction – a "walk" through the map will generate a Gray code, since each cell is different from its adjacent one by a single bit only. Moreover, If we return to the point of origin, we just created a cyclic Gray code.

Draw yourself some 4×4 K-Maps and start playing around with the idea. Remember the K-map is like a toroid, moving off the map to the left pops us back in on the right side and in an analogous way for up/down, right/left and down/up.

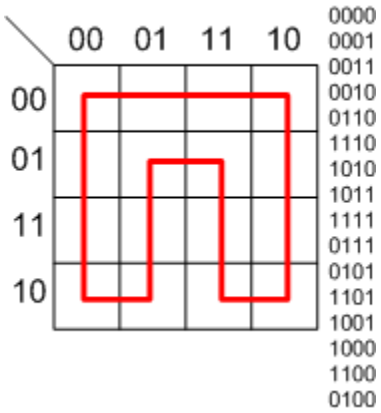
Here for instance is the good old "reflected Gray code" which is usually used in many applications which require "a Gray code". Notice the different toggling cycles of the columns in the outcome sequence – 2-2-2-2-2-2-2-2-2…,4-4-4-4…,8-8… and 8-8….



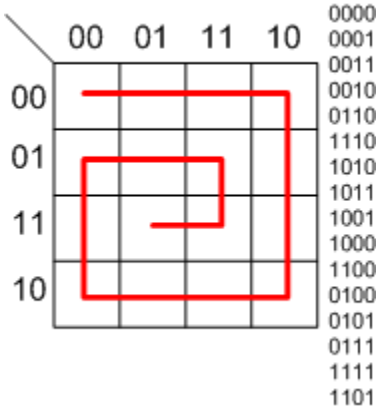
What if we take a slightly different tour through the map? Notice how now the 3 LSB columns have been rotated.



Let's try another way to walk the K-map, but maybe this time a *little* less symmetric (only one axis of symmetry). Look how now the toggling cycles of the columns became rather strange – no more something like 4-4-4-4… but rather 4-2-4-6… and other weird cycles.



What if we need (for whatever strange reason) a non cyclic one? there is nothing easier than that. The start and the end point are **not** adjacent! which makes the sequence **not** a cyclic one.



As you see there are many, many different Gray codes around. Sometimes it is just nice playing around with some combinations. For practical implementations, the only time I personally

needed the non standard Gray code was when using a non power of 2 Gray code counter – a topic which was already discussed here.

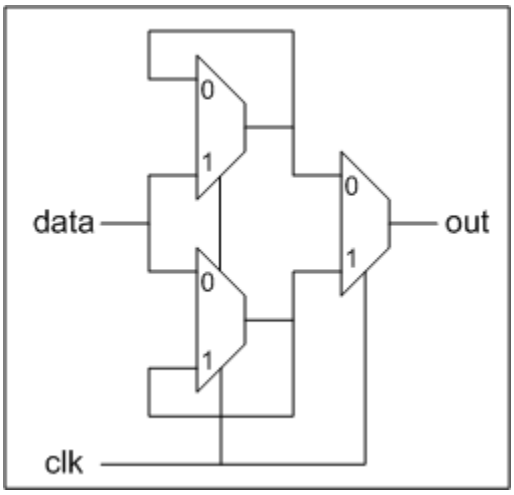
Posted in Gray Codes | Tagged Gray Codes, K-Maps, Recreational Math | 1 Comment »

Another Look at the Dual Edge Flip Flop

September 22, 2008

After writing the solution to one of the puzzles and after contemplating about our dear friend the dual edge flip flop, I noticed something very interesting.

If you look carefully at the implementation of the flip flop which is made out of MUXes, you will see that it is very easy to make a posedge or negedge flip flop by just exchanging the MUX feedback connection.
I wondered if it would be possible to construct a dual edge flip flop with MUXes.
Turns out it is quite possible and requires only one more MUX!



I find the above circuit to be pretty neat because of its symmetry.

Anyways, I wondered if I was the first one to think of this trick; Turns out that… well NO. A short search on the web showed me that someone already invented and wrote a paper about this circuit, check it out here.

I am not aware of any library utilizing this design for a standard cell (if you have different information please comment or send me an email). What is this good for? I guess you could use this neat trick in an ECO, since a lot of times MUX structures are readily available.

Posted in Cool Circuits | Leave a Comment »

On Replication and Wire Length

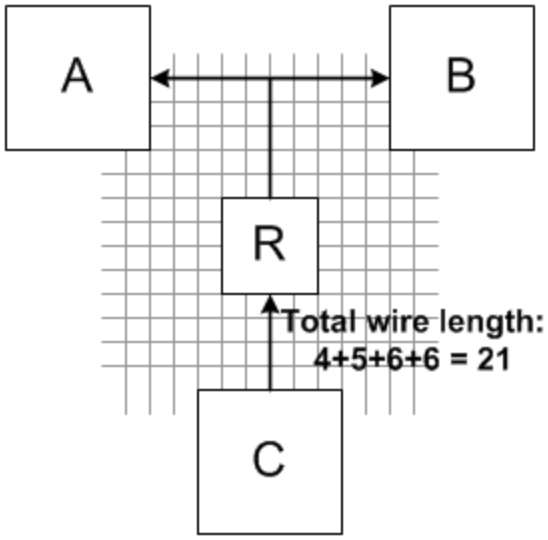
September 12, 2008

It is for some reason a common view, that when using replication you also **have** to pay in increased wire length. It looks reasonable isn't it? After all, you now have more blocks to wire into and out of and therefore total wire length should increase, right? Well, not really…

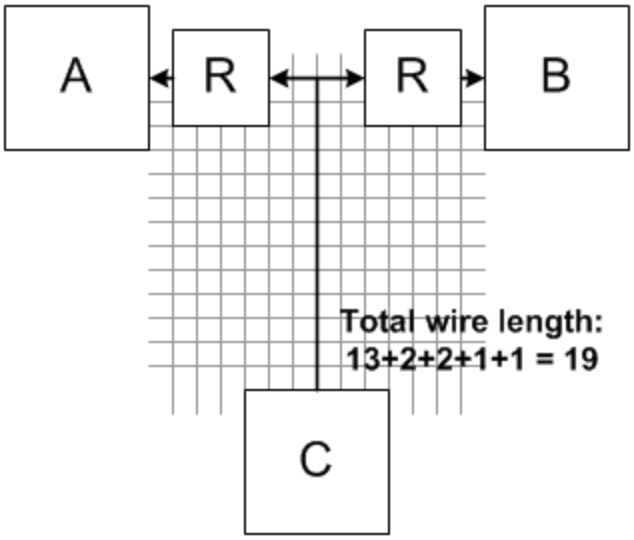
In some cases this might be true, but in most cases wire length should decrease. Wiring in a chip obeys taxicab geometry laws, so it is a bit less intuitive than usual.

Here is a simple example showing how wire length can decrease after replication. Sure, I chose the block placements and the replicated block (R) size to be in my favor, but this is not a rigorous math proof.

Before replication



After replication



Notice how blocks (A) and (B) are now actually farther apart. This leaves more room for other critical logic to be placed in the precious place near the center. On the other hand, after replication we now have one really long wire going out of block (C).

Bottom line: don't be afraid to use replication when you can, it has many advantages and not only for improving timing.

Posted in Layout | Leave a Comment »

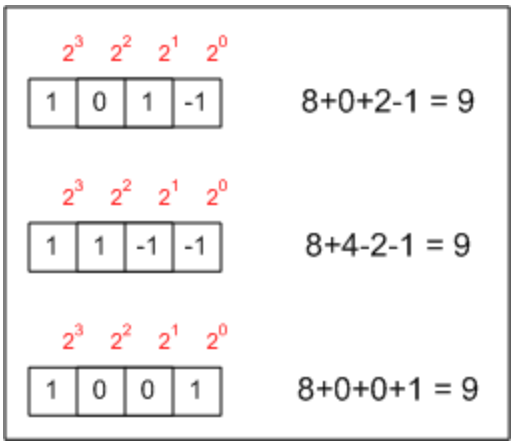
The Signed Digit Redundant Number System

September 5, 2008

Time for a new post on an arithmetical topic.

We all love the good old binary number system and some of us even consider round numbers to be 32, 64, 128, 256, …
Here is another important number system – the signed digit tri-nary number system, which is also a redundant number system. This means that we could have several representations for the very same number.
In the signed digit system we use -1,0,1 instead of just 0,1 for each digit.

It is best explained by an example; The picture below shows three different representations of the number 9 in tri-nary signed digit.



So why do we need to know another number system, and what is it useful for in digital design of ASICs and (especially) FPGAs?

Turns out that using the signed digit number system, one can add without needing to propagate the carry – i.e. in constant time!!!
Hold on your horses, it is not all bright and sunny. The result still needs to be converted back into the good old binary representation and that is **not** done in constant time, but dependent on the width of the number (although there are various techniques that help optimize the process).

If you are doing DSP applications, especially in FPGAs, where digital filter design is involved – using the signed digit number system can come in handy…

Posted in Digital Arithmetic | 7 Comments »

Why You Don't See a Lot of Verilog or VHDL on This Site

August 31, 2008

I get a lot of emails from readers from all over the world. Many want me to help them with their latest design or problem. This is OK, after all this is what this site is all about – giving tips, tricks and helping other designers making their steps through the complex world of ASIC digital design.

Many ask me for solutions directly in Verilog or VHDL. Although this would be pretty simple to give, I try to make sure **NOT** to do so. The reason is that it is my personal belief that thinking of a design in terms of Verilog or VHDL directly is a mistake and leads to poorer designs. I am aware that some readers may differ, but I repeatedly saw this kind of thinking leading to bigger, power hungry and faulty designs.

Moreover, I am in the opinion that for design work it is not necessary to know all the ins and outs of VHDL or Verilog (this is different if you do modeling, especially for a mixed signal project).

Eventually we all have to write code, but if you would look at my code you'd see it is extremely simple. For example, I rarely use any "for" statement and strictly try not using arrays.

Another important point on the subject is for you guys who interview people for a new position in your company. Please don't ask your candidates to write something in VHDL as an interview question. I see and hear about this mistake over and over again. The candidate should know how to **think in hardware** terms; It is of far lesser importance if he can generate some sophisticated code.
If he/she knows what he is aiming for in hardware terms he/she will be a much better designer than a Verilog or VHDL wiz who doesn't know what his code will be synthesized into. This is btw a very typical problem for people who come from CS background and go for design.

Posted in Coding Style, General | 3 Comments »

Max Area = 0 ?

August 24, 2008

You are working on a design, you simulated the thing and it looks promising, first synthesis run also looks clean – jobs done right? wrong!

Many ASIC designers do not care for the area of their blocks. It has to meet the max_transition, max_capacitance and timing requirements but who cares about the area? Well if you are an engineer in soul, you should care.

I completely agree that it is a well accepted strategy not to constrain for area (or max_area = 0) when you first approach synthesis. But this doesn't mean you should ignore the synthesis area reports, even if die size is not an issue in your project.

Not thinking about the area of your design is definitely a bad habit. Given that your transition, capacitance and timing requirements are met you should aim for lower area for your designs. In many cases the tool will meet the timing requirements at the cost of huge logic duplication and parallelism. This is OK for the critical path, but if you could do better than this for the other paths why not just "help" the tool?

For example, try thinking of pre-scaling wide increment logic or pre-decode deep logical clouds with information that might be available a cycle before. This would add some flip-flops but you might find your area decreasing significantly.

There is almost no design that can't be improved, sometimes with a lot of engineering effort, but most designs have a lot of low hanging fruits. In my current project, I was working with one of my best engineers on optimizing some big blocks that were a legacy from another designer. In almost all blocks we were able to reduce the overall size by 30% and in some cases by over 50%!! This was not because the blocks were poorly designed, it is just that the previous designer cared less about area issues.

Bottom line – remember that smaller blocks mean:

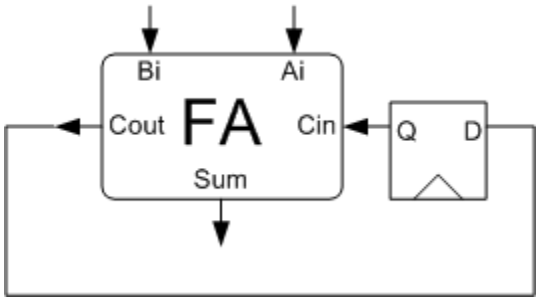
- *Other blocks are located closer*
- *– Shorter wires need to be driven through the chip*
- *– Less hardware*
- *– Lower power*
- *– Are just more neat* 

Posted in Synthesis | 2 Comments »

Arithmetic Tips and Tricks #2 – Another Look at a Slow Adder

August 18, 2008

Do you remember the old serial adder circuit below? A stream of bits comes in (LSB first) on the FA inputs, the present carry-out bit is registered and fed in the next cycle as a carry in. The sum comes in serially on the output (LSB first).



True, it is rather slow – it takes n cycles to add n bits. But hold on, check out the logic depth – one full adder only!! This means the clock can run **a lot** faster than your typical n-bit adder. Moreover, it is by far the smallest, cheapest and consumes the least power of all adders known to mankind.

Of course you gotta have this high speed clock available in the system already, and you still gotta know when to stop adding and to sample your result.
Taking all this into consideration, I am sure this old nugget can still be useful somewhere. If you already used it before, or have an idea, place a comment.

Posted in Architecture, Digital Arithmetic, Low Power | 1 Comment »

Real World Examples #3 – PRBS Look-ahead

August 8, 2008

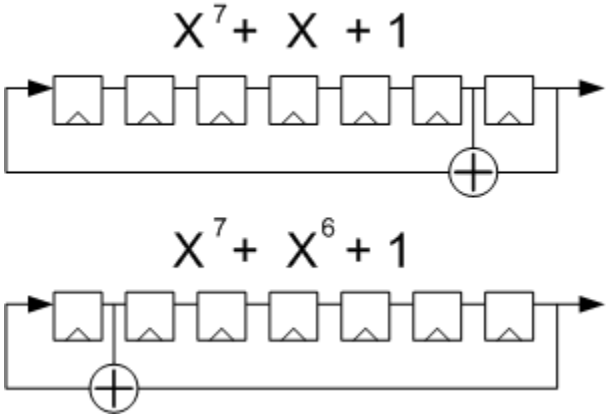
PRBS generation is very useful in many digital design applications as well as in DFT.
I am almost always confused when given a PRBS polynomial and asked to implement it, so I find it handy to visit this site.

This is all nice and well for simple PRBS patterns. In some systems however, the PHY is working in a much higher rate than the digital core (say n times higher). The data is collected in wide buses in the core and then serialized (n:1 serialization) and driven out of the chip by the PHY.

This means that if we do a normal PRBS in the core clock domain, we would not get a real PRBS pattern on the pin of the chip but rather a mixed up version of PRBS with repeating sub-patterns. Best way to see this is to experiment with it on paper.

To get a real PRBS on the pin we must calculate n PRBS steps in each core clock cycle. That is, execute the polynomial, then execute it again on the result and then again, n times.

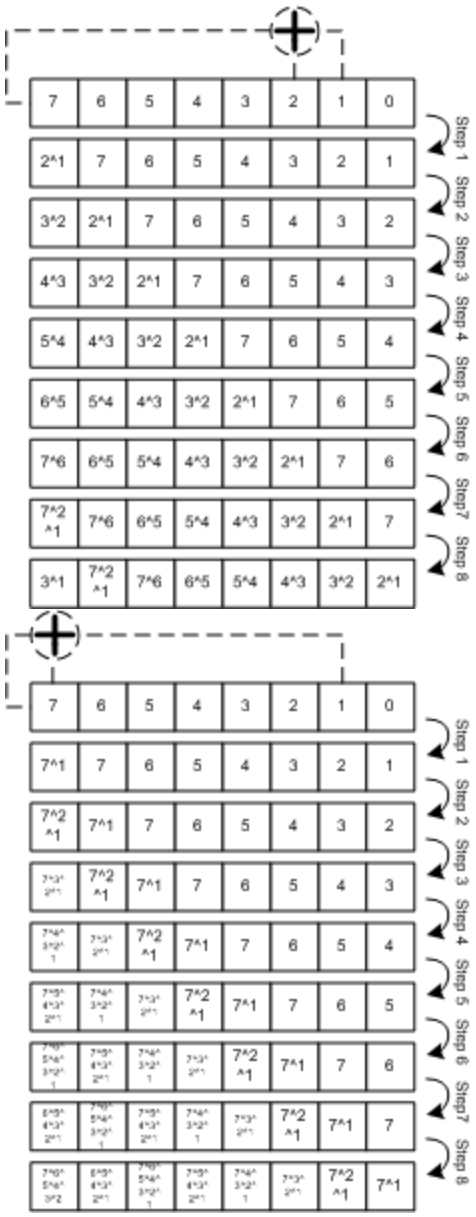
Let me describe a real life example I encountered not so long ago. The core was operating 8 times slower than the PHY and there was a requirement for a maximum length PRBS7 to be implemented.
There are a few maximum length polynomials for a PRBS7, here are two of them:



Both of these will generate a maximum length sequence of 127 different states. We now have to format it into 8 registers and hand it over to the PHY on each clock, But which of the two should we use? is there a speed/power/area advantage of one over the other? does it really matter?

Well, if you do a PRBS look-ahead, which is approximately the same order as your PRBS polynomial, then it really does matter. In our case we have to do a 8 look-ahead for a PRBS 7.

Compare the implementations of both polynomials below. For convenience both diagrams show the 8 intermediate steps needed for calculating the 8 look-ahead. In the circuit itself only the final result (the contents of the boxes in step 8) is used.



Because the XOR gate of the second polynomial is placed more close to where we have to shift in the new calculation of the PRBS, the amount of XORs (already too small in the second image to even notice) accumulate with each step. For the final step we have to use an XOR tree that basically XORs 7 of the 8 original bits – this is more in amount than the first implementation (even if you reuse some of the XORs in the logic) and the logic itself is deeper and thus the circuit becomes slower compared to the other implementation.

The first implementation requires at most a 3 input XOR for the calculation of look-ahead bit6 but the rest require only 2 input XOR gates.

Bottom line, if you do a PRBS look-ahead and have the possibility to choose a polynomial, choose one with lower exponents.

Posted in Real World Examples | 2 Comments »

This site is a "T-log"

August 1, 2008

I thought about it for a while and I would humbly like to introduce a new word to the English language – the word is ***T-log*** (pronounced tee-log) – a short for **Technical blog**. I am by the way aware that there are other uses for the acronym TLOG.

So why do I make such a big fuss about using the word "T-log" and why do I consider this site (and some others) a T-log rather than a blog?
Well, the main reason is that surfing the web for technical related blogs, you will find a lot of informative sites which deal with opinions (e.g "behind the scenes" issues, industry news or just giving their opinions on this or that topic), but the pure technical content is not there. This is actually great and this is what makes these blogs interesting to read.
However, this site is not like that. I try to give almost only technical information in the form of digital design techniques and to contribute from my own personal experience (maybe because I am too dry and can't generate interesting posts as other bloggers do).

The bottom line is that I prefer this site not to be called a blog but rather something else – so I experiment with coining the word tlog – who knows maybe this will catch up and we will soon see a wikipedia entry… just don't forget where you saw this first…

To wrap things up, let me recommend a cool t-log making its first steps on the web. It is run by a regular reader of this t-log who decided he also has something to say – check him out here: ASIC digital arithmetic.

Posted in General | 1 Comment »

Puzzle #13 – A Google Interview Question

July 25, 2008

The following puzzle was given to me by a friend who claimed it was given in a Google interview. If you can confirm or debunk this claim just post a comment – until then I am sure the headline will generate some traffic 😀.

The original question as it was given to me was:
Given an array with 2n+1 integer elements, n elements appear twice in arbitrary places in the array and a single integer appears only once somewhere inside. Find the lonely integer with O(n) operations and O(1) extra memory.

Now let's transform this to a more digital-design-like problem. Given an SRAM of depth N and some arbitrary width K, which is filled with 2n+1 non zero values (for completeness – the rest of the 2^N – (2n+1) are all zeroes). n elements appear twice – in different places in the SRAM, while a single value appears only once.

Design a circuit with the minimum amount of hardware to find the value which appears only once.

Posted in Interview Questions, Puzzles | 22 Comments »

Polymorphic Circuits

July 14, 2008

Here is a neat and not so new idea that I came across last year – **"Polymorphic Circuits"**. The basic concept is logic gates which under specific operating conditions behave in a certain way while under different operating conditions behave in another way. For example a circuit when operated with a 2 volt supply might act as an OR gate but when supplied with 1 volt will become an AND gate, or another example might be a circuit which in room temperature behaves as an XOR gate while at 120 degrees, the very same circuit operates as a NAND gate.

This concept just screams for new application (I guess mainly in security) but I was not able to think of something specific so far. Feel free to shoot ideas around in the comments section of this post.

In the meantime more details can be found on this paper (just skip to the images at the end of the paper to get the idea), or this paper.

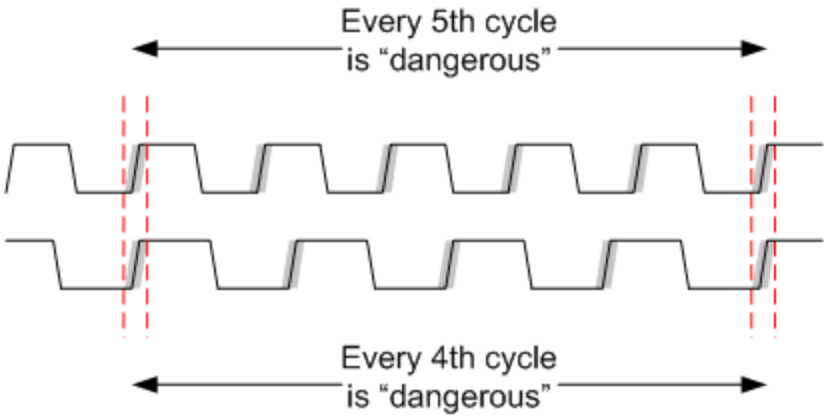
Posted in Cool Circuits | Tagged Cool Circuits, polymorphic | 3 Comments »

Predictive Synchronizers

July 7, 2008

As we discussed many times before, synchronization of signals involves also latency issues. Sometimes these latency issues are quite a mess. This post will go over the principle of operation of predictive synchronizers, which offer a specific solution for a very specific case.

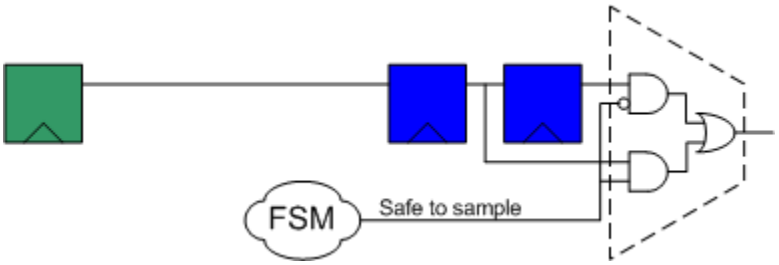
Let's start by describing the conditions for this specific case. For the sake of explanation let us assume we have two clock domains with different clock periods. On top we have a certain limited or capped jitter component defined by our spec.
Taking the conservative approach, we would always use a full two flip flop synchronizer. However, a closer look at a typical waveform reveals something interesting.



The figure above shows both clocks. The limited jitter as defined by our spec, is shown in gray. Notice how only during specific periods a full synchronizer needs to be used. For the upper clock each 5th cycle is a “dangerous” one, while for the lower clock each 4th is problematic. The time window in which these danger zones occur is **predictable**.

In general we could count the clock cycles, and then, when the *next* clock edge occurs in the “danger zone” we could switch and use a full synchronizer circuit, otherwise a single flop is enough.

A circuit which implements this idea can be seen below. The potentially metastable node is blocked by the FSM during the “danger time” and the synchronizer output is taken, otherwise the normal, first flop’s output, is taken. The logic at the output is basically that of a MUX.



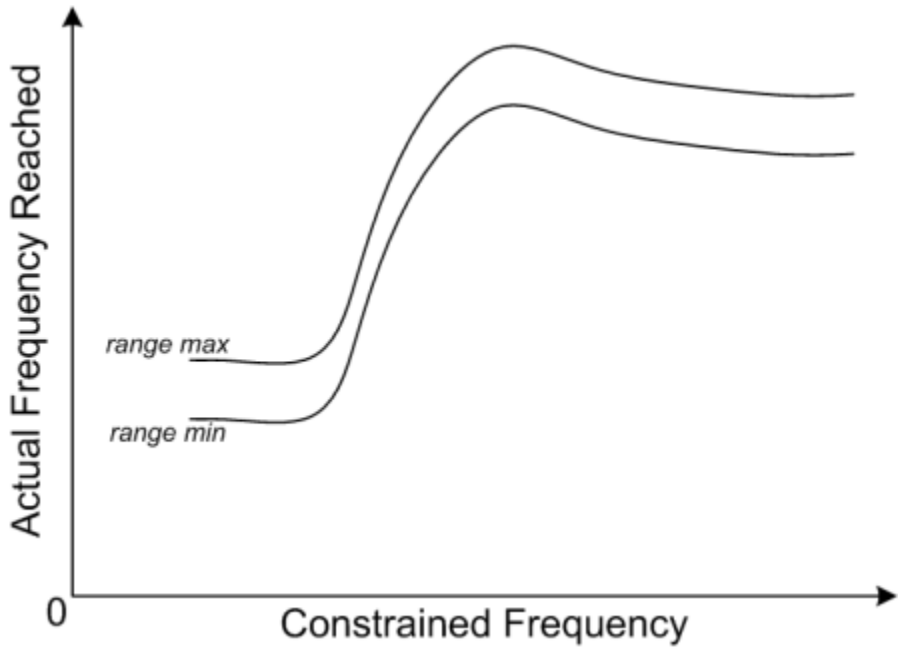
Posted in Architecture | Tagged Latency, Synchronizer | 5 Comments »

Why Not Just Over-Constrain My Design?

June 25, 2008

This is a question often raised by beginners when trying to squeeze performance from their designs.
So, why over-constraining a design does not necessarily improve performance. The truth is that I don’t *really* know. I assume it is connected to some internal variables and measuring algorithms inside the synthesis tool and the fact that they give up trying to improve the performance because they reached a certain local minimum in some n-variable space (really!).

But empirically, I (and many others) have found out that you can not get the best performance by just over-constraining your design in an unrealistic manner. It has to be somehow closely related to the actual maximum speed that can be reached. The graph below sums up this problem pretty neatly.



As seen above, there is a certain min-max range for the performance frequency that can be reached and its peak is **not** the result of the highest frequency constrained!
The flat region on the left of the figure is the speed reached without any optimization, that is, right after mapping your HDL into gates. As we move towards the right, we see actual speed improvement as we constrain for higher speeds. Then a peak is reached and constraining for higher speeds results in poorer performance.

I worked relatively less with FPGAs in my carrier but I have seen this phenomenon there as well. Take it to your attention.

Posted in Synthesis | Tagged constrains | 10 Comments »

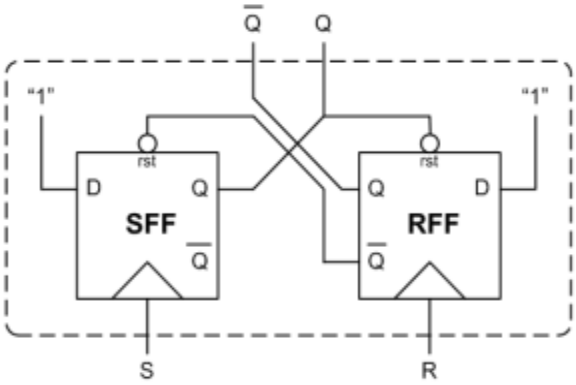
Edge Triggered SR Latch

June 18, 2008

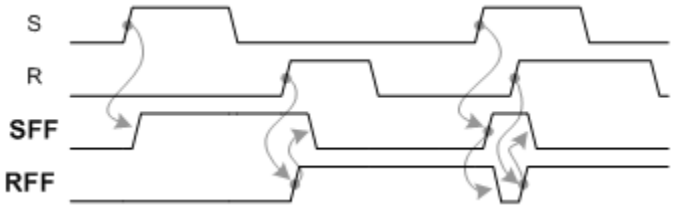
I never really used an edge triggered SR latch in any of my circuits before, but I dug this from my bag of circuit goodies and it is just too nice not to share (does it show that I am designing circuits for too long?)

The basic idea is to use two regular run-of-the-mill flip flops and combine them to a single “SR latch like” construction which is **edge** triggered.

The circuit is displayed below, and I just can't help it – admiring a circuit with some sort of cross coupling…



And a corresponding timing diagram:



Posted in Cool Circuits | Tagged Cool Circuits | 1 Comment »

Non-Power-of-2 Gray Counter Design

June 9, 2008

So… you want to design a counter with a cycle which is different than a power of 2. You would like to use a Gray counter because of its advantages and just because it is simply beautiful, but alas, your cycle length is not a power of two – what to do?
This post will try to give you a sort of recipe of how to design such a non-power-of-2 Gray counter and the reasoning behind.

First, if your cycle length is an odd number, you are in trouble since this is just not possible to construct a counter with the Gray properties and with an odd cycle length. A simple way to see why it is so, is to notice that a Gray counter changes its parity with each count because only one bit changes at a time.
This naturally means that the parity toggles, but since we have an odd number of states and if we started with even parity – the last state will also have odd parity, and when we wrap around the parity won't change! Assuming that the first and last states are different, this means that 2 bits must change at a time, thus contradicting the Gray hypothesis.

OK, so we limited ourselves to an even amount of states, is it possible now? It is! We could ask our friend Google and come up with some info and even some patents, but the best discussion on the subject that I found was written by Clive Maxfield here.

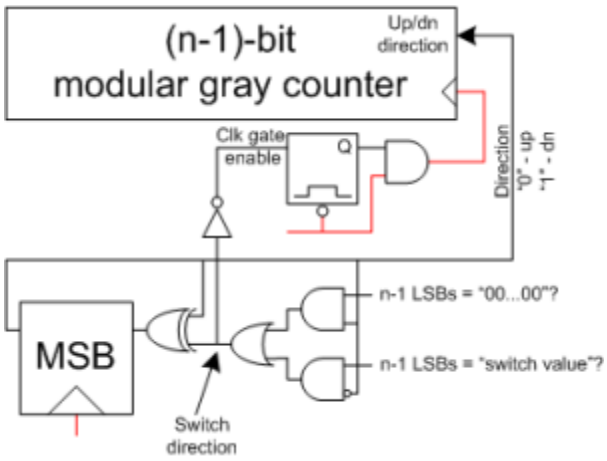
When approaching this problem, what (hopefully) should immediately struck us, is that we have to somehow use the reflection property of the Gray code (This method among others is discussed by Clive as well). Let's take a deeper look at the 4-bit Gray code below.



The pairs of states which have identical distance from the axis of reflection are only different by their MSB. This in turn, means that we could eliminate pairs-at-a-time around the axis of reflection, and arrive to our desired number of states for the counter. Moreover, we notice that the (n-1) LSBs count up to a certain value then change direction and count down again. This property remains true even if we remove any amount of pairs around the axis of reflection.

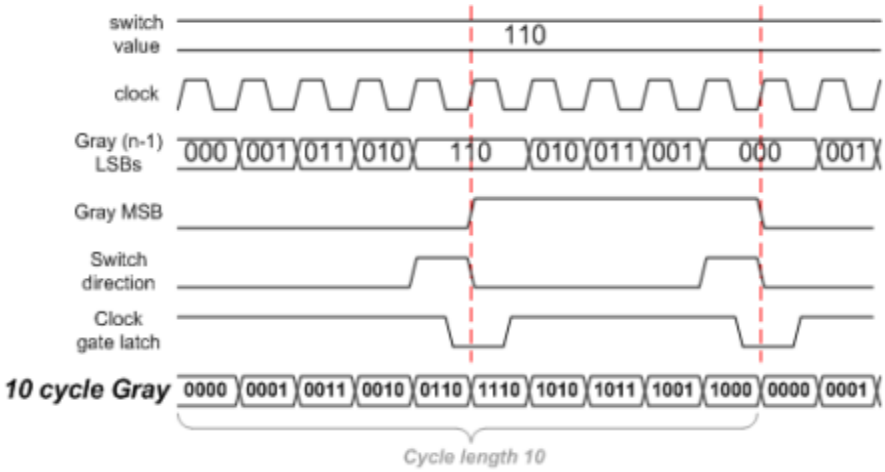
What we have to do now, is to find this "switching value", when we reach it on the up-count, toggle the direction bit – **which is also our MSB**, and block the (n-1) LSBs Gray counter for this direction switch cycle (otherwise 2 bits would change). We now count down to the initial state (all zeros). When we reach it, we again have to switch direction and block the counter and so on ad infinitum.

We can use the modular up/down Gray counter I described here, here and here. for our (n-1) LSBs. We have to find a priori the "switching value", which is the (n-1) bit Gray value of our number of counter states divided by 2. For Example, if you want a 10 state Gray counter then: 10/2 = 5, therefore we need the 5th Gray value of a normal 3 bit Gray code, which turns out to be 110
The rest of the circuit is depicted in the figure below:



It is important to see that we use the minimal possible memory elements required for the Gray counter (i.e. no extra states to remember or pipeline) and that during "direction switching" we gate the clock for the (n-1) LSBs up/down Gray counter using an ordinary clock gate construct. If we look carefully we see that the "direction switching" logic is basically a mux structure with the select being the direction bit.

A timing diagram of the above circuit for a 10 state Gray counter is also depicted below for clarity.



Posted in Architecture, Gray Codes | 2 Comments »

Puzzle #12 – Count and Add in Base -2

June 3, 2008

It has really been a long time since a new puzzle appeared on the blog.
This one is a neat little puzzle that was pretty popular as an interview question. I tried to expand on it a bit, so let's see where this goes.

The basic idea is: can you count in base -2? There is no typo here – it is **minus 2**. So far the original puzzle, now my small contribution…
Once you realize how to do this, try to build a logical circuit that performs addition *directly* (i.e. no conversions) in base -2.

Good luck!

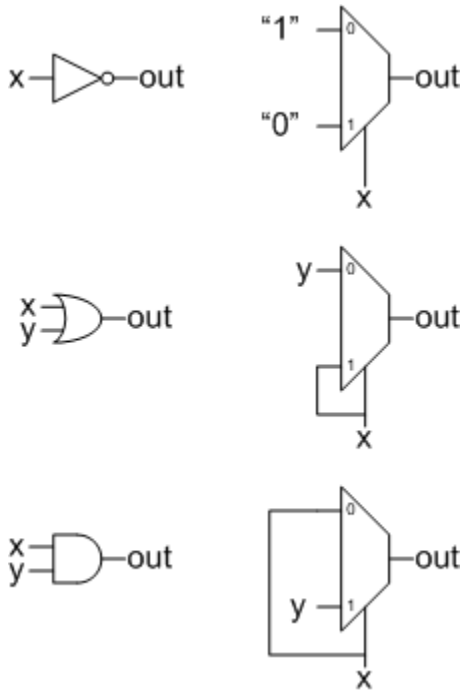
Posted in Interview Questions, Puzzles | Tagged Puzzles | 17 Comments »

Puzzle #10 – Mux Logic – Solution

May 29, 2008

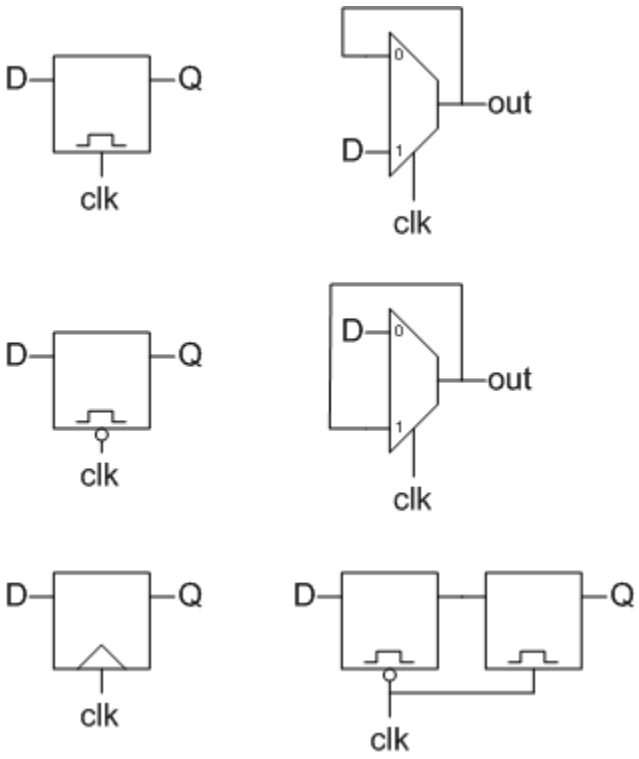
Puzzle #10 – Mux Logic, still didn't get an official solution so here goes.
If you are not familiar with the puzzle itself, as usual I ask you to follow the link and reread its description.

To solve this puzzle let's first take a look at the combinational parts of the circuits. If we could build an OR gate and a NOT gate from MUXes it would be enough to make any combinational circuit we wish (this is because OR and NOT are a complete logic system, same as AND and NOT, or just NOR or NAND).
The figure below shows how to build NOT, OR and AND gates from a single MUX.



Next in line we have to somehow build the flipflop in the circuit. We could build a latch from a single MUX quite easily if we feedback the output to one of the MUX inputs. The figure below

will make everything clearer. Notice that we could easily construct a latch which is transparent while its clock input is high or low by just changing the input the feedback wire is connected to. We then use two latches, one transparent low the other transparent high to construct a flipflop.



As a final note, some use the versatility of the MUX structure to their advantage by spreading MUX structures as spare cells. Later if an ECO is needed one can build combinational as well as sequential elements just from those single MUX structures.

Posted in Interview Questions, Puzzles | Tagged MUX, Puzzle | 2 Comments »

Low Power Methodology Manual

May 24, 2008

I recently got an email from Synopsys. It was telling me I can download a personalized copy of a "Low Power Methodology Manual". Now, to tell you the truth, I sometimes get overly suspicious of those emails (not necessarily from Synopsys) and I didn't really expect to find real value in the manual – **boy, was I wrong, big time!**

Here you get a very nice book (as pdf file), which has extremely practical advice. It does not just spend ink on vague concepts – you get excellent explanations with examples. And mind you, this is all for **free**.

Just rush to their site and download this excellent reference book that should be on the table of each digital designer.

The Synopsys link here.

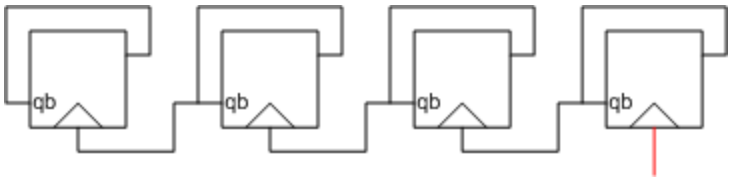
The hard copy version can be bought here or here.

Posted in General, Low Power | 7 Comments »

Replace Your Ripple Counters

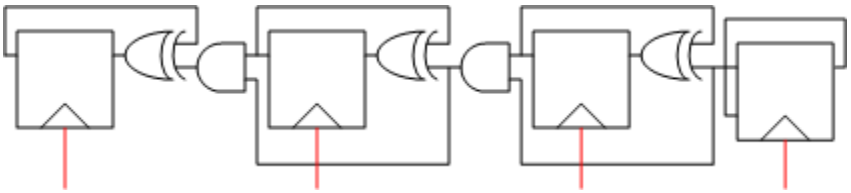
May 23, 2008

I was recently talking to some friends, and they mentioned some problems they encountered after tape out. Turns out that, the suspicious part of the design was done full custom and the designers thought it would be best to save some power and area and use asynchronous ripple counters like the one pictured below. The problem was that those counters were later fed into a semi-custom block – the rest is history.



Asynchronous ripple counters are nice and great but you really have to be careful with them. They are asynchronous because not all bits change at the same time. For the MSB to change the signal has to ripple through all the bits to its right, changing them first. The nice thing about them is that they are cheap in area and power. This is why they are so attractive in fast designs, but this is also why they are very dangerous because the ripple time through the counter can approach the order of magnitude of the clock period. This means that a digital circuit that depends on the asynchronous ripple counter as an input might violate the setup-hold window of the capturing flop behind it. To sum up, just because it is coming from a flop doesn't mean it has to be synchronous.

If you can, even if you are a full custom designer, I strongly recommend replacing your ripple counters with the following almost identical circuit.



It is based on T-flops (toggle flip flops are just normal flops with an XOR of the current state and the input, which is also called the toggle signal) and from the principle of operation is almost the

same, although here instead of generating the clock edge for the next stage, we generate a toggle signal when all previous (LSB) are “1″. Notice that the counter is synchronous since the clock signal (marked red) arrives simultaneously to all flops.

Posted in Architecture, Cool Circuits | Tagged Counters, synchronous design | Leave a Comment »

Latch Based Design Robustness

May 19, 2008

Latch based design is usually not given enough attention by digital designers. There are many reasons for that, some are very well based, other reasons are just because latch based design is just unknown and looked at as a strange beast.

I intend to have a serious of posts concerning latch based design issues. I have to admit that most of my design experience was gained doing flip flop based designs, but the latch based designs I did were always interesting and challenging. If any of you readers have some interesting latch based design examples please send them over to my email and I will include them in later posts.

Just to start the ball rolling, here is a very interesting paper on the robustness of latch based design with comparison to flip flop based design. If you don’t have the time to go through the entire paper just look at figure 1 and its description.

I also added this paper to the list of recommended reading list.

Posted in Architecture, Latch-based Design | Tagged Architecture, Latch, Robustness | 3 Comments »

Happy Birthday!

May 16, 2008

Wow, time goes by so fast. I just noticed that this blog celebrates its first birthday! Exactly one year ago it went “on the air”.
If you liked the posts, found them helpful or just have some nice things to say – just comment on this post.

Thanks,

Nir

Posted in General | 10 Comments »

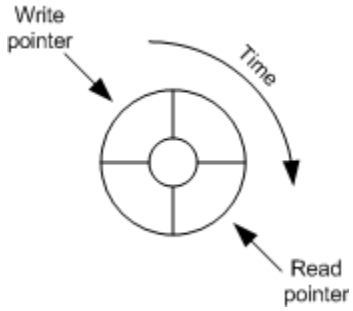
Ring Buffers

May 14, 2008

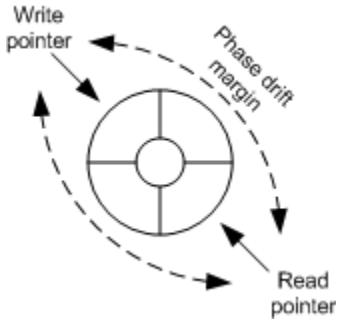
On the last technical post I discussed the problem of transferring information serially between two different clock domains with similar frequency but with drifting phase.
This post will try to explain how this issue is being solved.

When approaching this problem, we have to remember that the phase might drift over time and we have to quantify this drift before the design starts. Modeling the channel beforehand is very helpful here.
Once we know the needed margin, we can approach the design of the ring buffer.

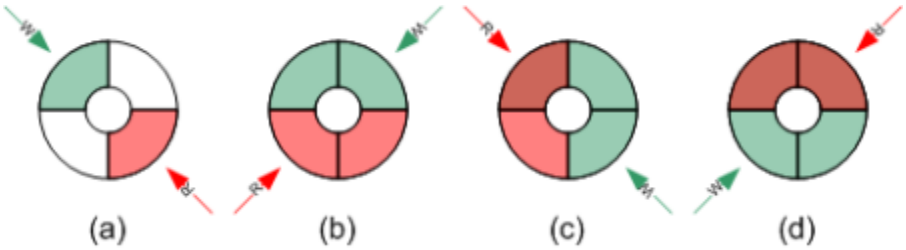
The ring buffer is a FIFO with both ends "tied together" as depicted below. Pointers designate the read and write position and are moved with each respected clock signal in the direction of the arrow (in the figure below – clockwise). Remember, the read and write pointers move at different times but the overall rate of change of both is similar. This means that in some moment one can move ahead of the other, and in another it can lag behind, but over time the the amount of clock edges is the same.



The tolerance of the ring buffer is represented below with the dashed arrows. The read and write clocks can drift in time up to a point just before they meet and cross each other.



The series of images below depicts how the read and write pointers move with time and how the buffer is filled with new information (green) and how it is read (red). Notice how the first two reads will generate garbage because it reads out information that was not written into the buffer.



One of the most complicated issues is the start-up of the ring buffer, because both clock domains are unrelated. A certain “start” signal has to be generated and “tell” both pointers to start to advance. If this is not done carefully enough, one pointer will start to advance ahead of time and thus “bite away” some of the margin we designed for. This problem is even more complicated, when a lot of channels with different ring buffers are operated in parallel.

In one of the next posts we will explore a simple technique that enables us to determine if the ring buffer failed and the information read is actually one which is not updated.

Posted in Architecture | Tagged Ring Buffers | 1 Comment »

Long Break…

May 10, 2008

Due to a long and severe sickness I was unable to update the blog for such a long time.
I promise to catch up soon, with more interesting posts – hold on…

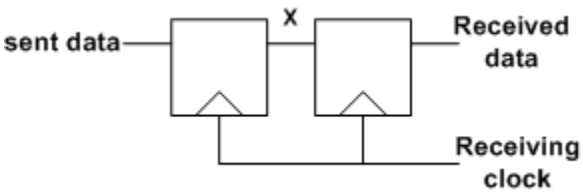
Posted in General | 1 Comment »

Clock Domain Crossing – An Important Problem

April 21, 2008

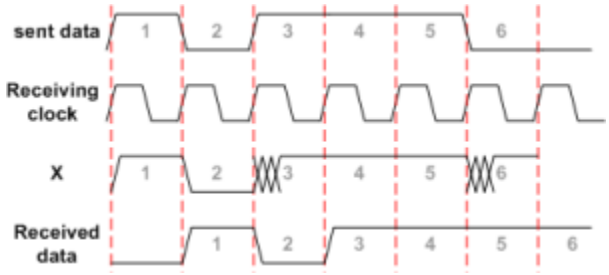
Sometimes, when crossing clock domains, synchronizers are just not enough.

Imagine sending data serially over a single line and receiving it on the other side from the output of a common synchronizer as shown bellow.

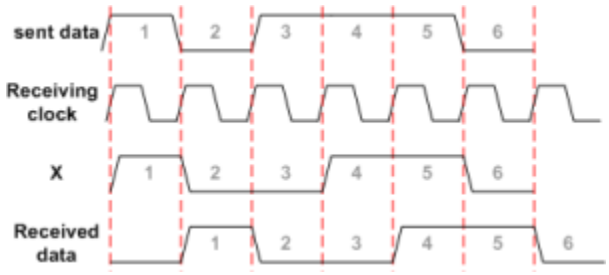


Assuming one clock cycle is enough to recover from metastability under the given operating conditions, what seems to be the main problem is not the integrity of the signal – i.e. making sure it is not propagating metastability through the rest of the circuit – but rather the correctness of the data.

Let's observe the waveform below. The red vertical lines represent the sampling point of the incoming signal. We see from the waveform that since sometimes we sample during a transition – in effect violating the setup-hold window – the output of the first sampling flop (marked "**x**") goes metastable. This metastability does not propagate further into the circuit, it is effectively blocked by the second flop, but since the result of recovery from metastability is not certain (see previous post) the outcome might be a corrupt data.
In this specific example we see that net **x** goes metastable after sampling the 3rd bit but recovers correctly. In a later sampling, for the 6th bit we see that the recovered outcome is not correct and as a result the output data is wrong.



Another interesting case is when both the send clock and the receive clock are frequency locked but their phase might drift in time or the clock signals might experience occasional jitter.
In that case, a bit might "stretch" or "shrink" and can be accidentally sampled twice or not sampled at all.
The waveform below demonstrates the problem. Notice how bit 2, was stretched and sampled twice.



To sum up, never use a simple synchronizer structure to transfer information serially between clock domains, even if they are frequency locked. You might be in more trouble than you initially thought.

On the next post we will discuss how to solve this problem with ring buffers (sometimes mistakenly called FIFOs).

Posted in Architecture | Tagged Clock Domain Crossing, Synchronization | 3 Comments »

Another FSM Design Tool

April 17, 2008

For those who don't read through the comments. Harry the ASIC guy commented on the last post about an FSM design environment from Paul Zimmer. You can find more details here.

Posted in General | Tagged CAD | 2 Comments »

Visual FSM Design Tool

April 9, 2008

I am still not convinced visual FSM design tools make such a big difference but this one looks pretty cool.
I haven't really went through all the features and details, so if anyone has some more details/recommendations/complaints about it, just email me or simply comment on this post.

Posted in CAD | Tagged CAD, FSM | 1 Comment »

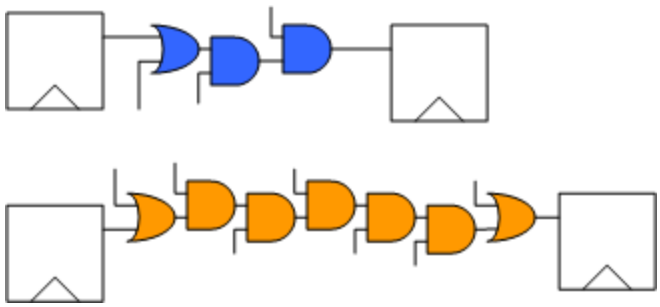
The Principle Behind Multi-Vdd Designs

April 2, 2008

Multi-Vdd design is a sort of buzz word lately. There are still many issues involved before it could become a real accepted and supported design methodology, but I wanted to write a few words on the principle behind the multi-Vdd approach.

The basic idea is that by lowering the operating voltage of a logic gate we naturally also cut the power dissipation through the gate.
The price we pay is that gates operated by lower voltage are somewhat slower (exact factor is dependent on many factors).

The basic idea is to identify the non-critical paths and to power the gates in those paths with a lower voltage. Seen below are two paths, there is obviously less logic through the blue path than through the orange one and is therefore a candidate for being supplied with lower Vdd.



The idea looks elegant but as always the devil is in the details. There are routing overheads for the different power grids, level shifters must be introduced when two different Vdd logic paths converge to create a new logical function, new power source for the new Vdd must be designed and most important of all, there has to be *support present by the CAD tools* – if that doesn't exist this technique will be buried.

Posted in Low Power | Tagged CAD, Emerging Methodologies, Low Power | 3 Comments »

Puzzle #9 – The Snail – Solution

March 27, 2008

I will keep this post short. First make sure you take a look at the original puzzle – link here.

The shortest sequence is 6 bits long and is "100110″ (or its inverse "011001″). The smallest amount of bits needed to determine a direction is 5, i.e. any 5 consecutive bits seen by the snail could help him determine the direction home.

Posted in Interview Questions, Puzzles | Tagged Interview Questions, Puzzles | 6 Comments »

Puzzle #8 – Clock Frequency Driver – Solution

March 26, 2008

It's been a while since I posted some puzzles or solutions to puzzles. I noticed that I concentrated lately more on tricky circuits and fancy ideas but neglected the puzzle section. Some readers asked me to post some more puzzles. Before I can do that, I have to first clear the list of all unsolved puzzles.

The clock frequency driver puzzle drew little attention compared to the others and I got only one complete and correct solution for it.
What follows is my own solution which I hope will be easily understood.

The requirement was to have a NAND gate as the last output stage with one input driven by a rising edge triggered memory element and the other by a falling edge triggered memory element.

A look at the NAND gate truth table reveals that somehow the inputs have to toggle between "11″ (to generate a logical "0″) and "10″, "00″ or "01″ (to generate the logical "1″) **on each and every clock edge!**

I will now describe the solution for a certain case while the value in brackets will represent the analogous opposite case.
This in tern means (and without loss of generality) that with each rising[falling] clock edge the output state of both flops should be "11″. On the falling[rising] edge we should have the states "00″, "01″ or "10″.

The state "00″ can be immediately eliminated because the transition "00″ –> "11″ means we have to have both bits change on the rising[falling] edge together.
we are left with the following possible cases for the transitions ("r" marks a rising edge transition, "f" a falling edge transition):

1. "10″ r–> "11″ f–> "10″
2. "10″ r–> "11″ f–> "01″

Looking at the first option reveals that the rightmost bit needs to change on the rising edge from "0″ to "1″ and on the falling edge from "1″ to "0″ – this is not possible or in contradiction to the rules.
The second option looks promising – the rightmost bit changes from "0″ to "1″ on the rising edge, the left most from "1″ to "0″ on the falling edge – so far so good… but, let us continue the pattern:

"10″ r–> "11″ f–> "01″ r–> "11″

Each second state **has** to be "11″. After continuing the sequence for one more step we see that now the rightmost bit changes from "0″ to "1″ on the **rising** edge, but the immediate previous transition had it change on the **falling** edge, therefore we get again a contradiction!

We conclude that having a NAND on the output is impossible.

As mentioned before *Mark Wachsler* sent his own solution long time ago. Here it is in is own words:

I'm assuming the question is, is it possible to do something like this:

```
always @ (posedge clock) p <= something;
always @ (negedge clock) n <= something else;
assign out = ~ (p & n);
```

and have out toggle on every transition of the clock?

If so, the answer is no.

Proof by contradiction:
1. Assume it can be done: out toggles on every transition of the clock.
2. We know p and n never change simultaneously, so for out to toggle, either p or n must be 1.
3. So it may never be the case that p == 0 and n == 0.
3. Since they can't both be zero, and they never change simultaneously, at least one of them must always be 1.
4. But if n is always one, out can't have a transition on negedge. And if p is always one, out can't have a transition on posedge.
5. Therefore there are some clock edges on which out doesn't toggle.

So it can't be done.

Posted in Puzzles | Leave a Comment »

Cyclic Combinational Circuits

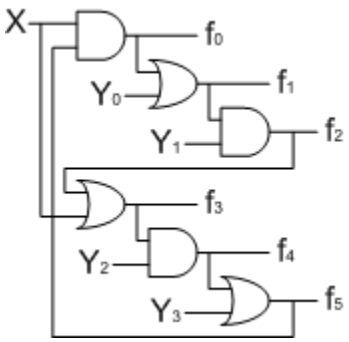
March 7, 2008

As one of my strange hobbies, I sometimes try to search the web for interesting PHD thesis works. I came across this one a while back and thought it would be interesting to share.

We always hear how bad combinational cyclic loops are. Design Compiler even generates a reports to help us detect them. In the normal ASIC flow combinational loops are very dangerous, hard to analyze and characterize for timing. But here comes this Dissertation work by Marc Riedel and highlights a special set of cyclic combinational circuits which offer several important advantages.

I will try to explain the basic principle by going through an example, but make sure to read his PHD thesis, it is well written and easily understood.

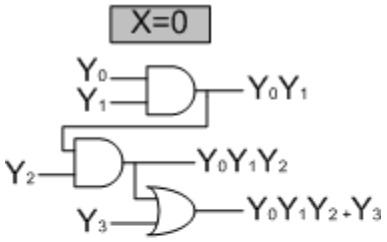
As an example we will look at the very simple case depicted below:



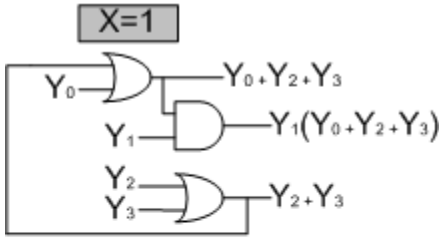
Notice that it has 5 inputs: X, Y0, Y1, Y2, Y3 and has 6 outputs f0..f5. Notice also the symmetry or duality between the AND/OR gates which have the X input connected into them. The basic

principle being, that if X = "0″ the cycle will be broken at the top AND gate and if X = "1″, the cycle would be broken in the middle OR gate. This in turn will "create" two "different" circuits depending on the value of X. In essence we have physically a combinational loop BUT we guarantee that whatever value X has, this loop will be **logically** broken!

Both cases are shown below.



$f_0 = 0$
$f_1 = Y_0$
$f_2 = Y_0Y_1$
$f_3 = Y_0Y_1$
$f_4 = Y_0Y_1Y_2$
$f_5 = Y_0Y_1Y_2 + Y_3$



$f_0 = Y_2 + Y_3$
$f_1 = Y_0 + Y_2 + Y_3$
$f_2 = Y_1(Y_0 + Y_2 + Y_3)$
$f_3 = 1$
$f_4 = Y_2$
$f_5 = Y_2 + Y_3$

If we factor in X into the equations we get the following dependencies for all the outputs on all the inputs.

$f_0 = X(Y_2 + Y_3)$
$f_1 = Y_0 + X(Y_2 + Y_3)$
$f_2 = Y_1(Y_0 + X(Y_2 + Y_3))$
$f_3 = X + Y_0Y_1$
$f_4 = Y_2(X + Y_0Y_1)$
$f_5 = Y_2(X + Y_0Y_1) + Y_3$

The above example is one of the simplest of all and was just presented to show the principle. In this specific circuit you could also short Y0 and Y2, Y1 and Y3 and get a 3 input circuit where each of the inputs has the same behavior as X in the example (shown in page 12 in the PDF file of the thesis).

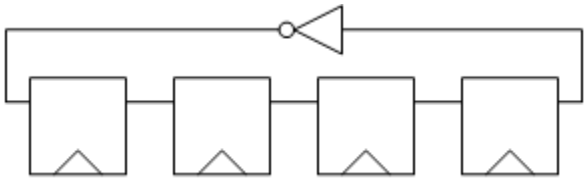
The thesis goes on to show how such circuits can be used with different advantages. The thesis is bears the date May 2004 – I hope that significant advances have been made in this area in the last 4 years. This idea is too beautiful to just let it accumulate dust or being discarded by the CAD industry…

Posted in Cool Circuits | 2 Comments »

Johnson Counter Recovery Circuits

February 25, 2008

In a previous post I discussed the Johnson counter (diagram below). It was mentioned that if a bit accidentally flips in the wrong place in the counter (due to wrong reset behavior, noise, etc.) it will rotate through the counter indefinitely.



In a robust Johnson counter there is a mechanism for self-correction of these errors. This post discussed in detail how to resolve such single bit errors with minimum hardware overhead.

Let's assume that for some odd reason *within* a run of "1″s or "0″s a single bit had flipped. If we now take a snapshot of 3 consecutive bits in the counter as the bits are rotating, we will eventually discover two **forbidden** states: "010″ and "101″. All the other six possible states for 3 consecutive bits are legal – as seen in the table below:

state	-/X
000	-
001	-
010	X
011	-
100	-
101	X
110	-
111	-

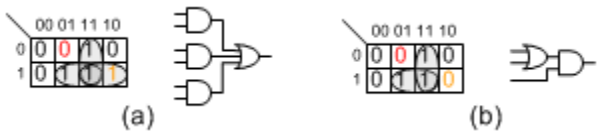
The basic idea, is to try to identify those rogue states and fix them by flipping the middle, erroneous bit and pushing the result to the next state. Naturally, we have to make sure that we keep the normal behavior of the circuit as well.
We will examine two solutions (a) and (b). One more efficient (hardware wise) than the other.

Let's start with approach (a). With this approach we try to correct **both** forbidden state. The table bellow shows a snapshot of 3 consecutive bits in the "state" column. One is marked in red the other in orange. The column "next(a)" contains the value to be shifted into the 3rd bit – e.g. if "011″ is encountered then the middle "1″ will be pushed unchanged to the bit to its right, however if the state "010″ will be detected, the middle "1″ will be flipped to a "0″ and pushed into the right thus correcting a forbidden state.

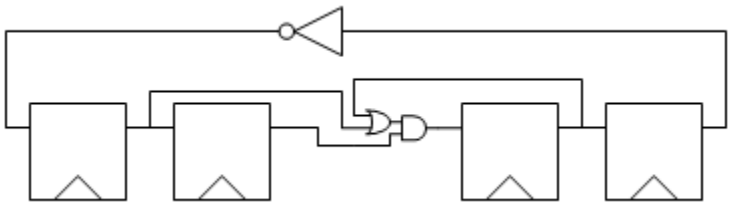
state	-/X	$next_{(a)}$	$next_{(b)}$
000	-	0	0
001	-	0	0
010	X	0	0
011	-	1	1
100	-	0	0
101	X	1	0
110	-	1	1
111	-	1	1

The second approach (b) corrects only *the single* forbidden state "010″. Then how come this solves the problem? Approach (b) relies on the fact that state "010″ is the inverse state of "101″. It is enough to correct state "010″ since state "101″ will reach the end of the counter, then will be flipped bit by bit and will eventually appear as "010″ in the next cycle through the counter!

The next diagram shows the different hardware implementation for both solutions. While I can be blamed of being petty, solution (b) is definitely cheaper.



The final, self-correcting 4-bit Johnson counter is shown below.



It is important to note that this circuit recovers from a single bit error. If we had a 7-bit Johnson counter and 2 adjacent bits would flip in a middle of a run (unlikely but still possible), we would

not detect it with the above circuit. For correcting 2 adjacent flips a wider "snapshot" of 4-bits is needed, and the circuit will naturally become more complex.

It is considered good design practice to have at least a single bit self-correcting circuit, as the one above, for each Johnson counter being used.

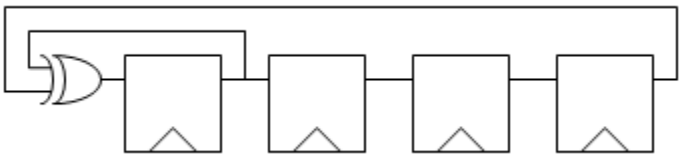
Posted in Cool Circuits | 6 Comments »

De Bruijn and Maximum Length LFSR Sequences

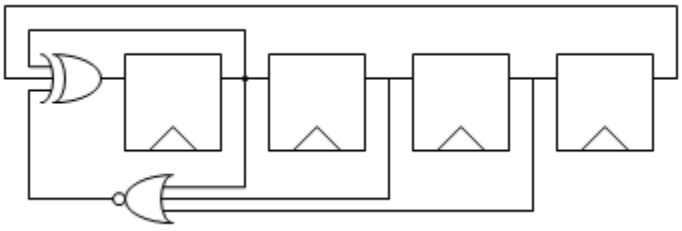
February 15, 2008

In the previous post I mentioned that a maximum length LFSR can be modified quite easily to generate the *"all zero"* state. The resulting sequence is then called a **De Bruijn** code. It has many uses in our business but also in remote areas like card tricks!!

The normal maximum length LFSR circuit cannot generate the "all zero" state or the trivial state, because it will get stuck in this state forever. The picture below shows as an example a 4-bit maximum length LFSR circuit.



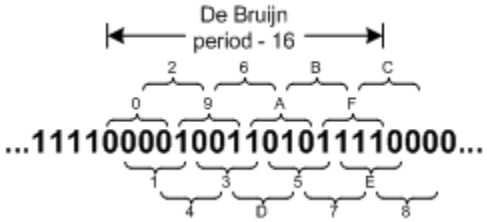
The trick is to try to insert the "all zero" state in between the "00..01″ and the "10..00″ states. Normally after the "00..01″ state the next value to be pushed in from the left would be a "1″, so the state "10..00″ could be generated. If we would like to squeeze the "00..00″ state next, we need to flip this to "0″. Then we have to make sure that for the next cycle a "1″ will be pushed. This is done by detecting when the entire set of registers – less the rightmost one – are all zero, and using this as another input in the XOR feedback path. The result is a 2^n counting space sequence which is called a De Bruijn sequence. The figure of the completed De Bruijn counter is shown below.



Since it is a bit hard to follow the words, you can convince yourself by following the sequence – the table below will be of some help.

Maximum length LFSR	De Bruijn Sequence
1111	1111
0111	0111
1011	1011
0101	0101
1010	1010
1101	1101
0110	0110
0011	0011
1001	1001
0100	0100
0010	0010
0001	0001
1000	0000
1100	1000
1110	1100
	1110

If you plot a single bit of the De Bruijn counter over time (doesn't matter which bit) you will see something similar to the next figure. Notice how over time, observing 4 consecutive bits in the sequence not only gives a unique result (until it wraps around) but also gives all possible 4-bit combinations! A simple LFSR will only fulfill the first criteria.



If you like LFSRs and want to investigate a bit more, there is an excellent (albeit quite "heavy to digest") book from Solomon Golomb called shift register sequences. The book is from the 1960s!! who said all good things in our industry are new…

Posted in Cool Circuits | 6 Comments »

Real World Examples #2 – Fast Counters

February 11, 2008

This is something which will be obvious to the "old school" people, because it was used a lot in the past.

A few weeks ago a designer who was working on a very, very small block asked for my advice on the implementation of counters. The problem was that he was using a 7-bit counter, defined in RTL as *cntr <= cntr +1;*
The synthesis tool generated a normal binary counter, but unfortunately it could not fulfill the

timing requirements – a few GHz.
(Don't ask me why this was not done in full-custom to begin with…)

Now, the key to solving this problem was to notice that in this specific design **only the terminal count was necessary**. This meant that all intermediate counter states were not used anywhere else, but the circuits purpose was to determine if certain amount of clock cycles have occured.

This brings us to the question: "Under these conditions, is there a cheaper/faster counter than the usual binary counter?"
Well I wouldn't write this post if the answer was negative… so obviously the answer is "Yes" – this is our old friend the LFSR!
LFSRs can be also used as counters, and they are being used in two very common, specific ways:

1. As a terminal counter – counter needs to measure a certain amount of clock edges. It counts to a specific value and then cycles over or resets
2. As a FIFO pointer – where the actual value itself is not of great importance but the order of increment needs to be deterministic and/or the relationship to another pointer of the same nature

Back in the age of prehistorical chip design (the 1970s), when designers really had to think hard for every gate, LFSRs were a very common building block and were often used as counters.

A slight disadvantage, is that the counting space of a full length n-bit LFSR is not 2^n but rather (2^n)-1. This sounds a bit petty on my side but believe me it can be annoying. Fear not! There is a very easy way to transform the state space to a full 2^n states. (can you find how???)

So next time you need a very fast counter, or when you need pointers for your FIFO structure – consider your good old friend the LFSR. Normally with just a single XOR gate as glue logic to your registers, you achieve (almost) the same counting capabilities given to you by the common binary counter.

Posted in Real World Examples | 6 Comments »



Ultimate Technical Interview Question – Take 2

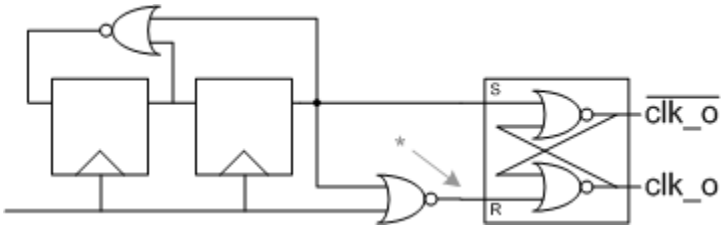
January 31, 2008

Allow me to quote from Martin Gardner's excellent, excellent book Mathematical Carnival (chapter 17):

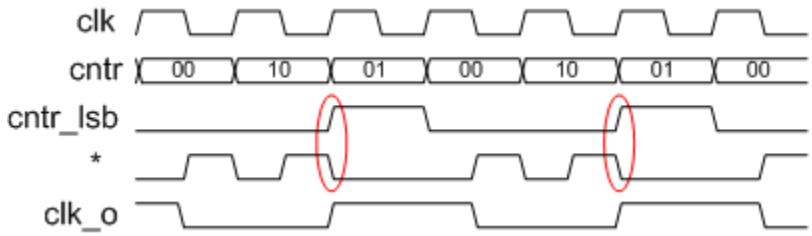
```
When a mathematical puzzle is found to contain a major flaw - when the answer
is wrong, when there is no answer, or when, contrary to claims, there is more
than one answer or a better answer - the puzzle is said to be "cooked".
```

From the number of hits, it looks like the last post was quite popular. Therefore, I decided to give the problem some more thought and to try to find more minimal solutions – or as defined in the above quote *"to cook this problem"*.

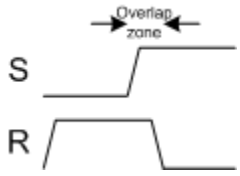
My initial hunch was to try and utilize an SR latch somehow. After all it is a memory element for the price of only two gates. I just had a feeling there is someway to do it like that.
I decided to leave the count-to-3 circuitry, cause if we want to do a divide by 3, we somehow have to count…
Here is what I first came up with:



The basic idea is to use the LSB of the counter to set the SR flop and to reset the SR flop with a combination of some states and the **low** clock.
Here is the timing diagram that corresponds to the circuit above.

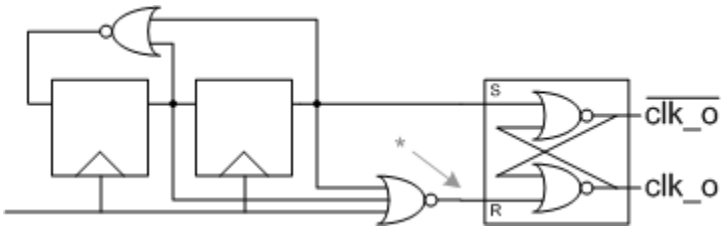


But! not everything is bright. The timing diagram is not marked red for nothing.
In an ideal world the propagation time through the bottom NOR gate would be zero. This would mean that exactly when the S pin of the SR latch goes high the R pin of the flop goes low – which means both pins are never high at the same time. Just as a reminder, if both inputs of an SR latch are high, we get a race condition and the outputs can toggle – not something you want on your clock signal. Back to the circuit… In our case, the propagation time through the bottom NOR gate is **not** zero, and the S pin of the latch will first go high, then – only after some time, the R pin will go low. In other words we will have on overlap time where both R and S pin of the latch will be high.

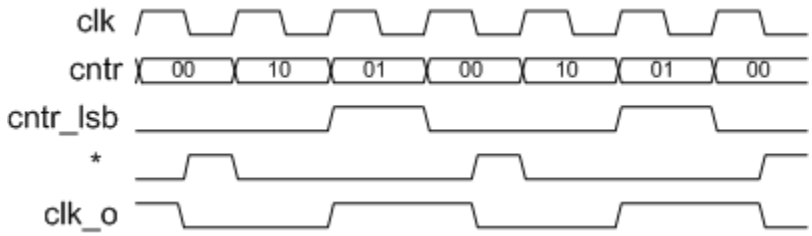


Looking back at the waveform, it would be nice if we could eliminate the second pulse in each set of two pulses on the R pin of the latch (marked as a * on the waveform). This means we just have to use the pulse which occurs during the "00″ state of the counter.
This is easy enough, since we have to use the "00″ from the counter and the "0″ from the clock itself – this is just the logic for a 3 input NOR gate!

The complete and corrected circuit looks like this now:



And the corresponding waveform below. Notice how the S and R inputs of the SR latch are not overlapping.



Posted in Interview Questions | Leave a Comment »

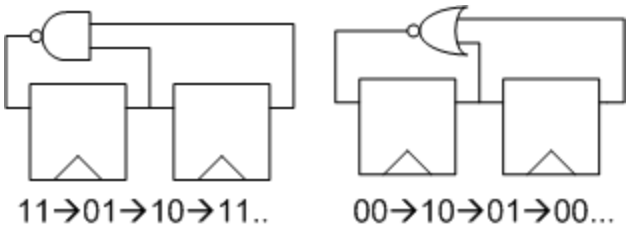
Ultimate Technical Interview Question – The Standard Solution

January 24, 2008

OK, so I am getting tons of email with requests to post a solution for this question which was initially posted here.
I am going to post now what I consider the "standard minimal solution", but some of you have come up with some neat and tricky ways, which I will save for future a post.
The basic insight was to notice that if you are doing a divide by 3 **and** wanna keep the duty cycle at 50% you have to use the falling edge of the clock as well.
The trick is how to come up with a minimal design, implementing as little as possible flip-flops, logic and guaranteeing glitch free divided clock.

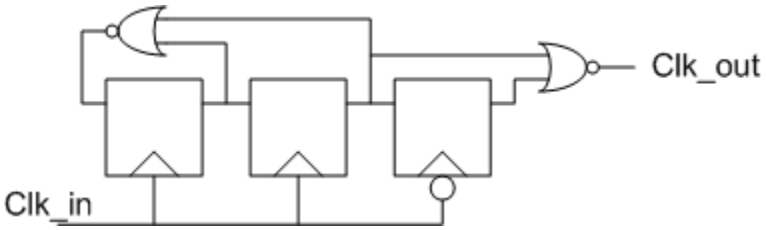
Most solutions that came in, utilized 4 or 5 flip flops plus a lot more logic than I believe is necessary. The solution, which I believe is minimal requires 3 flops – two working on the rising edge of the clock and generating a count-to-3 counter and an additional flop working on the falling edge of the clock.

A count-to-3 counter can be achieved with 2 flops and a NOR or a NAND gate only, as depicted below. These counters are also very robust and do not have a "stuck state".

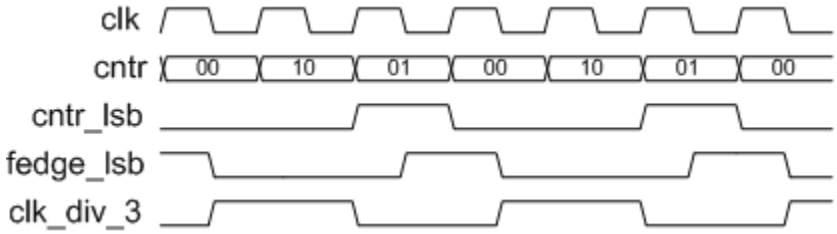


The idea now is to use the falling edge of the clock to sample one of the counter bits and generate simply a delayed version of it.
We will then use some more logic (preferably as little as possible) to combine the rising edge bits and falling edge bit in a way that will generate a divide by 3 output (with respect to out incoming clock).

The easiest way (IMHO) to actually solve this, is by drawing the wave forms and simply playing around. Here is what I came up with, which I believe to be *the optimal solution for this approach* – but you are more than welcome to question me!



and here is also the wave form diagram that describes the operation of the circuit, I guess it is self-explanatory.



One more interesting point about this implementation is that it does not require reset! The circuit will wake up in some state and will arrive a steady state operation that will generate a divide by 3 clock on its own. We discussed some of those techniques in the past when talking about ring counters – link to that post here.

Posted in Interview Questions | 5 Comments »

Low-Power Design Book

January 16, 2008

Everybody is talking low power design now. I try to give some tips here and there on this blog – mainly from the digital design or RTL point of view.

This book (google books link): Low-Power CMOS Circuits: Technology, Logic Design and CAD Tools By Christian Piguet has really something for everyone. Whether you are an analog designer, digital designer, architect or even a CAD guy – read it. It is heavy on examples, which immediately gets my points.

I found the low-power RTL chapter very informative and it even covers some of the stuff I addressed in this blog.
Check it out, it is worth your time!

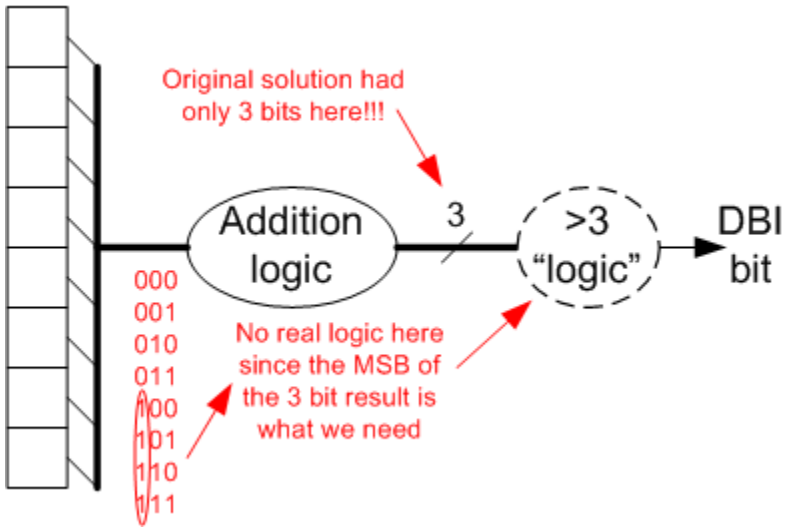
Posted in General | Leave a Comment »

Real World Examples #1 – DBI Bug Solution

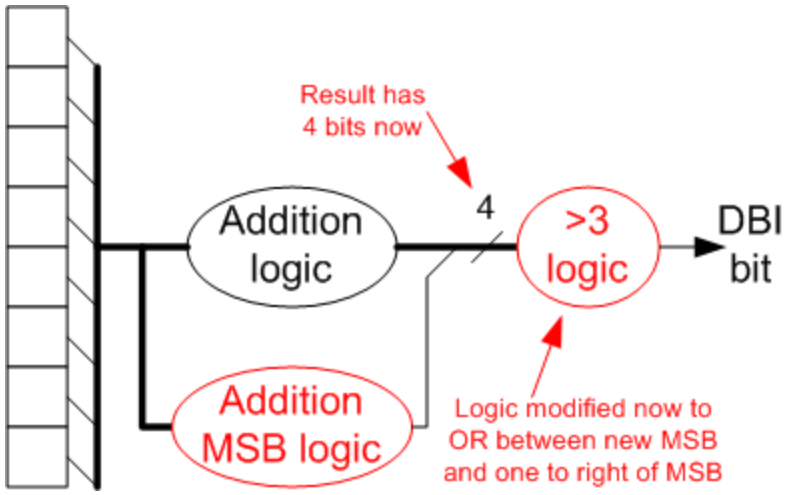
January 7, 2008

In the previous post I presented the problem. If you haven't read it, go back to it now cause it will make this entire explanation simpler.

Given the RTL code that was described, the synthesizer will generate something of this sort:

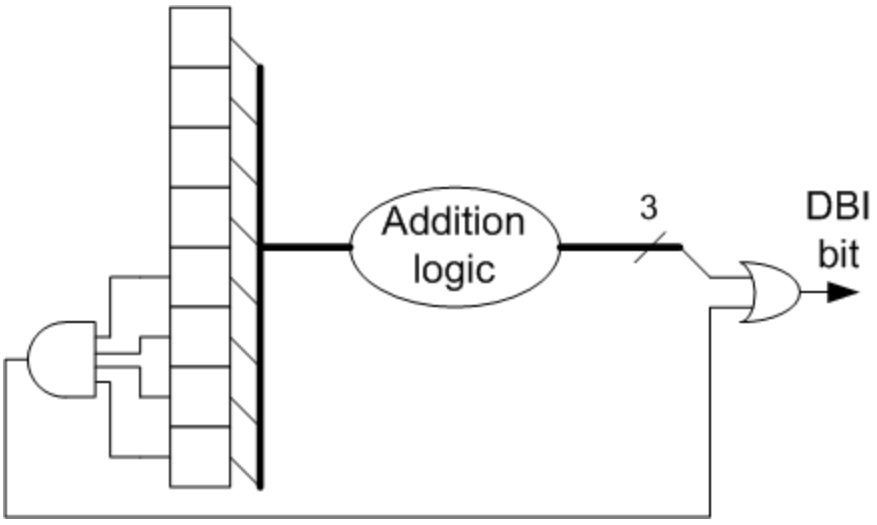


A straight forward approach, to solve the problem, would be to try to generate the MSB of the addition logic and do the comparison on the 4-bit result. This logic cloud would (probably) be created if we would make the result vector to be 4-bit wide in the first place. It would look something like this:



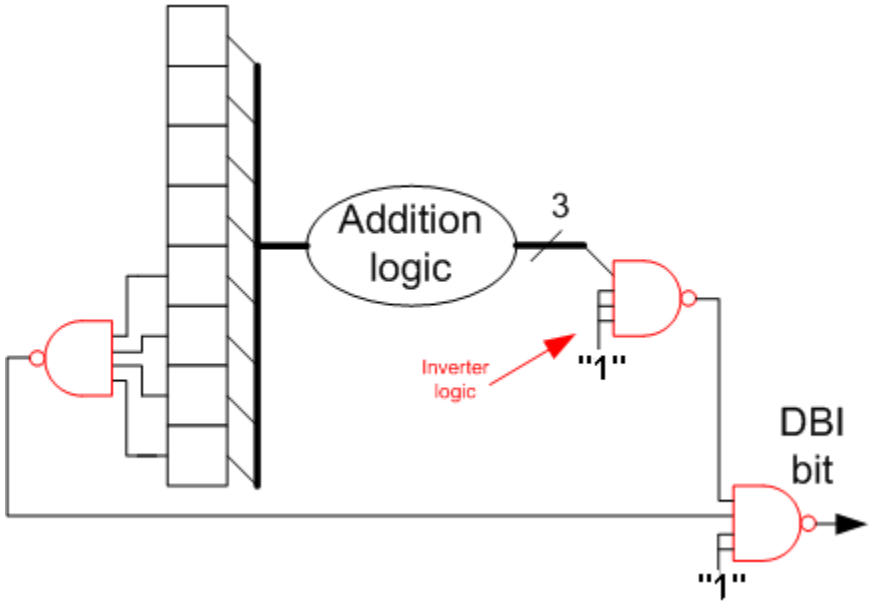
This looks nice on the paper, but press the pause button for a second and think – what is really hiding behind the MSB logic? You could probably re-use some of the addition logic already present, but you would have to do some digging in the layout netlist and make sure you got the right nets. On top of that, you would probably need to introduce some logic involving XORs (because of the nature of the addition). This is quite simple if you get to use any gate you wish, but it becomes complex when you got only NANDs and NORs available. It is *possible* from a logical point of view, but since you need to employ several spare cells, you might run into timing problems since the spare cells are spread all over and are not necessarily in the vicinity of your logic. Therefore, a solution with the least amount of gates is recommended!

So let's rethink the problem. We know that the circuit works for 0-7 "1"s but fails only for the case of 8 "1"s. We also know that in that case the circuit behaves as if there were 0 "1"s. Remember we go 4 input NANDs and NORs to our disposal. *We could take **any** 4 bits of the vector, AND them and OR them with the current result.* It's true, we do not identify 8 "1"s but in a case of 8 "1"s the AND result of any 4 bits will be high and together with the OR it will give the correct result. On other cases the output of this AND will be low and pass the correct result via the old circuit! There is a special case where there are exactly 4 bits on and these are the bits that are fed into our added AND gate, but in this case we have to anyway assert the DBI bit.
The above paragraph was relatively complicated so here is a picture to describe it:



It is important to notice that with this solution, the newly introduced AND gate is driven directly from the flip-flops of the vector. This makes it *much easier* to locate in the layout netlist, since flip-flop names are not changed at all (or very slightly changed).

Here is the above circuit implemented with 4 input NAND gates only (marked in red). This is also the final solution that was implemented.



Closing words – this example is aimed to show that when doing ECOs one has to really put effort and try to look for the cheapest and simplest solution. Every gate counts, and a lot of tricks need to be used. This is also the true essence of our work, but let's not get philosophical…

Posted in Real World Examples | Leave a Comment »

Real World Examples #1 – The DBI bug

January 3, 2008

OK, back after the long holidays (which were spent mainly in bed due to severe sickness, both of myself and my kids…) with some new ideas.
I thought it would be interesting to pick up some real life examples and blog about them. I mainly concentrated so far on design guide lines, tricky puzzles and general advice. I guess it would benefit many if we dive into the real world a bit. So – I added a new category called (in a very sophisticated way) "real life examples", which all this stuff will be tagged under.
Let's start with the first one.

The circuit under scrutiny, was supposed to calculate a DBI (Data Bus Inversion) bit for an 8-bit vector. Basically, on this specific application, if the 8-bit vector had more than 4 "1″s a DBI bit should have gone high, otherwise it should have stayed low.
The RTL designer decided to add all the bits up and if the result was 4 or higher the DBI bit was asserted – this is not a bad approach in itself and usually superior to LUT.

The pseudo code looked something like that:

```
assign sum_of_ones = data[0] + data[1] + data[2] + data[3] + data[4] +
data[5] + data[6] + data[7];
assign dbi_bit = (sum_of_ones > 3);
```

The problem was that accidentally the designer chose "sum_of_ones" to be **3-bit wide only!**
This meant that if the vector was all "1″s, the adder logic that generates the value for sums_of_ones would wrap around and give a value of "000″, which in turn would not result in the DBI bit being asserted as it should. During verification and simulation the problem was not detected for some reason (a thing in itself to question), but we were now facing with a problem we needed to fix as cheaply as possible. We decided to try a metal fix.

The $50K (or whatever the specific mask set cost was) question is how do you fix this as fast as possible with as little overhead as possible, assuming you have only 4 input NAND and 4 input NOR gates available?
Answer in the next post

Posted in Real World Examples | Leave a Comment »

Back with updates soon

December 29, 2007

just a short note. I did not post for a while, so just to let you all know – new updates will come in the new year (no, not in September…)

Posted in Uncategorized | 1 Comment »

Hands-on Arithmetic Operators

December 10, 2007

Here is a cool site that a colleague sent me – link here.
Scroll down and browse through the chapters. You can interactively play with different arithmetic operators and their implementations using the applets in the site. I found the special purpose adders to be especially interesting.

Posted in Digital Arithmetic, General | Leave a Comment »

ECO Flow

December 5, 2007

Here is a useful checklist you should use when doing your ECOs.

1. **RTL bug fix**

   Correct your bug in RTL, run simulations for the specific test cases and some your general golden tests. See if you corrected the problem and more important didn't destroy any correct behavior.

2. **Implement ECO in Synthesis netlist**

   Using your spare cells and/or rewiring, implement the bug fix directly in the synthesis verilog netlist. Remember you do not re-synthesize the entire design, you are patching it locally.

3. **Run equivalence check between synthesis and RTL**

   Using your favorite or available formal verification tool, run an equivalence check to see if the code you corrected really translates to the netlist you patched. Putting it simply – the formal verification tool runs through the entire state space and tries to look for an input vector that will create 2 different states in the RTL code and the synthesis netlist. If the two designs are equivalent you are **sure** that your RTL simulations would also have the same result (logically speaking) as the synthesis netlist.

4. **Implement ECO in layout netlist**

   You will now have to patch your layout netlist as well. Notice that this netlist is very different than the synthesis netlist. It usually has extra buffers inserted for edge shaping or hold violation correction or maybe even totally differently logically optimized.
   This is the real thing, a change here has to take into account the actual position of the

cells, the actuall names etc. Try to work with the layout expert in close proximity. Make sure you know and understand the floorplan as well – it is very common to connect a logic gate which is on the other side of the chip just because it is logically correct, but in reality it will violate timing requirements.

5. **Run equivalence check between layout and synthesis**

   This is to make sure the changes you made in the layout netlist are logically equivalent to the synthesis. Some tools and company internal flows enable a direct comparison of the layout netlist to the RTL. In many it is not so and one has to go through the synthesis netlist change as well

6. **Layout to GDS / gate level simulations / STA runs on layout netlist (all that backend stuff…)**

   Let the layout guys do their magic. As a designer you are usually not involved in this step.
   However, depending on your timing closure requirements, run STA on the layout netlist to see if everything is still ok. This step might be the most crucial since even a very small change might create huge timing violations and you would have to redo your work.
   Gate level simulations are also recommended, depending on your application and internal flow.

Posted in General, Layout, Synthesis | 3 Comments »



Spare Cells

November 26, 2007

What are spare cells and why the heck do we need them?

Spare cells are basically elements embedded in the design which are not driving anything. The idea is that maybe they will enable an easy (metal) fix without the need of a full redesign.

Sometimes not everything works after tape-out, a counter might not be reseted correctly, a control signal needs to be additionally blocked when another signal is high etc. These kind of problems could be solved easily if "only I would have another AND gate here…"
Spare cells aim to give a **chance** of solving those kind of problems. Generally, the layout guys try to embed in the free spaces of the floor-plan some cells which are not driving anything. There is almost always free space around, and adding more cells doesn't cost us in power (maybe in leakage in newer technologies), area (this space is anyhow there) or design time (the processes is 99% automatic).
Having spare cells might mean that we are able to fix a design for a few 10K dollars (sometimes less) rather than a few 100K.

So which spare cells should we use? It is always a good idea to have a few free memory elements, so I would recommend on a few flip-flops. Even a number as low as 100 FF in a 50K FF design is usually ok. Remember, you are *not* trying to build a new block, but rather to have a cheap possibility for a solution by rewiring some gates and FFs.
What gates should we through in? If you remember some basic boolean algebra, you know that NANDs and NORs can create any boolean function! This means that integrating only NANDs or NORs as spare cells would be sufficient. Usually, both NANDs and NORs are thrown in for more flexibility. 3 input, or even better 4 input NANDs and NORs should be used.

A small trick is tying the inputs of all NANDs to a logical "1″ and all inputs of the NORs to a logical "0″. This way if you decide to use only 2 of the 4 inputs the other inputs do not affect the output (check it yourself), this in turn means less layout work when tying and untying the inputs of those spare cells.

The integration of spare cells is usually done *after* the synthesis step and in the verilog netlist basically looks like an instantiation of library cells. This should not done before, since the synthesis tool will just optimize all those cells away as they drive nothing. The layout guy has to somehow by feeling (or black magic) spread the spare cells around in an even way.

I believe that when an ECO (Engineering Change Order) is needed and a metal-fix is considered – this is where our real work as digital designers start. I consider ECOs, and in turn the use of spare cells to solve or patch a problem, as the epitome our usage of skills, experience, knowledge and creativity!

More on ECOs will be written in the future…

Posted in General, Layout, Synthesis, Uncategorized | 5 Comments »

Send Your Problem

November 13, 2007

Being the generous person that I am I decided to open a new section in this blog called "Send Your Problem" or SYP.

If you have a design problem of any sort, that you think would interest others, or just that you need help with – send it over – yes, you heard it right. I will try to do the best I can to pick up the most difficult/interesting problems and post them with some solutions (hopefully).

I have limited time, and I really do it on my own spare time so be patient if it takes me time to answer. I will of course, try to address all questions, at least by email.

Posted in Send Your Problem | 27 Comments »

Micro-Architecture Template

November 13, 2007

After a somewhat long pause here is the uArch template as promised.

- **Part 1 – Block name, Owner, Version control**

As usual for any important document, it must have an owner, version control etc. Not much need to be explained here.

- **Part 2 – Interface signal list**

This is were order starts to give us some advantage. Every interface signal should be listed here, with its width, whether it is an input or output (regardless of the naming convention you use), description of what it is about and don't forget the comments.
I usually like to add information on signals which I know will come handy for other designers, for example – if a system clock is gated low, or the amount of pulses a certain signal should expect for normal operation. The list can be endless, but remember to fill in information which is helpful to the designers interfacing to you. Here is a template for the signal list table.

Signal Name	Width	I/O	Functionality description	Comments
Sys_clk_i	1	I	General system clock – 500MHZ	DCD expected

- **Part 3 – Overview**

Here, you want to describe what the block is supposed to do – e.g. this block controls a dual port RAM blah blah, or this is an FSM which controls this and that… The idea here is to give enough information for people to recognize the functionality in a glance.

- **Part 4 – Detailed Functional description of key circuitry with drawings**

This is the heart of it all. If you don't put enough effort here than forget about this ever being useful. Try to have a detailed diagram (not code!) of how your block is structures – same style that you see here on this blog. You don't necessarily have to draw every wire, but you should employ a qualitative approach.
Special care should be taken to describe in detail the critical path within the block, special interface signals (say if a signal is delivered on the falling edge for a normally rising edge design, the circuitry should be shown)
It is a good habit to have *all* interface signals present on your drawings. I also recommend to

have each flop present on the drawing. This is especially useful for data path designs. This is not as much work as would seem, usually if you do calculations on a wide bus you just need to draw a single flop (again qualitative approach).

- **Part 5 – Verification – list of assertions, formal verification rules, etc.**

This is becoming more and more important the more large the block becomes and the more complex the functionality is. Take your time and describe "rules" (e.g. 2 cycles after signal A goes down, signal B should also go down.
You can go to much detail here, but try to extract the essence of the block and describe the rules for their correct behavior.
If you got someone writing formal verification rules or assertions for you, she/he will kiss your toes for writing this section.

- **Part 6 – Comments**

All the important stuff that didn't go in the upper 5 sections should go here.

Posted in General | Leave a Comment »



On the Importance of Micro-Architecture

October 24, 2007

This post will be a bit different. I will try to tell you about my philosophy on how to approach a design, hopefully I am able to convince you why I believe it is right, and why I believe this approach makes one a much better designer (at least it worked for me).
So if you are looking for special circuits, cool tricks or design tips, you won't find them in this post.

Back in the 1990s, when I started in ASIC digital design, I used to get a task, think about it a bit and then wanted immediately to rush, code and synthesize the thing. My boss back then practically forced me to write a micro-architecture document. This in essence meant to give a precise visual description of all pipeline stages, inputs, outputs and a rough idea on the logic in between (one need not draw each gate in an adder for example).

I hated it big time. I thought it was so obvious and trivial. Why the heck do I need to have a list describing all inputs and outputs? it's anyways in the code. Why do I need to have a detailed drawing of the block with pipeline stages, arithmetic operations etc? it is also in the code. Well I was wrong, very wrong.

Only when working on a large project you understand how much the time invested in writing a micro-architecture document pays off. Doesn't matter how proficient you are in VHDL or Verilog, it is not easy understanding what you did 3 months ago, it is easier looking at a diagram and getting the general idea in a blink. uArch also helps aligning different designers within a big

project. It will help you optimize your design, because you will see all pipeline stages and you will get an overview on the block. you will see where it is possible to add more logic and where it must be cut. If you are experienced enough, you could often detect the critical path before even synthesizing your code.

This is the main reason why you see mostly diagrams on this blog and *not* code. HDL code is a way to describe what we want to achieve, some people confuse it with the goal itself. In my humble opinion it is extremely important to make this distinction.

Bottom line – most will think it is time spent for nothing. From my own personal experience, you will actually design your blocks faster like that, with less bugs and other people could actually understand what you did. **Try it**.

On the next post I will give a rough template on how I think a uArch document should look like and what it should contain.

Posted in General | 1 Comment »



Null Convention Logic

October 11, 2007

It is extremely rare in our industry that totally new approaches for Logic circuit design are taken. I don't know the exact reasons and I really don't want to get into the "fight" between tool vendors and engineers.

Null Convention Logic, is a totally different approach to circuit design. It is asynchronous in its heart (I guess half of the readers of this post just dropped now).
It is not new and being currently pushed by its developers in Theseus Research.

They published a book, which I really recommend reading. It is not very practical with the current mainstream tools and flows but it is a *very* interesting reading that will open your eyes to new approaches in logic design.
You can get a good introduction to the book's content by reading this paper. It is fairly technical and would need a few good hours to digest and grasp the meaning behind, especially given the fact that it is so much different than what we are used to – forget about AND, OR and NOT gates…

Book link here.

Posted in Cool Circuits, General | 8 Comments »

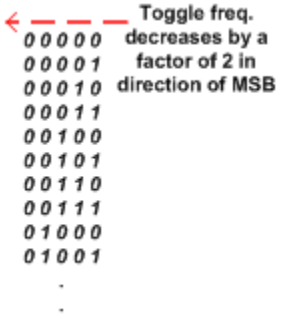


Pre-scaled Counters

October 8, 2007

It is obvious that as a normal binary counter increases in width its maximum operation frequency drops. The critical path going through the carry chain up to the last half-adder element is purely combinational and increases with size. But what if our target frequency is fixed (as is usually the case) and we need to build a very wide counter? Here come to the rescue a variant of the normal binary counter – the pre-scaled counter.

Pre-scaled counters are based on the observation (or fact) that in the binary counting sequence, the LSB will toggle in the highest frequency (half that of the clock when working with the rising edge only). The next bit in line will toggle in half that frequency, the next with half of the previous and so on.
In general, the n-th bit will toggle with frequency 2^(n+1) lower than the clock (we assume that bit 0 is the LSB here). A short look at the figure below will convince you.

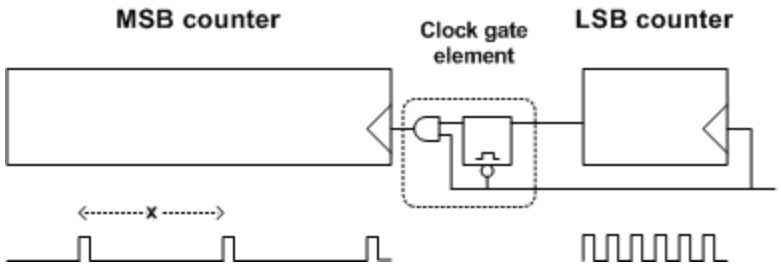


We can use this to our advantage by "partitioning" the counter we wish to build. In essence we make the target clock frequency of operation independent of the counter size! This means that given that our clock frequency enables us to have a single flop toggling plus minimal levels of logic, one could in theory build am extremely wide counter.

If you really insist, the above statement is not 100% correct (for reasons of clock distribution and skew, carry collect logic of a high number of partition stages, etc.) , but for all practical reasons it is true and useful. Just don't try to build a counter with googolplex bits.

The basic technique for a 2-partition is shown below. We have an LSB counter which operates at clock frequency. Its width is set so it could still operate with the desired clock frequency. Once this counter rolls over an enable signal is generated for the MSB counter to make a single increment. Notice how we also keep the entire MSB counter clock gated since we know it cannot change its state.

The distance between the filtered clock edges (marked as "X") of the MSB counter is determined by the width of the LSB counter. This should be constrained as a multi-cycle path with period X when doing synthesis.



The technique could be extended to a higher amount of "partitions" but then we must remember that the enable for each higher order counter is derived from **all** enable signals of all previous stages.

An interesting variant is trying to generate and up/down counter which is width independent. It is not so complicated and if you have an idea on how to implement it, just comment.

Posted in Cool Circuits | 6 Comments »

Everything About Scan

September 28, 2007

Back from vacation…

I really wanted to devote a set of posts on scan and its importance for us digital designers. I planned, wrote up a list of topics and problems I wanted to highlight, reworked everything and then searched the web…

Dang! Somebody already did it better and clearer than I could have ever done that!

All I can do is recommend on reading and re-reading those two articles – It will pay off. Links follow…

Part 1
Part 2

Posted in DFT | 4 Comments »

Puzzle #11 – Not Just Another Hats Problem

September 16, 2007

Here is another puzzle for you to ponder during the upcoming week. It would seem a bit far fetched from our usual digital design stuff, but the solution is somewhat related to the topics discussed in this blog. Moreover, it is simply a neat puzzle.

A group of 50 people are forming a column so person #1 is in front of all, followed by person #2 and so on up to person #50.
Person #50 can see all the people in front of him (#49..#1), person #49 can see only #48..#1 and so on.
The 50 people are now given hats in random. Each hat can be either black or white. The distribution of the hats is totally random (i.e. they might be all black or all white and not necessarily 25-25)

The people now take turn in guessing what color hat they are wearing – They are just allowed to say "white" or "black", *nothing more!*. Person #50 starts and they continue in order down to person #1. If the person happens to guess the color of his own hat the group receives $1000.
What is the best strategy the 50 people should agree on *before the experiments starts* to maximize the amount of money they should expect? And what is the sum of money they should expect to earn from this experiment?
(you can do better than pure chance, or much better than $25,000…)

For the experts

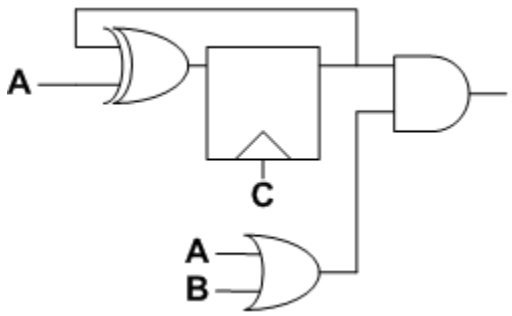
What if the experiment is done with hats which are red, black or white? what about 4 colors? What would be the maximum color of hats that will still guarantee the amount from the original variant? and how?

Posted in Puzzles | 14 Comments »

Puzzle #10 – Mux Logic

September 15, 2007

Your company is pretty tight on budget this year and it happens to have only Muxes to design with.
You are required to design a circuit equivalent to the one below, using only Mux structures.



Posted in Interview Questions, Puzzles | 16 Comments »

Going on Vacation…

September 15, 2007

I will be on vacation for a week and half (up to september 25th) and will have relatively limited access to the web. Therefore, I will leave you with a series of puzzles just for fun. I will try to make at least one of them hard enough…

Posted in General | Leave a Comment »

Puzzle #9 – The Snail

September 10, 2007

It's been a while since I posted a nice puzzle and since I know they are so popular, here is a relatively simple one. It was used in job interviews btw (the last line will boost the amount of views for this post…)

A snail leaves his warm house and takes a crawl through the forest leaving behind him on the ground a trail of "0″s and "1″s. He takes a very complicated route crossing his path several times. At one point he becomes tired and disoriented and wishes to go back home. He sees his own path of "0″s and "1″s on the ground which he is about to cross (i.e. not the trail ending in his tail) and wonders whether to follow the trail towards the left or towards the right.
What is the shortest repeating code of "0″s and "1″s he should leave as he crawls in order to easily and deterministically track the way back home? What is the minimum amount of bits he needs to observe (or the sample length of the code)?

Posted in Interview Questions, Puzzles | 11 Comments »

A Concise Guide to Why and How to Split your State Machines

September 8, 2007

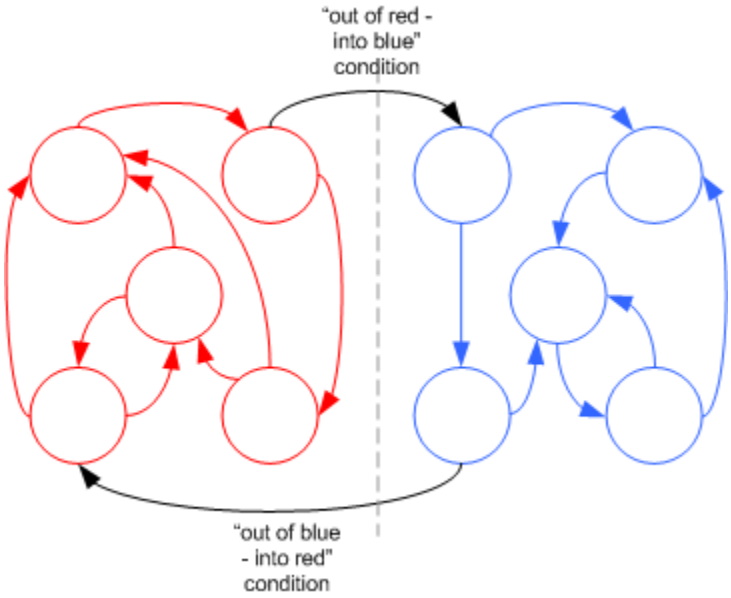
So, why do we really care about state machine partitioning? Why can't I have my big fatty FSM with 147 states if I want to?

Well, smaller state machines are:

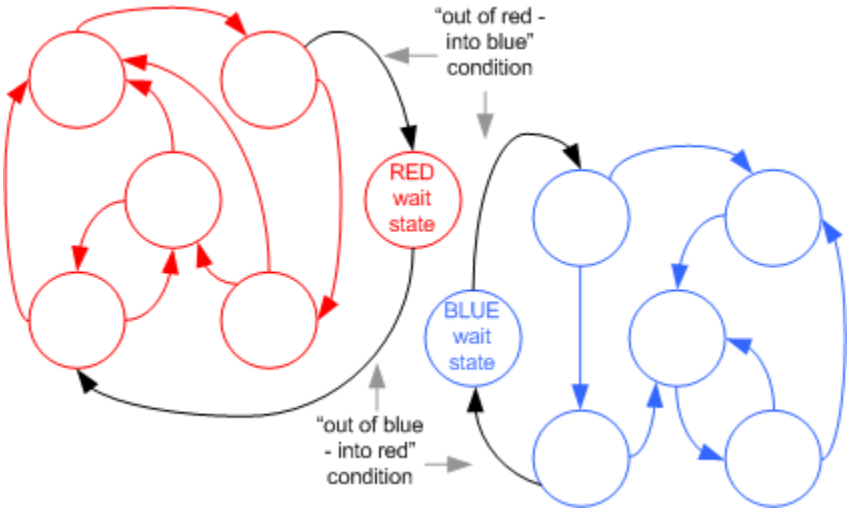
1. Easier to debug and probably less buggy
2. More easily modified
3. Require less decoding
4. Are more suitable for low power applications
5. Just nicer…

There is no rule of thumb stating the correct size of an FSM. Moreover, a lot of times it just doesn't make sense to split the FSM – So when can we do it? or when should we do it? Part of the answer lies in a deeper analysis of the FSM itself, its transitions and most important, the probability of occupying specific states.

Look at the diagram below. After some (hypothetical) analysis we recognize that in certain modes of operation, we spend either a lot of time among the states marked in red or among the states marked in blue. Transitions between the red and blue areas are possible but are less frequent.



The trick now, is to look at the *entire* red zone as one state for a new "blue" FSM, and vice versa for the a new "red" FSM. We basically split the original FSM into two completely separate FSMs and add to each of the FSM a new state, which we will call a "wait state". The diagram below depicts our new construction.



Notice how for the “red” FSM transitioning in and out of the new “wait state” is exactly equivalent (same conditions) to switching in and out of the red zone of the original FSM. Same goes for the blue FSM but the conditions for going in and out of the “wait state” are naturally reversed.

OK, so far so good, but what is this good for? For starters, it would probably be easier now to choose state encodings for each separate FSM that will reduce switching (check out this post on that subject). However, the sweetest thing is that when we are in the “red wait state” we could gate the clock for the rest of the red FSM and all its dependent logic! This is a significant bonus, since although previously such strategy would have been possible, it would just be by far more complicated to implement. The price we pay is additional states which will sometimes lead to more flip-flops needed to hold the current state.

As mentioned before, it is not wise to just blindly partition your FSMs arbitrarily. It is important to try to look for patterns and recognize “regions of operation”. Then, try to find transitions in and out of this regions which are relatively simple (ideally one condition to go in and one to go out). This means that sometimes it pays to include in a “region” one more state, just to make the transitioning in and out of the “region” simpler.

Use this technique. It will make your FSMs easy to debug, simple to code and hopefully will enable you to introduce low power concepts more easily in your design.

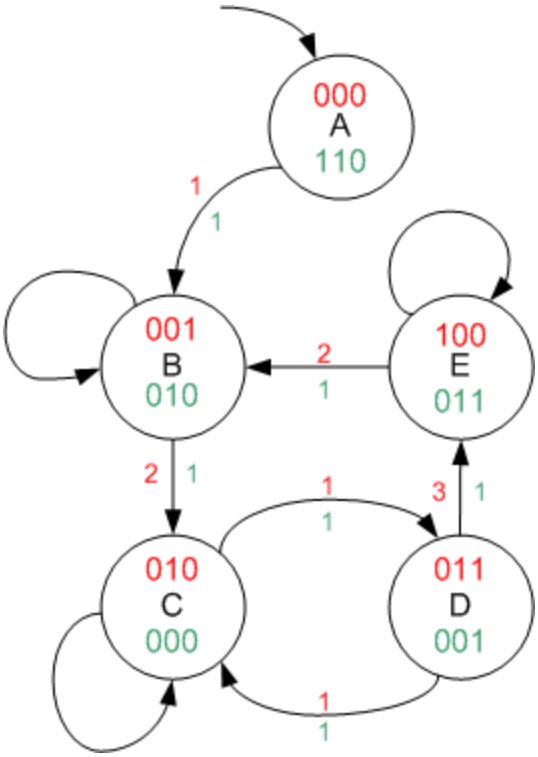
Posted in Coding Style, Low Power | Leave a Comment »

FSM State Encoding – More Switching Reduction Tips

September 4, 2007

I promised before to write some words on reducing switching activity by cleverly assigning the states of an FSM, so here goes…

Look at the example below. The FSM has five states **“A”-”E”**. Most naturally, one would just sequentially enumerate them (or use some enumeration scheme given by VHDL or Veriog – which is easier for debugging purposes).
In the diagram the sequential enumeration is marked in **red**. Now, consider only the topology of the FSM – i.e. without any reference to the probability of state transitions. You will notice that the diagram states (pun intended) in red near each arc the amount of bits switching for this specific transition. For example, to go from state **“E”** (*100*) to state **“B”** (*001*), two bits will toggle.



But could we choose a better enumeration scheme that will reduce the amount of switching? Turns out that yes (don't tell anybody but I forced this example to have a better enumeration 😀). If you look at the **green** state enumeration you will clearly see that at most only one bit toggles for every transition.

If you sum up all transitions (assuming equal probability) you would see that the green implementation toggles exactly half the time as the red. An interesting point is that we need only to consider states **"B" – "E"**, because once state **"A"** is exited it can never be returned to (this is sometimes being referred to as "black hole" or "a pit").

The fact that we chose the states enumeration more cleverly doesn't only mean that we reduced switching in the actual flip-flops that hold the state itself, but *we also reduce glitches/hazards in all the combinational logic that is dependent on the FSM!* The latter point is extremely important since those combinational clouds can be huge in comparison to the n flops that hold the state of the FSM.

The procedure on choosing the right enumeration deserve more words but this will become a too lengthy post. In the usually small FSMs that the average designer handles on a daily basis, the most efficient enumeration can be easily reached by trial and error. I am sure there is somewhere some sort of clever algorithm that given an FSM topology can spit out the best enumeration. If you are aware of something like that, please send me an email.

Posted in Coding Style, Gray Codes, Low Power | 3 Comments »

10,000 Views …

September 3, 2007

Last week this blog crossed the 10,000 views mark in about 3+ months or so of being "on the air". That number represents total views not counting my own visits to the blog (if it would include that, the number would for sure triple)

Just wanted to say thanks to everybody who is reading, sending emails, suggesting and even complaining (though there are not many of those).
Please don't be ashamed and continue sending emails. First, it is fun to get them and second, it helps me improve and write about what is really interesting for you guys.

Posted in General | Leave a Comment »



Low Power Buses – More Tricks for Switching Reduction

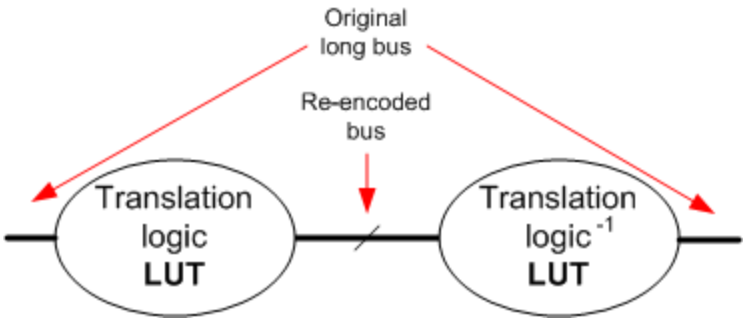
August 31, 2007

Viewing the search engine key words which people use to get to this blog, I can see that low power tips and tricks are one of the most interesting topics for people. Before I start this post, it is important to mention that although there is almost always something to do, the price can be great. Price doesn't always mean in area or complexity, sometimes it is just your own precious time. You can spend tons of time thinking on a very clever architecture or encoding for a bus but you might miss your dead lines all together.
OK, enough of me blubbering about "nonsense", let's get into some more switching reduction tricks.

Switching reduction means less dynamic power consumption, this has little to do with static power or leakage current reduction. When thinking of architectures or designing a block in HDL (verilog or VHDL) this is the main point we can tackle though. There is much less we could do about static power reduction by using various HDL tricks. This can be left to the system architect, our standard cell library developers or our FPGA vendor.

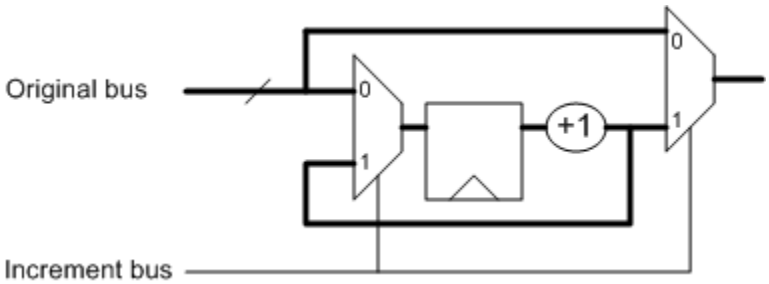
- **Bus States Re-encoding**

Buses usually transfer information across a chip, therefore in a lot of cases they are wide and long. Reduction of switching on a wide or long bus is of high importance. Assume you already have a design in a late stage which is already pretty well debugged. Try running some real life cases and extract what are the *most common transitions* that occur on the bus. If we got a 32-bit bus that switches a lot between 00…00 and 11…11 we know it is bad. It is a good idea to re-encode the state 11…11 into 00…01, for example. Then, decode it back on the other side. We would save the switching of 31 bits in this case. This is naturally a very simple case, but analyze your system, these things happen in real life and are relatively easy to solve – *even on a late stage of a design!* If you read this blog for sometime now, you probably know that I prefer visualization. The diagram below summarizes the entire paragraph.



- **Exploiting Special Cases – Identifying Patterns**

Imagine this, you have a system which uses a memory. During many operation stages you have to dump some contents into or out of the memory element. This is done by addressing the memory address by address in a sequential manner. We probably can't do much about the data, since it is by nature random but what about the address bus? We see a pattern that repeats over and over again: an address is followed by the next. We could *add* another line which tells the other side to increment the previous address given to him. This way we save the entire switching on the bus when sweeping through the entire address range.
The diagram below gives a qualitative solution of how an approach like this would work. If you are really a perfectionist, you could gate the clock to the bus sampling flops which reserve the previous state, because their value is only important when doing the increments. You would just have to be careful on some corner cases.



Generally speaking it is always a good idea to recognize patterns and symmetry and exploit it when transmitting information on a bus. Sometimes it can be the special numbering system being used, or a specific sequence which is often used on a bus or a million different other things. The point is to identify the trade off between investing a lot of investigations and the simplicity of the design.

On one of the future posts, we will investigate how we could use the same switching reduction techniques for FSM state assignments, so stay tuned.

Posted in Low Power | 1 Comment »

Puzzle #6 – The Spy- Solution

August 28, 2007

This puzzle created some interest, but apart from one non-complete solution which demonstrates the principle only, I didn't receive any other feedback. Here is my own solution, which is different than the one given in the Winkler book. Naturally, I believe my solution is easier to understand, but please get the Winkler book, it is really that good and you could decide for yourself.

Now for the reason you are reading this post… the solution… oh, if you don't remember the puzzle, please take a few moments to re-read it and understand what it is all about.

I will (try to) prove that 16 different symbols can be transmitted by flipping a single bit of the 15 which are transmitted daily.
First, for convenience reasons we define the 15-bit transmission as a 14-0 vector.
We will now define four parity functions **P0,P1,P2,P3**, as follows:

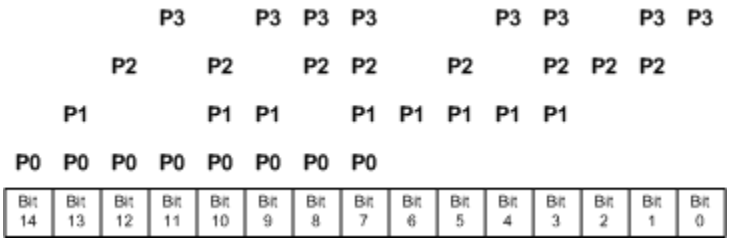
P0 = Bit 14 ^ Bit 13 ^ Bit 12 ^ Bit 11 ^ Bit 10 ^ Bit 9 ^ Bit 8 ^ Bit 7

P1 = Bit 13 ^ Bit 10 ^ Bit 9 ^ Bit 7 ^ Bit 6 ^ Bit 5 ^ Bit 4 ^ Bit 3

P2 = Bit 12 ^ Bit 10 ^ Bit 8 ^ Bit 7 ^ Bit 5 ^ Bit 3 ^ Bit 2 ^ Bit 1

P3 = Bit 11 ^ Bit 9 ^ Bit 8 ^ Bit 7 ^ Bit 4 ^ Bit 3 ^ Bit 1 ^ Bit 0

Why these specific functions will be clear in a moment.
Let's try to view them in a more graphical way by marking above each bit in the vector (with the symbols **P0..P3**) iff this bit affects the calculation of the respective "P-function". For example, bit 11 is included in the formula for **P0** and **P3** therefore we mark the column above it with **P0** and **P3**.
So far so good, but a closer look (diagram below) on the distribution of the "P-functions" reveals why and how they were constructed.

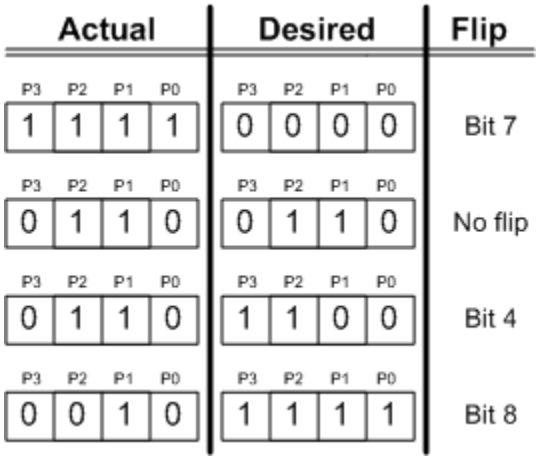


The "P-functions" were constructed in such a way that we have 1 bit which affects only the calculation of **P0**, one bit which affects only **P0 and P1**, one which affects only **P0 and P2** and so on… Try to observe the columns above each of the bits in the vector – *they span all the possible combinations!*

From here the end is very close.
The operators on the receiving side have to calculate the P0..P3 functions and assemble them into a 4-bit "word".

All the spy has to do, is calculate the actual "P-functions" given by today's random transmission and get a 4-bit "word". The spy compares this to the 4-bit "word" she wants to transmit and discovers the difference – *or in other words: the bits which need to be flipped in order to arrive from the actual "P-functions" to the desired "P-functions".* She then looks up in the diagram above and flips exactly that bit which corresponds to exactly the "P-functions" that she needs to flip. A single bit flip will also toggle the corresponding "P-function/s".

Since the above wording may sound a big vague, here is a table with some examples:



Actual				Desired				Flip
P3	P2	P1	P0	P3	P2	P1	P0	
1	1	1	1	0	0	0	0	Bit 7
P3	P2	P1	P0	P3	P2	P1	P0	
0	1	1	0	0	1	1	0	No flip
P3	P2	P1	P0	P3	P2	P1	P0	
0	1	1	0	1	1	0	0	Bit 4
P3	P2	P1	P0	P3	P2	P1	P0	
0	0	1	0	1	1	1	1	Bit 8

I have to say this again, This is really one of the most beautiful and elegant puzzles I came across. It is definitely going into "the notebook"…

Posted in Puzzles | 4 Comments »

The Coolest Binary Adder You Have Ever Seen…

August 26, 2007

I have to admit, I never thought I would ever link from this blog to youtube, but given the nature of the following contraption I believe you will agree it was a must…

This is by far the coolest binary adder you have ever seen – link here.
It has almost everything inside, a reset "pin", carry out "pin" etc.
If you are into wood working you could visit the builder's site and see exactly how this can be done – visit him here.

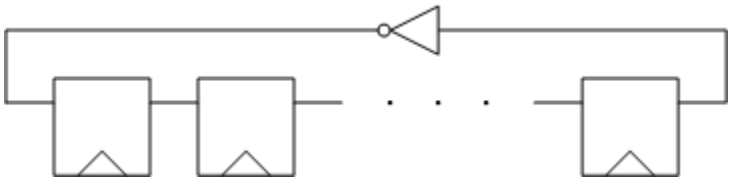
I also saw a mechanical binary adder in the Deutsches Mueseum, but it was based on water! I might try to get a video of that one running in the future, since the museum is 400 meters from my house. If you ever visit Munich and you don't go there – shame on you!!!

Posted in General | 1 Comment »

The Johnson Counter

August 20, 2007

Johnson counters, or Möbius counters (sometimes referred to with that name because of the abstract similarity to the famous Möbius strip) are extremely useful.
The Johnson counter is made of a simple shift register with an inverted feedback – as can be seen below.



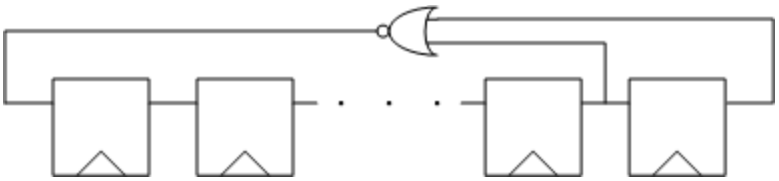
Johnson counters have 2N states (where N is the number of flip-flops) compared to 2^N states for a normal binary counter.
Since each time only a single bit is changing – Johnson counter states form a sort of a Gray code.
The next picture shows the 12 states of a 6 bit Johnson counter as an example.

000000
100000
110000
111000
111100
111110
111111
011111
001111
000111
000011
000001

Johnson counters are extremely useful in modeling, since by using any of the taps one could generate a clock like pattern with many different phases. You could easily see that by looking at the columns in the picture above, they all have 6 consecutive "1″s followed by 6 consecutive "0″s, but all in a different phase.

Decoding the state of the counter is extremely easy. A single 2 input gate which detects the border between the "1″s and the "0″s is enough. One needs not compare the entire length of the counter to some value.

One can also generate an odd length *"sort-of"* Johnson counter. The easiest way is by using a NOR feedback from the last two stages of the shift register as shown below.



The last picture shows the 11 states of the modified 6 flip-flop Johnson counter. Looking at the states sequence it is immediately noticeable that the 11…11 stage is skipped. We also lose the Gray property of the counter like that, since on a single case both the last and first bits will change simultaneously. But looking at the columns, which represent the different taps, we see that we kept the same behavior on each column (with respect to the signal shape) but the duty cycle is not 50% anymore – that is obvious because we no longer have an even amount of states.

000000
100000
110000
111000
111100
111110
011111
001111
000111
000011
000001

This post is becoming a bit too long to present all the cool things that can be done with Johnson counters, but a very important issue is robustness. In a future post we will see that serious designers do not use just a simple inverter for the feedback path, but also include some sort of self correction mechanism. This is necessary, because if a forbidden state creeps in (wrong reset behavior, crosstalk, etc) it will stay forever in the counter – sort of like the same problem we had in one of the previous posts on the ring counter. There are ways to get over this problem, and I will try to analyze them in a future post. Stay tuned…

Posted in Cool Circuits | 9 Comments »

Puzzle #8 – Clock Frequency Driver

August 13, 2007

Take the clock frequency circuit I posted about here. As I mentioned the XOR gate at the output might cause some duty cycle distortion with some libraries, due to the fact that most XOR gates are not built to be symmetrical with respect to transition delay.
Now, assume your library has a perfectly symmetrical NAND gate. Could you modify the circuit so the XOR will be replaced by a NAND gate and still have a clock frequency at the output (You are of course allowed to add more logic on other parts of the circuit).

If not, give a short explanation why not. If yes send a circuit description/diagram.

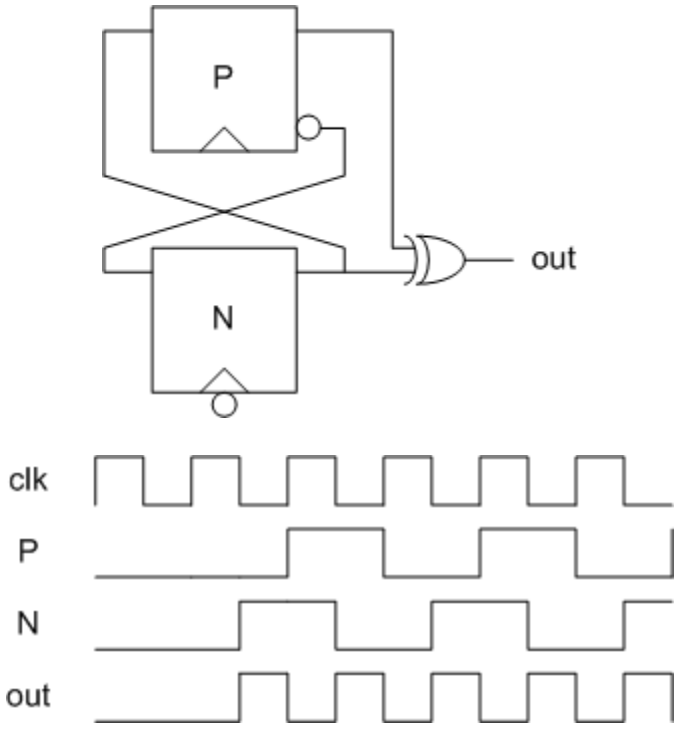
Posted in Interview Questions, Puzzles | 4 Comments »

Driving A Clock Frequency Signal From A Register

August 9, 2007

Usually in semi-custom flows it is a big no-no to use the clock in the data path. Sometimes it is necessary though, to drive a signal with the frequency of the clock to be used in some part of the design or driving it onto a pad. Normally, logical changes occur only on the rising edge of the clock and thus with half the frequency of the clock.

Here is a cool little circuit that will drive a signal which toggles at the clock frequency but is still driven from a register. It is very robust and upon wake up from a reset state will drive a clock like signal with the opposite phase of the clock but with the same frequency. To use the same phase as the clock itself, replace the XOR with an XNOR at the output.



If this circuit should be used as a clock for another block, consider the fact that the XOR gate at the output might introduce duty cycle distortion (DCD) due to the fact that many standard cell library XOR gates do not have a symmetrical behavior when transitioning from a logical 0 to a logical 1 and vice versa.

As an afterthought, it might be interesting to look at the similarities between this circuit and the "Double Edge Flip-Flop" I descried before.

Posted in Cool Circuits | 1 Comment »

Everything You Wanted to Know About Specman Verification and Never Dared to Ask

August 7, 2007

My friend Avidan Efody, has a site full of tons of advice, tips and tricks concerning verification with Specman. No, it is not "plug your buddy's blog" section, but if verification is what you do, and you never been there before – shame on you – you should visit it ASAP.

You can find it here.

Posted in General | Leave a Comment »

Puzzle #7 – Transitions – Solution

August 3, 2007

This one was solved pretty quickly. Basically I was trying to trick you. The idea was to try to create the impression an infinite amount of memory is necessary to hold all the 0–>1 and 1–>0 transitions. In practice there cannot be 2 consecutive 0–>1 transitions (or vice versa) since if the input goes from 0 to 1 before the next 0–>1 transition it must change to a 0 and thus have a 1–>0 transition!
The FSM can have only three states: "exactly one more 0–>1″, "equal amount of 0–>1 and 1–>0″ or "exactly one more 1–>0″.

Posted in Interview Questions, Puzzles | Leave a Comment »

Arithmetic Tips & Tricks #1

August 1, 2007

Every single one of us had sometime or another to design a block utilizing some arithmetic operations. Usually we use the necessary operator and forget about it, but since we are "hardware men" (should be said with pride and a full chest) we know there is much more going under the hood. I intend to have a series of posts dealing specifically with arithmetic implementation tips and tricks. There are plenty of them, I don't know all, probably not even half. So if you got some interesting ones please send them to me and I will post them with credits.

Let's start. This post will explain 2 of the most obvious and simple ones.

- **Multiplying by a constant**

Multipliers are extremely area hungry and thus when possible should be eliminated. One of the classic examples is when multiplying by a constant.
Assume you need to multiply the result of register A by a factor, say 5. Instead of instantiating a multiplier, you could "shift and add". 5 in binary is 101, just add A to A00 (2 trailing zeros, have the effect of multiplying by 4) and you have the equivalent of multiplying by 5, since what you basically did was 4A+A = 5A.
This is of course very simplistic, but when you write your code, make sure the constant is not passed on as an argument to a function. It might be that the synthesis tool knows how to handle it, but why take the risk.

- **Adding a bounded value**

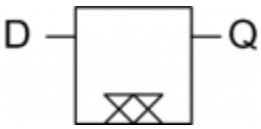
Sometimes (or even often), we need to add two values where one is much smaller than the other and *bounded*. For example adding a 3 bit value to a 32 bit register. The idea here is not to be neat and pad the 3 bit value by leading zeros and create by force a 32 bit register from it. Why? adding two 32 bit values instantiates full adder logic on all the 32 bits, while adding 3 bits to 32 will infer a full adder logic on the 3 LSBs and an increment logic (which is much faster and cheaper) on the rest of the bits. I am quite positive that today's synthesis tools know how to handle this, but again, it is good practice to always check the synthesis result and see what came up. If you didn't get what you wanted it is easy enough to force it by coding it in such a way.

Posted in Coding Style, Digital Arithmetic, General, Low Power | 4 Comments »

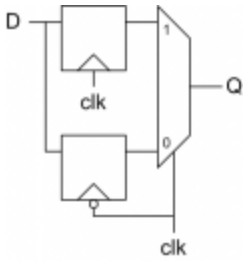
The Double Edge Flip Flop

July 31, 2007

Sometimes it is necessary to use both the rising and the falling edge of the clock to sample the data. This is sometimes needed in many DDR applications (naturally). The double edge flop is sometimes depicted like that:

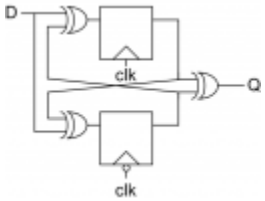


The most simple design one can imagine (at least me…), would be to use two flip flops. One sensitive to the rising edge of the clock, the other to the falling edge and to MUX the outputs of both, using the clock itself as the select. This approach is shown below:



What's wrong with the above approach? Well in an ideal world it is OK, but we have to remember that semi-custom tools/users don't like to have the clock in the data path. This requirement is justified and can cause a lot of headaches later when doing the clock tree synthesis and when analyzing the timing reports. It is a good idea to avoid such constructions unless they are absolutely necessary. This recommendation applies also for the reset net – try not combining the reset net into your logic clouds.

Here is a cool circuit that can help solve this problem:



I will not take the pleasure from you of drawing the timing diagrams yourself and realizing how and why this circuit works, let me just say that IMHO this is a darn cool circuit!

Searching the web a bit I came across a paper which describes practically the same idea by Ralf Hildebrandt. He names it a "Pseudo Dual-Edge Flip Flop", you can find his short (but more detailed) description, including a VHDL code, here.

Posted in Cool Circuits | 5 Comments »

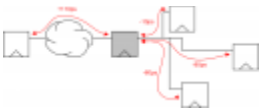
Replication

July 25, 2007

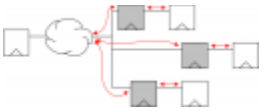
Replication is an extremely important technique in digital design. The basic idea is that under some circumstances it is useful to take the same logic cloud or the same flip-flops and produce more instances of them, even though only a single copy would normally be enough from a *logical* point of view.
Why would I want to spend more area on my chip and create more logic when I know I could do without it?

Imagine the situation on the picture below. The darkened flip-flop has to drive 3 other nets all over the chip and due to the physical placement of the capturing flops it can not be placed close

by to all of them. The layout tool finds as a compromise some place in the middle, which in turn will generate a negative slack on all the paths.



We notice that in the above example the logic cloud just before the darkened flop has a **positive** slack or in other words, “some time to give”. We now use this and produce a copy of the darkened flop, but this time closer to each of the capturing flops.



Yet another option, is to duplicate the *entire* logic cloud plus the sending flop, as pictured below. This will usually generate even better results.



Notice that we also reduce the fan out of the driving flop, thus further improving on timing.

It is important to take care about while writing the HDL code, that the paths are really separated. This means when you want to replicate flops and logic clouds make sure you give the registers/signals/wires different names. It is a good idea to keep some sort of naming convention for replicated paths, so in the future when a change is made on one path, it would be easy enough to mirror that change on the other replications.

There is no need to mention that when using this technique we pay in area and power – but I will still mention it

Posted in Coding Style, General, Layout | Leave a Comment »

Puzzle #7 – Transitions

July 17, 2007

It’s time for puzzle #7.

An FSM receives an endless stream of “0″s and “1″s. The stream can not be assumed to have certain properties like randomness, transition density or the like.

Is it possible to build a state machine, which at any given moment outputs whether there were more 0–>1 or 1–>0 transitions so far?

If yes, describe briefly the FSM. If no, give a short proof.

Posted in Interview Questions, Puzzles | 4 Comments »

2 Lessons on PRBS Generators and Randomness

July 10, 2007

The topic of "what is random" is rather deep and complicated. I am far from an authority on the subject and must admit to be pretty ignorant about it. However, this post will deal with two very simple but rather common errors (or misbehaviors) of random number generators usage.

LFSR width and random numbers for your testbench

Say you designed a pretty complicated block or even a system in HDL and you wish to test it by injecting some random numbers to the inputs (just for the heck of it). For simplicity reasons lets assume your block receives an integer with a value between 1 and 15. You think to yourself that it would be pretty neat to use a 4-bit LFSR which generates all possible values between 1 and 15 in a pseudo-random order and just repeat the sequence over and over again. Together with the other type of noise in the system you inject, this should be pretty thorough, right? Well, not really!

Imagine for a second how the sequence looks like, each number will always be followed by *another specific* number in this sequence! For example, you will never be able to verify a case where the same number is injected immediately again into the block!
To verify all other cases (at least for all different pairs of numbers) you would need to use an LFSR with a larger width (How much larger?). What you need to do then is to pick up only 4 bits of this bigger LFSR and inject them to your block.

I know this sounds very obvious, but I have seen this basic mistake done several times before – by me and by others as well (regardless of their experience level).

PRBS and my car radio "mix" function

On sunny days I ride my bicycle to work, but on rainy days I chicken out and use the car for the 6km I have to go. Since I don't often like what is on the radio, I decided to go through my collection of CDs and choose the 200 or so songs I would like to listen to in the car and burn them as mp3s on a single CD (Don't ask how much time this took). Unfortunately, if you just pop in the CD and press play, the songs play in alphabetical order. Luckily enough, my car CD player has a "mix" option. So far so good, but after a while I started to notice that when using the "mix" option, always song 149 is followed by song 148, which in turn is followed by song 18,

and believe me this is annoying to the bone. The whole idea of "mixing" is that you don't know what to expect next!

I assume that the "mix" function is accomplished by some sort of PRBS generator, which explains the deterministic order of song playing. But my advice to you if you design a circuit of this sort (for a CD player, or whatever), is to introduce some sort of true randomness to the system. For example, one could time the interval between power-up of the radio and the first human keystroke on the CD player and use this load the PRBS generator as a seed value, thus producing a different starting song for the play list each time. This however, does not solve the problem of the song playing order being deterministic. But given such a "random" number from the user once could use it to generate an offset for the the PRBS generator making it "jump" an arbitrary number of steps instead of the usual one step.

My point was not to indicate that this is the most clever way to do things, but I do think that with little effort one could come up with slightly more sophisticated systems, that make a big difference.

Posted in General | 1 Comment »

The Ultimate Interview Question for Logic Design – A Mini Challenge

July 9, 2007

I had countless interviews, with many different companies, large corporations and start ups. For some reason in almost all interviews, which were done in Israel, a single question popped up more often than others (maybe it is an Israeli High-Tech thing…).

Design a clock divide-by-3 circuit with 50% duty cycle

The solution should be easy enough even for a beginner designer. Since this is such a popular question, and since I am getting a decent amount of readers lately, I thought why not make a small challenge – try to find a solution to this problem with minimum hardware.

Please send me your solutions by email – can be found on the about me page.

Posted in Interview Questions | 5 Comments »

Puzzle #5 – Binary-Gray counters – solution

July 5, 2007

The binary-Gray puzzle from last week generated some flow of comments and emails. Basically, the important point to notice is the amount each counter toggles while going through a complete counting cycle.

For Gray coded counter, by definition only one bit changes at a time. Therefore, for an n stage counter we get 2^n toggling events for a complete counting cycle.

For binary coded n-bit counter, we have 2^(n+1)-2 toggling events for a complete counting cycle. you could verify this by

1. Taking my word for it (don't – check it yourself)
2. Writing down manually the results for a few simple cases and convince yourself it is so
3. Calculate the general case, but you have to remember something about how to calculate the sum of a simple series (best way)

Anyways, given the above assumptions and the fact that per bit the Gray counter consumes 3 times more power (2 times more would also just work, but the difference would be a constant), the Gray counter will always consume more power.
$3*2^n > 2^{(n+1)} - 2$

Posted in Puzzles | Leave a Comment »

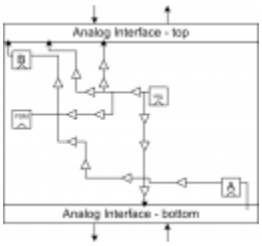
Some Layout Considerations

July 1, 2007

I work on a fairly large chip. The more reflect on what could have been done better, the more I realize how important floor planning is and how important is the concept work of identifying long lines within the chip and tackling these problems in the architectural planning phase.

The average digital designer will be happy if he finished his HDL coding, simulated it and verified it is working fine. Next he will run it through synthesis to see if timing is OK and job done, right? wrong! There are many problems that simply can't surface during synthesis. To name a few: routing congestion, cross talk effects and parasitics etc. This post will try concentrate on another issue which is much easier to understand, but when encountering it, it is usually too late in the design to be able to do something radical about it – the physical placement of flip-flops.

The picture below shows a hypothetical architecture of a design, which is very representative of the problems I want to describe.



Flop A is forced to be placed closed to the analog interface at the bottom, to have a clean interface to the digital core. In the same way Flop B is placed near the top, to have a clean interface to the analog part at the top. The signal between them, needs to physically cross the entire chip. The layout tools will place many buffers to have clean sharp edges, but in many cases timing is violated. If this signal has to go through during one clock period, you are in trouble. Many times it is not the case, and pipeline stages can be added along the way, or a multi-cycle path can be defined.
Most designers choose to introduce pipeline stages and to have a cleaner synthesis flow (less special constraints).

The other example shown in the diagram is a register that has loads all over the design. It drives signals in the analog interfaces as well as some state machines in the core itself. Normally, this is not a single wire but an entire bus and pipelining this can be very expensive. In a typical design there are hundreds of registers controlling state machines and settings all over the chip, with wires criss crossing by the thousands. Locating the bad guys should be done as soon as possible.

Some common solutions are:

1. Using local decoding as described on this post
2. Reducing the width of your register bus (costs in register read/write time)
3. Defining registers as quasi-static – changeable only during the power up sequence, static during normal operation

Posted in Layout | Leave a Comment »

Resource Sharing vs. Performance

June 27, 2007

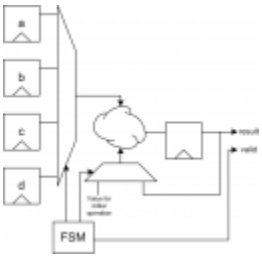
I wanted to spend a few words on the issue of resource sharing vs. performance. I believe it is trivial for most engineers but a few extra words won't do any harm I guess.
The issue is relevant most evidently when there is a need to perform a "heavy" or "expensive" calculation on several inputs in a repeated way.

The approaches usually in consideration are: building a balanced tree structure, sequencing the operations, or a combination of the two.

A tree structure architecture is depicted below. The logic cloud represents the "heavy" calculation. One can see immediately that the operation on a,b and c,d is done in parallel and thus saves latency on the expense of instantiating the logic cloud twice.



The other common solution, depicted below, is to use the logic cloud only once but introducing a state machine which controls a MUX, that determines which values will be calculated on the next cycle. The overhead of designing this FSM is minimal (and even less). The main saving is in using the logic cloud only once. Notice that we pay here in throughput and latency! With some more thinking, one could also save a calculation cycle by introducing another MUX in the feedback path, and using one of the inputs just for the first calculation, thereafter using always the feedback path.

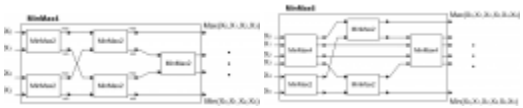


Posted in Architecture, General | Tagged Faster Design, Resource | 1 Comment »

Puzzle #4 – Solution

June 24, 2007

Here are the block diagrams for the solution of the MinMax problem.



Posted in Interview Questions, Puzzles | Leave a Comment »

Puzzle #6 – The Spy – (A real tough one…)

June 22, 2007

This one I heard a while back and saw that a version of it also appears in Peter Winkler's excellent book Mathematical Puzzles – A Connoisseur's Collection. Here is the version that appears in the book:

A spy in an enemy country wants to transmit information back to his home country.
The spy wants to utilize the enemy country's daily morning radio transmission of 15-bits (which is also received in his home country). The spy is able to infiltrate the radio station 5 minutes before transmission time, analyze the transmission that is about to go on air, and can either leave as it is, or flip a single bit somewhere in the transmission (a flip of more than one bit would make the original transmission too corrupt).

how much information can the spy transmit to his operators?

remember:

- The transmission is most likely a different set of 15-bits each day but can also repeat the last day's transmission. Best, assume it is random
- The spy is allowed to change a maximum of 1 bit in any position
- The spy has agreed on an algorithm/strategy with his operators before he was sent to the enemy country
- No other information or communication is available. the communication is strictly one way
- The spy sees for the first time the intended daily transmission 5 minutes before it goes on the air, he does not hold a list of all future transmissions
- The information on the other end should be extracted in a **deterministic** way

I believe this one is too tough for an interview question – it took me well over an hour to come up with a solution (well, that actually doesn't say much…). Anyways, this is definitely one of my favorite puzzles.

Posted in Puzzles | 15 Comments »



Non-Readable Papers

June 19, 2007

I actually enjoy surfing the web and reading technical papers which are somewhat related to my work. A lot of the good stuff appears in books, but if you want to find the coolest techniques and breakthrough ideas, they naturally first appear in technical papers.

I have to admit I don't like the format used by the standard technical papers, some of them seem to be made non-readable on purpose. Here is a real paper that can compete for the dubious title of being *the* most non-readable paper around.

Here is one of my papers. Before you continue, stop and try digesting what was written…

If you made through the first page, consider yourself a hero. That "technical paper" was generated *automatically* using SCIgen.

I bet a lot of people would be impressed if you present a list of papers generated by this service. A sort of a high-tech "emperor's new cloths" syndrome – no one wants to admit he doesn't understand a technical paper describing some "major" work in his own field…

Posted in General, Personal | Leave a Comment »

Puzzle #3 – Solution

June 19, 2007

This post is written only for completeness reasons. The answer to puzzle #3 was almost immediately given in the comments. I will just repeat it here.
The important observations are that XOR (X,X) = 0 and that XOR(X,0) = X The solution is therefore:

```
Operation       Result
---------------------------------
X = XOR(,)      X^Y,Y
Y = XOR(,)      X^Y,X^Y^Y = X
X = XOR(,)      X^X^Y = Y,X  done!
```

Posted in Interview Questions, Puzzles | Leave a Comment »

Low Power Techniques – Reducing Switching

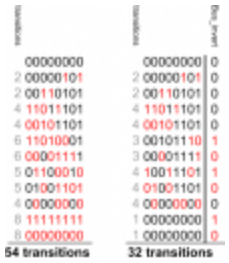
June 15, 2007

In one of the previous posts we discussed a cool technique to reduce leakage current. This time we will look at dynamic power consumption due to switching and some common techniques to reduce it.

Usually, with just a little bit of thinking, reduction of switching activity is quite possible. Let's look at some examples.

- **Bus inversion**

Bus inversion is an old technique which is used a lot in communication protocols between chip-sets (memories, processors, etc.), but not very often between modules within a chip. The basic idea is to add another line to the bus, which signals whether to invert the entire bus (or not). When more than half of the lines needs to be switched the bus inversion line is asserted. Here is a small example of a hypothetical transaction and the comparison of amount of transitions between the two schemes.



If you studied the above example a bit, you could immediately see that I manipulated the values in such a way that a significant difference in the total amount of transitions is evident.

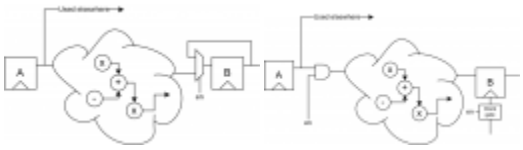
- **Binary Number Representation**

The two most common binary number representation in applications are 2′s complement and signed magnitude, with the former one usually preferred. However, for some very specific applications signed digit shows advantages in switching. Imagine you have a sort of integrator, which does nothing more than summing up values each clock cycle. Imagine also that the steady state value is around 0, but fluctuations above and below are common. If you would use 2′s complement going from 0 to -1 will result in switching of the entire bit range (-1 in 2′s complement is represented by 111….). If you would use signed digit, only 2 bits will switch when going from 0 to -1.



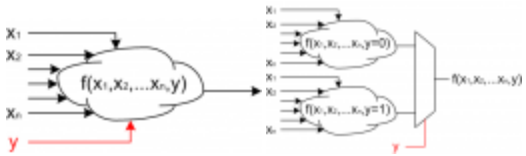
- **Disabling/Enabling Logic Clouds**

When handling a heavy logic cloud (with wide adders, multipliers, etc.) it is wise to enable this logic only when needed.
Take a look at the diagrams below. On the left implementation, only the flop at the end of the path – flop “B” has an enable signal, since flop “A” could not be gated (its outputs are used someplace else!) the entire logic cloud is toggling and wasting power. On the right (no pun intended) implementation, the enable signal was moved before the logic cloud and just for good measures, the clock for flop “B” was gated.



- **High Activity Nets**

This trick is usually completely ignored by designers. This is a shame since only power saving tools which can drive input vectors on your design and run an analysis of the active nets, might be able to resolve this.
The idea here is to identify the nets which have high activity among other very quiet nets, and to try to push them as deep as possible in the logic cloud.



On the left, we see a logic cloud which is a function of X1..Xn,Y. X1..Xn change with very low frequency, while Y is a high activity net. On the implementation on the right, the logic cloud was duplicated, once assuming Y=0 and once for Y=1, and then selecting between the 2 options depending on the value of Y. Often, the two new logic clouds will be reduced in size since Y has a fixed value there.

Posted in Coding Style, Low Power | 3 Comments »

A Short Note on Automatic Clock Gates Insertion

June 13, 2007

As we discussed before, clock gating is one of the most solid logic design techniques, which one can use when aiming for low power design.
It is only natural that most tools on the market support an automatic clock gating insertion option. Here is a quote from a synopsys article describing their power compiler tool

…Module clock gating can be used at the architectural level to disable the clock to parts of the design that are not in use. Synopsys' Power Compiler™ helps replace the clock gating logic inserted manually, gating the clock to any module using an Integrated Clock Gating (ICG) cell from the library. The tool automatically identifies such combinational logic…

But what does it really mean? What is this combinational logic that the tool "recognizes"?

The answer is relatively simple. Imagine a flip-flop with an enable signal. Implementation wise, this is done with a normal flip-flop and a MUX before with a feedback path to preserve the logical value of the flop when the enable is low. This is equivalent to a flop with the MUX removed and the enable signal controlling the enable of a clock gate cell, which in turn drives the clock for the flip-flop.

The picture below is better than any verbal explanation.



Posted in Synthesis | 4 Comments »

Puzzle #5 – Binary-Gray

June 12, 2007

Assuming you have an n-bit binary counter, made of n identical cascaded cells, which hold the corresponding bit value. Each of the binary cells dissipates a power of P units only when it toggles.
You also have an n-bit Gray counter made of n cascaded cells, which dissipates 3P units of power per cell when it toggles.

You now let the counters run through an entire cycle (2^n different values) until they return to their starting position. Which counter burns more power?

Posted in Gray Codes, Interview Questions, Puzzles | 13 Comments »



Low Power – Clock Gating Is Not The End Of It…

June 12, 2007

A good friend of mine, who works for one of the micro-electronics giants, told me how *low power* is the buzz word today. They care less about speed/frequency and more about minimizing power consumption.

He exposed me to a technique in logic design I was not familiar with. It is basically described in this paper. Let me just give you the basic idea.

The main observation is that even when not active, logic gates have different leakage current values depending on their inputs. The example given in the article shows that a NAND gate can have its leakage current reduced by almost a factor of 2.5 depending on the inputs!
How is this applied in reality? Assume that a certain part of the design is clock gated, this means all flip-flops are inactive and in turn the logic clouds between them. By "muxing" a different value at the output of the flop, which is logic dependent, we could minimize the leakage through the logic clouds. When waking up, we return to the old stored value.

The article, which is not a recent work by the way, describes a neat and cheap way of implementing a storage element with a "sleep mode" output of either logic "1″ or logic "0″. Notice that the "non-sleep mode" or normal operation value is still kept in the storage element. The cool thing is, that this need not really be a true MUX in the output of the flop – after finalizing the design an off-line application analyzes the logic clouds between the storage elements and determines what values are needed to be forced during sleep mode at the output of each flop. Then, the proper flavor of the storage element is instantiated in place (either a "sleep mode" logic "0″ or a "sleep mode" logic "1″).

It turns out that the main problem is the analysis of the logic clouds and that the complexity of this problem is rather high. There is also some routing overhead for the "sleep mode" lines and of course a minor area overhead.

I am interested to know how those trade-offs are handled. As usual, emails and comments are welcome.

Bottom line – this is a way cool technique!!!

Posted in Cool Circuits, General, Low Power | 2 Comments »

Puzzle #4 – The min-max question

June 8, 2007

Here is a question you are bound to stumble upon in one of your logic design job interviews, why? I don't know, I personally think it is pretty obvious, but what do I know…

MinMax2 is a component with 2 inputs – A and B, and 2 outputs – Max and Min. You guessed it, you connect the 2 n-bit numbers at the inputs and the component drives the Max output with the bigger of the two and the Min output with the smaller of the two.

Your job is to design a component – **MinMax4**, with 4 inputs and 4 outputs which sorts the 4 numbers using only **MinMax2** components. Try to use as little as possible **MinMax2** components.

If you made it so far, try making a **MinMax6** component from **MinMax2** and **MinMax4** components.

For bonus points – how many different input sequences are needed to verify the *logical* behavior of MinMax4?

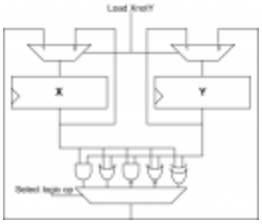
Posted in Interview Questions, Puzzles | 11 Comments »

Puzzle #3

June 4, 2007

OK, you seem to like them so here is another puzzle/interview question.

In the diagram below both X and Y are n-bit wide registers. With each clock cycle you could select a bit-wise basic operation between X and Y and load it to either X or Y, while the other register keeps its value.
The problem is to exchange the contents of X and Y. Describe the values of the "select logic op" and "load XnotY" signals for each clock cycle.



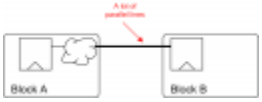
Posted in Interview Questions, Puzzles | 6 Comments »

Big Chips – Some Low Power Considerations

June 2, 2007

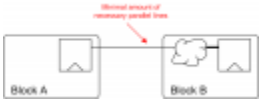
As designers, especially ones who only code in HDL, we don't normally take into account the physical size of the chip we are working on. There are many effects which surface only past the synthesis stage and when approaching the layout.

As usual, let's look at an example. Consider the situation described on the diagram below.



Imagine that block A and B are located physically far from one another, and could not be placed closer to one another. If the speeds we are dealing with are relatively high, it may very well be that the flight time of the signals from one side of the chip to another, already becomes too critical and even a flop to flop connection without any logic in between will violate setup requirements!
Now, imagine as depicted that many signals are sent across the chip. If you need to pipeline, you would need to pipeline a lot of parallel lines. This may result in a lot of extra flip-flops. Moreover, your layout tool will have to put in a lot of buffers to keep sharp edged signals. From architectural point of view, decoding globally may sound attractive at first, since you only need to do it once but can lead to a very power hungry architecture.

The alternative is to send as less long lines as possible across the chip, As depicted below.



With this architecture block B decodes the logic locally. If the lines sent to block B, need also to be spread all over the chip, we definitely pay in duplicating the logic for each target block.

There is no strict criteria to decide when to take the former or the latter architectures, as there is no definite crossover point. I believe this is more of a feeling and experience thing. It is just important to have this in mind when working on large designs.

Posted in Layout, Low Power | 1 Comment »

Synchronization of Buses

June 1, 2007

I know, I know, it is common knowledge that we never synchronize a bus. The reason being the uncertainty of when and how the meta-stability is resolved. You can read more about it in one of my previous posts.

A cool exception of when bus synchronization would be safe, is when you guarantee that:

1. On the sender side, one bit only changes at a time – Gray code like behavior
2. On the receiver (synchronized bus) side, the sampling clock is fast enough to allow only a single bus change

Just remember that both conditions must be fulfilled.

It is important to note that this can still be dangerous when the sender and receiver have the same frequency but phase is drifting! why???

Are there any other esoteric cases where one could synchronize a bus? comments are welcome!

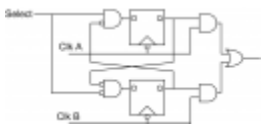
Posted in General, Gray Codes | Tagged Bus, Gray Code, Synchronization | 4 Comments »

Clock Muxing

May 29, 2007

Glitch free clock muxing is tricky. Some designers take it on the safe side and disable both clocks, do the switch and enable the clocks back on. Actually, I do not intend to discuss all the details of glitch-free clock muxing, a nice and very readable article can be found here.

If you finished reading the article above and are back with me, I want you to take a closer look at the second implementation mentioned. Here is a copy of the circuit for your convenience



The key question addressed by the author of the article is what happens if the select signal violates setup and hold conditions on one of the flip-flops? Apparently the flip-flop would go meta-stable and a glitch might occur, right? After all why was the synchronizer introduced in the 3rd circuit on the article. Well take a closer look!!

On closer look we see that both flip-flops operate on the *falling* edge of the clock, this means that a meta-stable state can occur when the clock is transitioning from a high to a low. But, since after the transition the clock is low, the AND gate immediately after the flop will block the unstable flop value for the entire low period of the clock. Or in other words the meta-stability has the entire low period of the clock to resolve and will not propagate through during this time. Isn't that absolutely cool??!!

I have to admit that upon seeing this circuit for the first time I missed this point, only after reading one of the application notes at Xilinx it dawned on me. The link can be found here (item #6)

Posted in Cool Circuits | 5 Comments »



The 5 Types of Technical Interview Questions

May 28, 2007

As I mentioned before, one of the most popular topics of this blog is the "interview questions" section. The following post tries to sort out the different types of *technical* interview questions one should expect.

- **The Logic Puzzle**

The logic puzzle is a favorite of many interviewers. The basic premise is that you are given a relatively tough logical puzzle (not necessarily related to digital design) which naturally, you should aim to solve. I used to belong to this school of thought and when interviewing people for a job used to try a few puzzles on them. The reason behind giving a non design related puzzle is that you want to try to assess how the person handles a problem which he never encountered before. The problem with this approach in my opinion is that the majority of puzzles have a trick or a shortcut to the answer, which makes them so elegant and differ from "normal" questions. These shortcut are not always easily detected under the pressure of an interview, moreover, who says that if you know how to solve a mathematical puzzle you know how to design good circuits?
Tips: If you do get this kind of question and you heard the puzzle before – admit it. If you encounter difficulties remember to think out loud.
Bottom line: I love puzzles, especially tough mathematical ones, but still I do not think it is the right approach to test for a job position.

- **The "We Don't Know the Answer to This One As Well" Question**

I actually got this one in an interview once. I can only guess that the interviewer either hopes that one of the candidates will solve the problems he (the interviewer) was unable to, or to see whether the candidate encounters the same problems/issues/pitfalls the interviewer has already experienced. I believe those kind of questions are well suited for a complicated algorithm or state machine design. I can see the merits of asking such a question, as the thought process of the

candidate is the interesting point here.
Tips: Think out loud. Maybe you can't say how something should be done, but if something can't be done in a certain way, say why it is not a good idea to do so.
Bottom Line: This could be an interesting approach to test candidates – I just never tried it myself…

- **The "Design A…" Question**

This type of question is the most common among them all. In my opinion, it is also the most effective for a job interview. The question directly deals with issues encountered at the job's environment. If the interviewer is smart, he will ask a sort of incremental question, adding more details as you move along. This is very effective because he can very easily "feel" the ground and detect what are the weak and strong points of the candidate. Many of the questions start simple and as you move along the interviewer will try to throw in problems or obstacles.
Tips: Study some good solid principles of digital design (e.g. synchronization issues, synthesis optimization, DFT etc.). When you get stuck, ask for help – since the question is usually incremental it is better to get some help in the beginning than to screw the entire thing up.
Bottom Line: The best and most fair way to test a candidate.

- **The "Code me A … in Verilog/VHDL" Question**

you might come across this kind of question sometime in the middle of the interview, where you interviewer tries to see how much hands-on experience you got.
Tips: Learn the basic constructs of an HDL i.e. learn how a flip-flop is described, Latch, combinational always/process etc.
Bottom Line: I believe this is a stupid approach for an interview question. In my opinion, the concept and principle of how to design a circuit is much more important than the coding (which we all anyway cut-and-paste…)

- **The "Tell Us About a Design You Made" Question**

This should be pretty obvious. Just remember to talk about a relatively small design you did – nobody has time or interest to hear about 4000 lines of code you had in a certain block. A very important point is to understand tricky points and to be able to say *why* you designed it like you did. Not less important is to know why you *didn't* choose certain strategies.
Tips: Be well prepared, if you can't tell about a design you did in detail, chances are you left a bad impression.
Bottom Line: This question is inevitable – expect it.

Posted in Interview Questions | 1 Comment »

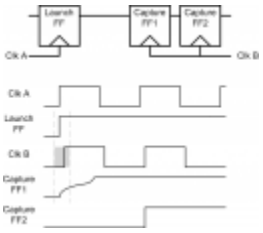
Synchronization, Uncertainty and Latency

May 28, 2007

I noticed that most of the hits coming from search engines to this blog contain the word “synchronization” or “interview questions”. I guess people think this is a tricky subject. Therefore another post on synchronization wouldn’t hurt…

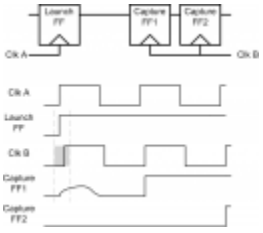
- **Synchronization**

Why do we need to synchronize signals at all? Signals arriving unrelated to the sampling clock might violate setup or hold conditions thus driving the output of the capturing flip-flop into a meta-stable state, or simply put, undefined. This means we could not guarantee the validity of the data at the output of the flip-flop. We do know, that since a flip-flop is a bi-stable device – after some (short) time the output will resolve either a logic “0″ or a logic “1″. The basic idea is to block the undefined (or meta-stable) value during this settling time from propagating into the rest of the circuit and creating havoc in our state machines. The simplest implementation is to use a shift register construction as pictured



- **Uncertainty**

We must remember, that regardless of the input transition, a meta-stable signal can resolve to either a logic “0″ or a logic “1″ after the settling time. The picture below is almost identical to the first, but here capture FF1 settled into a logic ’0″ state. On the next clk B rising edge it will capture a static “1″ value and thus change. Compare the timing of capture FF1 and capture FF2 in both diagrams. We see there is an inherent uncertainty on *when* capture FF2 assumes the input data. This uncertainty is one clk B period for the given synchronizer circuit.



- **Latency**

Sometimes, the uncertainty described can hurt the performance of a system. A trick which I don’t see used so often, is to use the falling edge triggered flop as one of the capture flops. This reduces the uncertainty from 1-2 capturing clock cycles to 1-1.5 capturing clock cycles. Sometimes though, there is no meaning to this uncertainty, it becomes more meaningful when there is only a phase difference between the 2 clock domains

Posted in Architecture, General | Tagged Synchronization, Synchronizer, Uncertainty | 2 Comments »

The “Bible” of Digital Design

May 26, 2007

This post will be very short.
Question – What is *the* book on digital design?
Answer – “CMOS VLSI design: A Circuits and Systems Perspective (3rd edition)”. If you don’t own it don’t call yourself a serious designer

Amazon link here

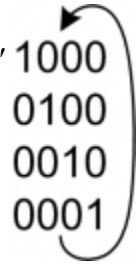
Posted in General | 1 Comment »

Designing Robust Circuits

May 25, 2007

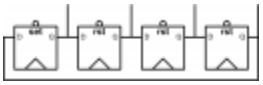
There are many ways to design a certain circuit. Moreover, there are many trade-offs like power, area, speed etc.
In this post we will discuss a bit about robustness and as usual, we will use a practical, real life example to state our point.

When one talks about robustness in digital design, one usually means that if a certain type of failure occurs during operation the circuit does not need outside “help” in order to return to a defined or at least allowed state. Maybe this is a bit cryptic so let’s look at a very simple example – a ring counter.



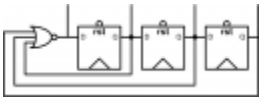
As pictured on the right a 4 bit ring counter has 4 different states, with only a single “1″ in each state. “counting” is performed by shifting or more correctly rotating the “1″ to one direction with each rising clock edge. Ring counters have many uses, one of the most common is as a pointer for a synchronous FIFO. Because of their simplicity, one finds them many times in high speed full custom designs. Ring counters have only a subset of all possible states as *allowed or legal states*. For example, the state “1001″ is not allowed.

A very simple implementation for a ring counter is the one depicted below. The 4 flip-flops are connected in a circular shift register fashion. Three of the registers have an asynchronous reset pin while the left most has an asynchronous set pin. When going into the reset state the ring counter will assume the state “1000″.



Now, imagine that for some reason (inappropriate reset removal, cross talk noise etc.) the state "1100″ appeared in the above design – an illegal state. From now on, the ring counter will always toggle between illegal states and this situation will continue until the next asynchronous reset is de-asserted. If a system is noisy, and such risk is not unthinkable, hard reseting the entire system just to bring the counter to a known state might be disastrous.

Let's inspect a different, more robust design of a ring counter in the picture below.



With the new implementation the NOR gate is functioning as the left most output. But more important, the NOR gate will drive "0″s into the 3-bit shift register until all 3-bits are "0″, then a "1″ will be driven. If we look at a forbidden or illegal state like "0110″, we see that the new circuit will go through the following states: "0110″–>"0011″–>"0001″ until it independently reaches a legal state! This means we might experience an unwanted behavior for a few cycles but we would not need to reset the circuit to bring it back to a legal state.

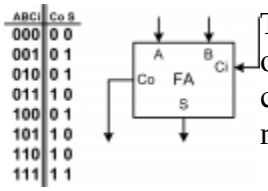
In a later post, when discussing Johnson counters, we will see this property again.

Posted in Coding Style | 2 Comments »

Puzzle #2 – Solution

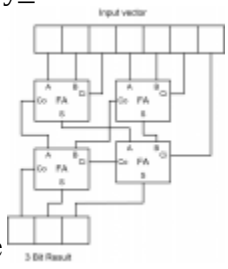
May 23, 2007

4 full-adder units are necessary to count the amount of "1″s in a 7-bit vector.



The most important thing to notice is that a full-adder "counts" the amount of "1″s of it's inputs. If you are not convinced , then a brief look in the component's truth table will prove this to you. The output is a binary represented 2-bit number.

The next picture shows how to connect the four full-adders in the desired way. The first stage generates two 2-bit numbers, each represents the amount of "1″s among its respected three input bits. The second stage adds those two binary numbers together and uses the carry_in of one full-adder for the 7th bit.



As I mentioned when I posted the puzzle, I used this in an actual design. In clock and data recovery circuits (CDRs) it is necessary to integrate the amount of "ups" and "downs" a phase detector outputs (if this tells you nothing, please hold on till the CDR post I am planning). Basically, you receive two vectors of a given length, one represents "ups" the other "downs". You have to sum up the

amount of "1″s in each vector and subtract one from the other. Summing up the amount of "1″s is done using this full-adder arrangement. Another way would be using a LUT (posts on LUTs are planned as well…).

Posted in Interview Questions, Puzzles | Leave a Comment »

Late Arriving Signals

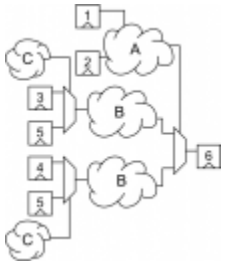
May 23, 2007

As I mentioned before, it is my personal opinion that many digital designers put themselves more and more further away from the physical implementation of digital circuits and concentrate more on the HDL implementations. A relatively simple construction like the one I am about to discuss, is already quite hard to debug directly in HDL. With a visual aid of how the circuit looks like, it is much easier (and faster) to find a solution.

The classic example we will discuss is that of a late arriving signal. Look at the picture below. The critical path through the circuit is along the red arrow. Let's assume that there is a setup violation on FF6.
Let's also assume that in this example the logic cloud marked as "A", which in turn controls the MUX that chooses between FF3 and FF4, is quite heavy. The combination of cloud "A" and cloud "B" plus the MUXes in sequence is just too much. But we have to use the result of "A" *before* calculating "B"! What can be done?



The most important observation is that we could duplicate the entire logic that follows "A". We assume for the duplicated blocks that one time the result of "A" was a logic "0″ and in another logic "1″. Later we could choose between the two calculations. Another picture will make it clearer.
Notice how the MUX that selected between FF3 and FF4 has vanished. There is now a MUX that selects between FF3 and FF5 ("A" result was a "0″) and a MUX in the parallel logic that selects between FF4 and FF5 ("A" result was a "1″) .
At the end of the path we introduced a new MUX which selects between the two calculations we made, this time depending on cloud "A". It is easy to see that although this implementation takes more area due to the duplicated logic, the calculation of the big logic clouds "A" and "B" is done in parallel rather than in series.



This technique is relatively easy to implement and to spot if you have a circuit diagram of your design. Also do not count on the synthesis tool to do this for you. It might be able to do it with relatively small structures but when those logic clouds get bigger, you should implement this trick on your own – you will see improvements in timing (and often in synthesis run time). What you pay for is area and maybe power – nothing comes for free…

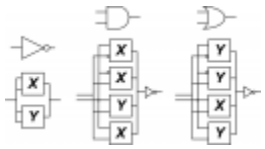
Posted in Coding Style, Synthesis | Leave a Comment »

Puzzle #1 – Solution

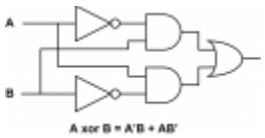
May 23, 2007

The key observation to the solution of this puzzle is to note that the *outputs* of components can be connected together given than only one drives a non high-Z value. If you realized that 90% of the way to solving this puzzle is behind you.

The second step is to realize a "NOT" gate using both the "X" and "Y" components. When you know how to do that an "OR" and an "AND" gate realization are quite simple.
The figure below sums up the construction of "NOT", "OR" and "AND gates from various instances of "X" and "Y".



The next step is quite straightforward. We combine the gates we constructed and make an "XOR" gate as follows:



This is by no means the most efficient solution in terms of minimum "X" and "Y" components.

Posted in Interview Questions, Puzzles | 5 Comments »

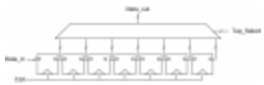
Do You Think Low Power???

May 20, 2007

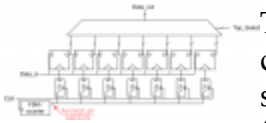
There is almost no design today, where low power is not a concern. Reducing power is an issue which can be tackled on many levels, from the system design to the most fundamental implementation techniques.

In digital design, clock gating is the back bone of low power design. It is true that there are many other ways the designer can influence the power consumption, but IMHO clock gating is the easiest and simplest to introduce without a huge overhead or compromise.

Here is a simple example on how to easily implement low power features.



The picture on the right shows a very simple synchronous FIFO. That FIFO is a very common design structure which is easily implementable using a shift register. The data is being pushed to the right with each clock and the tap select decides which register to pick. The problem with this construction is that with each clock *all* the flip-flops potentially toggle, and a clock is driven to all. This hurts especially in data or packet processing applications where the size of this FIFO can be in the range of thousands of flip-flops!!



The correct approach is instead of moving the entire data around with each clock, to "move" the clock itself. Well not really move, but to keep only one specific cell (or row in the case of vectors) active while all the other flip-flops are gated. This is done by using a simple counter (or a state machine for specific applications) that rotates a "one hot" signal – thus enabling only one cell at a time. Notice that the data_in signal is connected to all the cells in parallel,. When new data arrives only the cell which receives a clock edge in that moment will have a new value stored.

Posted in Coding Style, Low Power | 1 Comment »

Your Comments Are Welcome…

May 19, 2007

The title of this post is self explanatory. I would be happy to get emails from you on almost any subject related to this blog.
Let me know what you want to see, what you don't want to see or what you want to see changed.
My email can be found on the bottom of the about me page.

Posted in Uncategorized | 4 Comments »

Puzzle #2

May 19, 2007

OK, here is another nice puzzle, which actually has applications in real life!
This one was given to me on the same IBM interview sometime around 10 years ago.

Here goes.

Again we are dealing with the poor engineers in the land of Logicia. For some sort of fancy circuitry, a 7-bit binary input is received. As a result it should give the amount of "1″s present in this vector. For example, for the inputs 1100110 and 1001110 the result should be the same and equal to 100 (4 in binary). This time however, the only components they have on their hands are *Full Adders*. Describe the circuit with minimum amount of parts.

This puzzle is fairly easy, and as I mentioned before has found some practical uses in some of my designs. More on this when I'll give the answer.

Posted in Interview Questions, Puzzles | 5 Comments »

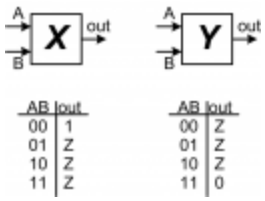
Puzzle #1

May 18, 2007

Since I am a big fan of puzzles, I will try to post here from time to time a few digital design related puzzles.

This particular one was given to me in an interview at IBM over 10 years ago.

Due to the war in the land of Logicia there is a shortage of XOR gates. Unfortunately, the only logic gates available are two weird components called "X" and "Y". The truth table of both components is presented below – Z represents a High-Z value on the output.
Could you help the poor engineers of Logicia to build an XOR gate?



AB	Iout
00	1
01	Z
10	Z
11	Z

AB	Iout
00	Z
01	Z
10	Z
11	0

Posted in Interview Questions, Puzzles | 8 Comments »

Another Synchronization Pitfall...

May 18, 2007

Many are the headaches of a designer doing multi clock domain designs. The basics that everyone should know when doing multi clock domain designs are presented in this paper. I would like to discuss on this post a lesser known problem, which is overlooked by most designers. Just as a small anecdote, this problem was encountered by a design team led by a friend of mine. The team was offered a 2 day vacation reward for anyone tracking and solving the weird failures that they experienced. I guess this already is a good reason to continue reading…

OK, we all know that when sending a control signal (better be a single one! – see the paper referenced above) from one clock domain to another, we must synchronize it at the other end by using a two stage shift register (some libraries even have a "sync cell" especially for this purpose).

Take a look at the hypothetical example below



Apparently all is well, the control signal, which is an output of some combinational logic, is being synchronized at the other end.
So what is wrong?
In some cases the combinational logic might generate a hazard, depending on the inputs. Regardless whether it is a static one (as depicted in the timing diagram) or a dynamic one, it is possible that exactly that point is being sampled at the other end. Take a close look at the timing diagram, the glitch was recognized as a "0″ on clk_b's side although it was not intended to be.

The solution to this problem is relatively easy and involves adding another sampling stage clocked with the sending clock as depicted below. Notice how this time the control signal at the other end was not recognized as a "0″. This is because the glitch had enough time to settle until the next rising edge of clk_a.



In general, the control signal sent between the two clock domains should present a strict behavior during switching- either 1–>0 or a 0–>1. Static hazards (1–>0–>1 or 0–>1–>0) or Dynamic hazards (1–>0–>1–>0 or 0–>1–>0–>1) are a cause for a problem.

Just a few more lines on synchronization faults. Quite often they might pop up in only *some* of the designs. You might have 2 identical chips, one will show a problem the other not. This can be due to slight process variations that make some logic faster or slower, and in turn generate a hazard exactly at the wrong moment.

Posted in Architecture, Coding Style | Tagged Design pitfalls, Synchronization, Synchronizer | 1 Comment »

Eliminating Unnecessary MUX structures

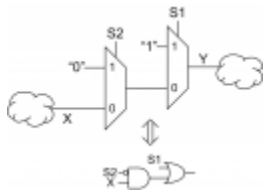
May 16, 2007

You will often hear engineers in our business saying something along these lines:
"I first code, and then let synthesis find the optimal implementation" or "synthesis tools are so good these days, there is no use in spending time on thinking in the circuit level…". Well not me – sorry!! I am a true fan of "helping" or "directing" the synthesis.
The example I will discuss on this post, is a real life example that occurred while reviewing a fellow engineer's work.

The block in discussion is quite a heavy one, with very tight timing requirements and complicated functionality (aren't they all like that…). Somewhere in the code I encountered this if-else-if statement (Verilog):

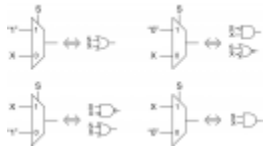
```
if (s1)
   y = 1'b1;
else if (s2)
         y = 1'b0;
      else
         y = x;
```

Now, if this would have stood on its own, it would not have risen much suspicion. But this statement happened to be part of the critical path. On first look, the if-else-if ladder is translated into a set of cascaded muxes, but looking carefully at it, one can simplify it into two gates (or even one complex gate in most libraries) as shown below.



I do not say that a good synthesis tool is not able to simplify this construction, and I have to admit I do not really know what is going on inside the optimization process – this seems to be some sort of black magic of our art – but fact is, that timing improved after describing the if-else-if statement explicitly as an or-and combination.
The reason can be, as depicted, that the muxes are being "dragged" somehow into the logic clouds just before and after them in hope of simplifying them there. *I just don't know!*
A good sign to spot when such simplification is easily possible, is when you have an if-else-if ladder or a case statement with constants on the right hand side (RHS). It does make the code a bit less readable, but IMHO it is worth it.

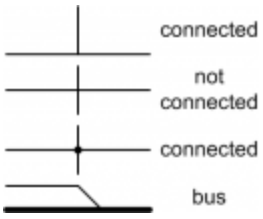
Here is a short summery of some common mux constructs with fixed inputs and their simplified forms.



Posted in Coding Style | 1 Comment »

A Short Note on Drawings Conventions

May 15, 2007



Posted in General | Leave a Comment »

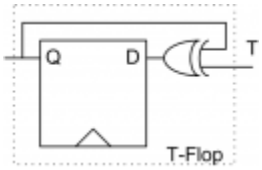
Counting in Gray – Part III – Putting Everything Together

May 14, 2007

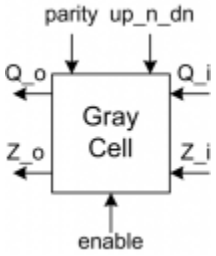
In the last post we built the basis for our Gray counter implementation. In this post we will combine all observations and create a "Gray bit cell" which could be instantiated as many times as one wishes to create Gray counters which count up or down and are of any desired length.

As mentioned before, the basic idea is to build a "Gray bit cell". Naturally it has a single bit output, but the cell also has to get the information from all previous cells whether or not a pattern was identified and whether it has to toggle or not.

The latter point reminds us that we will have to use T-Flops for the implementation, since the patterns we observed in the previous post only concern when a certain Gray bit toggles and not its absolute value. The most basic implementation of a T-Flop is presented on the figure on the right.

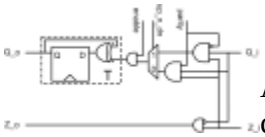


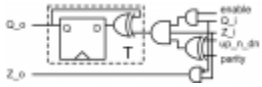
The abstract view of the Gray cell is presented to the left. Both the clock and reset inputs have been omitted. The cell inputs and outputs are:



- Q_o – Gray value of the specific bit (n)
- Q_i – The previous – n-1 – Gray bit value
- Z_i – All Gray bits n-2 down to 0 are "0"
- Z_o – All Gray bits n-1 down to 0 are "0"
- parity – Parity bit – or more correctly inverted parity
- up_n_dn – If "1" count up, if "0" count down
- enable – enable counting

Two implementations of the Gray cell are depicted below, the left one being more intuitive than the right, but the right one is more compact. Both implementations are logically identical.

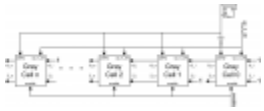




All that is left now is to see how to connect the Gray cells in series to produce a Gray up/down counter.
In the final picture the Gray cells were connected to form a Gray counter. Notice that some cells are connected in a special way:

- Cell 0 – Q_i and Z_i are both connected to "1", The parity input is inverted and Z_o left unconnected
- Cell 1 – Z_i connected to "1"
- Cell n (MSB) – Q_i is connected to "1", Z_o left unconnected

A few more words on the parity bit. In the given implementation it is generated by a normal D-Flop with its Qbar output connected to its D input. The same functionality can be achieved without this extra D-Flop, by using an Xor tree on the outputs of the Gray counter – remember our first observation from the previous post? the parity changes with each count.



That concludes this series of posts on Gray counters, but don't worry I promise there will be more interesting stuff coming on Gray codes.

Posted in Gray Codes | 1 Comment »

Counting in Gray – Part II – Observations

May 13, 2007

In the last post we discussed the different approaches, their advantages and disadvantages in terms of implementation, design requirements etc. We finished with the promise to have a solution for counting in Gray code, with registered outputs and which is could easily be described in HDL.

In this post we will observe some interesting facts concerning mirrored Gray codes, which in turn will lead us to our implementation.

Let's start.

One of the most important and basic things we can see when observing Gray codes, is that with each increment or decrement ***the parity of the entire number changes***. This is pretty obvious since each time only a single bit changes.

The Next observation is the “toggling period” of each of the bits in Gray representation. Bit 0, or the LSB has a “toggle period” of 2 – i.e. it flips each 2 counts. Bit 1 (one to the left of the LSB) has a “toggle period” of 4. In General with each move towards the MSB side, the toggle period doubles. An Exception is the MSB which has the same toggle period has the bit to its immediate right.
The top figure on the right demonstrates this property for a 5 bit Gray code.



The reason why this is true can be easily understood if we consider the way mirrored Gray codes are being constructed (which I assume is well known). Notice that this fact just tells us only the toggle period of each bit, not *when* it should toggle! To find this out, we will need our third observation.

Let us now look at when each bit flips with respect to its position. In order to help us, we will have to recall our first observation – parity changes with each count. The bottom figure on the right reveals the hidden patterns.



> In General: Gray bit n will toggle in the next cycle, when the bit to its immediate right is “1″ and all the other bits to its right are 0 – or in other words a 100…00 pattern
> The only exception is the MSB which toggles when all the bits to its right except the one to its immediate right are “0″ – or a X00…00 pattern

sounds complicated? look in the picture again, the pattern will just pop out to you.

You can take my word for it or check for yourself, anyways the rules for counting backwards (or down), in Gray, are:

> The LSB toggles when the parity bit is “0″
> For all the other bits: Gray bit n will toggle in the next cycle, when the bit to its immediate right is “1″, all the other bits to its right are 0 and the parity bit is “1″ – or in other words a 100…01 pattern

On the next post we will see how to use those observations to create a simple “gray bit cell”, which will be used as our building block for the final goal – the up/down Gray counter.

Posted in Gray Codes | 2 Comments »

Counting in Gray – Part I – The Problem

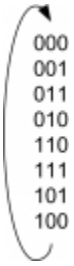
May 10, 2007

I love Gray codes, there I said it. I love to try to find different and weird applications for them. But Gray codes are one of those things where most designers heard of and know the principle they use – but when coming to implement a circuit based on Gray codes, especially when simple arithmetic is involved, things get complicated for them.

I don't really blame them since that stuff can get relatively tricky. Maybe it is best to show with an example…
This paper is a *must read* for any digital designer trying to design an asynchronous FIFO. All the major issues, corner cases and pitfalls are mentioned there, and I just can't recommend it enough.

But… what caught my attention was the implementation of the Gray counters in the design (page 2, section 2.0). Before we get into what was written, maybe a presentation of the problem is in place. Counting (i.e. only +1 or -1 operations on a vector is considered) in binary is relatively straight forward. We all learned to do this, and use it. The problem is how do you count in "Gray code" – i.e. given the 3-bit gray code number 111, what is the next number in line? (answer: 101)

The figure below shows the Gray code counting scheme for 3-bit "mirrored" Gray code (the most commonly used)



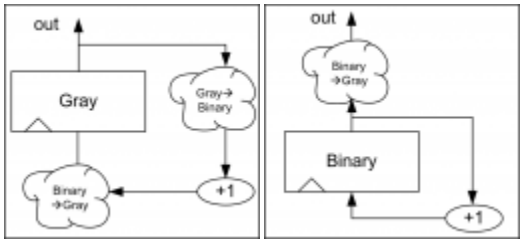
Look at any line, can you figure out what will be the next line based only on the line you look at??? If you think you know try figuring out what comes after 11011000010 ??? 😀

There are 2 very common approaches to solve this problem:

1. Convert to binary –> do a "+1" –> convert back to Gray
2. Use a Look-up-Table to decode the next state

Both have severe disadvantages.

Let's look through them one at a time.
Option 1, can be implemented in principle in two different ways (the plot thickens…)



The implementation on the left has the big advantage that the Gray output is registered, i.e. the values stored in the Flip-Flop are truly Gray. This is necessary when the output is used in an asynchronous interface (e.g. as a FIFO pointer).
The implementation on the right is faster though, with the disadvantage of the output being combinational.

The advantage of both implementations is that they are relatively compact to describe in HDL, even for wide counters and very flexible – e.g. one can add a "-1″ functionality quite easily.

Option 2, is basically a big LUT that describes the next Gray state of the counter.
The outputs will be truly registered, the implementation relatively fast, but very tedious to describe in HDL and prone to errors. just imagine a 7-bit Gray counter implemented as a big case statement with 128 lines. Now imagine that you would want to add a backward counting (or "-1″) operation.

The natural question asked is, isn't there a better implementation that gives us the best of both worlds? Registered outputs, fast and easily described in HDL. The answer is a big "YES", and I will show how to do it on my next post. That implementation will be even easy enough for entering it in schematic tools and using it in a full-custom environment!

hold on…

Posted in Gray Codes | 1 Comment »

First Post…

May 9, 2007

Hi, I really suck in writing so I will get straight to the point.
This weblog will mainly be of interest to fellow Electrical Engineers with emphasis on the different aspects of Digital Design.
I will try to contribute from my experience and understanding and try to present some tips, tricks and just plain cool ideas from my field. Some things will be relatively basic, other more advanced.
In general, I intend to update the site once a week or so, since most stuff will be technical it would be quite hard to come up with something new each day – that, plus the fact that I am lazy…

Hopefully in time a small database of goodies will be accumulated.
I will certainly make mistakes and sometimes even post complete nonsense, so I hope you guys will correct me and be understanding.

Nir

p.s. I also admit the name "Adventures in ASIC Digital Design" is pretty lame but I just couldn't come up with something better.

1. Why does the present VLSI circuits use MOSFETs instead of BJTs?
Answer

Compared to BJTs, MOSFETs can be made very small as they occupy very small silicon area on IC chip and are relatively simple in terms of manufacturing. Moreover digital and memory ICs can be implemented with circuits that use only MOSFETs i.e. no resistors, diodes, etc.

2. What are the various regions of operation of MOSFET? How are those regions used?
Answer

MOSFET has three regions of operation: the cut-off region, the triode region, and the saturation region.
The cut-off region and the triode region are used to operate as switch. The saturation region is used to operate as amplifier.

3. What is threshold voltage?
Answer

The value of voltage between Gate and Source i.e. V_{GS} at which a sufficient number of mobile electrons accumulate in the channel region to form a conducting channel is called threshold voltage (V_t is positive for NMOS and negative for PMOS).

4. What does it mean "the channel is pinched off"?
Answer

For a MOSFET when V_{GS} is greater than V_t, a channel is induced. As we increase V_{DS} current starts flowing from Drain to Source (triode region). When we further increase V_{DS}, till the voltage between gate and channel at the drain end to become V_t, i.e. $V_{GS} - V_{DS} = V_t$, the channel depth at Drain end decreases almost to zero, and the channel is said to be pinched off. This is where a MOSFET enters saturation region.

5. Explain the three regions of operation of a MOSFET.
Answer

Cut-off region: When $V_{GS} < V_t$, no channel is induced and the MOSFET will be in cut-off region. No current flows.
Triode region: When $V_{GS} \geq V_t$, a channel will be induced and current starts flowing if $V_{DS} > 0$. MOSFET will be in triode region as long as $V_{DS} < V_{GS} - V_t$.
Saturation region: When $V_{GS} \geq V_t$, and $V_{DS} \geq V_{GS} - V_t$, the channel will be in saturation mode, where the current value saturates. There will be little or no effect on MOSFET when V_{DS} is further increased.

6. What is channel-length modulation?
Answer

In practice, when V_{DS} is further increased beyond saturation point, it does has some effect on the characteristics of the MOSFET. When V_{DS} is increased the channel pinch-off point starts moving

away from the Drain and towards the Source. Due to which the effective channel length decreases, and this phenomenon is called as Channel Length Modulation.

7. Explain depletion region.
Answer

When a positive voltage is applied across Gate, it causes the free holes (positive charge) to be repelled from the region of substrate under the Gate (the channel region). When these holes are pushed down the substrate they leave behind a carrier-depletion region.

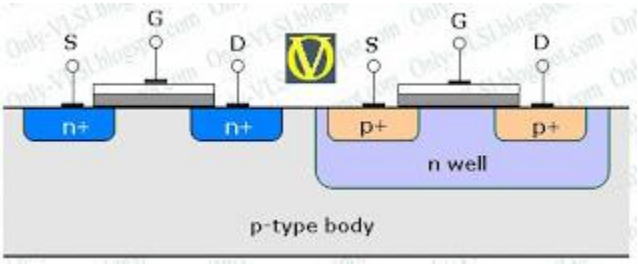
8. What is body effect?
Answer

Usually, in an integrated circuit there will be several MOSFETs and in order to maintain cut-off condition for all MOSFETs the body substrate is connected to the most negative power supply (in case of PMOS most positive power supply). Which causes a reverse bias voltage between source and body that effects the transistor operation, by widening the depletion region. The widened depletion region will result in the reduction of channel depth. To restore the channel depth to its normal depth the VGS has to be increased. This is effectively seen as change in the threshold voltage - V_t. This effect, which is caused by applying some voltage to body is known as body effect.

9. Give various factors on which threshold voltage depends.
Answer

As discussed in the above question, the V_t depends on the voltage connected to the Body terminal. It also depends on the temperature, the magnitude of V_t decreases by about 2mV for every 1^oC rise in temperature.

10. Give the Cross-sectional diagram of the CMOS.
Answer



Synchronous Reset VS Asynchronous Reset

Why Reset?

A Reset is required to initialize a hardware design for system operation and to force an ASIC

into a known state for simulation.

A reset simply changes the state of the device/design/ASIC to a user/designer defined state. There are two types of reset, what are they? As you can guess them, they are Synchronous reset and Asynchronous reset.

Synchronous Reset

A synchronous reset signal will only affect or reset the state of the flip-flop on the active edge of the clock. The reset signal is applied as is any other input to the state machine.

Advantages:

- The advantage to this type of topology is that the reset presented to all functional flip-flops is fully synchronous to the clock and will always meet the reset recovery time.
- Synchronous reset logic will synthesize to smaller flip-flops, particularly if the reset is gated with the logic generating the d-input. But in such a case, the combinational logic gate count grows, so the overall gate count savings may not be that significant.
- Synchronous resets provide some filtering for the reset signal such that it is not effected by glitches, unless they occur right at the clock edge. A synchronous reset is recommended for some types of designs where the reset is generated by a set of internal conditions. As the clock will filter the logic equation glitches between clock edges.

Disadvantages:

- The problem in this topology is with reset assertion. If the reset signal is not long enough to be captured at active clock edge (or the clock may be slow to capture the reset signal), it will result in failure of assertion. In such case the design needs a pulse stretcher to guarantee that a reset pulse is wide enough to be present during the active clock edge.
- Another problem with synchronous resets is that the logic synthesis cannot easily distinguish the reset signal from any other data signal. So proper care has to be taken with logic synthesis, else the reset signal may take the fastest path to the flip-flop input there by making worst case timing hard to meet.
- In some power saving designs the clocked is gated. In such designed only asynchronous reset will work.
- Faster designs that are demanding low data path timing, can not afford to have extra gates and additional net delays in the data path due to logic inserted to handle synchronous resets.

Asynchronous Reset

An asynchronous reset will affect or reset the state of the flip-flop asynchronously i.e. no matter what the clock signal is. This is considered as high priority signal and system reset happens as soon as the reset assertion is detected.

Advantages:

- High speeds can be achieved, as the data path is independent of reset signal.
- Another advantage favoring asynchronous resets is that the circuit can be reset with or without a clock present.
- As in synchronous reset, no work around is required for logic synthesis.

Disadvantages:

- The problem with this type of reset occurs at logic de-assertion rather than at assertion like in synchronous circuits. If the asynchronous reset is released (reset release or reset removal) at or near the active clock edge of a flip-flop, the output of the flip-flop could go metastable.
- Spurious resets can happen due to reset signal glitches.

Conclusion

Both types of resets have positives and negatives and none of them assure fail-proof design. So there is something called "Asynchronous assertion and Synchronous de-assertion" reset which can be used for best results. (which will be discussed in next post).

3 Comments

Labels: ASIC, Digital Design, Important Concepts, VLSI design

FPGA vs ASIC

Definitions

FPGA: A Field-Programmable Gate Array (FPGA) is a semiconductor device containing programmable logic components called "logic blocks", and programmable interconnects. Logic blocks can be programmed to perform the function of basic logic gates such as AND, and XOR, or more complex combinational functions such as decoders or mathematical functions. *For complete details click here.*

ASIC: An application-specific integrated circuit (ASIC) is an integrated circuit designed for a particular use, rather than intended for general-purpose use. Processors, RAM, ROM, etc are examples of ASICs.

FPGA vs ASIC

Speed
ASIC rules out FPGA in terms of speed. As ASIC are designed for a specific application they can be optimized to maximum, hence we can have high speed in ASIC designs. ASIC can have hight speed clocks.

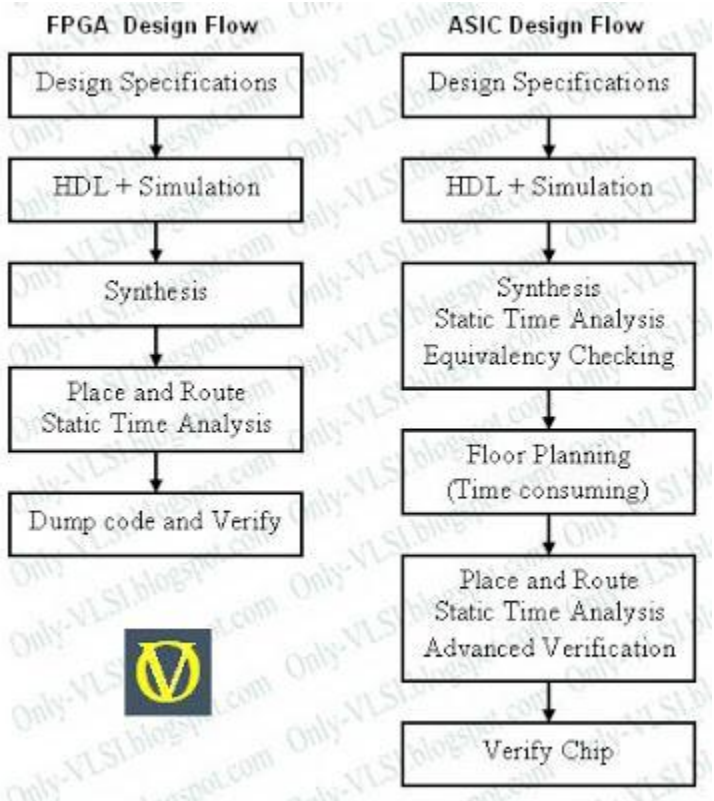
Cost
FPGAs are cost effective for small applications. But when it comes to complex and large volume

designs (like 32-bit processors) ASIC products are cheaper.

Size/Area
FPGA are contains lots of LUTs, and routing channels which are connected via bit streams(program). As they are made for general purpose and because of re-usability. They are in-general larger designs than corresponding ASIC design. For example, LUT gives you both registered and non-register output, but if we require only non-registered output, then its a waste of having a extra circuitry. In this way ASIC will be smaller in size.

Power
FPGA designs consume more power than ASIC designs. As explained above the unwanted circuitry results wastage of power. FPGA wont allow us to have better power optimization. When it comes to ASIC designs we can optimize them to the fullest.

Time to Market
FPGA designs will till less time, as the design cycle is small when compared to that of ASIC designs. No need of layouts, masks or other back-end processes. Its very simple: Specifications -- HDL + simulations -- Synthesis -- Place and Route (along with static-analysis) -- Dump code onto FPGA and Verify. When it comes to ASIC we have to do floor planning and also advanced verification. The FPGA design flow eliminates the complex and time-consuming floor planning, place and route, timing analysis, and mask / re-spin stages of the project since the design logic is already synthesized to be placed onto an already verified, characterized FPGA device.



Type of Design
ASIC can have mixed-signal designs, or only analog designs. But it is not possible to design

them using FPGA chips.

Customization
ASIC has the upper hand when comes to the customization. The device can be fully customized as ASICs will be designed according to a given specification. Just imagine implementing a 32-bit processor on a FPGA!

Prototyping
Because of re-usability of FPGAs, they are used as ASIC prototypes. ASIC design HDL code is first dumped onto a FPGA and tested for accurate results. Once the design is error free then it is taken for further steps. Its clear that FPGA may be needed for designing an ASIC.

Non Recurring Engineering/Expenses
NRE refers to the one-time cost of researching, designing, and testing a new product, which is generally associated with ASICs. No such thing is associated with FPGA. Hence FPGA designs are cost effective.

Simpler Design Cycle
Due to software that handles much of the routing, placement, and timing, FPGA designs have smaller designed cycle than ASICs.

More Predictable Project Cycle
Due to elimination of potential re-spins, wafer capacities, etc. FPGA designs have better project cycle.

Tools
Tools which are used for FPGA designs are relatively cheaper than ASIC designs.

Re-Usability
A single FPGA can be used for various applications, by simply reprogramming it (dumping new HDL code). By definition ASIC are application specific cannot be reused.

Dynamic Gates

Posted on October 4, 2012

Dynamic gates use clock for their normal operation as opposed to the static gates, which don't use clocks.

Dynamic gates use NMOS or PMOS logic. It doesn't use CMOS logic like regular static gates. Because it uses either NMOS or PMOS logic and not CMOS logic, it usually has fewer transistors compared to static gates. Although there are extra transistors given that it uses clocks.

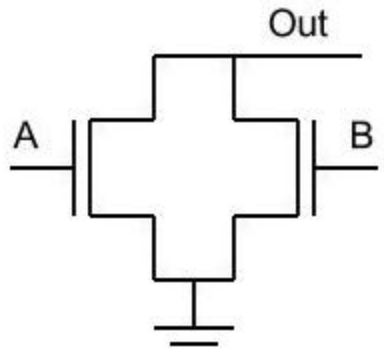


Figure : NMOS pull down logic for NOR gate.

The figure shows the pull down NMOS logic for a NOR gate. This pull down structure is used in the dynamic gates.

How dynamic gates work :

In static gates, inputs switch and after a finite input to output delay, output possibly switches to the expected state.

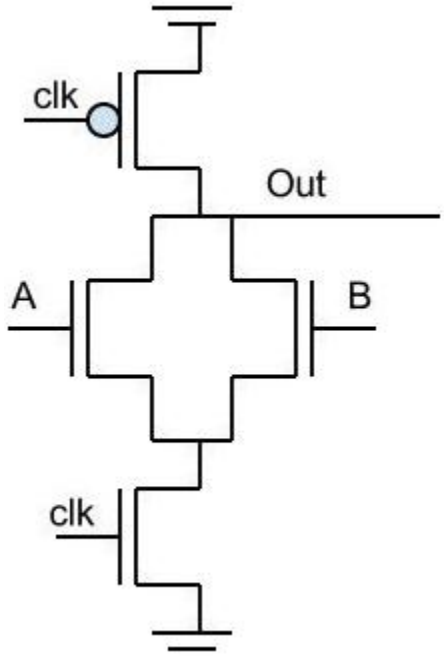


Figure : Dynamic NOR gate.

As you can see in the figure above, dynamic gate is made using NMOS pull down logic along with clock transistors on both pull up and pull down paths.

We know that clock has two phases, the low phase and the high phase. Dynamic gate has two operating phases based on the clock phases. During the low clock phase, because of the pmos gate on the pull up network, the output of dynamic gate is pre-charged to high phase. This is the pre-charge state of dynamic gate.

When the clock is at high phase, the output of dynamic gate may change based on the inputs, or it may stay pre-charged depending on the input. The phase of the dynamic gates, when the clock is high, is called the evaluate phase. As it is essentially evaluating what the output should be during this phase.

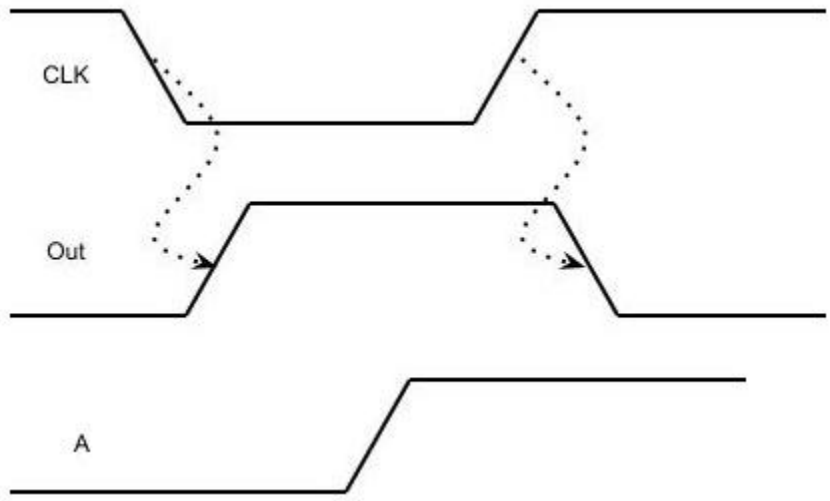


Figure : Dynamic NOR waveforms when input 'A' is high.

As seen in the waveforms above, as soon as CLK goes low, it pre-charges output node 'Out' high. While in the pre-charge state, NOR input 'A' goes high. When CLK goes high, and evaluation phase begins, 'Out' is discharged to low as input 'A' is high. Input 'B' is not shown in the waveform as it is not relevant to this case.

If both inputs 'A' and 'B' were to remain low, output node would be held high during the pre-charge.

This technique of always priming or pre-charging output to be with, is a way to minimize switching of the output node, because if with a new set of inputs, output was supposed to be high, it wouldn't have to switch, as it is already pre-charged. Output only has to switch in the case where it has to be low.

But obviously such reduction in output switching doesn't come free, as it means introducing the clocks and the extra pre-charge face, where output is not ready to be sampled.

One of the biggest concerns with dynamic gates, is the crowbar current. It needs to be ensured that the clock input to the pull up and pull down is the same node, because of pull up and pull

down clocks are coming from different sources, there is a higher likelihood of both pull up and pull down transistors to be on at the same time and hence the crowbar current.

Dynamic gates burn more power because of the associated clocks. Clock signal switches continuously, hence there is more dynamic power dissipated.

The biggest benefit of dynamic gates is that they can be cascaded together and their pull down only property can be leveraged to have a very fast delay through a chain of multiple stage dynamic gates.

Posted in Circuits, CMOS theory | Leave a reply

NMOS and PMOS logic

Posted on August 16, 2012

CMOS is the short form for the Complementary Metal Oxide Semiconductor. Complementary stands for the fact that in CMOS technology based logic, we use both p-type devices and n-type devices.

Logic circuits that use only p-type devices is referred to as PMOS logic and similarly circuits only using n-type devices are called NMOS logic. Before CMOS technology became prevalent, NMOS logic was widely used. PMOS logic had also found its use in specific applications.

Lets understand more how NMOS logic works. As per the definition, we are only allowed to use the n – type device as building blocks. No p-type devices are allowed. Lets take an example to clarify this. Following is the truth table for a NOR gate.

A	B	Out
0	0	1
0	1	0
1	0	0
1	1	0

Figure : NOR truth table.

We need to come up the a circuit for this NOR gate, using n-mos only transistors. From our understanding of CMOS logic, we can think about the pull down tree, which is made up of only n-mos gates.

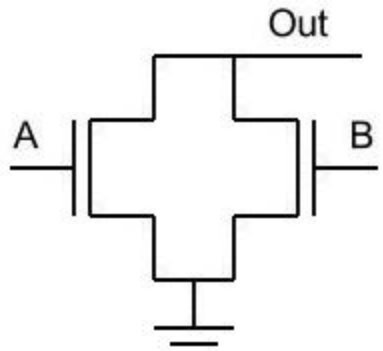


Figure : NOR pulldown logic.

Here we can see that when either of the inputs ‘A’ or ‘B’ is high, the output is pulled down to the ground. But this circuit only reflects the negative logic, or the partial functionality of NOR gate when at least one of the inputs is high. This doesn’t represent the case where both input area low, the first row of the truth table. For an equivalent CMOS NOR gate, there would be pull up tree made up of p-mos devices.

But here we are referring to NMOS logic and we are not allowed to have p-mos devices. How could we come up with the pull up logic for our NOR gate ? The answer is a resistor. Essentially when both n-mos transistor are turned off, we want ‘out’ node to be pulled up and held at VDD. A resistor tied between VDD and ‘out’ node would achieve this. There could be other possible elaborate schemes to achieve the same using n-mos transistors for pulling up purpose, but an n-mos as a resistor is used to pull up the output node.

Of course you see some immediate drawbacks. You can see that when at least one of the pull down n-mos is on, there is a static bias current flowing from VDD to the ground even in the steady state. Which is why such circuits dissipate almost an order of magnitude more power compared to CMOS equivalent. Not only that, this type of circuit is very susceptible to the input noise glitches.

Any n-mos device can be made into a resistor by making it permanently on. N-mos device has inherent resistance and we can achieve the desired resistance by modulating the width of n-mos transistor.

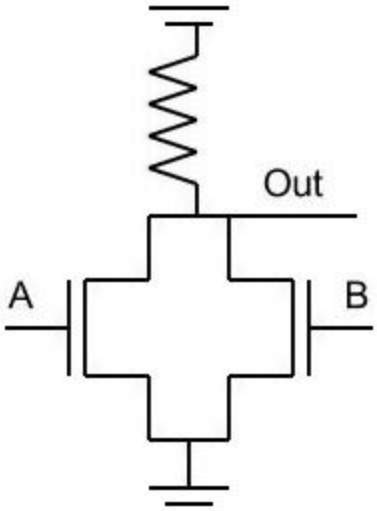


Figure : NMOS logic NOR gate.

The above figure shows the NOR gate made using NMOS logic. Similarly any gate can also be made using PMOS logic.

Posted in CMOS theory | Leave a reply

Verilog Races

Posted on July 27, 2012

In Verilog certain type of assignments or expression are scheduled for execution at the same time and order of their execution is not guaranteed. This means they could be executed in any order and the order could be change from time to time. This non-determinism is called the race condition in Verilog.

For the purpose of refreshing your memory here is the Verilog execution order again, which we had discussed in a prior post.

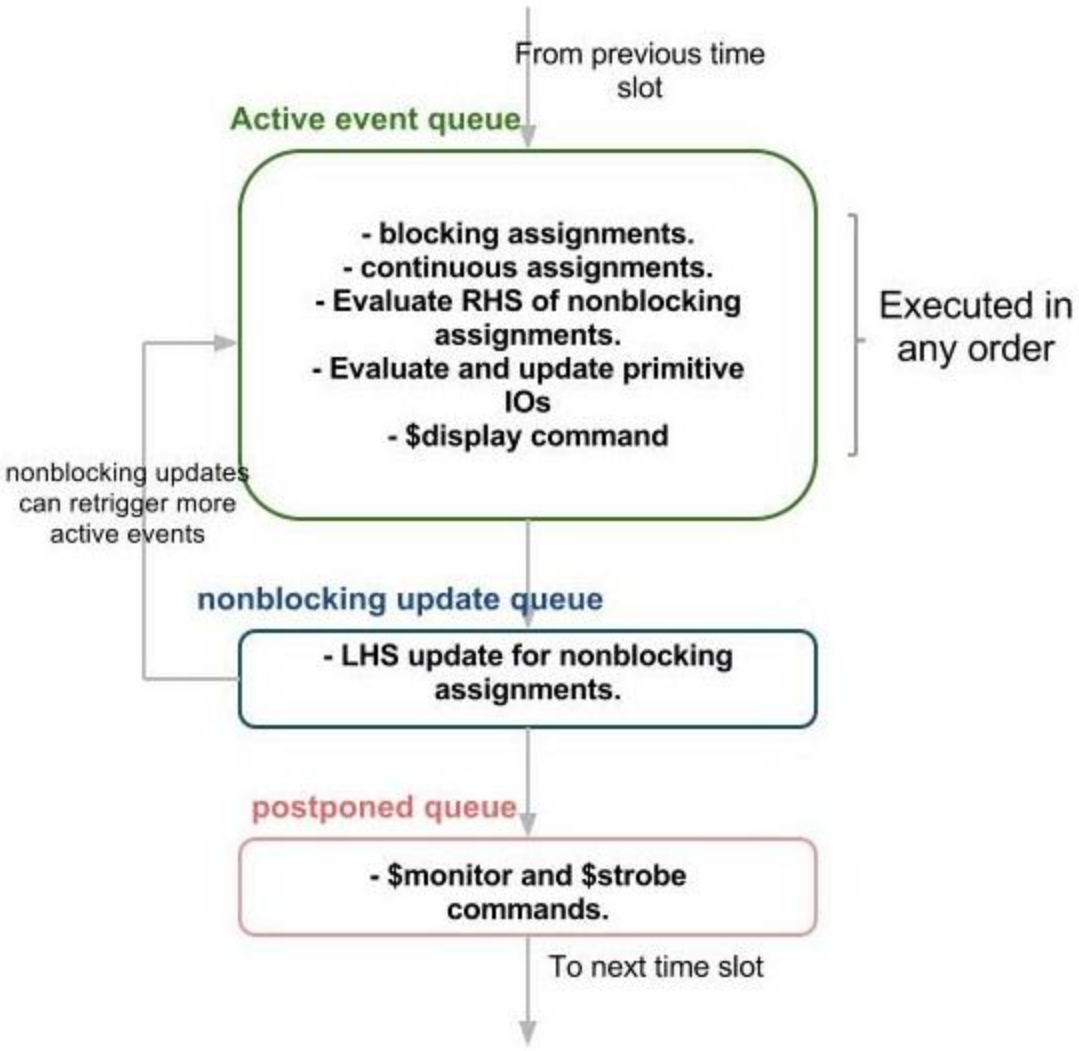


Figure : Verilog execution order.

If you look at the active event queue, it has multiple types of statements and commands with equal priority, which means they all are scheduled to be executed together in any random order, which leads to many of the races..

Lets look at some of the common race conditions that one may encounter.

1) Read-Write or Write-Read race condition.

Take the following example :

```
always @(posedge clk)
x = 2;
```

```
always @(posedge clk)
y = x;
```

Both assignments have same sensitivity (posedge clk), which means when clock rises, both will be scheduled to get executed at the same time. Either first ‘x’ could be assigned value ’2′ and then ‘y’ could be assigned ‘x’, in which case ‘y’ would end up with value ’2′. Or it could be other way around, ‘y’ could be assigned value of ‘x’ first, which could be something other than ’2′ and then ‘x’ is assigned value of ’2′. So depending on the order final value of ‘y’ could be different.

How can you avoid this race ? It depends on what your intention is. If you wanted to have a specific order, put both of the statements in that order within a ‘begin’…’end’ block inside a single ‘always’ block. Let’s say you wanted ‘x’ value to be updated first and then ‘y’ you can do following. Remember blocking assignments within a ‘begin’ .. ‘end’ block are executed in the order they appear.

```
always @(posedge clk)
begin
x = 2;
y = x;
end
```

2) Write-Write race condition.

```
always @(posedge clk)
x = 2;

always @(posedge clk)
x = 9;
```

Here again both blocking assignments have same sensitivity, which means they both get scheduled to be executed at the same time in ‘active event’ queue, in any order. Depending on the order you could get final value of ‘x’ to be either ’2′ or ’9′. If you wanted a specific order, you can follow the example in previous race condition.

3) Race condition arising from a ‘fork’…’join’ block.

```
always @(posedge clk)
fork
x = 2;
y = x;
join
```

Unlike ‘begin’…’end’ block where expressions are executed in the order they appear, expression within ‘fork’…’join’ block are executed in parallel. This parallelism can be the source of the race condition as shown in above example.

Both blocking assignments are scheduled to execute in parallel and depending upon the order of their execution eventual value of ‘y’ could be either ’2′ or the previous value of ‘x’, but it can not be determined beforehand.

4) Race condition because of variable initialization.

reg clk = 0

initial
clk = 1

In Verilog ‘reg’ type variable can be initialized within the declaration itself. This initialization is executed at time step zero, just like initial block and if you happen to have an initial block that does the assignment to the ‘reg’ variable, you have a race condition.

There are few other situations where race conditions could come up, for example if a function is invoked from more than one active blocks at the same time, the execution order could become non-deterministic.

-SS.

Posted in Digital Design, Verilog | Leave a reply

Max Fanout of a CMOS Gate

Posted on July 25, 2012

When it comes to doing digital circuit design, one has to know how to size gates. The idea is to pick gate sizes in such a way that it gives the best power v/s performance trade off. We refer to concept of ‘fanout’ when we talk about gate sizes. Fanout for CMOS gates, is the ratio of the load capacitance (the capacitance that it is driving) to the input gate capacitance. As capacitance is proportional to gate size, the fanout turns out to be the ratio of the size of the driven gate to the size of the driver gate.

Fanout of a CMOS gate depends upon the load capacitance and how fast the driving gate can charge and discharge the load capacitance. Digital circuits are mainly about speed and power tradeoff. Simply put, CMOS gate load should be within the range where driving gate can charge or discharge the load within reasonable time with reasonable power dissipation.

Our aim is to find out the nominal fanout value which gives the best speed with least possible power dissipation. To simplify our analysis we can focus on the leakage power, which is proportional to the width or size of the gate. Hence our problem simplifies to, how can we get the smallest delay through gates, while choosing smallest possible gate sizes.

Typical fanout value can be found out using the CMOS gate delay models. Some of the CMOS gate models are very complicated in nature. Luckily there are simplistic delay models, which are fairly accurate. For sake of comprehending this issue, we will go through an overly simplified delay model.

We know that I-V curves of CMOS transistor are not linear and hence, we can't really assume transistor to be a resistor when transistor is ON, but as mentioned earlier we can assume transistor to be resistor in a simplified model, for our understanding. Following figure shows a NMOS and a PMOS device. Let's assume that NMOS device is of unit gate width 'W' and for such a unit gate width device the resistance is 'R'. If we were to assume that mobility of electrons is double that of holes, which gives us an approximate P/N ratio of 2/1 to achieve same delay(with very recent process technologies the P/N ratio to get same rise and fall delay is getting close to 1/1). In other words to achieve the same resistance 'R' in a PMOS device, we need PMOS device to have double the width compared to NMOS device. That is why to get resistance 'R' through PMOS device device it needs to be '2W' wide.

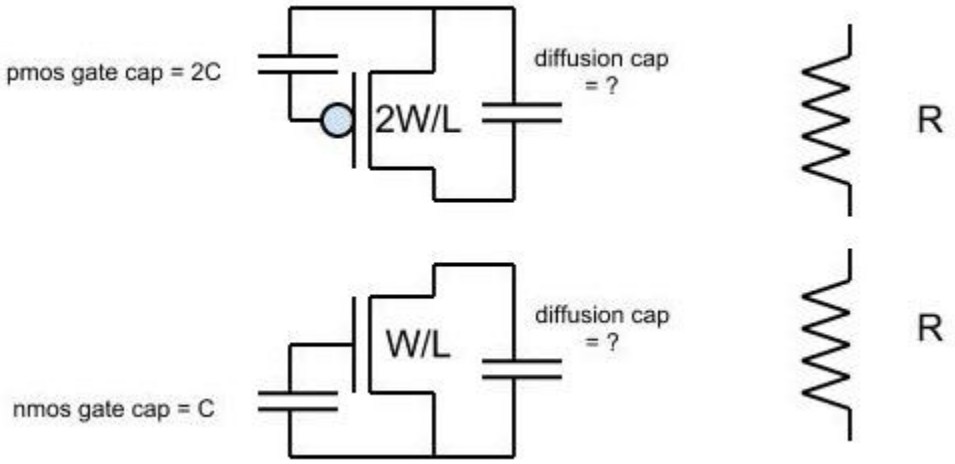


Figure 1. R and C model of CMOS inverter

Our model inverter has NMOS with width 'W' and PMOS has width '2W', with equal rise and fall delays. We know that gate capacitance is directly proportional to gate width. Lets also assume that for width 'W', the gate capacitance is 'C'. This means our NMOS gate capacitance is 'C' and our PMOS gate capacitance is '2C'. Again for sake of simplicity lets assume the diffusion capacitance of transistors to be zero.

Lets assume that an inverter with 'W' gate width drives another inverter with gate width that is 'a' times the width of the driver transistor. This multiplier 'a' is our fanout. For the receiver inverter(load inverter), NMOS gate capacitance would be a*C as gate capacitance is

proportional to the width of the gate.

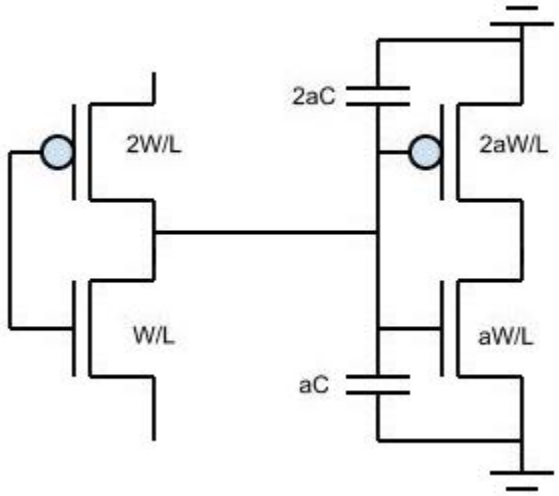


Figure 2. Unit size inverter driving ‘a’ size inverter

Now let’s represent this back to back inverter in terms of their R and C only models.

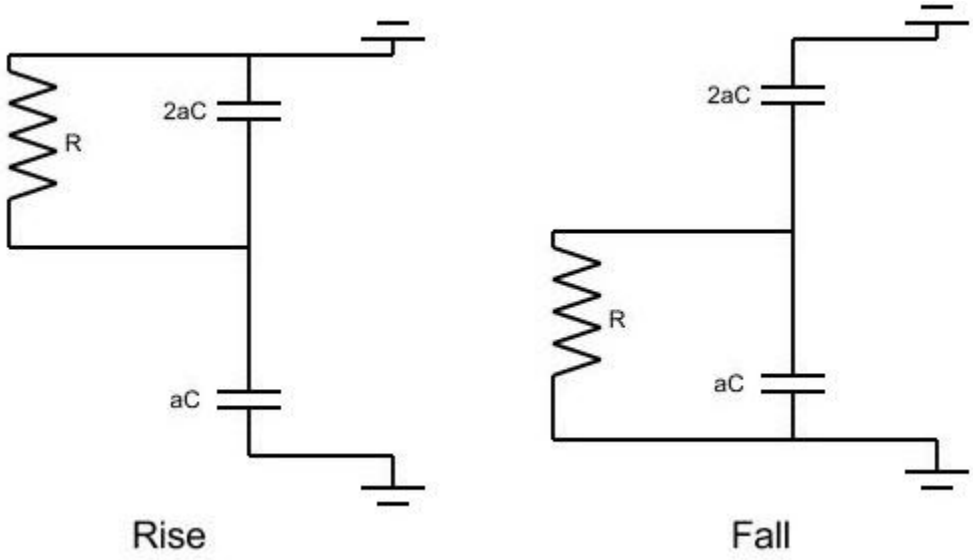


Figure 3. Inverter R & C model

For this RC circuit, we can calculate the delay at the driver output node using Elmore delay approximation. If you can recall in Elmore delay model one can find the total delay through multiple nodes in a circuit like this : Start with the first node of interest and keep going downstream along the path where you want to find the delay. Along the path stop at each node and find the total resistance from that node to VDD/VSS and multiply that resistance with total Capacitance on that node. Sum up such R and C product for all nodes.

In our circuit, there is only one node of interest. That is the driver inverter output, or the end of resistance R. In this case total resistance from the node to VDD/VSS is ‘R’ and total capacitance on the node is ‘aC+2aC=3aC’. Hence the delay can be approximated to be ‘R*3aC= 3aRC’

Now to find out the typical value of fanout ‘a’, we can build a circuit with chain of back to back inverters like following circuit.

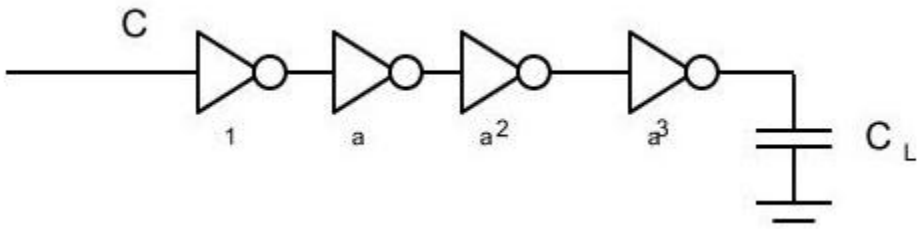


Figure 4. Chain of inverters.

Objective is to drive load CL with optimum delay through the chain of inverters. Lets assume the input capacitance of first inverter is ‘C’ as shown in figure with unit width. Fanout being ‘a’ next inverter width would ‘a’ and so forth.

The number of inverters along the path can be represented as a function of CL and C like following.

Total number of inverters along chain D = Loga(CL/C) = ln(CL/C)/ln(a)

Total delay along the chain D = Total inverters along the chain * Delay of each inverter.

Earlier we learned that for a back to back inverters where driver inverter input gate capacitance is ‘C’ and the fanout ration of ‘a’, the delay through driver inverter is 3aRC

Total delay along the chain D = ln(CL/C)/ln(a) * 3aRC

If we want to find the minimum value of total delay function for a specific value of fanout ‘a’, we need to take the derivative of ‘total delay’ with respect to ‘a’ and make it zero. That gives us the minima of the ‘total delay’ with respect to ‘a’.

D = 3*RC*ln(CL/C)*a/ln(a)

dD/da = 3*RC* ln(CL/C) [(ln(a) -1)/ln2(a)] = 0

For this to be true

(ln(a) -1) = 0

Which means : ln(a) = 1, the root of which is a = e.

This is how we derive the fanout of ‘e’ to be an optimal fanout for a chain of inverters.
If one were to plot the value of total delay ‘D’ against ‘a’ for such an inverter chain it looks like following.

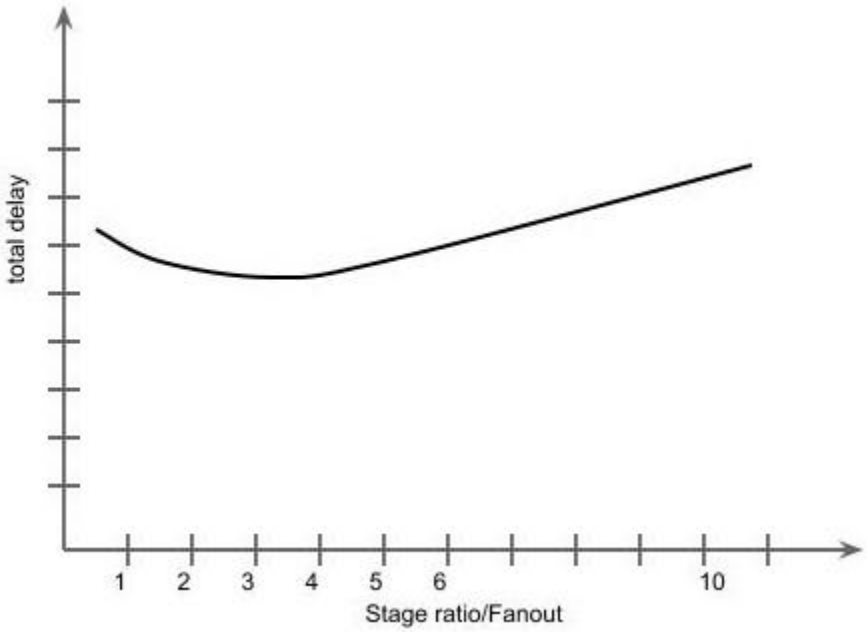


Figure 5. Total delay v/s Fanout graph

As you can see in the graph, you get lowest delay through a chain of inverters around ratio of ‘e’. Of course we made simplifying assumptions including the zero diffusion capacitance. In reality graph still follows similar contour even when you improve inverter delay model to be very accurate. What actually happens is that from fanout of 2 to fanout of 6 the delay is within less than 5% range. That is the reason, in practice a fanout of 2 to 6 is used with ideal being close to ‘e’.

One more thing to remember here is that, we assumed a chain of inverter. In practice many times you would find a gate driving a long wire. The theory still applies, one just have to find out the effective wire capacitance that the driving gate sees and use that to come up with the fanout ratio.

-SS.

Posted in Circuits, CMOS theory | Leave a reply

Inverted Temperature Dependence.

Posted on July 21, 2012

It is known that with increase in temperate, the resistivity of a metal wire(conductor) increases. The reason for this phenomenon is that with increase in temperature, thermal vibrations in lattice

increase. This gives rise to increased electron scattering. One can visualize this as electrons colliding with each other more and hence contributing less to the streamline flow needed for the flow of electric current.

There is similar effect that happens in semiconductor and the mobility of primary carrier decreases with increase in temperature. This applies to holes equally as well as electrons.

But in semiconductors, when the supply voltage of a MOS transistor is reduced, and interesting effect is observed. At lower voltages the delay through the MOS device decreases with increasing temperature, rather than increasing. After all common wisdom is that with increasing temperature the mobility decreases and hence one would have expected reduced current and subsequently reduced delay. This effect is also referred to as low voltage Inverted Temperature Dependence.
Lets first see, what does the delay of a MOS transistor depend upon, in a simplified model.

Delay = (Cout * Vdd)/ Id [approx]

Where
Cout = Drain Cap
Vdd = Supply voltage
Id = Drain current.

Now lets see what drain current depends upon.

$Id = \mu(T) * (Vdd - Vth(T))\alpha$

Where
μ = mobility
Vth = threshold voltage
α = positive constant (small number)

One can see that Id is dependent upon both mobility μ and threshold voltage Vth. Let examine the dependence of mobility and threshold voltage upon temperature.

$\mu(T) = \mu(300) (300/T)m$
$Vth(T) = Vth(300) - \kappa(T - 300)$
here '300' is room temperature in kelvin.

Mobility and threshold voltage both decreases with temperature. But decrease in mobility means less drain current and slower device, whereas decrease in threshold voltage means increase in drain current and faster device.

The final drain current is determined by which trend dominates the drain current at a given voltage and temperature pair. At high voltage mobility determines the drain current where as at lower voltages threshold voltage dominates the darin current.

This is the reason, at higher voltages device delay increase with temperature but at lower voltages, device delay increases with temperature.

-SS.

Posted in CMOS theory, sta | Leave a reply

Synchronous or Asynchronous resets ?

Posted on July 18, 2012

Both synchronous reset and asynchronous reset have advantages and disadvantages and based on their characteristics and the designers needs, one has to choose particular implementation.

Synchronous reset :

Advantages :

- This is the obvious advantage. synchronous reset conforms to synchronous design guidelines hence it ensures your design is 100% synchronous. This may not be a requirement for everyone, but many times it is a requirement that design be 100% synchronous. In such cases, it will be better to go with synchronous reset implementation.

- Protection against spurious glitches. Synchronous reset has to set up to the active clock edge in order to be effective. This provides for protection against accidental glitches as long these glitches don't happen near the active clock edges. In that sense it is not 100% protection as random glitch could happen near the active clock edge and meet both setup and hold requirements and can cause flops to reset, when they are not expected to be reset.

This type of random glitches are more likely to happen if reset is generated by some internal conditions, which most of the time means reset travels through some combinational logic before it finally gets distributed throughout the system.

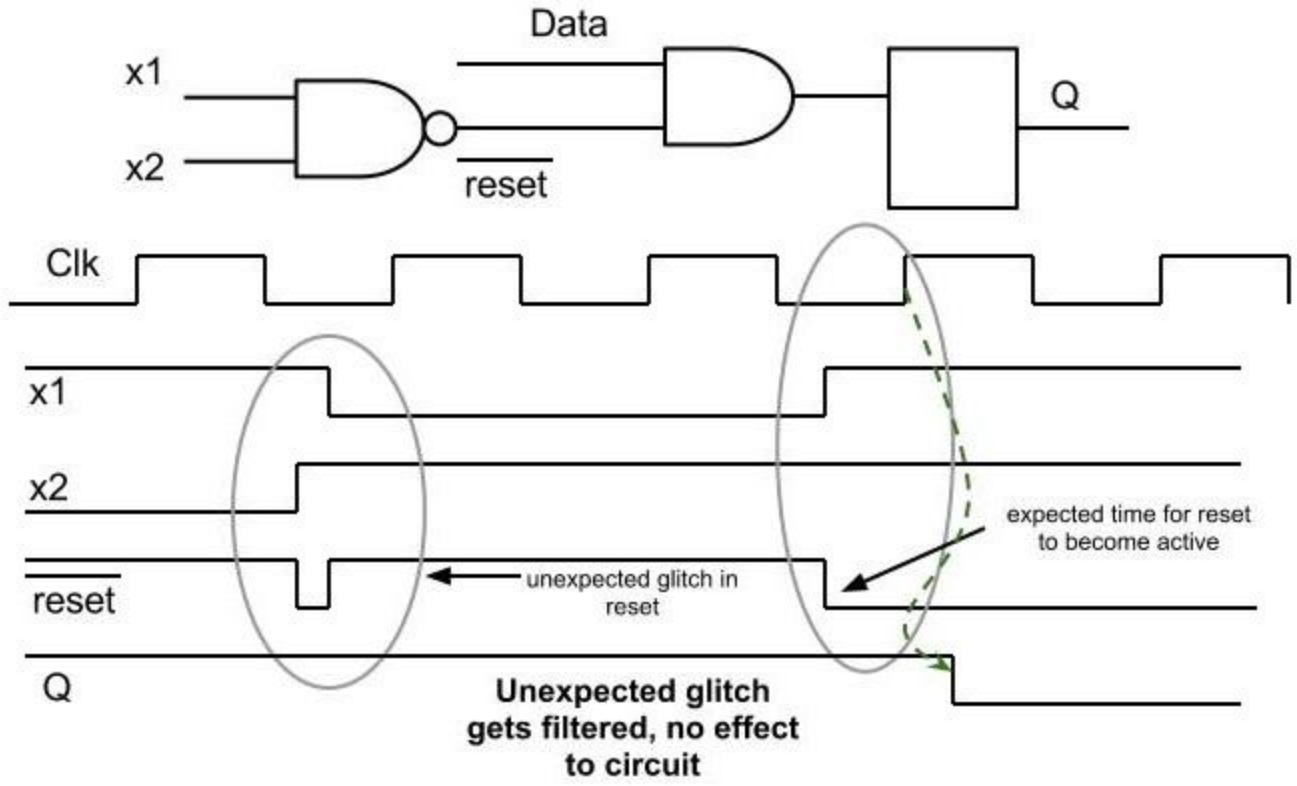


Figure : Glitch with synchronous reset

As shown in the figure, x1 and x2 generate (reset)bar. Because of the way x1 and x2 transition during the first clock cycle we get a glitch on reset signal, but because reset is synchronous and because glitch did not happen near the active clock edge, it got filtered and we only saw reset take effect later during the beginning of 4th clock cycle, where it was expected.

- One advantage that is touted for synchronous resets is smaller flops or the area savings. This is really not that much of an advantage. In terms of area savings it is really a wash between synchronous and asynchronous resets.

Synchronous reset flops are smaller as reset is just and-ed outside the flop with data, but you need that extra and gate per flop to accommodate reset. While asynchronous reset flop has to factor reset inside the flop design, where typically one of the last inverters in the feedback loop of the slave device is converted into NAND gate

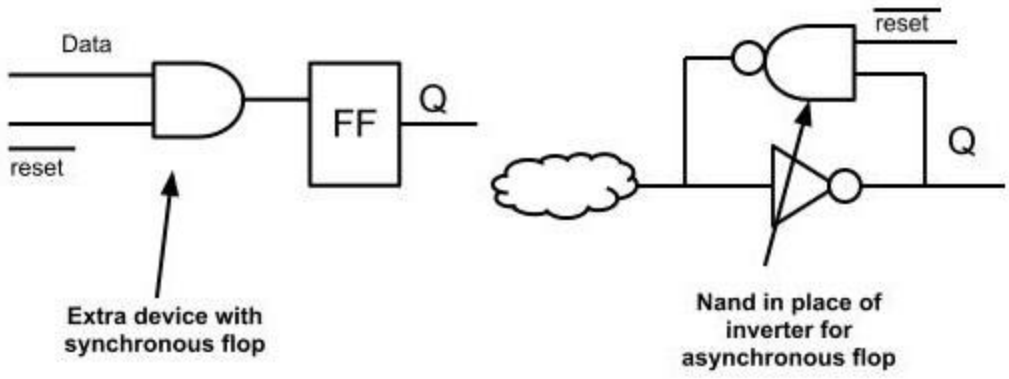


Figure : Synchronous v/s Asynchronous reset flop comparison.

Disadvantages :

- Wide enough pulse of the reset signal. We saw that being synchronous, reset has to meet the setup to the clock. We saw earlier in the figure that spurious glitches gets filtered in synchronous design, but this very behavior could be a problem. On the flip side when we do intend the reset to work, the reset pulse has to be wide enough such that it meets setup to the active edge of the clock for the all receivers sequentials on the reset distribution network.

- Another major issue with synchronous is clock gating. Designs are increasingly being clock gated to save power. Clock gating is the technique where clock is passed through an and gate with an enable signal, which can turn off the clock toggling when clock is not used thus saving power. This is in direct conflict with reset. When chip powers up, initially the clocks are not active and they could be gated by the clock enable, but right during the power up we need to force the chip into an known set and we need to use reset to achieve that. Synchronous reset will not take into effect unless there is active edge and if clock enable is off, there is no active edge of the clock.

Designer has to carefully account for this situation and design reset and clock enabling strategy which accounts for proper circuit operation.

- Use of tri-state structures. When tri-state devices are used, they need to be disabled at power-up. Because, when inadvertently enabled, tri-state device could crowbar and excessive current could flow through them and it could damage the chip. If tri-state enable is driven by a synchronous reset flop, the flop output could not be low, until the active edge of the clock arrives, and hence there is a potential to turn on tri-state device.

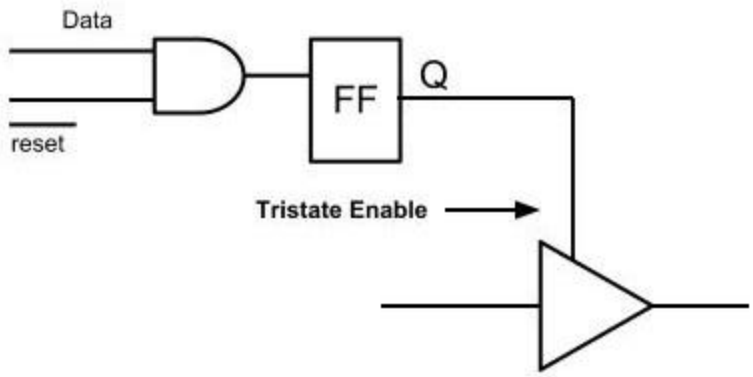


Figure : Tri-state Enable.

Asynchronous reset :

Advantages :

- Faster data path. Asynchronous reset scheme removes that AND gate at the input of the flop, thus saving one stage delay along the data path. When you are pushing the timing limits of the chip. This is very helpful.

- It has obvious advantage of being able to reset flops without the need of a clock. Basically assertion of the reset doesn't have to setup to clock, it can come anytime and reset the flop. This could be double edged sword as we have seen earlier, but if your design permits the use of asynchronous reset, this could be an advantage.

Disadvantages :

- Biggest issue with asynchronous reset is reset de-assertion edge. Remember that when we refer to reset as 'asynchronous', we are referring to only the assertion of reset. You can see in figure about synchronous and asynchronous reset comparison, that one of the way asynchronous reset is implemented is through converting one the feedback loop inverters into NAND gate. You can see that when reset input of the NAND gate, goes low it forces the Q output to be low irrespective of the input of the feedback loop. But as soon as you deassert reset, that NAND gate immediately becomes an inverter and we are back to normal flop, which is susceptible to the setup and hold requirements. Hence de-assertion of the reset could cause flop output to go metastable depending upon the relative timing between de-assertion and the clock edge. This is also called reset recovery time check, which asynchronous reset have to meet even if they are asynchronous ! You don't have this problem in synchronous reset, as you are explicitly forced to check both setup and hold on reset as well as data, as both are AND-ed and fed to the flop.

- Spurious glitches. With asynchronous reset, unintended glitches will cause circuit to go into reset state. Usually a glitch filter has to be introduced right at the reset input port. Or one may have to switch to synchronous reset.

- If reset is internally generated and is not coming directly from the chip input port, it has to be excluded for DFT purposes. The reason is that, in order for the ATPG test vectors to work correctly, test program has to be able to control all flop inputs, including data, clock and all resets. During the test vector application, we can not have any flop get reset. If reset is coming externally, test program hold it at its inactive value. If master asynchronous reset is coming externally, test program also holds it at inactive state, but if asynchronous reset is generated internally, test program has no control on the final reset output and hence the asynchronous reset net has to be removed for DFT purpose.

One issue that is common to both type of reset is that reset release has to happen within one cycle. If reset release happen in different clock cycles, then different flops will come out of reset in different clock cycles and this will corrupt the state of your circuit. This could very well happen with large reset distribution trees, where by some of receivers are closer to the master distribution point and others could be farther away.

Thus reset tree distribution is non-trivial and almost as important as clock distribution. Although you don't have to meet skew requirements like clock, but the tree has to guarantee that all its branches are balanced such that the difference between time delay of any two branches is not more than a clock cycle, thus guaranteeing that reset removal will happen within one clock cycle and all flops in the design will come out of reset within one clock cycle, maintaining the coherent state of the design.

To address this problem with asynchronous reset, where it could be more severe, the master asynchronous reset coming off chip, is synchronized using a synchronizer, the synchronizer essentially converts asynchronous reset to be more like synchronous reset and it becomes the master distributor of the reset (head of reset tree). By clocking this synchronizer with the clock similar to the clock for the flops(last stage clock in clock distribution), we can minimize the risk of reset tree distribution not happening within one clock.

-SS.

Posted in Digital Design, sta | Leave a reply

Verilog execution order

Posted on July 18, 2012

Following three items are essential for getting to the bottom of Verilog execution order.

1) Verilog event queues.

2) Determinism in Verilog.

3) Non determinism in Verilog.

Verilog event queues :

To get a very good idea of the execution order of different statements and assignments, especially the blocking and non-blocking assignments, one has to have a sound comprehension of inner workings of Verilog.

This is where Verilog event queues come into picture. Sometime it is called stratified event queues of Verilog. It is the standard IEEE spec about system Verilog, as to how different events are organized into logically segmented events queues during Verilogsimulation and in what order they get executed.

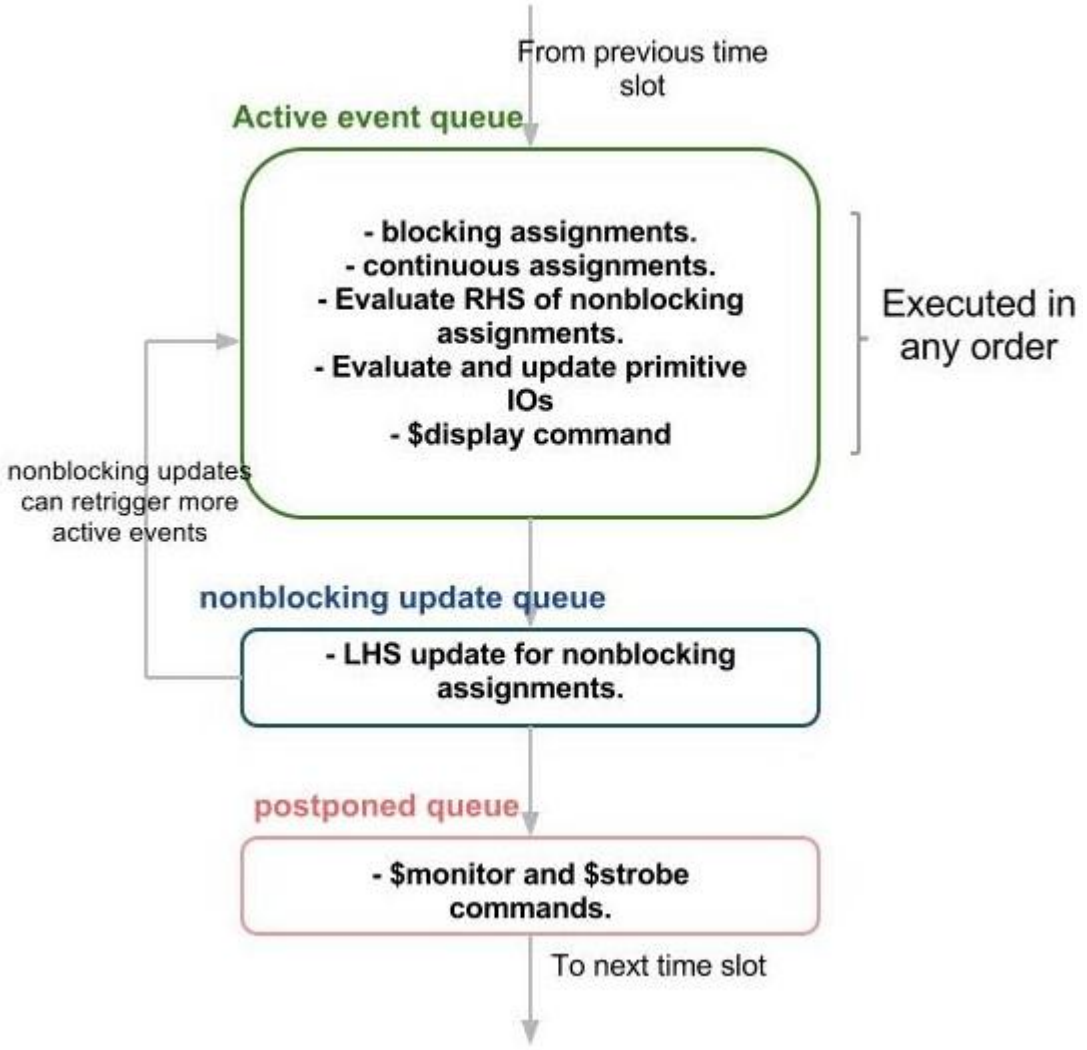


Figure : Stratified Verilog Event Queues.

As per standard the event queue is logically segmented into four different regions. For sake of simplicity we're showing the three main event queues. The "Inactive" event queue has been omitted as #0 delay events that it deals with is not a recommended guideline.

As you can see at the top there is 'active' event queue. According to the IEEE Verilog spec, events can be scheduled to any of the event queues, but events can be removed only from the

“active” event queue. As shown in the image, the ‘active’ event queue holds blocking assignments, continuous assignments. primitive IO updates and $write commands. Within “active” queue all events have same priority, which is why they can get executed in any order and is the source of nondeterminism in Verilog.
There is a separate queue for the LHS update for the nonblocking assignments. As you can see that LHS updates queue is taken up after “active” events have been exhausted, but LHS updates for the nonblocking assignments could re-trigger active events.

Lastly once the looping through the “active” and non blocking LHS update queue has settled down and finished, the “postponed” queue is taken up where $strobe and $monitor commands are executed, again without any particular preference of order.

At the end simulation time is incremented and whole cycle repeats.

Determinism in Verilog.

Based on the event queue diagram above we can make some obvious conclusions about the determinism.

- $strobe and $monitor commands are executed after all the assignment updates for the current simulation unit time have been done, hence $strobe and $monitor command would show the latest value of the variables at the end of the current simulation time.

- Statements within a begin…end block are evaluated sequentially. This means the statements within the begin…end block are executed in the order they appear within the block. The current block execution could get suspended for execution of other active process blocks, but the execution order of any being..end block does not change in any circumstances.

This is not to be confused with the fact that nonblocking assignment LHS update will always happen after the blocking assignments even if blocking assignment appears later in the begin..end order. Take following example.

```
initial                                  begin
x                          =                 0
y                          <=                3
z                          =                 8
end
```

When we refer of execution order of these three assignments.

1) First blocking statement is executed along with other blocking statements which are active in other processes.

2) Secondly for the nonblocking statement only RHS is evaluated, it is crucial to understand that the update to variable ‘y’ by value of ’3′ doesn’t happen yet. Remember that nonblocking statement execution happens in two stages, first stage is the evaluation of the RHS and second

step is update of LHS. Evaluation of RHS of nonblocking statement has same priority as blocking statement execution in general. Hence in our example here, second step is the evaluation of RHS of nonblocking statement and

3) third step is execution of the last blocking statement ‘z = 8′. The last step here will be the update to ‘y’ for the nonblocking statement. As you can see here the begin .. end block maintains the execution order in so far as the within the same priority events.

4) last step would be the update of the LHS for the nonblocking assignment, where ‘y’ will be assigned value of 3.

- One obvious question that comes to mind, having gone through previous example is that what would be the execution order of the nonblocking LHS udpate !! In the previous example we only had one nonblocking statement. What if we had more than one nonblocking statement within the begin..end block. We will look at two variation of this problem. One where two nonblocking assignments are to two different variable and the two nonblocking assignments to same variable !!

First variation.

```
initial begin
x = 0
y <= 3
z = 8
p <= 6
end
```

For the above mentioned case, the execution order still follows the order in which statements appear.

1) blocking statement ‘x = 0′ is executed in a single go.

2) RHS of nonblocking assignment ‘y <= 3′ is evaluated and LHS update is scheduled.

3) blocking assignment ‘z = 8′ is executed.

4) RHS of nonblocking assignment ‘p <= 6′ is evaluated and LHS update is scheduled.

5) LHS update from the second nonblocking assignment is carried out.

6) LHS update from the last nonblocking assignment is carried out.

Second variation.

```
initial begin
```

```
x = 0
y <= 3
z = 8
y <= 6
end
```

For the above mentioned case, the execution order still follows the order in which statements appear.

1) blocking statement ‘x = 0′ is executed in a single go.

2) RHS of nonblocking assignment ‘y <= 3′ is evaluated and LHS update is scheduled.

3) blocking assignment ‘z = 8′ is executed.

4) RHS of nonblocking assignment ‘y <= 6′ is evaluated and LHS update is scheduled.

5) LHS update from the second nonblocking assignment is carried out, ‘y’ is 3 now.

6) LHS update from the last nonblocking assignment is carried out, ‘y’ is 6 now.

Non-determinism in Verilog.

One has to look at the active event queue in the Verilog event queues figure, to get an idea as to where the non-determinism in Verilog stems from. You can see that within the active event queue, items could be executed in any order. This means that blocking assignments, continuous assignments, primitive output updates, and $display command, all could be executed in any random order across all the active processes.

Non-determinism especially bits when race conditions occur. For example we know that blocking assignments across all the active processes will be carried out in random order. This is dandy as long as blocking assignments are happening to different variables. As soon as one make blocking assignments to same variable from different active processes one will run into issues and one can determine the order of execution. Similarly if two active blocking assignments happen to read from and write to the same variable, you’ve a read write race.

We’ll look at Verilog race conditions and overall good coding guidelines in a separate post.

-SS.

Posted in Digital Design, Verilog | Leave a reply

Interview preparation for a VLSI design position

Posted on June 9, 2012

Some people believe that explicitly preparing for job interview questions and answers is futile. Because when it comes to important matter of job interview, what counts is real knowledge of the field. It is not an academic exam, where text-book preparation might come handy. You just have to know the real deal to survive a job interview. Also it is not only about the technical expertise that gets tested during job interview, but it is also about your overall aptitude, your social skill, your analytical skill and bunch of other things which are at stake.

Agreed, that it is not as simple as preparing few specific technical questions will lend you the job. But author's perspective is that, one should prepare specific interview questions as a supplement to the real deal. One has to have the fundamental technical knowledge, the technical ability, but it doesn't hurt to do some targeted preparations for job interview. It is more of a brush up of things, revision of old knowledge, tackling of some well-known technical tricks and more importantly boosting your confidence in the process. There is no harm and it definitely helps a lot to do targeted preparation for interview. Not only one should prepare for technical questions, but there is a most often asked behavioral questions set also available. One would be surprised, how much the preparation really helps.

It really depends on which position you are applying. Chip design involves several different skill and ability area, including RTL design, synthesis, physical design, static timing analysis, verification, DFT and lot more. One has to focus on the narrow field relevant to the position one is interviewing for. Most of the job positions tend to be related to ASIC design or the digital design. There are a few position in the custom design, circuit design, memory design and analog or mixed signal design.

What helps is having CMOS fundamental understanding. More than you might realize. Secondly you need to know more about verilog, as you will be dealing with verilog as long as you are in semiconductor industry. Next would come the static timing analysis. You need to know about timing also as long as you are in semiconductor industry as every chip has to run at certain frequency. Knowing about DFT is very crucial as well, because every chip designed has one or the other form of testability features, because in submicron technology no chip is designed without DFT. Basically focus on verilog, timing and DFT and fundamentals about MOS is what you need to begin with.

After having done the de-facto preparation of VLSI interview questions, you can focus more on the specific niche or the focus area that you are interviewing for, which could be verification, analog design or something else.

Posted in General | Leave a reply

Latch using a 2:1 MUX

Posted on May 11, 2012

After the previous post about XNOR gate using 2:1 MUX, one might have thought that finally we exhausted the number of gates that we could make using 2:1 MUX. But that is not entirely true !!

There are still more devices that we can make using a 2:1 MUX. These are some of the favorite static timing analysis and logic design interview questions and they are about making memory elements using the 2:1 MUX.

We know the equation of a MUX is :

Out = S * A + (S)bar * B

We also know that level sensitive latch equation is

If (Clock)

Q = D [This means if Clock is high Q follows D]

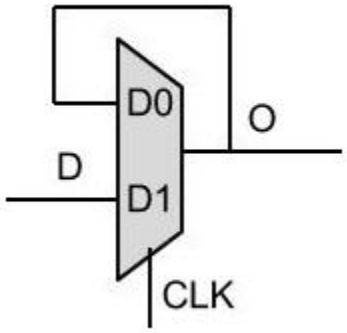
else

Q = Q [If clock is off, Q holds previous state]

We can rewrite this as following :

Q = Clock * D + (Clock)bar * Q

This means we can easily make a latch using 2:1 MUX like following.



Latch using a 2:1 MUX

When CLK is high it passes through D to O and when CLK is off, O is fed back to D0 input of mux, hence O appears back at the output, in other words, we retain the value of O when CLK is off. This is what exactly latch does.

Synchronous Reset VS Asynchronous Reset

Why Reset?

A Reset is required to initialize a hardware design for system operation and to force an ASIC into a known state for simulation.

A reset simply changes the state of the device/design/ASIC to a user/designer defined state. There are two types of reset, what are they? As you can guess them, they are Synchronous reset and Asynchronous reset.

Synchronous Reset

A synchronous reset signal will only affect or reset the state of the flip-flop on the active edge of the clock. The reset signal is applied as is any other input to the state machine.

Advantages:

- The advantage to this type of topology is that the reset presented to all functional flip-flops is fully synchronous to the clock and will always meet the reset recovery time.
- Synchronous reset logic will synthesize to smaller flip-flops, particularly if the reset is gated with the logic generating the d-input. But in such a case, the combinational logic gate count grows, so the overall gate count savings may not be that significant.
- Synchronous resets provide some filtering for the reset signal such that it is not effected by glitches, unless they occur right at the clock edge. A synchronous reset is recommended for some types of designs where the reset is generated by a set of internal conditions. As the clock will filter the logic equation glitches between clock edges.

Disadvantages:

- The problem in this topology is with reset assertion. If the reset signal is not long enough to be captured at active clock edge (or the clock may be slow to capture the reset signal), it will result in failure of assertion. In such case the design needs a pulse stretcher to guarantee that a reset pulse is wide enough to be present during the active clock edge.
- Another problem with synchronous resets is that the logic synthesis cannot easily distinguish the reset signal from any other data signal. So proper care has to be taken with logic synthesis, else the reset signal may take the fastest path to the flip-flop input there by making worst case timing hard to meet.
- In some power saving designs the clocked is gated. In such designed only asynchronous reset will work.
- Faster designs that are demanding low data path timing, can not afford to have extra gates and additional net delays in the data path due to logic inserted to handle synchronous resets.

Asynchronous Reset

An asynchronous reset will affect or reset the state of the flip-flop asynchronously i.e. no matter what the clock signal is. This is considered as high priority signal and system reset happens as soon as the reset assertion is detected.

Advantages:

- High speeds can be achieved, as the data path is independent of reset signal.
- Another advantage favoring asynchronous resets is that the circuit can be reset with or without a clock present.
- As in synchronous reset, no work around is required for logic synthesis.

Disadvantages:

- The problem with this type of reset occurs at logic de-assertion rather than at assertion like in synchronous circuits. If the asynchronous reset is released (reset release or reset removal) at or near the active clock edge of a flip-flop, the output of the flip-flop could go metastable.
- Spurious resets can happen due to reset signal glitches.

Conclusion

Both types of resets have positives and negatives and none of them assure fail-proof design. So there is something called "Asynchronous assertion and Synchronous de-assertion" reset which can be used for best results. (which will be discussed in next post).

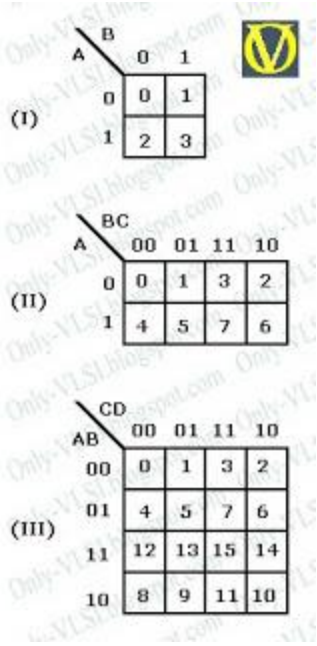
3 Comments

Labels: ASIC, Digital Design, Important Concepts, VLSI design

Boolean Expression Simplification

The k-map Method

The "Karnaugh Map Method", also known as k-map method, is popularly used to simplify Boolean expressions. The map method is first proposed by Veitch and then modified by Karnaugh, hence it is also known as "Veitch Diagram". The map is a diagram made up of squares (equal to 2 power number of inputs/variables). Each square represents a minterm, hence any Boolean expression can be represented graphically using a k-map.



The above diagram shows two (I), three (II) and four (III) variable k-maps. The number of squares is equal 2 power number of variables. Two adjacent squares will differ only by one variable. The numbers inside the squares are shown for understanding purpose only. The number shown corresponds to a minterm in the the Boolean expression.

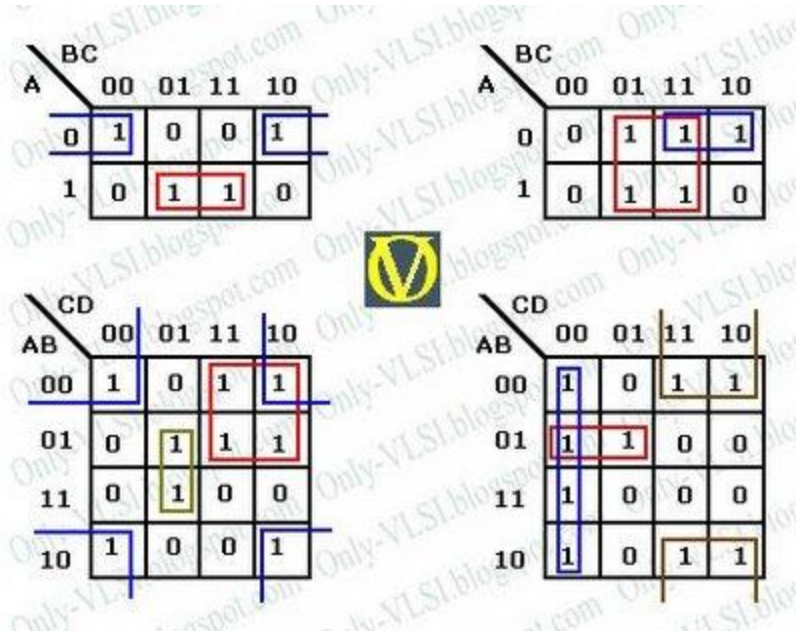
Simplification using k-map:

- Obtain the logic expression in canonical form.
- Identify all the minterms that produce an output of logic level 1 and place 1 in appropriate k-map cell/square. All others cells must contain a 0.
- Every square containing 1 must be considered at least once.
- A square containing 1 can be included in as many groups as desired.
- There can be isolated 1's, i.e. which cannot be included in any group.
- A group must be as large as possible. The number of squares in a group must be a power of 2 i.e. 2, 4, 8, ... so on.
- The map is considered to be folded or spherical, therefore squares at the end of a row or column are treated as adjacent squares.

The simplest Boolean expression contains minimum number of literals in any one in sum of products or products of sum. The simplest form obtained is not necessarily unique as grouping can be made in different ways.

Valid Groups

The following diagram illustrates the valid grouping k-map method.



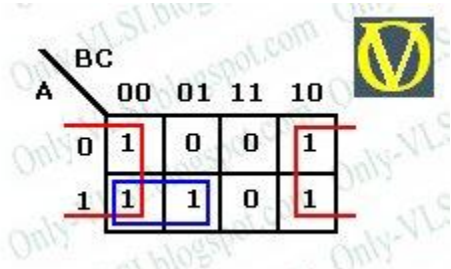
Simplification: Product of Sums

The above method gives a simplified expression in Sum of Products form. With slight modification to the above method, we can get the simplified expression in Product of Sums form. Group adjacent 0's instead of 1's, which gives us the complement of the function i.e. F'. The complement of obtained F' gives us the required expression F, which is done using the DeMorgan's theorem. See Example-2 below for better understanding.

Examples:

1. Simplify F(A, B, C) = Σ (0, 2, 4, 5, 6).

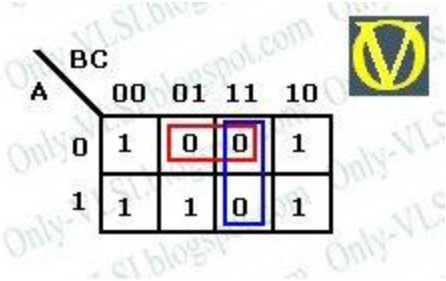
The three variable k-map of the given expression is:



The grouping is also shown in the diagram. Hence we get,
F(A, B, C) = AB' + C'

2. Simplify F(A, B, C) = Σ (0, 2, 4, 5, 6) into Product of Sums.

The three variable k-map of the given expression is:

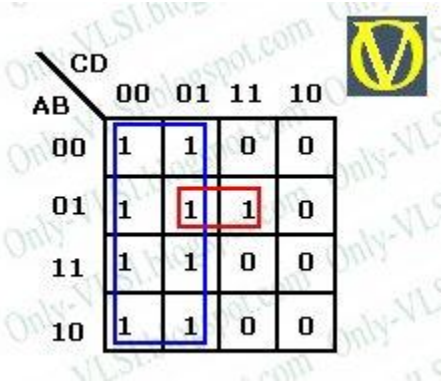


The 0's are grouped to get the F'.
F' = A'C + BC

Complementing both sides and using DeMorgan's theorem we get F,
F = (A + C')(B' + C')

3. Simplify F(A, B, C, D) = Σ(0, 1, 4, 5, 7, 8, 9, 12, 13)

The four variable k-map of the given expression is:



The grouping is also shown in the diagram. Hence we get,
F(A, B, C, D) = C' + A'BD

1 Comments

Labels: Digital Design

Finite State Machine

Definition

A machine consisting of a set of states, a start state, an input, and a transition function that maps input and current states to a next state. Machine begins in the start state with an input. It changes to new states depending on the transition function. The transition function depends on current states and inputs. The output of the machine depends on input and/or current state.

There are two types of FSMs which are popularly used in the digital design. They are

- Moore machine
- Mealy machine

Moore machine

In Moore machine the output depends only on current state.The advantage of the Moore model is a simplification of the behavior.

Mealy machine

In Mealy machine the output depend on both current state and input.The advantage of the Mealy model is that it may lead to reduction of the number of states.

In both models the next state depends on current state and input. Some times designers use mixed models. States will be encoded for representing a particular state.

Representation of a FSM

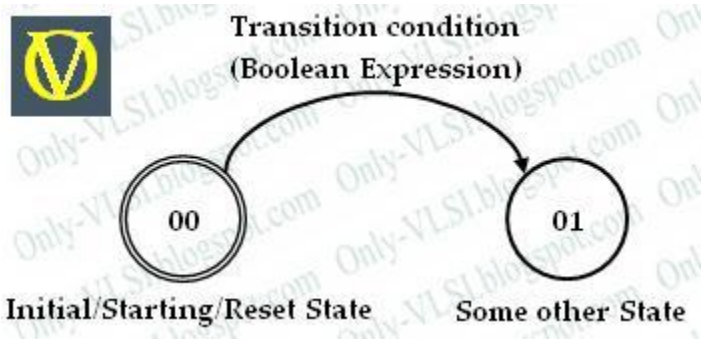
A FSM can be represented in two forms:

- Graph Notation
- State Transition Table

Graph Notation

- In this representation every state is a node. A node is represented using a circular shape and the state code is written within the circular shape.
- The state transitions are represented by an edge with arrow head. The tail of the edge shows current state and arrow points to next state, depending on the input and current state. The state transition condition is written on the edge.
- The initial/start state is sometime represented by a double lined circular shape, or a different colour shade.

The following image shows the way of graph notation of FSM. The codes *00* and *11* are the state codes. *00* is the value of initial/starting/reset state. The machine will start with *00* state. If the machine is reseted then the next state will be *00* state.



State Transition Table

The State Transition Table has the following columns:

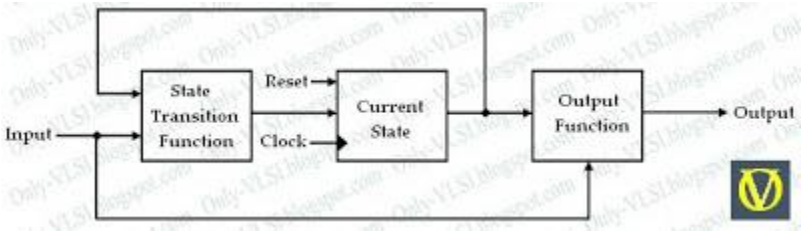
- Current State: Contains current state code
- Input: Input values of the FSM
- Next State: Contains the next state code
- Output: Expected output values

An example of state transition table is shown below.

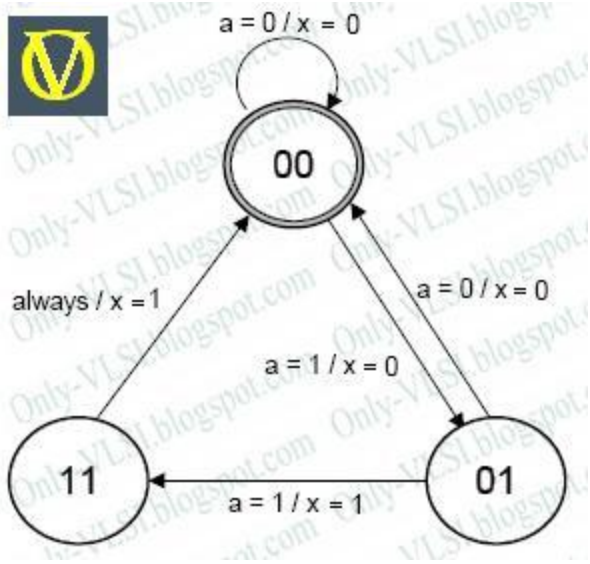
Current State	Input	Next State	Output
0 0	0	0 0	0
0 0	1	0 1	0
0 1	0	0 0	0
0 1	1	1 1	0
1 1	x	0 0	1

Mealy FSM

In Mealy machine the output depend on both current state and input.The advantage of the Mealy model is that it may lead to reduction of the number of states.



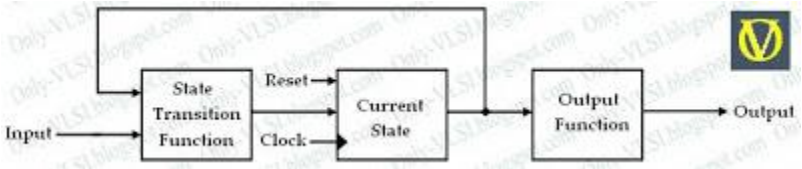
The block diagram of the Mealy FSM is shown above. The output function depends on input also. The current state function updates the current state register (number of bits depends on state encoding used).



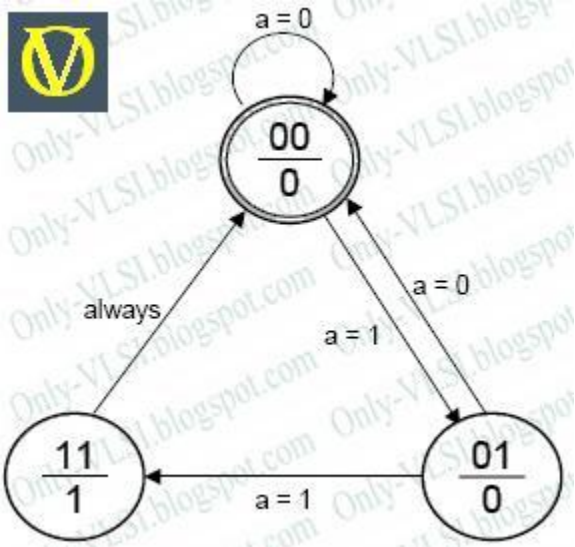
The above FSM shows an example of a Mealy FSM, the text on the arrow lines show (condition)/(output). '*a*' is the input and '*x*' is the output.

Moore FSM

In Moore machine the output depends only on current state.The advantage of the Moore model is a simplification of the behavior.



The above figure shows the block diagram of a Moore FSM. The output function doesn't depend on input. The current state function updates the current state register.



The above FSM shows an example of a Moore FSM. '*a*' is the input. Inside every circle the text

is (State code)/(output). Here there is only one output, in state '*11*' the output is '*1*'.

In both the FSMs the reset signal will change the contents of current state register to initial/reset state.

State Encoding

In a FSM design each state is represented by a binary code, which are used to identify the state of the machine. These codes are the possible values of the state register. The process of assigning the binary codes to each state is known as state encoding.
The choice of encoding plays a key role in the FSM design. It influences the complexity, size, power consumption, speed of the design. If the encoding is such that the transitions of flip-flops (of state register) are minimized then the power will be saved. The timing of the machine are often affected by the choice of encoding.
The choice of encoding depends on the type of technology used like ASIC, FPGA, CPLD etc. and also the design specifications.

State encoding techniques

The following are the most common state encoding techniques used.

- Binary encoding
- One-hot encoding
- Gray encoding

In the following explanation assume that there are *N* number of states in the FSM.

Binary encoding
The code of a state is simply a binary number. The number of bits is equal to $\log_2(N)$ rounded to next natural number. Suppose *N = 6*, then the number of bits are 3, and the state codes are:
S0 - 000
S1 - 001
S2 - 010
S3 - 011
S4 - 100
S5 - 101

One-hot encoding
In one-hot encoding only one bit of the state vector is asserted for any given state. All other state bits are zero. Thus if there are *N* states then *N* state flip-flops are required. As only one bit remains logic high and rest are logic low, it is called as One-hot encoding. If *N = 5*, then the number of bits (flip-flops) required are 5, and the state codes are:
S0 - 00001
S1 - 00010
S2 - 00100
S3 - 01000

S4 - 10000

To know more about one-hot encoding click here.

Gray encoding
Gray encoding uses the Gray codes, also known as reflected binary codes, to represent states, where two successive codes differ in only one digit. This helps is reducing the number of transition of the flip-flops outputs. The number of bits is equal to $\log_2(N)$ rounded to next natural number. If N = 4, then 2 flip-flops are required and the state codes are:
S0 - 00
S1 - 01
S2 - 11
S3 - 10

Designing a FSM is the most common and challenging task for every digital logic designer. One of the key factors for optimizing a FSM design is the choice of state coding, which influences the complexity of the logic functions, the hardware costs of the circuits, timing issues, power usage, etc. There are several options like binary encoding, gray encoding, one-hot encoding, etc. The choice of the designer depends on the factors like technology, design specifications, etc.

3 Comments

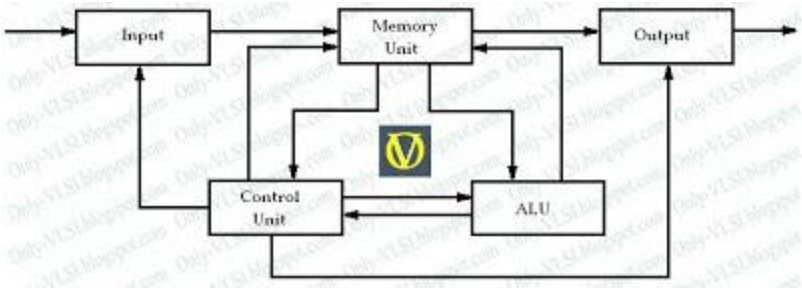
Labels: Digital Design, FSM

Introduction to Digital Logic Design

>> **Introduction**
>> **Binary Number System**
>> **Complements**
>> **2's Complement vs 1's Complement**
>> **Binary Logic**
>> **Logic Gates**

Introduction

The fundamental idea of digital systems is to represent data in discrete form (Binary: ones and zeros) and processing that information. Digital systems have led to many scientific and technological advancements. Calculators, computers, are the examples of digital systems, which are widely used for commercial and business data processing. The most important property of a digital system is its ability to follow a sequence of steps to perform a task called program, which does the required data processing. The following diagram shows how a typical digital system will look like.



Representing the data in ones and zeros, i.e. in binary system is the root of the digital systems. All the digital system store data in binary format. Hence it is very important to know about binary number system. Which is explained below.

Binary Number System

The binary number system, or base-2 number system, is a number system that represents numeric values using two symbols, usually 0 and 1. The base-2 system is a positional notation with a radix of 2. Owing to its straightforward implementation in digital electronic circuitry using logic gates, the binary system is used internally by all computers. Suppose we need to represent 14 in binary number system.
14 - 01110 - $0x2^4 + 1x2^3 + 1x2^2 + 1x2^1 + 0x2^0$
similarly,
23 - 10111 - $1x2^4 + 0x2^3 + 1x2^2 + 1x2^1 + 1x2^0$

Complements

In digital systems, complements are used to simplify the subtraction operation. There are two types of complements they are:
The r's Complement
The (r-1)'s Complement

Given:

- N a positive number.
- r base of the number system.
- n number of digits.
- m number of digits in fraction part.

The r's complement of N is defined as r^n - N for N not equal to 0 and 0 for N=0.

The (r-1)'s Complement of N is defined as $r^n - r^m$ - N.

Subtraction with r's complement:

The subtraction of two positive numbers (M-N), both are of base r. It is done as follows:
1. Add M to the r's complement of N.
2. Check for an end carry:
(a) If an end carry occurs, ignore it.

(b) If there is no end carry, the negative of the r's complement of the result obtained in step-1 is the required value.

Subtraction with (r-1)'s complement:

The subtraction of two positive numbers (M-N), both are of base r. It is done as follows:
1. Add M to the (r-1)'s complement of N.
2. Check for an end carry:
(a) If an end carry occurs, add 1 to the result obtained in step-1.
(b) If there is no end carry, the negative of the (r-1)'s complement of the result obtained in step-1 is the required value.

For a binary number system the complements are: 2's complement and 1's complement.

2's Complement vs 1's Complement

The only advantage of 1's complement is that it can be calculated easily, just by changing 0s into 1s and 1s into 0s. The 2's complement is calculated in two ways, (i) add 1 to the 1's complement of the number, and (ii) leave all the leading 0s in the least significant positions and keep first 1 unchanged, and then change 0s into 1s and 1s into 0s.

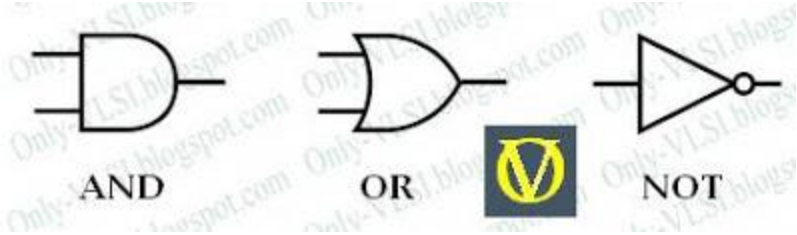
The advantages of 2's complement over 1's complement are:
(i) For subtraction with complements, 2's complement requires only one addition operation, where as for 1's complement requires two addition operations if there is an end carry.
(ii) 1's complement has two arithmetic zeros, all 0s and all 1s.

Binary Logic

Binary logic contains only two discrete values like, 0 or 1, true or false, yes or no, etc. Binary logic is similar to Boolean algebra. It is also called as boolean logic. In boolean algebra there are three basic operations: AND, OR, and NOT.
AND: Given two inputs x, y the expression x.y or simply xy represents "x AND y" and equals to 1 if both x and y are 1, otherwise 0.
OR: Given two inputs x, y the expression x+y represents "x OR y" and equals to 1 if at least one of x and y is 1, otherwise 0.
NOT: Given x, the expression x' represents NOT(x) equals to 1 if x is 0, otherwise 0. NOT(x) is x complement.

Logic Gates

A logic gate performs a logical operation on one or more logic inputs and produces a single logic output. Because the output is also a logic-level value, an output of one logic gate can connect to the input of one or more other logic gates. The logic gate use binary logic or boolean logic. AND, OR, and NOT are the three basic logic gates of digital systems. Their symbols are shown below.



AND and OR gates can have more than two inputs. The above diagram shows 2 input AND and OR gates. The truth tables of AND, OR, and NOT logic gates are as follows.

x	y	x AND y
0	0	0
0	1	0
1	0	0
1	1	1

x	y	x OR y
0	0	0
0	1	1
1	0	1
1	1	1

x	NOT(x)
0	1
1	0

FPGA vs ASIC

Definitions

FPGA: A Field-Programmable Gate Array (FPGA) is a semiconductor device containing programmable logic components called "logic blocks", and programmable interconnects. Logic blocks can be programmed to perform the function of basic logic gates such as AND, and XOR, or more complex combinational functions such as decoders or mathematical functions. *For complete details click here.*

ASIC: An application-specific integrated circuit (ASIC) is an integrated circuit designed for a particular use, rather than intended for general-purpose use. Processors, RAM, ROM, etc are examples of ASICs.

FPGA vs ASIC

Speed
ASIC rules out FPGA in terms of speed. As ASIC are designed for a specific application they can be optimized to maximum, hence we can have high speed in ASIC designs. ASIC can have hight speed clocks.

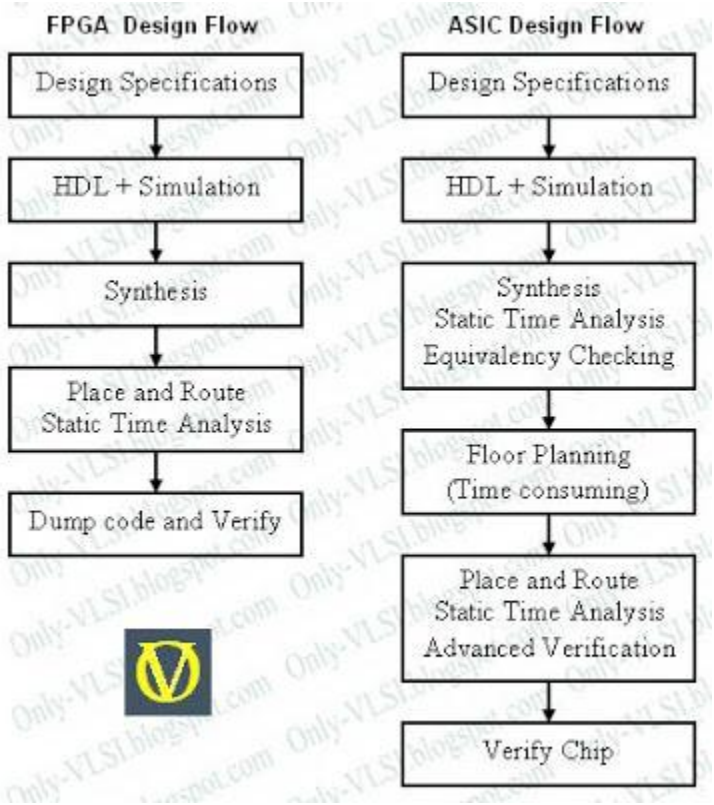
Cost
FPGAs are cost effective for small applications. But when it comes to complex and large volume designs (like 32-bit processors) ASIC products are cheaper.

Size/Area
FPGA are contains lots of LUTs, and routing channels which are connected via bit streams(program). As they are made for general purpose and because of re-usability. They are in-general larger designs than corresponding ASIC design. For example, LUT gives you both

registered and non-register output, but if we require only non-registered output, then its a waste of having a extra circuitry. In this way ASIC will be smaller in size.

Power
FPGA designs consume more power than ASIC designs. As explained above the unwanted circuitry results wastage of power. FPGA wont allow us to have better power optimization. When it comes to ASIC designs we can optimize them to the fullest.

Time to Market
FPGA designs will till less time, as the design cycle is small when compared to that of ASIC designs. No need of layouts, masks or other back-end processes. Its very simple: Specifications -- HDL + simulations -- Synthesis -- Place and Route (along with static-analysis) -- Dump code onto FPGA and Verify. When it comes to ASIC we have to do floor planning and also advanced verification. The FPGA design flow eliminates the complex and time-consuming floor planning, place and route, timing analysis, and mask / re-spin stages of the project since the design logic is already synthesized to be placed onto an already verified, characterized FPGA device.



Type of Design
ASIC can have mixed-signal designs, or only analog designs. But it is not possible to design them using FPGA chips.

Customization
ASIC has the upper hand when comes to the customization. The device can be fully customized as ASICs will be designed according to a given specification. Just imagine implementing a 32-bit processor on a FPGA!

Prototyping
Because of re-usability of FPGAs, they are used as ASIC prototypes. ASIC design HDL code is first dumped onto a FPGA and tested for accurate results. Once the design is error free then it is taken for further steps. Its clear that FPGA may be needed for designing an ASIC.

Non Recurring Engineering/Expenses
NRE refers to the one-time cost of researching, designing, and testing a new product, which is generally associated with ASICs. No such thing is associated with FPGA. Hence FPGA designs are cost effective.

Simpler Design Cycle
Due to software that handles much of the routing, placement, and timing, FPGA designs have smaller designed cycle than ASICs.

More Predictable Project Cycle
Due to elimination of potential re-spins, wafer capacities, etc. FPGA designs have better project cycle.

Tools
Tools which are used for FPGA designs are relatively cheaper than ASIC designs.

Re-Usability
A single FPGA can be used for various applications, by simply reprogramming it (dumping new HDL code). By definition ASIC are application specific cannot be reused.

3 Comments

Labels: ASIC, FPGA, Integrated Circuits

Field-Programmable Gate Array

A *Field-Programmable Gate Array* (FPGA) is a semiconductor device containing programmable logic components called "logic blocks", and programmable interconnects. Logic blocks can be programmed to perform the function of basic logic gates such as AND, and XOR, or more complex combinational functions such as decoders or mathematical functions. In most FPGAs, the logic blocks also include memory elements, which may be simple flip-flops or more complete blocks of memory.

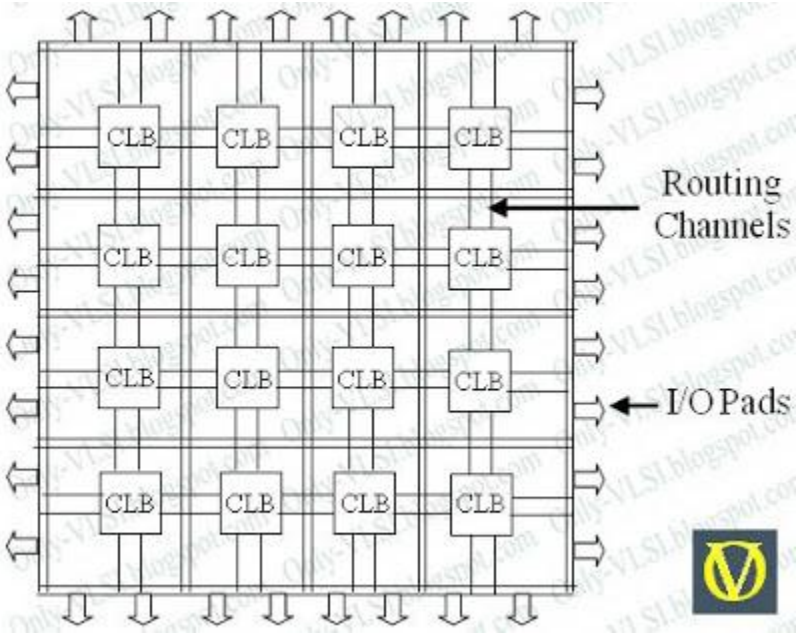
Applications

- ASIC prototyping: Due to high cost of ASIC chips, the logic of the application is first verified by dumping HDL code in a FPGA. This helps for faster and cheaper testing. Once the logic is verified then they are made into ASICs.

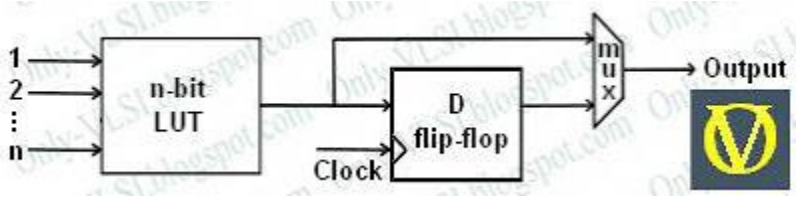
- Very useful in applications that can make use of the massive parallelism offered by their architecture. Example: code breaking, in particular brute-force attack, of cryptographic algorithms.
- FPGAs are sued for computational kernels such as FFT or Convolution instead of a microprocessor.
- Applications include digital signal processing, software-defined radio, aerospace and defense systems, medical imaging, computer vision, speech recognition, cryptography, bio-informatics, computer hardware emulation and a growing range of other areas.

Architecture

FPGA consists of large number of "configurable logic blocks" (CLBs) and routing channels. Multiple I/O pads may fit into the height of one row or the width of one column in the array. In general all the routing channels have the same width. The block diagram of FPGA architecture is shown below.



CLB: The CLB consists of an n-bit look-up table (LUT), a flip-flop and a 2x1 mux. The value n is manufacturer specific. Increase in n value can increase the performance of a FPGA. Typically n is 4. An n-bit lookup table can be implemented with a multiplexer whose select lines are the inputs of the LUT and whose inputs are constants. An n-bit LUT can encode any n-input Boolean function by modeling such functions as truth tables. This is an efficient way of encoding Boolean logic functions, and LUTs with 4-6 bits of input are in fact the key component of modern FPGAs. The block diagram of a CLB is shown below.



Each CLB has n-inputs and only one input, which can be either the registered or the unregistered LUT output. The output is selected using a 2x1 mux. The LUT output is registered using the flip-flop (generally D flip-flop). The clock is given to the flip-flop, using which the output is registered. In general, high fanout signals like clock signals are routed via special-purpose dedicated routing networks, they and other signals are managed separately.

Routing channels are programmed to connect various CLBs. The connecting done according to the design. The CLBs are connected in such a way that logic of the design is achieved.

FPGA Programming

The design is first coded in HDL (Verilog or VHDL), once the code is validated (simulated and synthesized). During synthesis, typically done using tools like Xilinx ISE, FPGA Advantage, etc, a technology-mapped net list is generated. The net list can then be fitted to the actual FPGA architecture using a process called place-and-route, usually performed by the FPGA company's proprietary place-and-route software. The user will validate the map, place and route results via timing analysis, simulation, and other verification methodologies. Once the design and validation process is complete, the binary file generated is used to (re)configure the FPGA. Once the FPGA is (re)configured, it is tested. If there are any issues or modifications, the original HDL code will be modified and then entire process is repeated, and FPGA is reconfigured.

Finite State Machine

Definition

A machine consisting of a set of states, a start state, an input, and a transition function that maps input and current states to a next state. Machine begins in the start state with an input. It changes to new states depending on the transition function. The transition function depends on current states and inputs. The output of the machine depends on input and/or current state.

There are two types of FSMs which are popularly used in the digital design. They are

- Moore machine
- Mealy machine

Moore machine

In Moore machine the output depends only on current state.The advantage of the Moore model is a simplification of the behavior.

Mealy machine

In Mealy machine the output depend on both current state and input.The advantage of the Mealy model is that it may lead to reduction of the number of states.

In both models the next state depends on current state and input. Some times designers use

mixed models. States will be encoded for representing a particular state.

Representation of a FSM

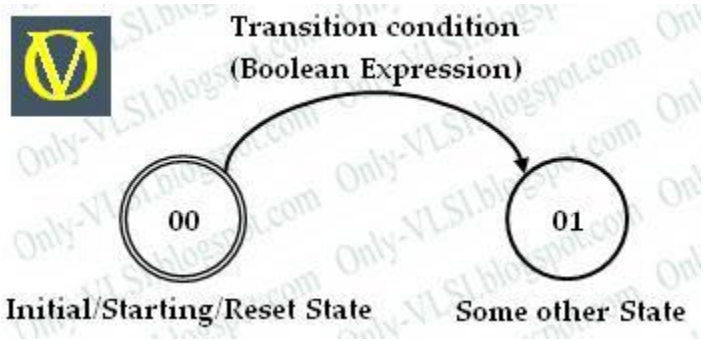
A FSM can be represented in two forms:

- Graph Notation
- State Transition Table

Graph Notation

- In this representation every state is a node. A node is represented using a circular shape and the state code is written within the circular shape.
- The state transitions are represented by an edge with arrow head. The tail of the edge shows current state and arrow points to next state, depending on the input and current state. The state transition condition is written on the edge.
- The initial/start state is sometime represented by a double lined circular shape, or a different colour shade.

The following image shows the way of graph notation of FSM. The codes *00* and *11* are the state codes. *00* is the value of initial/starting/reset state. The machine will start with *00* state. If the machine is reseted then the next state will be *00* state.



State Transition Table

The State Transition Table has the following columns:

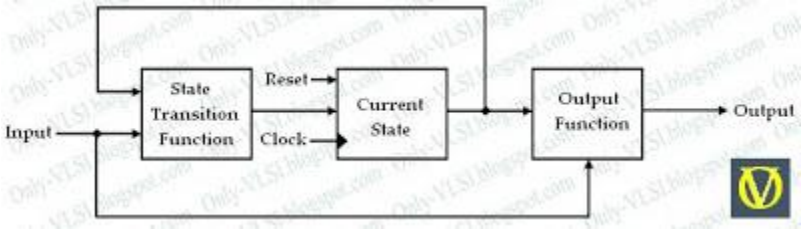
- Current State: Contains current state code
- Input: Input values of the FSM
- Next State: Contains the next state code
- Output: Expected output values

An example of state transition table is shown below.

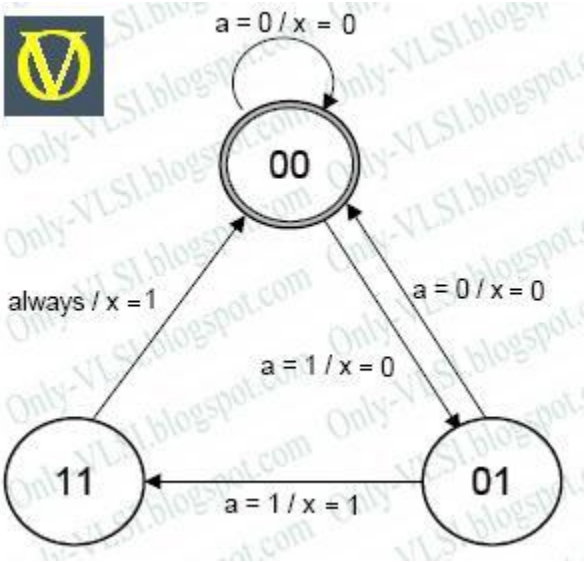
Current State	Input	Next State	Output
0 0	0	0 0	0
0 0	1	0 1	0
0 1	0	0 0	0
0 1	1	1 1	0
1 1	x	0 0	1

Mealy FSM

In Mealy machine the output depend on both current state and input.The advantage of the Mealy model is that it may lead to reduction of the number of states.



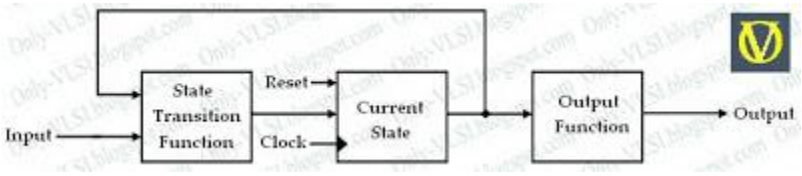
The block diagram of the Mealy FSM is shown above. The output function depends on input also. The current state function updates the current state register (number of bits depends on state encoding used).



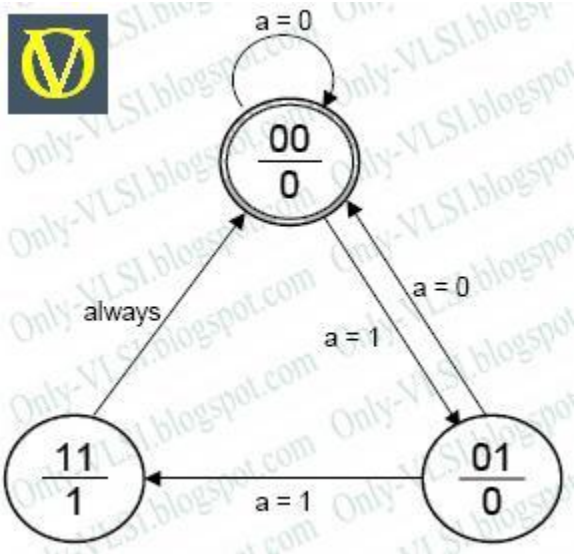
The above FSM shows an example of a Mealy FSM, the text on the arrow lines show (condition)/(output). '*a*' is the input and '*x*' is the output.

Moore FSM

In Moore machine the output depends only on current state.The advantage of the Moore model is a simplification of the behavior.



The above figure shows the block diagram of a Moore FSM. The output function doesn't depend on input. The current state function updates the current state register.



The above FSM shows an example of a Moore FSM. '*a*' is the input. Inside every circle the text is (State code)/(output). Here there is only one output, in state '*11*' the output is '*1*'.

In both the FSMs the reset signal will change the contents of current state register to initial/reset state.

State Encoding

In a FSM design each state is represented by a binary code, which are used to identify the state of the machine. These codes are the possible values of the state register. The process of assigning the binary codes to each state is known as state encoding.
The choice of encoding plays a key role in the FSM design. It influences the complexity, size, power consumption, speed of the design. If the encoding is such that the transitions of flip-flops (of state register) are minimized then the power will be saved. The timing of the machine are often affected by the choice of encoding.
The choice of encoding depends on the type of technology used like ASIC, FPGA, CPLD etc.

and also the design specifications.

State encoding techniques

The following are the most common state encoding techniques used.

- Binary encoding
- One-hot encoding
- Gray encoding

In the following explanation assume that there are *N* number of states in the FSM.

Binary encoding
The code of a state is simply a binary number. The number of bits is equal to $\log_2(N)$ rounded to next natural number. Suppose *N = 6*, then the number of bits are 3, and the state codes are:
S0 - 000
S1 - 001
S2 - 010
S3 - 011
S4 - 100
S5 - 101

One-hot encoding
In one-hot encoding only one bit of the state vector is asserted for any given state. All other state bits are zero. Thus if there are *N* states then *N* state flip-flops are required. As only one bit remains logic high and rest are logic low, it is called as One-hot encoding. If *N = 5*, then the number of bits (flip-flops) required are 5, and the state codes are:
S0 - 00001
S1 - 00010
S2 - 00100
S3 - 01000
S4 - 10000

To know more about one-hot encoding click here.

Gray encoding
Gray encoding uses the Gray codes, also known as reflected binary codes, to represent states, where two successive codes differ in only one digit. This helps is reducing the number of transition of the flip-flops outputs. The number of bits is equal to $\log_2(N)$ rounded to next natural number. If N = 4, then 2 flip-flops are required and the state codes are:
S0 - 00
S1 - 01
S2 - 11
S3 - 10

Designing a FSM is the most common and challenging task for every digital logic designer. One

of the key factors for optimizing a FSM design is the choice of state coding, which influences the complexity of the logic functions, the hardware costs of the circuits, timing issues, power usage, etc. There are several options like binary encoding, gray encoding, one-hot encoding, etc. The choice of the designer depends on the factors like technology, design specifications, etc.

3 Comments

Labels: Digital Design, FSM

One-hot Encoding

Designing a FSM is the most common and challenging task for every digital logic designer. One of the key factors for optimizing a FSM design is the choice of state coding, which influences the complexity of the logic functions, the hardware costs of the circuits, timing issues, power usage, etc. There are several options like binary encoding, gray encoding, one-hot encoding, etc. The choice of the designer depends on the factors like technology, design specifications, etc.

One-hot encoding

In one-hot encoding only one bit of the state vector is asserted for any given state. All other state bits are zero. Thus if there are *n* states then *n* state flip-flops are required. As only one bit remains logic high and rest are logic low, it is called as One-hot encoding.
Example: If there is a FSM, which has 5 states. Then 5 flip-flops are required to implement the FSM using one-hot encoding. The states will have the following values:
S0 - 10000
S1 - 01000
S2 - 00100
S3 - 00010
S4 - 00001

Advantages

- State decoding is simplified, since the state bits themselves can be used directly to check whether the FSM is in a particular state or not. Hence additional logic is not required for decoding, this is extremely advantageous when implementing a big FSM.
- Low switching activity, hence resulting low power consumption, and less prone to glitches.
- Modifying a design is easier. Adding or deleting a state and changing state transition equations (combinational logic present in FSM) can be done without affecting the rest of the design.
- Faster than other encoding techniques. Speed is independent of number of states, and depends only on the number of transitions into a particular state.
- Finding the critical path of the design is easier (static timing analysis).
- One-hot encoding is particularly advantageous for FPGA implementations. If a big FSM design is implemented using FPGA, regular encoding like binary, gray, etc will use fewer flops for the state vector than one-hot encoding, but additional logic blocks will be

required to encode and decode the state. But in FPGA each logic block contains one or more flip-flops (click here to know why?) hence due to presence of encoding and decoding more logics block will be used by regular encoding FSM than one-hot encoding FSM.

Disadvantages

- The only disadvantage of using one-hot encoding is that it required more flip-flops than the other techniques like binary, gray, etc. The number of flip-flops required grows linearly with number of states. *Example*: If there is a FSM with 38 states. One-hot encoding requires 38 flip-flops where as other require 6 flip-flops only.

Synchronous Reset VS Asynchronous Reset

Why Reset?

A Reset is required to initialize a hardware design for system operation and to force an ASIC into a known state for simulation.

A reset simply changes the state of the device/design/ASIC to a user/designer defined state. There are two types of reset, what are they? As you can guess them, they are Synchronous reset and Asynchronous reset.

Synchronous Reset

A synchronous reset signal will only affect or reset the state of the flip-flop on the active edge of the clock. The reset signal is applied as is any other input to the state machine.

Advantages:

- The advantage to this type of topology is that the reset presented to all functional flip-flops is fully synchronous to the clock and will always meet the reset recovery time.
- Synchronous reset logic will synthesize to smaller flip-flops, particularly if the reset is gated with the logic generating the d-input. But in such a case, the combinational logic gate count grows, so the overall gate count savings may not be that significant.
- Synchronous resets provide some filtering for the reset signal such that it is not effected by glitches, unless they occur right at the clock edge. A synchronous reset is recommended for some types of designs where the reset is generated by a set of internal conditions. As the clock will filter the logic equation glitches between clock edges.

Disadvantages:

- The problem in this topology is with reset assertion. If the reset signal is not long enough to be captured at active clock edge (or the clock may be slow to capture the reset signal), it will result in failure of assertion. In such case the design needs a pulse stretcher to guarantee that a reset pulse is wide enough to be present during the active clock edge.

- Another problem with synchronous resets is that the logic synthesis cannot easily distinguish the reset signal from any other data signal. So proper care has to be taken with logic synthesis, else the reset signal may take the fastest path to the flip-flop input there by making worst case timing hard to meet.
- In some power saving designs the clocked is gated. In such designed only asynchronous reset will work.
- Faster designs that are demanding low data path timing, can not afford to have extra gates and additional net delays in the data path due to logic inserted to handle synchronous resets.

Asynchronous Reset

An asynchronous reset will affect or reset the state of the flip-flop asynchronously i.e. no matter what the clock signal is. This is considered as high priority signal and system reset happens as soon as the reset assertion is detected.

Advantages:

- High speeds can be achieved, as the data path is independent of reset signal.
- Another advantage favoring asynchronous resets is that the circuit can be reset with or without a clock present.
- As in synchronous reset, no work around is required for logic synthesis.

Disadvantages:

- The problem with this type of reset occurs at logic de-assertion rather than at assertion like in synchronous circuits. If the asynchronous reset is released (reset release or reset removal) at or near the active clock edge of a flip-flop, the output of the flip-flop could go metastable.
- Spurious resets can happen due to reset signal glitches.

Conclusion

Both types of resets have positives and negatives and none of them assure fail-proof design. So there is something called "Asynchronous assertion and Synchronous de-assertion" reset which can be used for best results. (which will be discussed in next post).

3 Comments

Labels: ASIC, Digital Design, Important Concepts, VLSI design

One-hot Encoding

Designing a FSM is the most common and challenging task for every digital logic designer. One of the key factors for optimizing a FSM design is the choice of state coding, which influences the complexity of the logic functions, the hardware costs of the circuits, timing issues, power usage,

etc. There are several options like binary encoding, gray encoding, one-hot encoding, etc. The choice of the designer depends on the factors like technology, design specifications, etc.

One-hot encoding

In one-hot encoding only one bit of the state vector is asserted for any given state. All other state bits are zero. Thus if there are *n* states then *n* state flip-flops are required. As only one bit remains logic high and rest are logic low, it is called as One-hot encoding.
Example: If there is a FSM, which has 5 states. Then 5 flip-flops are required to implement the FSM using one-hot encoding. The states will have the following values:
S0 - 10000
S1 - 01000
S2 - 00100
S3 - 00010
S4 - 00001

Advantages

- State decoding is simplified, since the state bits themselves can be used directly to check whether the FSM is in a particular state or not. Hence additional logic is not required for decoding, this is extremely advantageous when implementing a big FSM.
- Low switching activity, hence resulting low power consumption, and less prone to glitches.
- Modifying a design is easier. Adding or deleting a state and changing state transition equations (combinational logic present in FSM) can be done without affecting the rest of the design.
- Faster than other encoding techniques. Speed is independent of number of states, and depends only on the number of transitions into a particular state.
- Finding the critical path of the design is easier (static timing analysis).
- One-hot encoding is particularly advantageous for FPGA implementations. If a big FSM design is implemented using FPGA, regular encoding like binary, gray, etc will use fewer flops for the state vector than one-hot encoding, but additional logic blocks will be required to encode and decode the state. But in FPGA each logic block contains one or more flip-flops (click here to know why?) hence due to presence of encoding and decoding more logics block will be used by regular encoding FSM than one-hot encoding FSM.

Disadvantages

- The only disadvantage of using one-hot encoding is that it required more flip-flops than the other techniques like binary, gray, etc. The number of flip-flops required grows linearly with number of states. *Example*: If there is a FSM with 38 states. One-hot encoding requires 38 flip-flops where as other require 6 flip-flops only.

0 Comments

Labels: FSM, Important Concepts

Random Access Memory

Random Access Memory (RAM) is a type of computer data storage. Its mainly used as main memory of a computer. RAM allows to access the data in any order, i.e random. The word random thus refers to the fact that any piece of data can be returned in a constant time, regardless of its physical location and whether or not it is related to the previous piece of data. You can access any memory cell directly if you know the row and column that intersect at that cell.

Most of the RAM chips are volatile types of memory, where the information is lost after the power is switched off. There are some non-volatile types such as, ROM, NOR-Flash.

SRAM: Static Random Access Memory

SRAM is static, which doesn't need to be periodically refreshed, as SRAM uses bistable latching circuitry to store each bit. SRAM is volatile memory. Each bit in an SRAM is stored on four transistors that form two cross-coupled inverters. This storage cell has two stable states which are used to denote 0 and 1. Two additional access transistors serve to control the access to a storage cell during read and write operations. A typical SRAM uses six MOSFETs to store each memory bit.

As SRAM doesnt need to be refreshed, it is faster than other types, but as each cell uses at least 6 transistors it is also very expensive. So in general SRAM is used for faster access memory units of a CPU.

DRAM: Dynamic Random Access Memory

In a DRAM, a transistor and a capacitor are paired to create a memory cell, which represents a single bit of data. The capacitor holds the bit of information. The transistor acts as a switch that lets the control circuitry on the memory chip read the capacitor or change its state. As capacitors leak charge, the information eventually fades unless the capacitor charge is refreshed periodically. Because of this refresh process, it is a dynamic memory.

The advantage of DRAM is its structure simplicity. As it requires only one transistor and one capacitor per one bit, high density can be achieved. Hence DRAM is cheaper and slower, when compared to SRAM.

Other types of RAM

FPM DRAM: Fast page mode dynamic random access memory was the original form of DRAM. It waits through the entire process of locating a bit of data by column and row and then reading the bit before it starts on the next bit.

EDO DRAM: Extended data-out dynamic random access memory does not wait for all of the processing of the first bit before continuing to the next one. As soon as the address of the first bit is located, EDO DRAM begins looking for the next bit. It is about five percent faster than FPM.

SDRAM: Synchronous dynamic random access memory takes advantage of the burst mode concept to greatly improve performance. It does this by staying on the row containing the requested bit and moving rapidly through the columns, reading each bit as it goes. The idea is

that most of the time the data needed by the CPU will be in sequence. SDRAM is about five percent faster than EDO RAM and is the most common form in desktops today.

DDR SDRAM: Double data rate synchronous dynamic RAM is just like SDRAM except that is has higher bandwidth, meaning greater speed.

DDR2 SDRAM: Double data rate two synchronous dynamic RAM. Its primary benefit is the ability to operate the external data bus twice as fast as DDR SDRAM. This is achieved by improved bus signaling, and by operating the memory cells at half the clock rate (one quarter of the data transfer rate), rather than at the clock rate as in the original DDR SRAM.

0 Comments

Labels: Important Concepts

Direct Memory Access

Direct memory access (DMA) is a feature of modern computers that allows certain hardware subsystems within the computer to access system memory for reading and/or writing independently of the central processing unit. Computers that have DMA channels can transfer data to and from devices with much less CPU overhead than computers without a DMA channel.

Principle of DMA

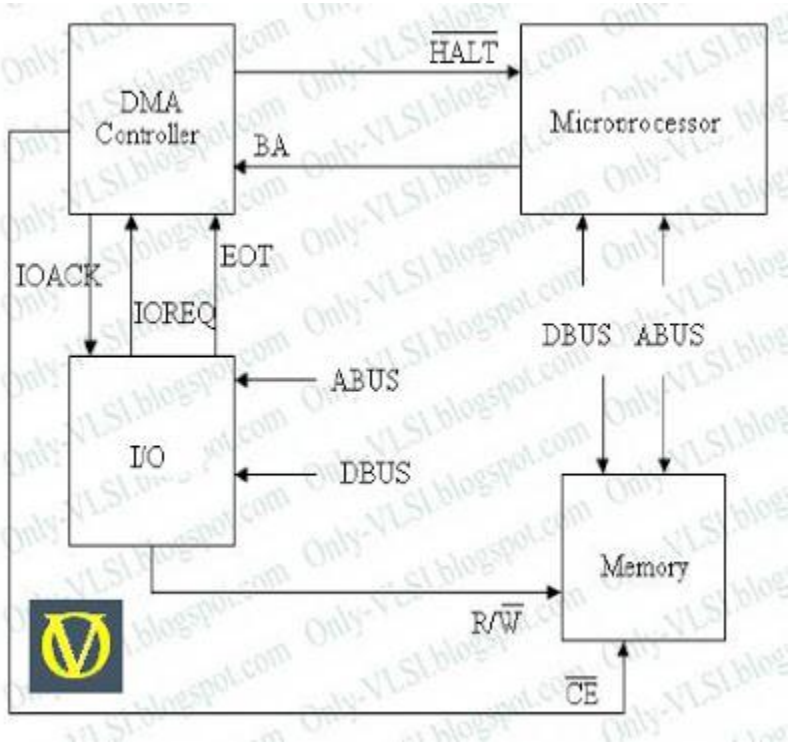
DMA is an essential feature of all modern computers, as it allows devices to transfer data without subjecting the CPU to a heavy overhead. Otherwise, the CPU would have to copy each piece of data from the source to the destination. This is typically slower than copying normal blocks of memory since access to I/O devices over a peripheral bus is generally slower than normal system RAM. During this time the CPU would be unavailable for any other tasks involving CPU bus access, although it could continue doing any work which did not require bus access.

A DMA transfer essentially copies a block of memory from one device to another. While the CPU initiates the transfer, it does not execute it. For so-called "third party" DMA, as is normally used with the ISA bus, the transfer is performed by a DMA controller which is typically part of the motherboard chipset. More advanced bus designs such as PCI typically use bus mastering DMA, where the device takes control of the bus and performs the transfer itself.

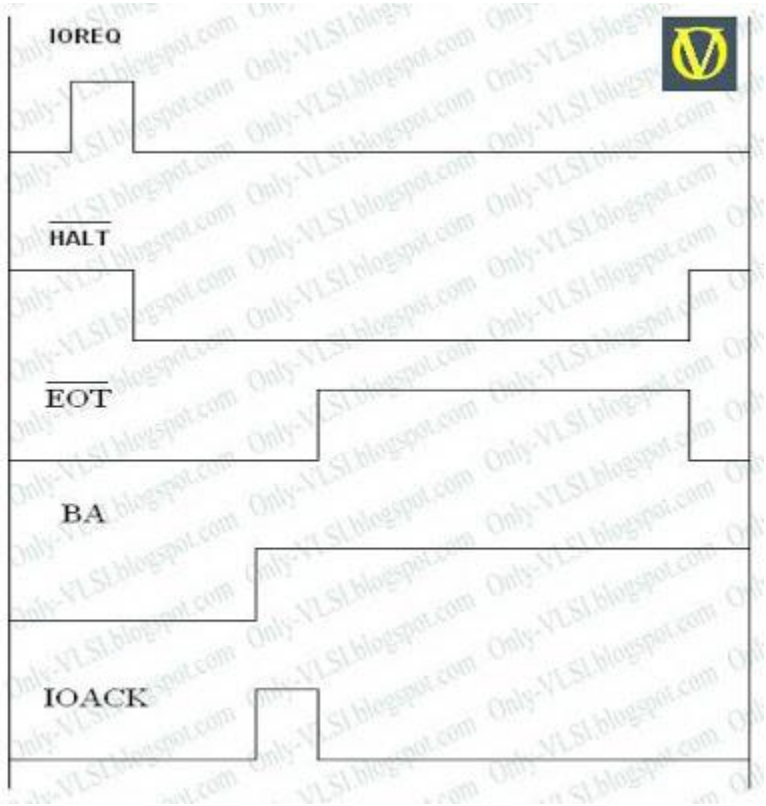
A typical usage of DMA is copying a block of memory from system RAM to or from a buffer on the device. Such an operation does not stall the processor, which as a result can be scheduled to perform other tasks. DMA is essential to high performance embedded systems. It is also essential in providing so-called zero-copy implementations of peripheral device drivers as well as functionalities such as network packet routing, audio playback and streaming video.

DMA Controller

The processing unit which controls the DMA process is known as DMA controller. Typically the job of the DMA controller is to setup a connection between the memory unit and the IO device, with the permission from the microprocessor, so that the data can be transferred with much less processor overhead. The following figure shows a simple example of hardware interface of a DMA controller in a microprocessor based system.



Functioning (Follow the timing diagram for better understanding).
Whenever there is a IO request (IOREQ) for memory access from a IO device. The DMA controller sends a Halt signal to microprocessor. Generally halt signal (HALT) is active low. Microprocessor then acknowledges the DMA controller with a bus availability signal (BA). As soon as BA is available, then DMA controller sends an IO acknowledgment to IO device (IOACK) and chip enable (CE - active low) to the memory unit. The read/write control (R/W) signal will be give by the IO device to memory unit. Then the data transfer will begin. When the data transfer is finished, the IO device sends an end of transfer (EOT - active low) signal. Then the DMA controller will stop halting the microprocessor. ABUS and DBUS are address bus and data bus, respectively, they are included just for general information that microprocessor, IO devices, and memory units are connected to the buses, through which data will be transferred.



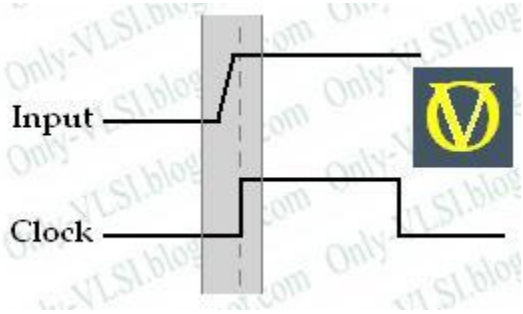
0 Comments

Labels: Important Concepts

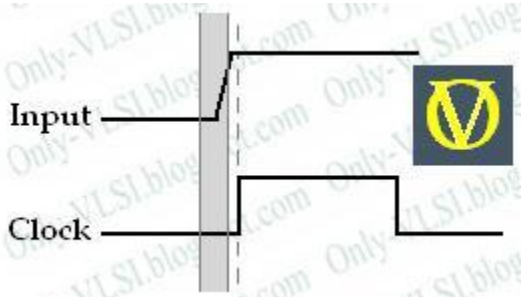
Setup and Hold TIme

Every flip-flop has restrictive time regions around the active clock edge in which input should not change. We call them restrictive because any change in the input in this regions the output may be the expected one (*see below). It may be derived from either the old input, the new input, or even in between the two. Here we define, two very important terms in the digital clocking. Setup and Hold time.

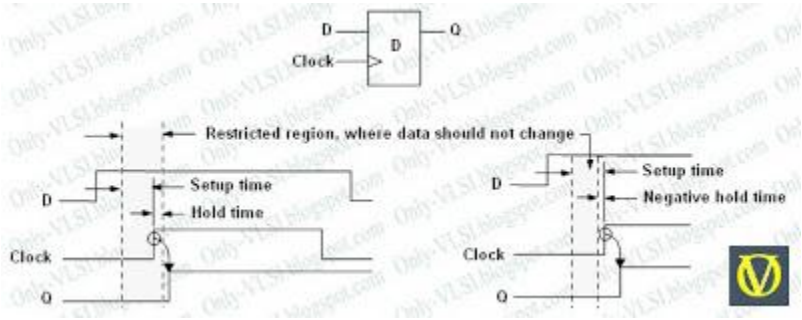
- The **setup time** is the interval before the clock where the data must be held stable.
- The **hold time** is the interval after the clock where the data must be held stable. Hold time can be negative, which means the data can change slightly before the clock edge and still be properly captured. Most of the current day flip-flops has zero or negative hold time.



In the above figure, the shaded region is the restricted region. The shaded region is divided into two parts by the dashed line. The left hand side part of shaded region is the setup time period and the right hand side part is the hold time period. If the data changes in this region, as shown the figure. The output may, follow the input, or many not follow the input, or may go to metastable state (where output cannot be recognized as either logic low or logic high, the entire process is known as metastability).



The above figure shows the restricted region (shaded region) for a flip-flop whose hold time is negative. The following diagram illustrates the restricted region of a D flip-flop. *D* is the input, *Q* is the output, and *clock* is the clock signal. If *D* changes in the restricted region, the flip-flop may not behave as expected, means *Q* is unpredictable.



To avoid setup time violations:

- The combinational logic between the flip-flops should be optimized to get minimum delay.
- Redesign the flip-flops to get lesser setup time.
- Tweak launch flip-flop to have better slew at the clock pin, this will make launch flip-flop to be fast there by helping fixing setup violations.
- Play with clock skew (useful skews).

To avoid hold time violations:

- By adding delays (using buffers).
- One can add lockup-latches (in cases where the hold time requirement is very huge, basically to avoid data slip).

* *may be expected one*: which means output is not sure, it may be the one you expect. You can also say "may not be expected one". "*may*" implies uncertainty. Thanks for the readers for their comments.

11 Comments

Labels: Important Concepts

Parallel vs Serial Data Transmission

Parallel and serial data transmission are most widely used data transfer techniques. Parallel transfer have been the preferred way for transfer data. But with serial data transmission we can achieve high speed and with some other advantages.

In parallel transmission n bits are transfered simultaneously, hence we have to process each bit separately and line up them in an order at the receiver. Hence we have to convert parallel to serial form. This is known as overhead in parallel transmission.

Signal skewing is the another problem with parallel data transmission. In the parallel communication, n bits leave at a time, but may not be received at the receiver at the same time, some may reach late than others. To overcome this problem, receiving end has to synchronize with the transmitter and must wait until all the bits are received. The greater the skew the greater the delay, if delay is increased that effects the speed.

Another problem associated with parallel transmission is crosstalk. When n wires lie parallel to each, the signal in some particular wire may get attenuated or disturbed due the induction, cross coupling etc. As a result error grows significantly, hence extra processing is necessary at the receiver.

Serial communication is full duplex where as parallel communication is half duplex. Which means that, in serial communication we can transmit and receive signal simultaneously, where as in parallel communication we can either transmit or receive the signal. Hence serial data transfer is superior to parallel data transfer.

Practically in computers we can achieve 150MBPS data transfer using serial transmission where as with parallel we can go up to 133MBPS only.

The advantage we get using parallel data transfer is reliability. Serial data transfer is less reliable than parallel data transfer.

SoC : System-On-a-Chip

System-on-a-chip (SoC) refers to integrating all components of an electronic system into a single integrated circuit (chip). A SoC can include the integration of:

- Ready made sub-circuits (IP)
- One or more microcontroller, microprocessor or DSP core(s)
- Memory components
- Sensors
- Digital, Analog, or Mixed signal components
- Timing sources, like oscillators and phase-locked loops
- Voltage regulators and power management circuits

The blocks of SoC are connected by a special bus, such as the AMBA bus. DMA controllers are used for routing the data directly between external interfaces and memory, by-passing the processor core and thereby increasing the data throughput of the SoC. SoC is widely used in the area of embedded systems. SoCs can be fabricated by several technologies, like, Full custom, Standard cell, FPGA, etc. SoC designs are usually power and cost effective, and more reliable than the corresponding multi-chip systems. A programmable SoC is known as PSoC.

Advantages of SoC are:

- Small size, reduction in chip count
- Low power consumption
- Higher reliability
- Lower memory requirements
- Greater design freedom
- Cost effective

Design Flow

SoC consists of both hardware and software(to control SoC components). The aim of SoC design is to develop hardware and software in parallel. SoC design uses pre-qualified hardware, along with their software (drivers) which control them. The hardware blocks are put together using CAD tools; the software modules are integrated using a software development environment. The SoC design is then programmed onto a FPGA, which helps in testing the behavior of SoC. Once SoC design passes the testing it is then sent to the place and route process. Then it will be fabricated. The chips will be completely tested and verified.

0 Comments

Labels: Integrated Circuits

Complex Programmable Logic Device

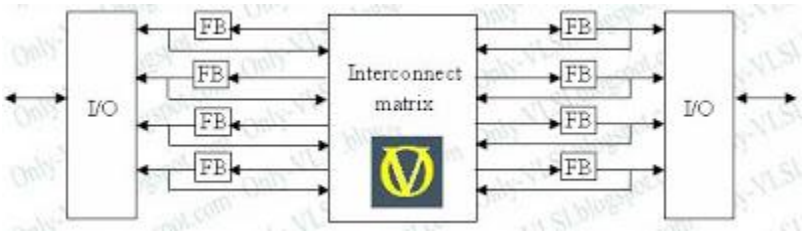
A complex programmable logic device (CPLD) is a semiconductor device containing programmable blocks called macro cell, which contains logic implementing disjunctive normal form expressions and more specialized logic operations. CPLD has complexity between that of PALs and FPGAs. It can has up to about 10,000 gates. CPLDs offer very predictable timing characteristics and are therefore ideal for critical control applications.

Applications

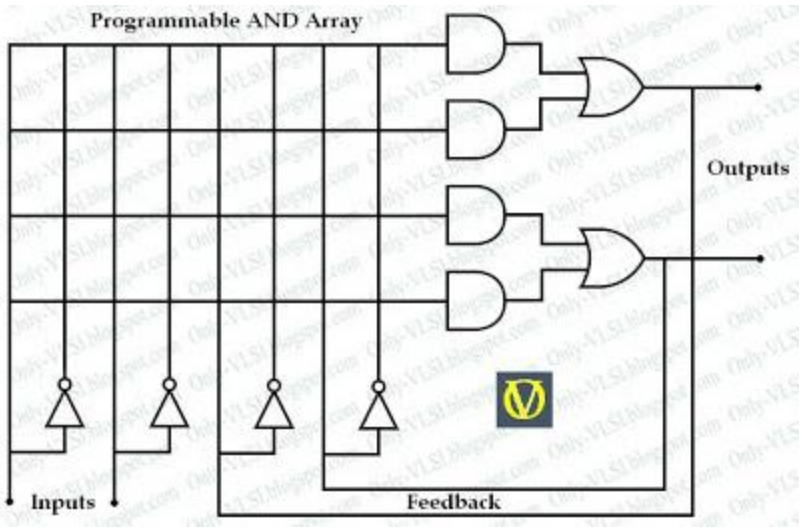
- CPLDs are ideal for critical, high-performance control applications.
- CPLD can be used for digital designs which perform boot loader functions.
- CPLD is used to load configuration data for an FPGA from non-volatile memory.
- CPLD are generally used for small designs, for example, they are used in simple applications such as address decoding.
- CPLDs are often used in cost-sensitive, battery-operated portable applications, because of its small size and low-power usage.

Architecture

A CPLD contains a bunch of programmable functional blocks (FB) whose inputs and outputs are connected together by a global interconnection matrix. The global interconnection matrix is reconfigurable, so that we can change the connections between the FBs. There will be some I/O blocks which allow us to connect CPLD to external world. The block diagram of architecture of CPLD is shown below.



The programmable functional block typically looks like the one shown below. There will be an array of AND gates which can be programed. The OR gates are fixed. But each manufacturer has their way of building the functional block. A registered output can be obtained by manipulating the feedback signals obtained from the OR ouputs.



CPLD Programming

The design is first coded in HDL (Verilog or VHDL), once the code is validated (simulated and synthesized). During synthesis the target device(CPLD model) is selected, and a technology-mapped net list is generated. The net list can then be fitted to the actual CPLD architecture using a process called place-and-route, usually performed by the CPLD company's proprietary place-and-route software. Then the user will do some verification processes. If every thing is fine, he will use the CPLD, else he will reconfigure it.

1 Comments

Labels: Integrated Circuits

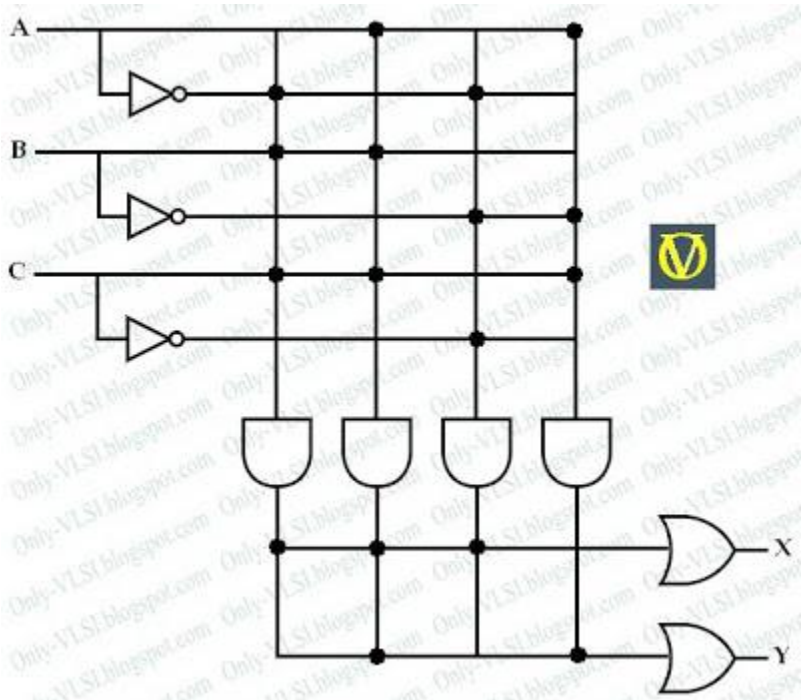
Programmable Logic Array

In Digital design, we often use a device to perform multiple applications. The device configuration is changed (reconfigured) by programming it. Such devices are known as programmable devices. It is used to build reconfigurable digital circuits. The following are the popular programmable device

- PLA - Programmable Logic Array
- PAL - Programmable Array Logic
- CPLD - Complex Programmable Logic Device (Click here for more details)
- FPGA - Field-Programmable Gate Array (Click here for more details)

PLA: Programmable Logic Array is a programmable device used to implement combinational logic circuits. The PLA has a set of programmable AND planes, which link to a set of programmable OR planes, which can then be conditionally complemented to produce an output. This layout allows for a large number of logic functions to be synthesized in the sum of products canonical forms.

Suppose we need to implement the functions: $X = A'BC + ABC + A'B'C'$ and $Y = ABC + AB'C$. The following figures shows how PLA is configured. The big dots in the diagram are connections. For the first AND gate (left most), *A* complement, *B*, and *C* are connected, which is first minterm of function *X*. For second AND gate (from left), *A*, *B*, and *C* are connected, which forms *ABC*. Similarly for *A'B'C'*, and *AB'C*. Once the minterms are implemented. Now we have to combine them using OR gates to the functions *X*, and *Y*.



One application of a PLA is to implement the control over a data path. It defines various states in an instruction set, and produces the next state (by conditional branching).

Note that the use of the word "Programmable" does not indicate that all PLAs are field-programmable; in fact many are mask-programmed during manufacture in the same manner as a ROM. This is particularly true of PLAs that are embedded in more complex and numerous integrated circuits such as microprocessors. PLAs that can be programmed after manufacture are called FPLA (Field-programmable logic array).

0 Comments

Labels: Integrated Circuits

FPGA vs ASIC

Definitions

FPGA: A Field-Programmable Gate Array (FPGA) is a semiconductor device containing programmable logic components called "logic blocks", and programmable interconnects. Logic blocks can be programmed to perform the function of basic logic gates such as AND, and XOR,

or more complex combinational functions such as decoders or mathematical functions. *For complete details click here.*

ASIC: An application-specific integrated circuit (ASIC) is an integrated circuit designed for a particular use, rather than intended for general-purpose use. Processors, RAM, ROM, etc are examples of ASICs.

FPGA vs ASIC

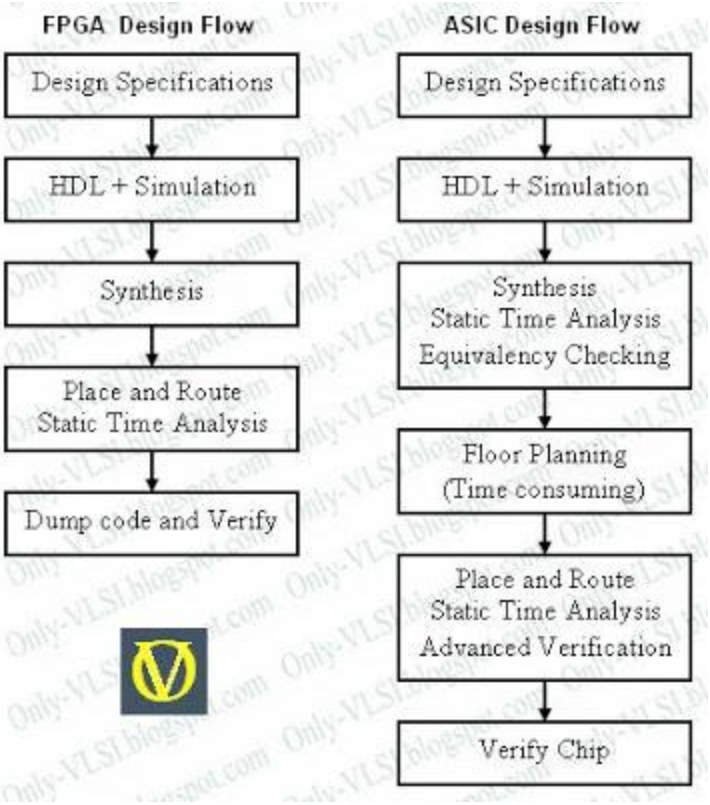
Speed
ASIC rules out FPGA in terms of speed. As ASIC are designed for a specific application they can be optimized to maximum, hence we can have high speed in ASIC designs. ASIC can have hight speed clocks.

Cost
FPGAs are cost effective for small applications. But when it comes to complex and large volume designs (like 32-bit processors) ASIC products are cheaper.

Size/Area
FPGA are contains lots of LUTs, and routing channels which are connected via bit streams(program). As they are made for general purpose and because of re-usability. They are in-general larger designs than corresponding ASIC design. For example, LUT gives you both registered and non-register output, but if we require only non-registered output, then its a waste of having a extra circuitry. In this way ASIC will be smaller in size.

Power
FPGA designs consume more power than ASIC designs. As explained above the unwanted circuitry results wastage of power. FPGA wont allow us to have better power optimization. When it comes to ASIC designs we can optimize them to the fullest.

Time to Market
FPGA designs will till less time, as the design cycle is small when compared to that of ASIC designs. No need of layouts, masks or other back-end processes. Its very simple: Specifications -- HDL + simulations -- Synthesis -- Place and Route (along with static-analysis) -- Dump code onto FPGA and Verify. When it comes to ASIC we have to do floor planning and also advanced verification. The FPGA design flow eliminates the complex and time-consuming floor planning, place and route, timing analysis, and mask / re-spin stages of the project since the design logic is already synthesized to be placed onto an already verified, characterized FPGA device.



Type of Design
ASIC can have mixed-signal designs, or only analog designs. But it is not possible to design them using FPGA chips.

Customization
ASIC has the upper hand when comes to the customization. The device can be fully customized as ASICs will be designed according to a given specification. Just imagine implementing a 32-bit processor on a FPGA!

Prototyping
Because of re-usability of FPGAs, they are used as ASIC prototypes. ASIC design HDL code is first dumped onto a FPGA and tested for accurate results. Once the design is error free then it is taken for further steps. Its clear that FPGA may be needed for designing an ASIC.

Non Recurring Engineering/Expenses
NRE refers to the one-time cost of researching, designing, and testing a new product, which is generally associated with ASICs. No such thing is associated with FPGA. Hence FPGA designs are cost effective.

Simpler Design Cycle
Due to software that handles much of the routing, placement, and timing, FPGA designs have smaller designed cycle than ASICs.

More Predictable Project Cycle
Due to elimination of potential re-spins, wafer capacities, etc. FPGA designs have better project

cycle.

Tools
Tools which are used for FPGA designs are relatively cheaper than ASIC designs.

Re-Usability
A single FPGA can be used for various applications, by simply reprogramming it (dumping new HDL code). By definition ASIC are application specific cannot be reused.

3 Comments

Labels: ASIC, FPGA, Integrated Circuits

Field-Programmable Gate Array

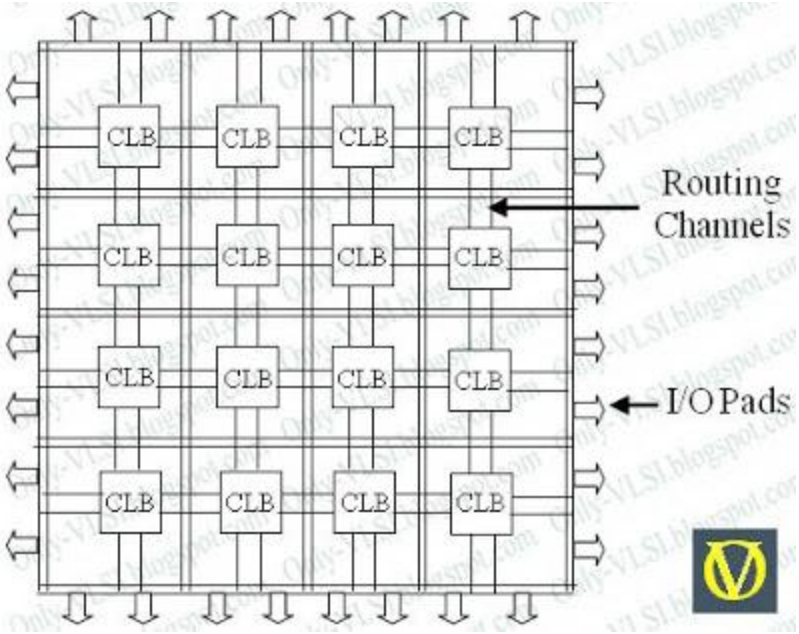
A *Field-Programmable Gate Array* (FPGA) is a semiconductor device containing programmable logic components called "logic blocks", and programmable interconnects. Logic blocks can be programmed to perform the function of basic logic gates such as AND, and XOR, or more complex combinational functions such as decoders or mathematical functions. In most FPGAs, the logic blocks also include memory elements, which may be simple flip-flops or more complete blocks of memory.

Applications

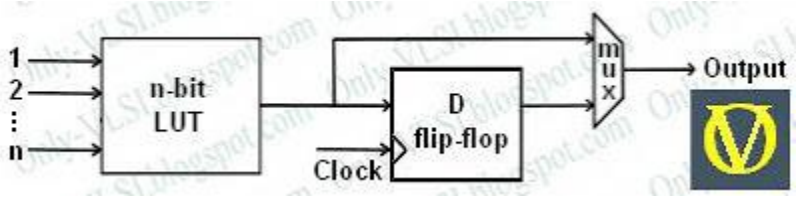
- ASIC prototyping: Due to high cost of ASIC chips, the logic of the application is first verified by dumping HDL code in a FPGA. This helps for faster and cheaper testing. Once the logic is verified then they are made into ASICs.
- Very useful in applications that can make use of the massive parallelism offered by their architecture. Example: code breaking, in particular brute-force attack, of cryptographic algorithms.
- FPGAs are sued for computational kernels such as FFT or Convolution instead of a microprocessor.
- Applications include digital signal processing, software-defined radio, aerospace and defense systems, medical imaging, computer vision, speech recognition, cryptography, bio-informatics, computer hardware emulation and a growing range of other areas.

Architecture

FPGA consists of large number of "configurable logic blocks" (CLBs) and routing channels. Multiple I/O pads may fit into the height of one row or the width of one column in the array. In general all the routing channels have the same width. The block diagram of FPGA architecture is shown below.



CLB: The CLB consists of an n-bit look-up table (LUT), a flip-flop and a 2x1 mux. The value n is manufacturer specific. Increase in n value can increase the performance of a FPGA. Typically n is 4. An n-bit lookup table can be implemented with a multiplexer whose select lines are the inputs of the LUT and whose inputs are constants. An n-bit LUT can encode any n-input Boolean function by modeling such functions as truth tables. This is an efficient way of encoding Boolean logic functions, and LUTs with 4-6 bits of input are in fact the key component of modern FPGAs. The block diagram of a CLB is shown below.



Each CLB has n-inputs and only one input, which can be either the registered or the unregistered LUT output. The output is selected using a 2x1 mux. The LUT output is registered using the flip-flop (generally D flip-flop). The clock is given to the flip-flop, using which the output is registered. In general, high fanout signals like clock signals are routed via special-purpose dedicated routing networks, they and other signals are managed separately.

Routing channels are programmed to connect various CLBs. The connecting done according to the design. The CLBs are connected in such a way that logic of the design is achieved.

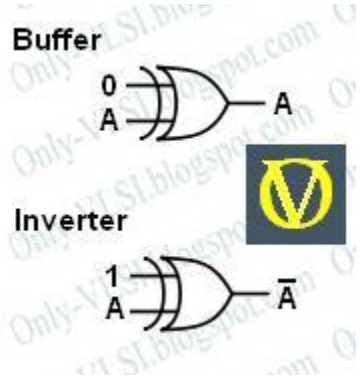
FPGA Programming

The design is first coded in HDL (Verilog or VHDL), once the code is validated (simulated and synthesized). During synthesis, typically done using tools like Xilinx ISE, FPGA Advantage, etc, a technology-mapped net list is generated. The net list can then be fitted to the actual FPGA architecture using a process called place-and-route, usually performed by the FPGA company's proprietary place-and-route software. The user will validate the map, place and route results via

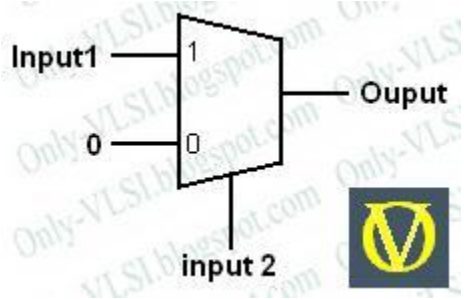
timing analysis, simulation, and other verification methodologies. Once the design and validation process is complete, the binary file generated is used to (re)configure the FPGA. Once the FPGA is (re)configured, it is tested. If there are any issues or modifications, the original HDL code will be modified and then entire process is repeated, and FPGA is reconfigured.

Digital Design Interview Questions - All in 1

1. How do you convert a XOR gate into a buffer and a inverter (Use only one XOR gate for each)?
Answer



2. Implement an 2-input AND gate using a 2x1 mux.
Answer



3. What is a multiplexer?
Answer

A multiplexer is a combinational circuit which selects one of many input signals and directs to the only output.

4. What is a ring counter?
Answer

A ring counter is a type of counter composed of a circular shift register. The output of the last

shift register is fed to the input of the first register. For example, in a 4-register counter, with initial register values of 1100, the repeating pattern is: 1100, 0110, 0011, 1001, 1100, so on.

5. Compare and Contrast Synchronous and Asynchronous reset.
Answer

Synchronous reset logic will synthesize to smaller flip-flops, particularly if the reset is gated with the logic generating the d-input. But in such a case, the combinational logic gate count grows, so the overall gate count savings may not be that significant. The clock works as a filter for small reset glitches; however, if these glitches occur near the active clock edge, the Flip-flop could go metastable. In some designs, the reset must be generated by a set of internal conditions. A synchronous reset is recommended for these types of designs because it will filter the logic equation glitches between clock.
Problem with synchronous resets is that the synthesis tool cannot easily distinguish the reset signal from any other data signal. Synchronous resets may need a pulse stretcher to guarantee a reset pulse width wide enough to ensure reset is present during an active edge of the clock, if you have a gated clock to save power, the clock may be disabled coincident with the assertion of reset. Only an asynchronous reset will work in this situation, as the reset might be removed prior to the resumption of the clock. Designs that are pushing the limit for data path timing, can not afford to have added gates and additional net delays in the data path due to logic inserted to handle synchronous resets.

Asynchronous reset: The major problem with asynchronous resets is the reset release, also called reset removal. Using an asynchronous reset, the designer is guaranteed not to have the reset added to the data path. Another advantage favoring asynchronous resets is that the circuit can be reset with or without a clock present. Ensure that the release of the reset can occur within one clock period else if the release of the reset occurred on or near a clock edge then flip-flops may go into metastable state.

6. What is a Johnson counter?
Answer

Johnson counter connects the complement of the output of the last shift register to its input and circulates a stream of ones followed by zeros around the ring. For example, in a 4-register counter, the repeating pattern is: 0000, 1000, 1100, 1110, 1111, 0111, 0011, 0001, so on.

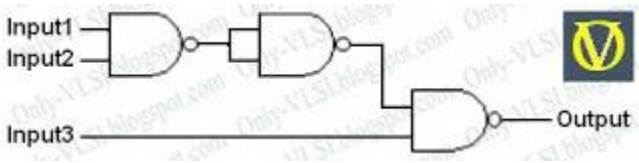
7. An assembly line has 3 fail safe sensors and one emergency shutdown switch.The line should keep moving unless any of the following conditions arise:
(1) If the emergency switch is pressed
(2) If the senor1 and sensor2 are activated at the same time.
(3) If sensor 2 and sensor3 are activated at the same time.
(4) If all the sensors are activated at the same time
Suppose a combinational circuit for above case is to be implemented only with NAND Gates. How many minimum number of 2 input NAND gates are required?
Answer

Solve it out!

8. In a 4-bit Johnson counter How many unused states are present?
Answer

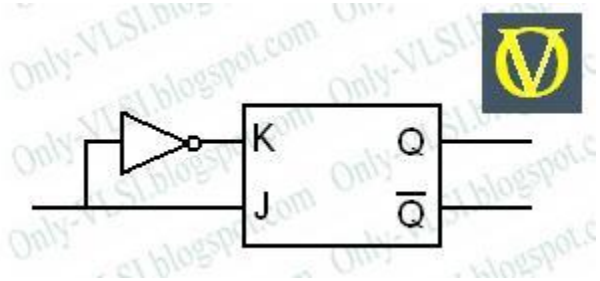
4-bit Johnson counter: 0000, 1000, 1100, 1110, 1111, 0111, 0011, 0001, 0000.
8 unused states are present.

9. Design a 3 input NAND gate using minimum number of 2 input NAND gates.
Answer



10. How can you convert a JK flip-flop to a D flip-flop?
Answer

Connect the inverted J input to K input.



11. What are the differences between a flip-flop and a latch?
Answer

Flip-flops are edge-sensitive devices where as latches are level sensitive devices.
Flip-flops are immune to glitches where are latches are sensitive to glitches.
Latches require less number of gates (and hence less power) than flip-flops.
Latches are faster than flip-flops.

12. What is the difference between Mealy and Moore FSM?
Answer

Mealy FSM uses only input actions, i.e. output depends on input and state. The use of a Mealy FSM leads often to a reduction of the number of states.
Moore FSM uses only entry actions, i.e. output depends only on the state. The advantage of the Moore model is a simplification of the behavior.

13. What are various types of state encoding techniques? Explain them.
Answer

One-Hot encoding: Each state is represented by a bit flip-flop). If there are four states then it requires four bits (four flip-flops) to represent the current state. The valid state values are 1000, 0100, 0010, and 0001. If the value is 0100, then it means second state is the current state.

One-Cold encoding: Same as one-hot encoding except that '0' is the valid value. If there are four states then it requires four bits (four flip-flops) to represent the current state. The valid state values are 0111, 1011, 1101, and 1110.

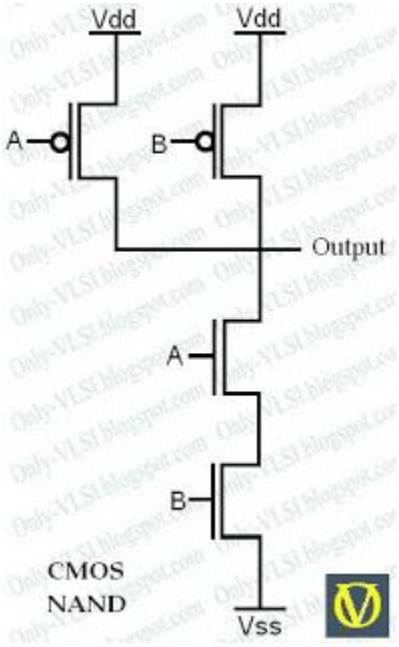
Binary encoding: Each state is represented by a binary code. A FSM having '2 power N' states requires only N flip-flops.

Gray encoding: Each state is represented by a Gray code. A FSM having '2 power N' states requires only N flip-flops.

14. Define Clock Skew , Negative Clock Skew, Positive Clock Skew.
Answer

Clock skew is a phenomenon in synchronous circuits in which the clock signal (sent from the clock circuit) arrives at different components at different times. This can be caused by many different things, such as wire-interconnect length, temperature variations, variation in intermediate devices, capacitive coupling, material imperfections, and differences in input capacitance on the clock inputs of devices using the clock.
There are two types of clock skew: negative skew and positive skew. Positive skew occurs when the clock reaches the receiving register later than it reaches the register sending data to the receiving register. Negative skew is the opposite: the receiving register gets the clock earlier than the sending register.

15. Give the transistor level circuit of a CMOS NAND gate.
Answer



16. Design a 4-bit comparator circuit.
Answer

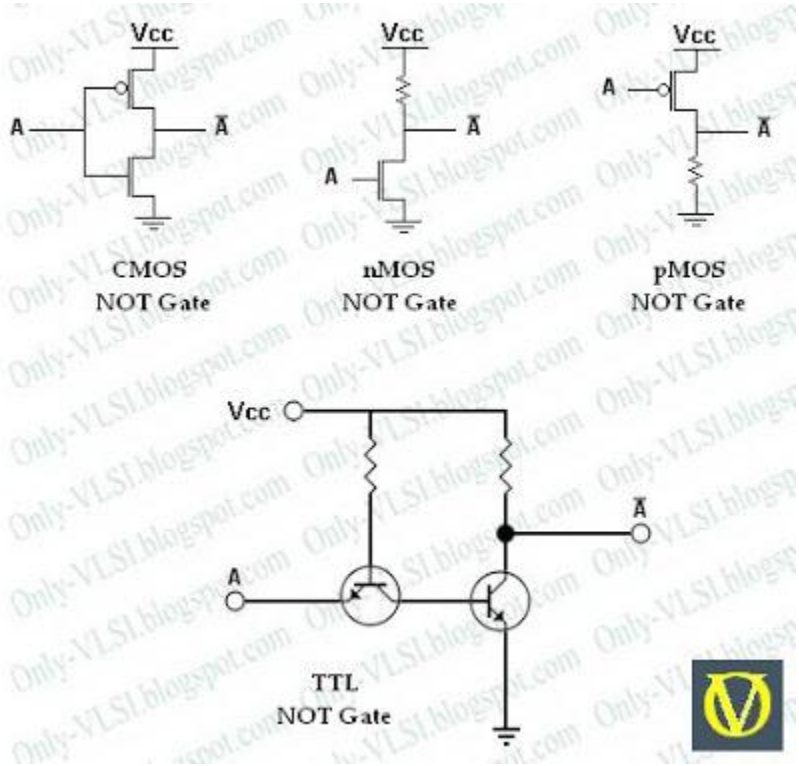
17. Design a Transmission Gate based XOR. Now, how do you convert it to XNOR (without inverting the output)?
Answer

18. Define Metastability.
Answer

If there are setup and hold time violations in any sequential circuit, it enters a state where its output is unpredictable, this state is known as metastable state or quasi stable state, at the end of metastable state, the flip-flop settles down to either logic high or logic low. This whole process is known as metastability.

19. Compare and contrast between 1's complement and 2's complement notation.
Answer

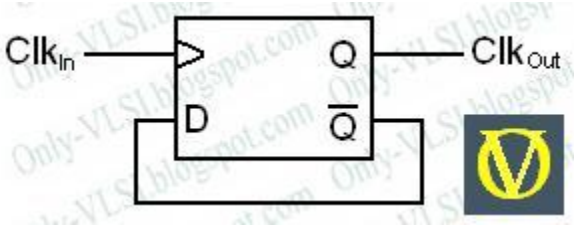
20. Give the transistor level circuit of CMOS, nMOS, pMOS, and TTL inverter gate.
Answer



21. What are set up time and hold time constraints?
Answer

Set up time is the amount of time before the clock edge that the input signal needs to be stable to guarantee it is accepted properly on the clock edge.
Hold time is the amount of time after the clock edge that same input signal has to be held before changing it to make sure it is sensed properly at the clock edge.
Whenever there are setup and hold time violations in any flip-flop, it enters a state where its output is unpredictable, which is known as as metastable state or quasi stable state. At the end of metastable state, the flip-flop settles down to either logic high or logic low. This whole process is known as metastability.

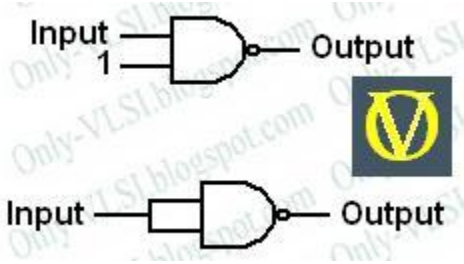
22. Give a circuit to divide frequency of clock cycle by two.
Answer



23. Design a divide-by-3 sequential circuit with 50% duty circle.
Answer

24. Explain different types of adder circuits.
Answer

25. Give two ways of converting a two input NAND gate to an inverter.
Answer



26. Draw a Transmission Gate-based D-Latch.
Answer

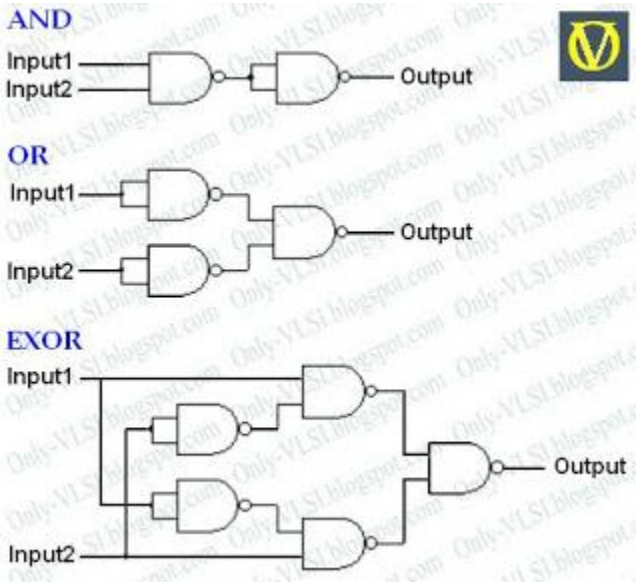
27. Design a FSM which detects the sequence 10101 from a serial line without overlapping.
Answer

28. Design a FSM which detects the sequence 10101 from a serial line with overlapping.
Answer

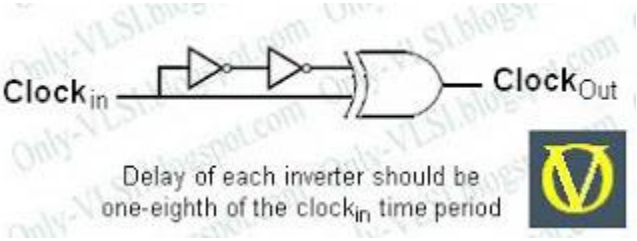
29. Give the design of 8x1 multiplexer using 2x1 multiplexers.
Answer

30. Design a counter which counts from 1 to 10 (Resets to 1, after 10).
Answer

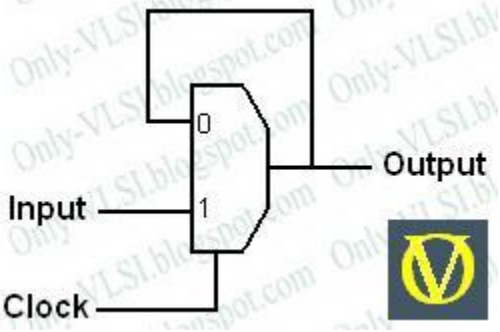
31. Design 2 input AND, OR, and EXOR gates using 2 input NAND gate.
Answer



32. Design a circuit which doubles the frequency of a given input clock signal.
Answer



33. Implement a D-latch using 2x1 multiplexer(s).
Answer



34. Give the excitation table of a JK flip-flop.
Answer

35. Give the Binary, Hexadecimal, BCD, and Excess-3 code for decimal 14.
Answer

14:
Binary: 1110
Hexadecimal: E
BCD: 0001 0100
Excess-3: 10001

36. What is race condition?
Answer

37. Give 1's and 2's complement of 19.
Answer

19: 10011
1's complement: 01100
2's complement: 01101

38. Design a 3:6 decoder.
Answer

39. If A*B=C and C*A=B then, what is the Boolean operator * ?
Answer

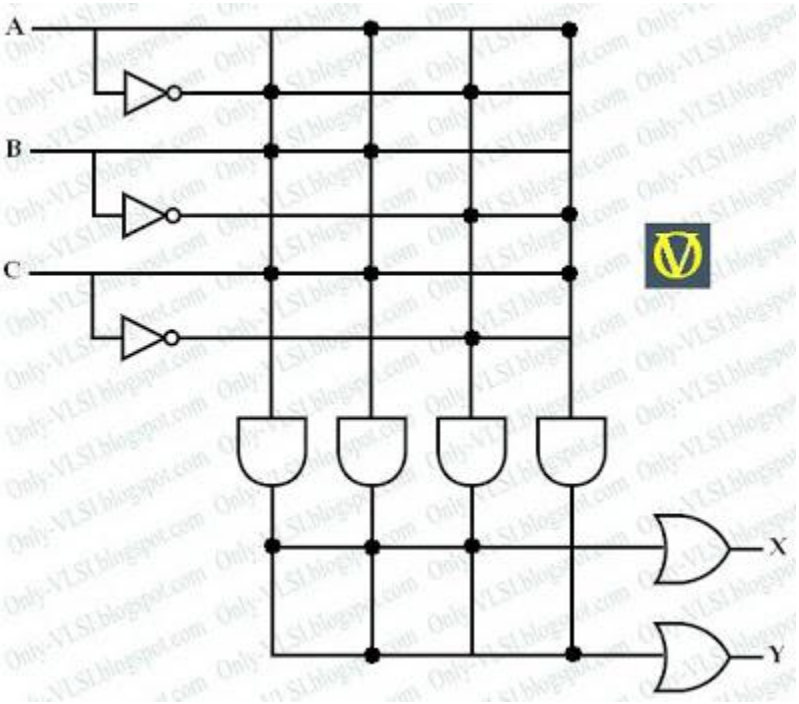
* is Exclusive-OR.

40. Design a 3 bit Gray Counter.
Answer

41. Expand the following: PLA, PAL, CPLD, FPGA.
Answer

PLA - Programmable Logic Array
PAL - Programmable Array Logic
CPLD - Complex Programmable Logic Device
FPGA - Field-Programmable Gate Array

42. Implement the functions: X = A'BC + ABC + A'B'C' and Y = ABC + AB'C using a PLA.
Answer



43. What are PLA and PAL? Give the differences between them.
Answer

Programmable Logic Array is a programmable device used to implement combinational logic circuits. The PLA has a set of programmable AND planes, which link to a set of programmable OR planes, which can then be conditionally complemented to produce an output.
PAL is programmable array logic, like PLA, it also has a wide, programmable AND plane. Unlike a PLA, the OR plane is fixed, limiting the number of terms that can be ORed together. Due to fixed OR plane PAL allows extra space, which is used for other basic logic devices, such as multiplexers, exclusive-ORs, and latches. Most importantly, clocked elements, typically flip-flops, could be included in PALs. PALs are also extremely fast.

44. What is LUT?
Answer

LUT - Look-Up Table. An n-bit look-up table can be implemented with a multiplexer whose select lines are the inputs of the LUT and whose inputs are constants. An n-bit LUT can encode any n-input Boolean function by modeling such functions as truth tables. This is an efficient way of encoding Boolean logic functions, and LUTs with 4-6 bits of input are in fact the key component of modern FPGAs.

45. What is the significance of FPGAs in modern day electronics? (Applications of FPGA.)
Answer

- ASIC prototyping: Due to high cost of ASIC chips, the logic of the application is first verified by dumping HDL code in a FPGA. This helps for faster and cheaper testing. Once the logic is verified then they are made into ASICs.
- Very useful in applications that can make use of the massive parallelism offered by their architecture. Example: code breaking, in particular brute-force attack, of cryptographic algorithms.
- FPGAs are sued for computational kernels such as FFT or Convolution instead of a microprocessor.
- Applications include digital signal processing, software-defined radio, aerospace and defense systems, medical imaging, computer vision, speech recognition, cryptography, bio-informatics, computer hardware emulation and a growing range of other areas.

46. What are the differences between CPLD and FPGA.
Answer

47. Compare and contrast FPGA and ASIC digital designing.
Answer

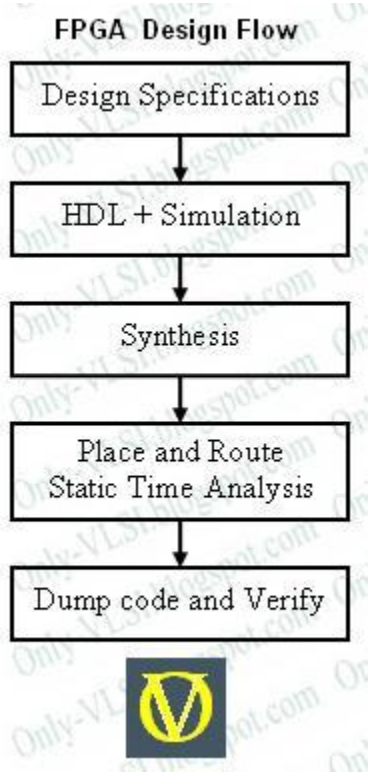
Click here.

48. Give True or False.
(a) CPLD consumes less power per gate when compared to FPGA.
(b) CPLD has more complexity than FPGA
(c) FPGA design is slower than corresponding ASIC design.
(d) FPGA can be used to verify the design before making a ASIC.
(e) PALs have programmable OR plane.
(f) FPGA designs are cheaper than corresponding ASIC, irrespective of design complexity.
Answer

49. Arrange the following in the increasing order of their complexity: FPGA,PLA,CPLD,PAL.
Answer

Increasing order of complexity: PLA, PAL, CPLD, FPGA.

50. Give the FPGA digital design cycle.
Answer



51. What is DeMorgan's theorem?
Answer

For N variables, DeMorgan's theorems are expressed in the following formulas:
(ABC..N)' = A' + B' + C' + ... + N' -- The complement of the product is equivalent to the sum of the complements.
(A + B + C + ... + N)' = A'B'C'...N' -- The complement of the sum is equivalent to the product of the complements.
This relationship so induced is called DeMorgan's duality.

52. F'(A, B, C, D) = C'D + ABC' + ABCD + D. Express F in Product of Sum form.
Answer

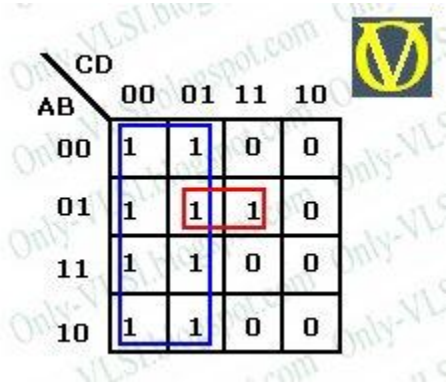
Complementing both sides and applying DeMorgan's Theorem:
F(A, B, C, D) = (C + D')(A' + B' + C)(A' + B' + C' + D')(D')

53. How many squares/cells will be present in the k-map of F(A, B, C)?
Answer

F(A, B, C) has three variables/inputs.
Therefore, number of squares/cells in k-map of F = $2^{(\text{Number of variables})} = 2^3 = 8$.

54. Simplify F(A, B, C, D) = S (0, 1, 4, 5, 7, 8, 9, 12, 13)
Answer

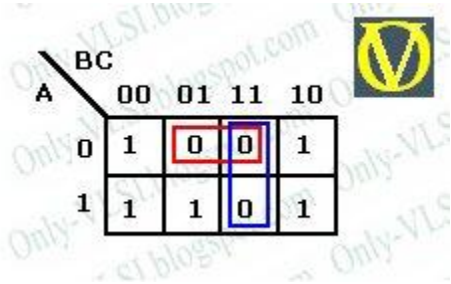
The four variable k-map of the given expression is:



The grouping is also shown in the diagram. Hence we get,
F(A, B, C, D) = C' + A'BD

55. Simplify F(A, B, C) = S (0, 2, 4, 5, 6) into Product of Sums.
Answer

The three variable k-map of the given expression is:



The 0's are grouped to get the F'.
F' = A'C + BC

Complementing both sides and using DeMorgan's theorem we get F,
F = (A + C')(B' + C')

56. The simplified expression obtained by using k-map method is unique. True or False. Explain your answer.
Answer

False. The simplest form obtained is not necessarily unique as grouping can be made in different ways.

57. Give the characteristic tables of RS, JK, D and T flip-flops.
Answer

RS flip-flop.

S R Q(t+1)
0 0 Q(t)
0 1 0
1 0 1
1 1 ?

JK flip-flop
J K Q(t+1)
0 0 Q(t)
0 1 0
1 0 1
1 1 Q'(t)

D flip-flop
D Q(t+1)
0 0
1 1

T flip-flop
T Q(t+1)
0 Q(t)
1 Q'(t)

58. Give excitation tables of RS, JK, D and T flip-flops.
Answer

RS flip-flop.

Q(t)	Q(t+1)	S	R
0	0	0	X
0	1	1	0
1	0	0	1
1	1	X	0

JK flip-flop

Q(t)	Q(t+1)	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

D flip-flop

Q(t)	Q(t+1)	D
0	0	0
0	1	1
1	0	0
1	1	1

T flip-flop

Q(t)	Q(t+1)	T
0	0	0
0	1	1
1	0	1
1	1	0

59. Design a BCD counter with JK flip-flops
Answer

60. Design a counter with the following binary sequence 0, 1, 9, 3, 2, 8, 4 and repeat. Use T flip-flops.
Answer

Digital Design Interview Questions - 6

1. What is DeMorgan's theorem?
Answer

For N variables, DeMorgan's theorems are expressed in the following formulas:
(ABC..N)' = A' + B' + C' + ... + N' -- The complement of the product is equivalent to the sum of the complements.
(A + B + C + ... + N)' = A'B'C'...N' -- The complement of the sum is equivalent to the product of the complements.
This relationship so induced is called DeMorgan's duality.

2. F'(A, B, C, D) = C'D + ABC' + ABCD + D. Express F in Product of Sum form.
Answer

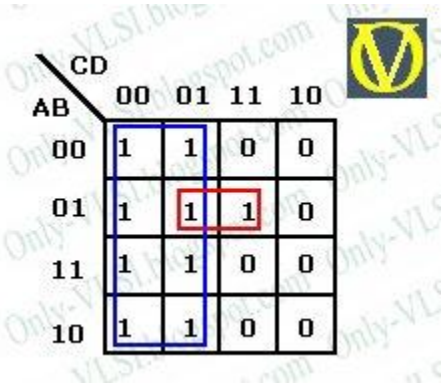
Complementing both sides and applying DeMorgan's Theorem:
F(A, B, C, D) = (C + D')(A' + B' + C)(A' + B' + C' + D')(D')

3. How many squares/cells will be present in the k-map of F(A, B, C)?
Answer

F(A, B, C) has three variables/inputs.
Therefore, number of squares/cells in k-map of F = $2^{(\text{Number of variables})} = 2^3 = 8$.

4. Simplify F(A, B, C, D) = Σ (0, 1, 4, 5, 7, 8, 9, 12, 13)
Answer

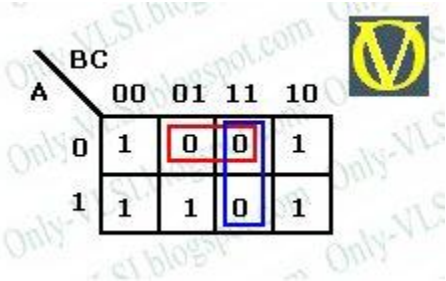
The four variable k-map of the given expression is:



The grouping is also shown in the diagram. Hence we get,
F(A, B, C, D) = C' + A'BD

5. Simplify F(A, B, C) = Σ (0, 2, 4, 5, 6) into Product of Sums.
Answer

The three variable k-map of the given expression is:



The 0's are grouped to get the F'.
F' = A'C + BC

Complementing both sides and using DeMorgan's theorem we get F,
F = (A + C')(B' + C')

6. The simplified expression obtained by using k-map method is unique. True or False. Explain your answer.
Answer

False. The simplest form obtained is not necessarily unique as grouping can be made in different ways.

7. Give the characteristic tables of RS, JK, D and T flip-flops.
Answer

RS flip-flop.
S R Q(t+1)
0 0 Q(t)
0 1 0
1 0 1
1 1 ?

JK flip-flop
J K Q(t+1)
0 0 Q(t)
0 1 0
1 0 1
1 1 Q'(t)

D flip-flop
D Q(t+1)
0 0
1 1

T flip-flop
T Q(t+1)
0 Q(t)
1 Q'(t)

8. Give excitation tables of RS, JK, D and T flip-flops.
Answer

RS flip-flop.
Q(t) Q(t+1) S R
0 0 0 X
0 1 1 0
1 0 0 1
1 1 X 0

JK flip-flop

Q(t)	Q(t+1)	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

D flip-flop

Q(t)	Q(t+1)	D
0	0	0
0	1	1
1	0	0
1	1	1

T flip-flop

Q(t)	Q(t+1)	T
0	0	0
0	1	1
1	0	1
1	1	0

9. Design a BCD counter with JK flip-flops
Answer

10. Design a counter with the following binary sequence 0, 1, 9, 3, 2, 8, 4 and repeat. Use T flip-flops.
Answer

Microprocessor Interview Questions - 5

1. Why are program counter and stack pointer 16-bit registers?
Answer

Program Counter (PC) and Stack Pointer (SP) are basically used to hold 16-bit memory addresses.PC stores the 16-bit memory address of the next instruction to be fetched. SP stores address of stack's starting block.

2. What happens during DMA transfer?
Answer

3. Define ISR.
Answer

An interrupt handler, also known as an interrupt service routine (ISR), is a callback subroutine in an operating system or device driver whose execution is triggered by the reception of an interrupt. Whenever there is an interrupt the processor jumps to ISR and executes it.

4. Define PSW.
Answer

The Program Status Word (PSW) is a register which contains information about the current program status used by the operating system and the underlying hardware. The PSW includes the instruction address, condition code, and other fields. In general, the PSW is used to control instruction sequencing and to hold and indicate the status of the system in relation to the program currently being executed. The active or controlling PSW is called the current PSW. By storing the current PSW during an interruption, the status of the CPU can be preserved for subsequent inspection. By loading a new PSW or part of a PSW, the state of the CPU can be initialized or changed.

5. What are the execution modes available in x86 processors?
Answer

* Real mode (16-bit)
* Protected mode (16-bit and 32-bit)
* Virtual 8086 mode (16-bit)
* Unreal mode (32-bit)
* System Management Mode (16-bit)
* Long mode (64-bit)

6. What is meant real mode?
Answer

Real mode is an execution/operating mode of 80286 and later x86-compatible CPUs. Real mode is characterized by a 20 bit segmented memory address space, where a maximum of 1 MB of memory can be addressed, direct software access to BIOS routines and peripheral hardware, and no concept of memory protection or multitasking at the hardware level. All x86 CPUs in the 80286 series and later start in real mode at power-on (earlier CPUs had only one operational mode, which is equivalent to real mode in later chips).

7. What is protected mode?
Answer

Protected mode allows system software to utilize features such as virtual memory, paging, safe multi-tasking, and other features designed to increase an operating system's control over application software.
When a processor that supports x86 protected mode is powered on, it begins executing instructions in real mode, in order to maintain backwards compatibility with earlier x86 processors. Protected mode may only be entered after the system software sets up several descriptor tables and enables the Protection Enable (PE) bit in the Control Register 0.

8. What is virtual 8086 mode?
Answer

Virtual real mode or VM86, allows the execution of real mode applications that are incapable of running directly in protected mode. It uses a segmentation scheme identical to that of real mode, and also uses 21-bit addressing - resulting in linear addressing - so it is subject to paging.

9. What is unreal mode?
Answer

Unreal mode, also known as big real mode, huge real mode, or flat real mode, is a variant of real mode. one or more data segment registers will be loaded with 32-bit addresses and limits.

10. What is the difference between ISR and a function call?
Answer

ISR has no return value, where as a function call has the return value.

VLSI Interview Questions - 6

1. Why is NAND gate preferred over NOR gate for fabrication?
Answer

2. Which transistor has higher gain: BJT or MOSFET and why?
Answer

3. Why PMOS and NMOS are sized equally in a transmission gates?
Answer

4. What is SCR?
Answer

5. In CMOS digital design, why is the size of PMOS is generally higher than that of the NMOS?
Answer

6. What is slack?

Answer

7. What is latch up?
Answer

8. Why is the size of inverters in buffer design gradually increased? Why not give the output of a circuit to one large inverter?
Answer

9. What is Charge Sharing? Explain the Charge Sharing problem while sampling data from a Bus?
Answer

10. What happens to delay if load capacitance is increased?
Answer

Microprocessor Interview Questions - 4

1. What is the size of flag register of 8086 processor?
Answer

2. How many pin IC 8086 is?
Answer

3. What is the Maximum clock frequency of 8086?
Answer

4. What is meant by instruction cycle?
Answer

5. What is Von Neumann architecture?
Answer

6. What is the main difference between 8086 and 8085?
Answer

7. What does EAX mean?
Answer

8. What type of instructions are available in instruction set of 8086?
Answer

9. How is Stack Pointer affected when a PUSH and POP operations are performed?
Answer

10. What are SIM and RIM instructions?
Answer

Microprocessor Interview Questions - 3

1. How many bits processor is 8086?
Answer

2. What are the sizes of data bus and address bus in 8086?
Answer

3. What is the maximum addressable memory of 8086?
Answer

4. How are 32-bit addresses stored in 8086?
Answer

5. What are the 16-bit registers that are available in 8086?
Answer

6. What are the different types of address modes available in 8086?
Answer

7. How many flags are available in flag register? What are they?
Answer

8. Explain the functioning of IP (instruction pointer).
Answer

9. What are the various types of interrupts present in 8086?
Answer

10. How many segments are present in 8086? What are they?
Answer

Digital Design Interview Questions - 5

1. Expand the following: PLA, PAL, CPLD, FPGA.
Answer

2. Implement the functions: X = A'BC + ABC + A'B'C' and Y = ABC + AB'C using a PLA.
Answer

3. What are PLA and PAL? Give the differences between them.

Answer

4. What is LUT?
Answer

5. What is the significance of FPGAs in modern day electronics? (Applications of FPGA.)
Answer

6. What are the differences between CPLD and FPGA.
Answer

7. Compare and contrast FPGA and ASIC digital designing.
Answer

8. Give True or False.
(a) CPLD consumes less power per gate when compared to FPGA.
(b) CPLD has more complexity than FPGA
(c) FPGA design is slower than corresponding ASIC design.
(d) FPGA can be used to verify the design before making a ASIC.
(e) PALs have programmable OR plane.
(f) FPGA designs are cheaper than corresponding ASIC, irrespective of design complexity.
Answer

9. Arrange the following in the increasing order of their complexity: FPGA,PLA,CPLD,PAL.
Answer

10. Give the FPGA digital design cycle.
Answer

2 Comments

Labels: Interview Questions

Digital Design Interview Questions - 4

1. Design 2 input AND, OR, and EXOR gates using 2 input NAND gate.
Answer

2. Design a circuit which doubles the frequency of a given input clock signal.
Answer

3. Implement a D-latch using 2x1 multiplexer(s).
Answer

4. Give the excitation table of a JK flip-flop.

Answer

5. Give the Binary, Hexadecimal, BCD, and Excess-3 code for decimal 14.
Answer

6. What is race condition?
Answer

7. Give 1's and 2's complement of 19.
Answer

8. Design a 3:6 decoder.
Answer

9. If A*B=C and C*A=B then, what is the Boolean operator * ?
Answer

10. Design a 3 bit Gray Counter.
Answer

4 Comments

Labels: Interview Questions

Verilog Interview Questions - 3

1. How are blocking and non-blocking statements executed?
Answer

2. How do you model a synchronous and asynchronous reset in Verilog?
Answer

3. What happens if there is connecting wires width mismatch?
Answer

4. What are different options that can be used with $display statement in Verilog?
Answer

5. Give the precedence order of the operators in Verilog.
Answer

6. Should we include all the inputs of a combinational circuit in the sensitivity list? Give reason.
Answer

7. Give 10 commonly used Verilog keywords.

Answer

8. Is it possible to optimize a Verilog code such that we can achieve low power design?
Answer

9. How does the following code work?

```
wire [3:0] a;
always @(*)
begin
case (1'b1)
 a[0]: $display("Its a[0]");
 a[1]: $display("Its a[1]");
 a[2]: $display("Its a[2]");
 a[3]: $display("Its a[3]");
 default: $display("Its default")
endcase
end
```

Answer

10. Which is updated first: signal or variable?
Answer

7 Comments

Labels: Interview Questions

VLSI Interview Questions - 5

This sections contains interview questions related to LOW POWER VLSI DESIGN.

1. What are the important aspects of VLSI optimization?
Answer

2. What are the sources of power dissipation?
Answer

3. What is the need for power reduction?
Answer

4. Give some low power design techniques.
Answer

5. Give a disadvantage of voltage scaling technique for power reduction.
Answer

6. Give an expression for switching power dissipation.
Answer

7. Will glitches in a logic circuit cause power wastage?
Answer

8. What is the major source of power wastage in SRAM?
Answer

9. What is the major problem associated with caches w.r.t low power design? Give techniques to overcome it.
Answer

10. Does software play any role in low power design?
Answer

1 Comments

Labels: Interview Questions

Digital Design Interview Questions - 1

1. How do you convert a XOR gate into a buffer and a inverter (Use only one XOR gate for each)?
Answer

2. Implement an 2-input AND gate using a 2x1 mux.
Answer

3. What is a multiplexer?
Answer

4. What is a ring counter?
Answer

5. Compare and Contrast Synchronous and Asynchronous reset.
Answer

6. What is a Johnson counter?
Answer

7. An assembly line has 3 fail safe sensors and one emergency shutdown switch.The line should keep moving unless any of the following conditions arise:
(1) If the emergency switch is pressed
(2) If the senor1 and sensor2 are activated at the same time.
(3) If sensor 2 and sensor3 are activated at the same time.
(4) If all the sensors are activated at the same time

Suppose a combinational circuit for above case is to be implemented only with NAND Gates. How many minimum number of 2 input NAND gates are required?
Answer

8. In a 4-bit Johnson counter How many unused states are present?
Answer

9. Design a 3 input NAND gate using minimum number of 2 input NAND gates.
Answer

10. How can you convert a JK flip-flop to a D flip-flop?
Answer

13 Comments

Labels: Interview Questions

VLSI Interview Questions - 4

1. Why is the number of gate inputs to CMOS gates (e.g. NAND or NOR gates)usually limited to four?
Answer

2. What are static and dynamic power dissipation w.r.t to CMOS gate?
Answer

3. Draw Vds-Ids curve for a MOSFET. Now, show how this curve changes (a) with increasing Vgs (b) considering Channel Length Modulation.
Answer

4. Which is fastest among the following technologies: CMOS, BiCMOS, TTL, ECL?
Answer

5. What is a transmission gate, and what is its typical use in VLSI?
Answer

6. Draw the cross section of nMOS or pMOS.
Answer

7. What should be done to the size of a pMOS transistor inorder to increase its threshold voltage?
Answer

8. Explain the various MOSFET Capacitances and their significance.
Answer

9. On what factors does the resistance of metal depend on?
Answer

10. Draw the layout a CMOS NAND gate.
Answer

1 Comments

Labels: Interview Questions

VLSI Interview Questions - 3

1. Explain the voltage transfer characteristics of a CMOS Inverter.
Answer

2. What should be done to the size of a nMOS transistor in order to increase its threshold voltage?
Answer

3. What are the advantages of CMOS technology?
Answer

4. Give the expression for CMOS switching power dissipation.
Answer

5. Why is static power dissipation very low in CMOS technology when compared to others?
Answer

6. What is velocity saturation? What are its effects?
Answer

7. Why are pMOS transistor networks generally used to produce high signals, while nMOS networks are used to product low signals?
Answer

8. Expand: DTL, RTL, ECL, TTL, CMOS, BiCMOS.
Answer

9. On IC schematics, transistors are usually labeled with two, or sometimes one number(s). What do each of those numbers mean?
Answer

10. How do you calculate the delay in a CMOS circuit?
Answer

2 Comments

Labels: Interview Questions

VLSI Interview Questions - 2

1. Explain the various MOSFET capacitance and give their significance.
Answer

2. What is the fundamental difference between a MOSFET and BJT ?
Answer

3. What is meant by scaling in VLSI design? Describe various effects of scaling.
Answer

4. What is early effect?
Answer

5. Compare and contrast analog and digital design.
Answer

6. What are various types of the number notations? Explain them.
Answer

7. Why are most interrupts active low?
Answer

8. Which is better: synchronous reset or asynchronous reset signal?
Answer

9. What is meant by 90nm technology?
Answer

10. Compare enhancement and depletion mode devices.
Answer

0 Comments

Labels: Interview Questions

Digital Design Interview Questions - 2

1. What are the differences between a flip-flop and a latch?
Answer

2. What is the difference between Mealy and Moore FSM?
Answer

3. What are various types of state encoding techniques? Explain them.
Answer

4. Define Clock Skew , Negative Clock Skew, Positive Clock Skew.
Answer

5. Give the transistor level circuit of a CMOS NAND gate.
Answer

6. Design a 4-bit comparator circuit.
Answer

7. Design a Transmission Gate based XOR. Now, how do you convert it to XNOR (without inverting the output)?
Answer

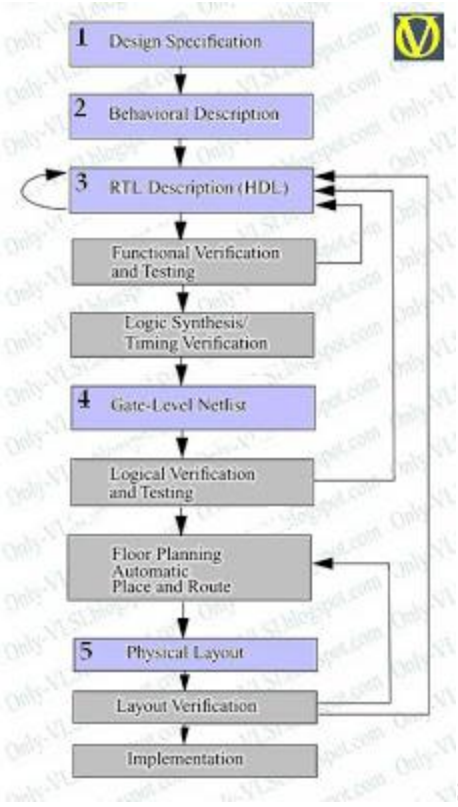
8. Define Metastability.
Answer

9. Compare and contrast between 1's complement and 2's complement notation.
Answer

10. Give the transistor level circuit of CMOS, nMOS, pMOS, and TTL inverter gate.
Answer

The VLSI Design Flow

The VLSI IC circuits design flow is shown in the figure below. The various level of design are numbered and the gray coloured blocks show processes in the design flow.



Specifications comes first, they describe abstractly the functionality, interface, and the architecture of the digital IC circuit to be designed.

- Behavioral description is then created to analyze the design in terms of functionality, performance, compliance to given standards, and other specifications.
- RTL description is done using HDLs. This RTL description is simulated to test functionality. From here onwards we need the help of EDA tools.
- RTL description is then converted to a gate-level netlist using logic synthesis tools. A gate-level netlist is a description of the circuit in terms of gates and connections between them, which are made in such a way that they meet the timing, power and area specifications.
- Finally a physical layout is made, which will be verified and then sent to fabrication.

Behavioral Modeling

>> **Introduction**
>> **The initial Construct**
>> **The always Construct**
>> **Procedural Assignments**
>> **Block Statements**
>> **Conditional (if-else) Statement**
>> **Case Statement**
>> **Loop Statements**
>> **Examples**

Introduction

Behavioral modeling is the highest level of abstraction in the Verilog HDL. The other modeling techniques are relatively detailed. They require some knowledge of how hardware, or hardware signals work. The abstraction in this modeling is as simple as writing the logic in C language. This is a very powerful abstraction technique. All that designer needs is the algorithm of the design, which is the basic information for any design.

Most of the behavioral modeling is done using two important constructs: initial and always. All the other behavioral statements appear only inside these two structured procedure constructs.

The initial Construct

The statements which come under the *initial* construct constitute the initial block. The initial block is executed only once in the simulation, at time 0. If there is more than one initial block. Then all the initial blocks are executed concurrently. The initial construct is used as follows:

```
initial
begin
reset = 1'b0;
clk = 1'b1;
end

or

initial
clk = 1'b1;
```

In the first initial block there are more than one statements hence they are written between begin and end. If there is only one statement then there is no need to put begin and end.

The always Construct

The statements which come under the *always* construct constitute the always block. The always block starts at time 0, and keeps on executing all the simulation time. It works like a infinite loop. It is generally used to model a functionality that is continuously repeated.

```
always
#5 clk = ~clk;

initial
clk = 1'b0;
```

The above code generates a clock signal clk, with a time period of 10 units. The initial blocks initiates the clk value to 0 at time 0. Then after every 5 units of time it toggled, hence we get a time period of 10 units. This is the way in general used to generate a clock signal for use in test benches.

```
always @(posedge clk, negedge reset)
begin
a = b + c;
    d = 1'b1;
end
```

In the above example, the always block will be executed whenever there is a positive edge in the clk signal, or there is negative edge in the reset signal. This type of always is generally used in implement a FSM, which has a reset signal.

```
always @(b,c,d)
begin
    a = ( b + c )*d;
    e = b | c;
end
```

In the above example, whenever there is a change in b, c, or d the always block will be executed. Here the list b, c, and d is called the *sensitivity list*.

In the Verilog 2000, we can replace always @(b,c,d) with always @(*), it is equivalent to include all input signals, used in the always block. This is very useful when always blocks is used for implementing the combination logic.

Procedural Assignments

Procedural assignments are used for updating *reg*, *integer*, *time*, *real*, *realtime*, and *memory* data types. The variables will retain their values until updated by another procedural assignment. There is a significant difference between procedural assignments and continuous assignments. Continuous assignments drive nets and are evaluated and updated whenever an input operand changes value. Where as procedural assignments update the value of variables under the control of the procedural flow constructs that surround them.

The LHS of a procedural assignment could be:

- *reg*, *integer*, *real*, *realtime*, or *time* data type.
- Bit-select of a *reg*, *integer*, or *time* data type, rest of the bits are untouched.
- Part-select of a *reg*, *integer*, or *time* data type, rest of the bits are untouched.
- Memory word.
- Concatenation of any of the previous four forms can be specified.

When the RHS evaluates to fewer bits than the LHS, then if the right-hand side is signed, it will be sign-extended to the size of the left-hand side.

There are two types of procedural assignments: blocking and non-blocking assignments.

Blocking assignments: A blocking assignment statements are executed in the order they are specified in a sequential block. The execution of next statement begin only after the completion

of the present blocking assignments. A blocking assignment will not block the execution of the next statement in a parallel block. The blocking assignments are made using the operator =.

```
initial
begin
    a = 1;
    b = #5 2;
    c = #2 3;
end
```

In the above example, a is assigned value 1 at time 0, and b is assigned value 2 at time 5, and c is assigned value 3 at time 7.

Non-blocking assignments: The nonblocking assignment allows assignment scheduling without blocking the procedural flow. The nonblocking assignment statement can be used whenever several variable assignments within the same time step can be made without regard to order or dependence upon each other. Non-blocking assignments are made using the operator <=.
Note: <= is same for less than or equal to operator, so whenever it appears in a expression it is considered to be comparison operator and not as non-blocking assignment.

```
initial
begin
    a <= 1;
    b <= #5 2;
    c <= #2 3;
end
```

In the above example, a is assigned value 1 at time 0, and b is assigned value 2 at time 5, and c is assigned value 3 at time 2 (because all the statements execution starts at time 0, as they are non-blocking assignments.

Block Statements

Block statements are used to group two or more statements together, so that they act as one statement. There are two types of blocks:

- Sequential block.
- Parallel block.

Sequential block: The sequential block is defined using the keywords *begin* and *end*. The procedural statements in sequential block will be executed sequentially in the given order. In sequential block delay values for each statement shall be treated relative to the simulation time of the execution of the previous statement. The control will pass out of the block after the execution of last statement.

Parallel block: The parallel block is defined using the keywords *fork* and *join*. The procedural statements in parallel block will be executed concurrently. In parallel block delay values for each statement are considered to be relative to the simulation time of entering the block. The delay control can be used to provide time-ordering for procedural assignments. The control shall pass

out of the block after the execution of the last time-ordered statement.

Note that blocks can be nested. The sequential and parallel blocks can be mixed.

Block names: All the blocks can be named, by adding *: block_name* after the keyword *begin* or *fork*. The advantages of naming a block are:

- It allows to declare local variables, which can be accessed by using hierarchical name referencing.
- They can be disabled using the *disable* statement (*disable block_name;*).

Conditional (if-else) Statement

The condition (if-else) statement is used to make a decision whether a statement is executed or not. The keywords *if* and *else* are used to make conditional statement. The conditional statement can appear in the following forms.

```
if ( condition_1 )
    statement_1;

if ( condition_2 )
    statement_2;
else
    statement_3;

if ( condition_3 )
    statement_4;
else if ( condition_4 )
    statement_5;
else
    statement_6;

if ( condition_5 )
begin
    statement_7;
    statement_8;
end
else
begin
    statement_9;
    statement_10;
end
```

Conditional (if-else) statement usage is similar to that if-else statement of C programming language, except that parenthesis are replaced by *begin* and *end*.

Case Statement

The case statement is a multi-way decision statement that tests whether an expression matches

one of the expressions and branches accordingly. Keywords *case* and *endcase* are used to make a case statement. The case statement syntax is as follows.

```
case (expression)
    case_item_1: statement_1;
    case_item_2: statement_2;
    case_item_3: statement_3;
    ...
    ...
    default: default_statement;
endcase
```

If there are multiple statements under a single match, then they are grouped using begin, and end keywords. The default item is optional.

Case statement with don't cares: casez and casex

casez treats high-impedance values (z) as don't cares. *casex* treats both high-impedance (z) and unknown (x) values as don't cares. Don't-care values (z values for *casez*, z and x values for *casex*) in any bit of either the case expression or the case items shall be treated as don't-care conditions during the comparison, and that bit position shall not be considered. The don't cares are represented using the ? mark.

Loop Statements

There are four types of looping statements in Verilog:

- forever
- repeat
- while
- for

Forever Loop

Forever loop is defined using the keyword forever, which Continuously executes a statement. It terminates when the system task $finish is called. A forever loop can also be ended by using the disable statement.

```
initial
begin
    clk = 1'b0;
    forever #5 clk = ~clk;
end
```

In the above example, a clock signal with time period 10 units of time is obtained.

Repeat Loop

Repeat loop is defined using the keyword repeat. The repeat loop block continuously executes the block for a given number of times. The number of times the loop executes can be mention using a constant or an expression. The expression is calculated only once, before the start of loop and not during the execution of the loop. If the expression value turns out to be z or x, then it is treated as zero, and hence loop block is not executed at all.

```
initial
begin
    a = 10;
    b = 5;
    b <= #10 10;
    i = 0;
    repeat(a*b)
    begin
        $display("repeat in progress");
        #1 i = i + 1;
    end
end
```

In the above example the loop block is executed only 50 times, and not 100 times. It calculates (a*b) at the beginning, and uses that value only.

While Loop

The while loop is defined using the keyword while. The while loop contains an expression. The loop continues until the expression is true. It terminates when the expression is false. If the calculated value of expression is z or x, it is treated as a false. The value of expression is calculated each time before starting the loop. All the statements (if more than one) are mentioned in blocks which begins and ends with keyword begin and end keywords.

```
initial
begin
    a = 20;
    i = 0;
    while (i < a)
    begin
    $display("%d",i);
    i = i + 1;
    a = a - 1;
    end
end
```

In the above example the loop executes for 10 times. (observe that a is decrementing by one and i is incrementing by one, so loop terminated when both i and a become 10).

For Loop

The For loop is defined using the keyword for. The execution of for loop block is controlled by a three step process, as follows:

1. Executes an assignment, normally used to initialize a variable that controls the number of times the for block is executed.
2. Evaluates an expression, if the result is false or z or x, the for-loop shall terminate, and if it is true, the for-loop shall execute its block.
3. Executes an assignment normally used to modify the value of the loop-control variable and then repeats with second step.

Note that the first step is executed only once.

```
initial
begin
    a = 20;
    for (i = 0; i < a; i = i + 1, a = a - 1)
    $display("%d",i);
end
```

The above example produces the same result as the example used to illustrate the functionality of the while loop.

Examples

1. Implementation of a 4x1 multiplexer.

```
module 4x1_mux (out, in0, in1, in2, in3, s0, s1);

output out;

// out is declared as reg, as default is wire

reg out;

// out is declared as reg, because we will
// do a procedural assignment to it.

input in0, in1, in2, in3, s0, s1;

// always @(*) is equivalent to
// always @( in0, in1, in2, in3, s0, s1 )

always @(*)
begin
  case ({s1,s0})
      2'b00: out = in0;
      2'b01: out = in1;
      2'b10: out = in2;
      2'b11: out = in3;
      default: out = 1'bx;
  endcase
end

endmodule
```

2. Implementation of a full adder.

```
module full_adder (sum, c_out, in0, in1, c_in);

output sum, c_out;
reg sum, c_out

input in0, in1, c_in;

always @(*)
  {c_out, sum} = in0 + in1 + c_in;

endmodule
```

3. Implementation of a 8-bit binary counter.

```
module ( count, reset, clk );

output [7:0] count;
reg [7:0] count;

input reset, clk;

// consider reset as active low signal

always @( posedge clk, negedge reset)
begin
  if(reset == 1'b0)
      count <= 8'h00;
  else
      count <= count + 8'h01;
end

endmodule
```

Implementation of a 8-bit counter is a very good example, which explains the advantage of behavioral modeling. Just imagine how difficult it will be implementing a 8-bit counter using gate-level modeling.
In the above example the incrementation occurs on every positive edge of the clock. When count becomes 8'hFF, the next increment will make it 8'h00, hence there is no need of any modulus operator. Reset signal is active low.

<< Previous **Home** **Next >>**

1 Comments

Labels: Verilog Tutorial

Tasks and Functions

>> **Introduction**
>> **Differences**
>> **Tasks**
>> **Functions**
>> **Examples**

Introduction

Tasks and functions are introduced in the verilog, to provide the ability to execute common procedures from different places in a description. This helps the designer to break up large behavioral designs into smaller pieces. The designer has to abstract the similar pieces in the description and replace them either functions or tasks. This also improves the readability of the code, and hence easier to debug. Tasks and functions must be defined in a module and are local to the module. Tasks are used when:

- There are delay, timing, or event control constructs in the code.
- There is no input.
- There is zero output or more than one output argument.

Functions are used when:

- The code executes in zero simulation time.
- The code provides only one output(return value) and has at least one input.
- There are no delay, timing, or event control constructs.

Differences

Functions	**Tasks**
Can enable another function but not another task.	Can enable other tasks and functions.
Executes in 0 simulation time.	May execute in non-zero simulation time.
Must not contain any delay, event, or timing control statements.	May contain delay, event, or timing control statements.
Must have at least one input argument. They can have more than one input.	May have zero or more arguments of type input, output, or inout.
Functions always return a single value. They cannot have output or inout arguments.	Tasks do not return with a value, but can pass multiple values through output and inout arguments.

Tasks

There are two ways of defining a task. The first way shall begin with the keyword task, followed

by the optional keyword automatic, followed by a name for the task, and ending with the keyword endtask. The keyword automatic declares an automatic task that is reentrant with all the task declarations allocated dynamically for each concurrent task entry. Task item declarations can specify the following:

- Input arguments.
- Output arguments.
- Inout arguments.
- All data types that can be declared in a procedural block

The second way shall begin with the keyword task, followed by a name for the task and a parenthesis which encloses task port list. The port list shall consist of zero or more comma separated ports. The task body shall follow and then the keyword endtask.

In both ways, the port declarations are same. Tasks without the optional keyword automatic are static tasks, with all declared items being statically allocated. These items shall be shared across all uses of the task executing concurrently. Task with the optional keyword automatic are automatic tasks. All items declared inside automatic tasks are allocated dynamically for each invocation. Automatic task items can not be accessed by hierarchical references. Automatic tasks can be invoked through use of their hierarchical name.

Functions

Functions are mainly used to return a value, which shall be used in an expression. The functions are declared using the keyword function, and definition ends with the keyword endfunction.

If a function is called concurrently from two locations, the results are non-deterministic because both calls operate on the same variable space. The keyword automatic declares a recursive function with all the function declarations allocated dynamically for each recursive call. Automatic function items can not be accessed by hierarchical references. Automatic functions can be invoked through the use of their hierarchical name.

When a function is declared, a register with function name is declared implicitly inside Verilog HDL. The output of a function is passed back by setting the value of that register appropriately.

Examples

1. Simple task example, where task is used to get the address tag and offset of a given address.

```
module example1_task;

input addr;
wire [31:0] addr;

wire [23:0] addr_tag;
wire [7:0] offset;

task get_tag_and_offset ( addr, tag, offset);
```

```
input addr;
output tag, offset;

begin
 tag = addr[31:8];
 offset = addr[7:0];
end
endtask

always @(addr)
begin
 get_tag_and_offset (addr, addr_tag, addr_offset);
end

// other internals of module

endmodule
```

2. Task example, which uses the global variables of a module. Here task is used to do temperature conversion.

```
module example2_global;

real t1;
real t2;

// task uses the global variables of the module

task t_convert;
begin
 t2 = (9/5)*(t1+32);
end
endtask

always @(t1)
begin
 t_convert();
end

endmodule
```

<< Previous **Home** **Next >>**

0 Comments

Labels: Verilog Tutorial

Dataflow Modeling

>> **Introduction**
>> **The *assign* Statement**
>> **Delays**
>> **Examples**

Introduction

Dataflow modeling is a higher level of abstraction. The designer no need have any knowledge of logic circuit. He should be aware of data flow of the design. The gate level modeling becomes very complex for a VLSI circuit. Hence dataflow modeling became a very important way of implementing the design.
In dataflow modeling most of the design is implemented using continuous assignments, which are used to drive a value onto a net. The continuous assignments are made using the keyword *assign*.

The *assign* statement

The *assign* statement is used to make continuous assignment in the dataflow modeling. The *assign* statement usage is given below:

```
assign out = in0 + in1; // in0 + in1 is evaluated and then
assigned to out.
```

Note:

- The LHS of assign statement must always be a scalar or vector net or a concatenation. It cannot be a register.
- Continuous statements are always active statements.
- Registers or nets or function calls can come in the RHS of the assignment.
- The RHS expression is evaluated whenever one of its operands changes. Then the result is assigned to the LHS.
- Delays can be specified.

Examples:

```
assign out[3:0] = in0[3:0] & in1[3:0];

assign {o3, o2, o1, o0} = in0[3:0] | {in1[2:0],in2}; // Use of
concatenation.
```

Implicit Net Declaration:

```
wire in0, in1;
assign out = in0 ^ in1;
```

In the above example out is undeclared, but verilog makes an implicit net declaration for out.

Implicit Continuous Assignment:

```
wire out = in0 ^ in1;
```

The above line is the implicit continuous assignment. It is same as,

```
wire out;
assign out = in0 ^ in1;
```

Delays

There are three types of delays associated with dataflow modeling. They are: Normal/regular assignment delay, implicit continuous assignment delay and net declaration delay.

Normal/regular assignment delay:

```
assign #10 out = in0 | in1;
```

If there is any change in the operands in the RHS, then RHS expression will be evaluated after 10 units of time. Lets say that at time t, if there is change in one of the operands in the above example, then the expression is calculated at t+10 units of time. The value of RHS operands present at time t+10 is used to evaluate the expression.

Implicit continuous assignment delay:

```
wire #10 out = in0 ^ in1;
```

is same as

```
wire out;
assign 10 out = in0 ^ in1;
```

Net declaration delay:

```
wire #10 out;
assign out = in;
```

is same as

```
wire out;
assign #10 out = in;
```

Examples

1. Implementation of a 2x4 decoder.

```
module decoder_2x4 (out, in0, in1);

output out[0:3];
input in0, in1;

// Data flow modeling uses logic operators.
assign out[0:3] = { ~in0 & ~in1, in0 & ~in1,
                    ~in0 & in1, in0 & in1 };

endmodule
```

2. Implementation of a 4x1 multiplexer.

```
module mux_4x1 (out, in0, in1, in2, in3, s0, s1);

output out;
input in0, in1, in2, in3;
input s0, s1;

assign out = (~s0 & ~s1 & in0)|(s0 & ~s1 & in1)|
             (~s0 & s1 & in2)|(s0 & s1 & in0);

endmodule
```

3. Implementation of a 8x1 multiplexer using 4x1 multiplexers.

```
module mux_8x1 (out, in, sel);

output out;
input [7:0] in;
input [2:0] sel;

wire m1, m2;

// Instances of 4x1 multiplexers.
mux_4x1 mux_1 (m1, in[0], in[1], in[2],
               in[3], sel[0], sel[1]);
mux_4x1 mux_2 (m2, in[4], in[5], in[6],
               in[7], sel[0], sel[1]);

assign out = (~sel[2] & m1)|(sel[2] & m2);

endmodule
```

4. Implementation of a Full adder.

```
module full_adder (sum, c_out, in0, in1, c_in);

output sum, c_out;
input in0, in1, c_in;

assign { c_out, sum } = in0 + in1 + c_in;

endmodule
```

1 Comments

Labels: Verilog Tutorial

Gate-Level Modeling

>> **Introduction**
>> **Gate Primitives**
>> **Delays**
>> **Examples**

Introduction

In Verilog HDL a module can be defined using various levels of abstraction. There are four levels of abstraction in verilog. They are:

- Behavioral or algorithmic level: This is the highest level of abstraction. A module can be implemented in terms of the design algorithm. The designer no need to have any knowledge of hardware implementation.
- Data flow level: In this level the module is designed by specifying the data flow. Designer must how data flows between various registers of the design.
- Gate level: The module is implemented in terms of logic gates and interconnections between these gates. Designer should know the gate-level diagram of the design.
- Switch level: This is the lowest level of abstraction. The design is implemented using switches/transistors. Designer requires the knowledge of switch-level implementation details.

Gate-level modeling is virtually the lowest-level of abstraction, because the switch-level abstraction is rarely used. In general, gate-level modeling is used for implementing lowest level modules in a design like, full-adder, multiplexers, etc. Verilog HDL has gate primitives for all basic gates.

Gate Primitives

Gate primitives are predefined in Verilog, which are ready to use. They are instantiated like modules. There are two classes of gate primitives: Multiple input gate primitives and Single input gate primitives.
Multiple input gate primitives include and, nand, or, nor, xor, and xnor. These can have multiple

inputs and a single output. They are instantiated as follows:

```
// Two input AND gate.
and and_1 (out, in0, in1);

// Three input NAND gate.
nand nand_1 (out, in0, in1, in2);

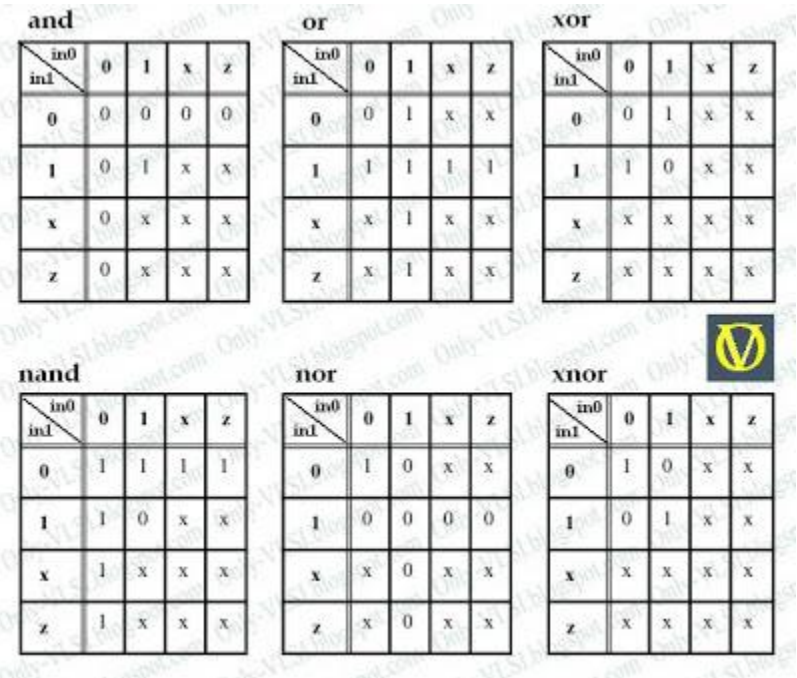
// Two input OR gate.
or or_1 (out, in0, in1);

// Four input NOR gate.
nor nor_1 (out, in0, in1, in2, in3);

// Five input XOR gate.
xor xor_1 (out, in0, in1, in2, in3, in4);

// Two input XNOR gate.
xnor and_1 (out, in0, in1);
```

Note that instance name is not mandatory for gate primitive instantiation. The truth tables of multiple input gate primitives are as follows:



and

in1 \ in0	0	1	x	z
0	0	0	0	0
1	0	1	x	x
x	0	x	x	x
z	0	x	x	x

or

in1 \ in0	0	1	x	z
0	0	1	x	x
1	1	1	1	1
x	x	1	x	x
z	x	1	x	x

xor

in1 \ in0	0	1	x	z
0	0	1	x	x
1	1	0	x	x
x	x	x	x	x
z	x	x	x	x

nand

in1 \ in0	0	1	x	z
0	1	1	1	1
1	1	0	x	x
x	1	x	x	x
z	1	x	x	x

nor

in1 \ in0	0	1	x	z
0	1	0	x	x
1	0	0	0	0
x	x	0	x	x
z	x	0	x	x

xnor

in1 \ in0	0	1	x	z
0	1	0	x	x
1	0	1	x	x
x	x	x	x	x
z	x	x	x	x

Single input gate primitives include not, buf, notif1, bufif1, notif0, and bufif0. These have a single input and one or more outputs. Gate primitives notif1, bufif1, notif0, and bufif0 have a control signal. The gates propagate if only control signal is asserted, else the output will be high impedance state (z). They are instantiated as follows:

```
// Inverting gate.
not not_1 (out, in);

// Two output buffer gate.
buf buf_1 (out0, out1, in);

// Single output Inverting gate with active-high control signal.
notif1 notif1_1 (out, in, ctrl);

// Double output buffer gate with active-high control signal.
bufif1 bufif1_1 (out0, out1, in, ctrl);

// Single output Inverting gate with active-low control signal.
notif0 notif0_1 (out, in, ctrl);

// Single output buffer gate with active-low control signal.
bufif0 bufif1_0 (out, in, ctrl);
```

The truth tables are as follows:

not

input	output
0	1
1	0
x	x
z	x

notif1

I \ C	0	1	x	z
0	z	1	H	H
1	z	0	L	L
x	z	x	x	x
z	z	x	x	x

notif0

I \ C	0	1	x	z
0	1	z	H	H
1	0	z	L	L
x	x	z	x	x
z	x	z	x	x

buf

input	output
0	0
1	1
x	x
z	x

bufif1

I \ C	0	1	x	z
0	z	0	L	L
1	z	1	H	H
x	z	x	x	x
z	z	x	x	x

bufif0

I \ C	0	1	x	z
0	0	z	L	L
1	1	z	H	H
x	x	z	x	x
z	x	z	x	x

Array of Instances:

```
wire [3:0] out, in0, in1;
and and_array[3:0] (out, in0, in1);
```

The above statement is equivalent to following bunch of statements:

```
and and_array0 (out[0], in0[0], in1[0]);
```

```
and and_array1 (out[1], in0[1], in1[1]);
and and_array2 (out[2], in0[2], in1[2]);
and and_array3 (out[3], in0[3], in1[3]);
```

>> Examples

Gate Delays:

In Verilog, a designer can specify the gate delays in a gate primitive instance. This helps the designer to get a real time behavior of the logic circuit.

Rise delay: It is equal to the time taken by a gate output transition to 1, from another value 0, x, or z.

Fall delay: It is equal to the time taken by a gate output transition to 0, from another value 1, x, or z.

Turn-off delay: It is equal to the time taken by a gate output transition to high impedance state, from another value 1, x, or z.

- If the gate output changes to x, the minimum of the three delays is considered.
- If only one delay is specified, it is used for all delays.
- If two values are specified, they are considered as rise, and fall delays.
- If three values are specified, they are considered as rise, fall, and turn-off delays.
- The default value of all delays is zero.

```
and #(5) and_1 (out, in0, in1);
// All delay values are 5 time units.

nand #(3,4,5) nand_1 (out, in0, in1);
// rise delay = 3, fall delay = 4, and turn-off delay = 5.

or #(3,4) or_1 (out, in0, in1);
// rise delay = 3, fall delay = 4, and turn-off delay = min(3,4)
= 3.
```

There is another way of specifying delay times in verilog, Min:Typ:Max values for each delay. This helps designer to have a much better real time experience of design simulation, as in real time logic circuits the delays are not constant. The user can choose one of the delay values using +maxdelays, +typdelays, and +mindelays at run time. The typical value is the default value.

and #(4:5:6) and_1 (out, in0, in1);
// For all delay values: Min=4, Typ=5, Max=6.

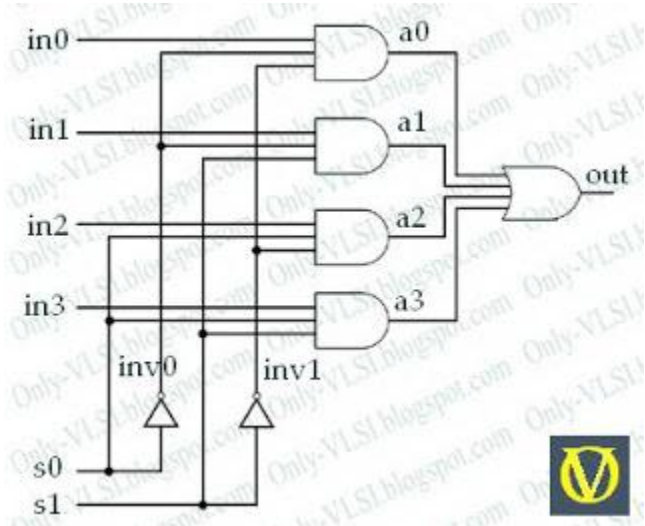
```
nand #(3:4:5,4:5:6,5:6:7) nand_1 (out, in0, in1);
// rise delay: Min=3, Typ=4, Max=5, fall delay: Min=4, Typ=5,
Max=6, turn-off delay: Min=5, Typ=6, Max=7.
```

In the above example, if the designer chooses typical values, then rise delay = 4, fall delay = 5, turn-off delay = 6.

Examples:

1. Gate level modeling of a 4x1 multiplexer.

The gate-level circuit diagram of 4x1 mux is shown below. It is used to write a module for 4x1 mux.




```
module 4x1_mux (out, in0, in1, in2, in3, s0, s1);

// port declarations
output out; // Output port.
input in0, in1, in2. in3; // Input ports.
input s0, s1; // Input ports: select lines.

// intermediate wires
wire inv0, inv1; // Inverter outputs.
wire a0, a1, a2, a3; // AND gates outputs.

// Inverters.
not not_0 (inv0, s0);
not not_1 (inv1, s1);

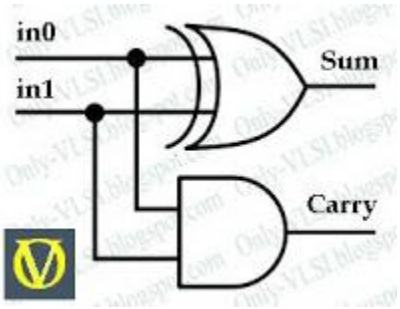
// 3-input AND gates.
and and_0 (a0, in0, inv0, inv1);
and and_1 (a1, in1, inv0, s1);
and and_2 (a2, in2, s0, inv1);
and and_3 (a3, in3, s0, s1);

// 4-input OR gate.
or or_0 (out, a0, a1, a2, a3);

endmodule
```

2. Implementation of a full adder using half adders.

Half adder:




```
module half_adder (sum, carry, in0, in1);

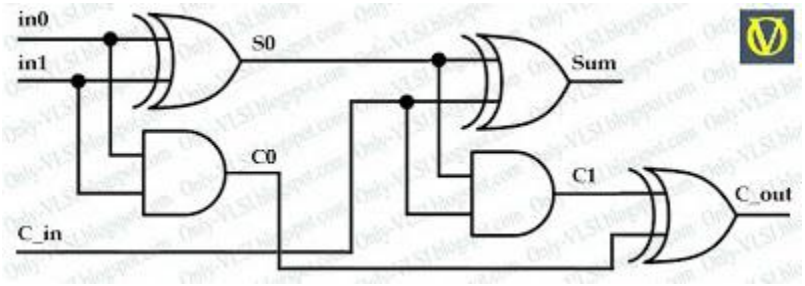
output sum, carry;
input in0, in1;

// 2-input XOR gate.
xor xor_1 (sum, in0, in1);

// 2-input AND gate.
and and_1 (carry, in0, in1);

endmodule
```

Full adder:




```
module full_adder (sum, c_out, ino, in1, c_in);

output sum, c_out;
input in0, in1, c_in;

wire s0, c0, c1;

// Half adder : port connecting by order.
half_adder ha_0 (s0, c0, in0, in1);

// Half adder : port connecting by name.
half_adder ha_1 (.sum(sum),
                 .in0(s0),
                 .in1(c_in),
                 .carry(c1));
```

```
// 2-input XOR gate, to get c_out.
xor xor_1 (c_out, c0, c1);

endmodule
```

<< Previous **Home** **Next >>**

0 Comments

Labels: Verilog Tutorial

Scheduling

The Verilog HDL is defined in terms of a discrete event execution model. A design consists of connected processes. Processes are objects that can be evaluated, that may have state, and that can respond to changes on their inputs to produce outputs. Processes include primitives, modules, initial and always procedural blocks, continuous assignments, asynchronous tasks, and procedural assignment statements.

The following definitions helps in better understanding of scheduling and execution of events:

- *Update event*: Every change in value of a net or variable in the circuit being simulated, as well as the named event, is considered as an update event.
- *Evaluation event*: Processes are sensitive to update events. When an update event is executed, all the processes that are sensitive to that event are evaluated in an arbitrary order. The evaluation of a process is also an event, known as an evaluation event.
- *Simulation time:* It is used to refer to the time value maintained by the simulator to model the actual time it would take for the circuit being simulated.

Events can occur at different times. In order to keep track of the events and to make sure they are processed in the correct order, the events are kept on an event queue, ordered by simulation time. Putting an event on the queue is called scheduling an event.

Scheduling events:

The Verilog event queue is logically segmented into five different regions. Each event will be added to one of the five regions in the queue but are only removed from the active region.

1. *Active events*: Events that occur at the current simulation time and can be processed in any order.
2. *Inactive events*: Events that occur at the current simulation time, but that shall be processed after all the active events are processed.

3. *Nonblocking assign update events*: Events that have been evaluated during some previous simulation time, but that shall be assigned at this simulation time after all the active and inactive events are processed.
4. *Monitor events*: Events that shall be processed after all the active, inactive, and nonblocking assign update events are processed.
5. *Future events*: Events that occur at some future simulation time. Future events are divided into future inactive events, and future nonblocking assignment update events.

The processing of all the active events is called a simulation cycle.

0 Comments

Labels: Verilog Tutorial

List of Operators

>> **Logical Operators**
>> **Relational Operators**
>> **Equality Operators**
>> **Arithmetic Operators**
>> **Bitwise Operators**
>> **Reduction Operators**
>> **Shift Operators**
>> **Conditional Operators**
>> **Replication Operators**
>> **Concatenation Operators**
>> **Operator Precedence**

Logical Operators

Symbol	Description	#Operators
!	Logical negation	One
||	Logical OR	Two
&&	Logical AND	Two

Relational Operators

Symbol	Description	#Operators
>	Greater than	Two
<	Less than	Two
>=	Greater than or equal to	Two

<=	Less than or equal to	Two

Equality Operators

Symbol	Description	#Operators
==	Equality	Two
!=	Inequality	Two
===	Case equality	Two
!==	Case inequality	Two

Arithmetic Operators

Symbol	Description	#Operators
+	Add	Two
-	Substract	Two
*	Multiply	Two
/	Divide	Two
**	Power	Two
%	Modulus	Two

Bitwise Operators

Symbol	Description	#Operators
~	Bitwise negation	One
&	Bitwise AND	Two
|	Bitwise OR	Two
^	Bitwise XOR	Two
^~ or ~^	Bitwise XNOR	Two

Reduction Operators

Symbol	Description	#Operators
&	Reduction AND	One
~&	Reduction NAND	One
|	Reduction OR	One
~|	Reduction NOR	One

^	Reduction XOR	One
^~ or ~^	Reduction XNOR	One

Shift Operators

Symbol	Description	#Operators
>>	Right shift	Two
<<	Left shift	Two
>>>	Arithmetic right shift	Two
<<<	Arithmetic left shift	Two

Conditional Operators

Symbol	Description	#Operators
?:	Conditional	Two

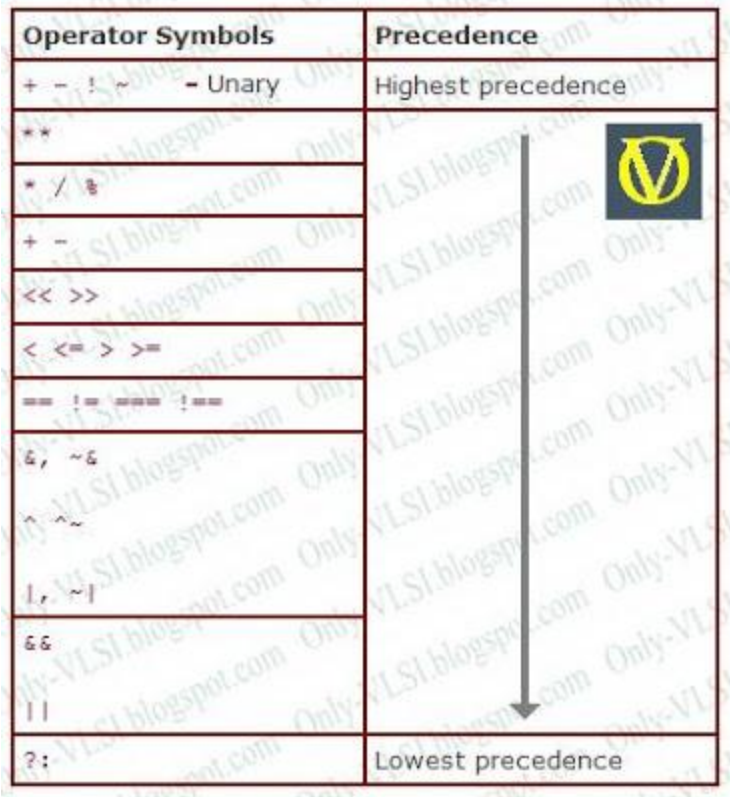
Replication Operators

Symbol	Description	#Operators
{ { } }	Replication	> One

Concatenation Operators

Symbol	Description	#Operators
{ }	Concatenation	> One

Operator Precedence



Operator Symbols	Precedence
+ - ! ~ - Unary	Highest precedence
**	
* / %	
+ -	
<< >>	
< <= > >=	
== != === !==	
&, ~& ^ ^~ |, ~|	
&& ||	
?:	Lowest precedence

<< Previous **Home** **Next >>**

0 Comments

Labels: Verilog Tutorial

Basics: Data Types

>> **Value Set**
>> **Nets**
>> **Registers**
>> **Integers**
>> **Real Numbers**
>> **Parameters**
>> **Vectors**
>> **Arrays**
>> **Strings**
>> **Time Data Type**

Value Set

The Verilog HDL value set consists of four basic values:

- 0 - represents a logic zero, or a false condition.
- 1 - represents a logic one, or a true condition.
- x - represents an unknown logic value.
- z - represents a high-impedance state.

The values 0 and 1 are logical complements of one another. Almost all of the data types in the Verilog HDL store all four basic values.

Nets

Nets are used to make connections between hardware elements. Nets simply reflect the value at one end(head) to the other end(tail). It means the value they carry is continuously driven by the output of a hardware element to which they are connected to. Nets are generally declared using the keyword *wire*. The default value of net (wire) is z. If a net has no driver, then its value is **z**.

Registers

Registers are data storage elements. They hold the value until they are replaced by some other value. Register doesn't need a driver, they can be changed at anytime in a simulation. Registers are generally declared with the keyword *reg*. Its default value is x. Register data types should not be confused with hardware registers, these are simply variables.

Integers

Integer is a register data type of 32 bits. The only difference of declaring it as integer is that, it becomes a signed value. When you declare it as a 32 bit register (array) it is an unsigned value. It is declared using the keyword *integer*.

Real Numbers

Real number can be declared using the keyword *real*. They can be assigned values as follows:
real r_1;

```
r_1 = 1.234; // Decimal notation.
r_1 = 3e4; // Scientific notation.
```

Parameters

Parameters are the constants that can be declared using the keyword *parameter*. Parameters are in general used for customization of a design. Parameters are declared as follows:

```
parameter p_1 = 123; // p_1 is a constant with value 123.
```

Keyword *defparam* can be used to change a parameter value at module instantiation. Keyword *localparam* is usedd to declare local parameters, this is used when their value should not be changed.

Vectors

Vectors can be a net or reg data types. They are declared as [high:low] or [low:high], but the left number is always the MSB of the vector.

```
wire [7:0] v_1; // v_1[7] is the MSB.
reg [0:15] v_2; // v_2[15] is the MSB.
```

In the above examples: If it is written as v_1[5:2], it is the part of the entire vector which contains 4 bits in order: v_1[5], v_1[4], v_1[3], v_1[2]. Similarly v_2[0:7], means the first half part of the vecotr v_2.
Vector parts can also be specified in a different way:
vector_name[start_bit+:width] : part-select increments from start_bit. In above example: v_2[0:7] is same as v_2[0+:8]. vector_name[start_bit-:width] : part-select decrements from start_bit. In above example: v_1[5:2] is same as v_1[5-:4].

Arrays

Arrays of reg, integer, real, time, and vectors are allowed. Arrays are declared as follows:

```
reg a_1[0:7];
real a_3[15:0];
wire [0:3] a_4[7:0]; // Array of vector
integer a_5[0:3][6:0]; // Double dimensional array
```

Strings

Strings are register data types. For storing a character, we need a 8-bit register data type. So if you want to create string variable of length *n*. The string should be declared as register data type of length *n*8*.

```
reg [8*8-1:0] string_1; // string_1 is a string of length 8.
```

Time Data Type

Time data type is declared using the keyword *time*. These are generally used to store simulation time. In general it is 64-bit long.

```
time t_1;
t_1 = $time; // assigns current simulation time to t_1.
```

There are some other data types, but are considered to be advanced data types, hence they are not

discussed here.

<< Previous **Home** **Next >>**

0 Comments

Labels: Verilog Tutorial

Ports

Modules communicate with external world using ports. They provide interface to the modules. A module definition contains list of ports. All ports in the list of ports must be declared in the module, ports can be one the following types:

- Input port, declared using keyword **input**.
- Output port, declared using keyword **output**.
- Bidirectional port, declared using keyword **inout**.

All the ports declared are considered to be as wire by default. If a port is intended to be a wire, it is sufficient to declare it as *output*, *input*, or *inout*. If output port holds its value it should be declared as *reg* type. Ports of type *input* and *inout* cannot be declared as *reg* because *reg* variables hold values and input ports should not hold values but simply reflect the changes in the external signals they are connected to.

Port Connection Rules

- Inputs: Always of type *net*(*wire*). Externally, they can be connected to *reg* or *net* type variable.
- Outputs: Can be of *reg* or *net* type. Externally, they must be connected to a *net* type variable.
- Bidirectional ports (*inout*): Always of type *net*. Externally, they must be connected to a *net* type variable.

Note:

- It is possible to connect internal and external ports of different size. In general you will receive a warning message for width mismatch.
- There can be unconnected ports in module instances.

Ports can declared in a module in C-language style:

```
module module_1( input a, input b, output c);
--
```

// Internals
--
endmodule

If there is an instance of above module, in some other module. Port connections can be made in two types.

Connection by Ordered List:

```
module_1 instance_name_1 ( A, B, C);
```

Connecting ports by name:

```
module_1 instance_name_2 (.a(A), .c(C), .b(B));
```

In connecting port by name, order is ignored.

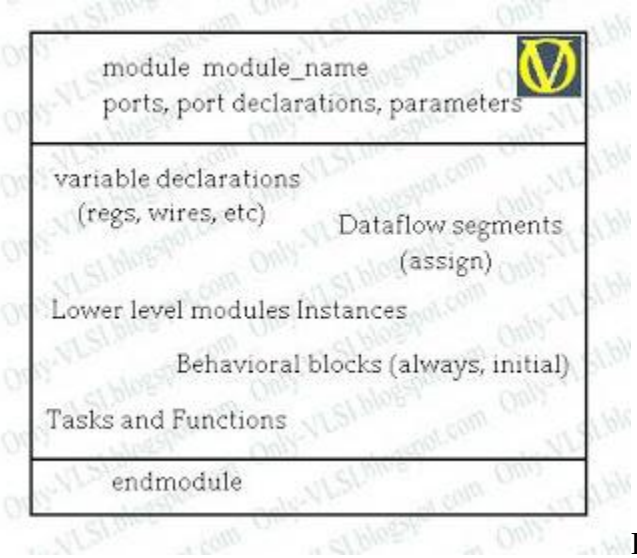
<< Previous **Home** **Next >>**

0 Comments

Labels: Verilog Tutorial

Modules

A module is the basic building block in Verilog HDL. In general many elements are grouped to form a module, to provide a common functionality, which can be used at many places in the design. Port interface (using input and output ports) helps in providing the necessary functionality to the higher-level blocks. Thus any design modifications at lower level can be easily implemented without affecting the entire design code. The structure of a module is show in the figure below.



Keyword ***module*** is used to begin a module

and it ends with the keyword ***endmodule***. The syntax is as follows:
module *module_name*

// internals
---
endmodule

Example: D Flip-flop implementation (Try to understand the module structure, ignore unknown constraints/statements).

```
module D_FlipFlop(q, d, clk, reset);

// Port declarations
output q;
reg q;
input d, clk, reset;

// Internal statements - Logic
always @(posedge reset or poseedge clk)
if (reset)
q < = 1'b0;
else
q < = d;

// endmodule statement
endmodule
```

Note:

- Multiple modules can be defined in a single design file with any order.
- See that the endmodule statement should not written as endmodule; (no ; is used).
- All components except module, module name, and endmodule are optional.
- The 5 internal components can come in any order.

<< Previous **Home** **Next >>**

0 Comments

Labels: Verilog Tutorial

Basics: Lexical Tokens

>> **Operators**
>> **Comments**
>> **Whitespace**
>> **Strings**
>> **Identifiers**
>> **Keywords**
>> **Number Specification**

Operators

There are three types of operators: unary, binary, and ternary, which have one, two, and three operands respectively.

Unary : Single operand, which precede the operand.
Ex: x = ~y
~ is a unary operator
y is the operand

binary : Comes between two operands.
Ex: x = y || z
|| is a binary operator
y and z are the operands

ternary : Ternary operators have two separate operators that separate three operands.
Ex: p = x ? y : z
? : is a ternary operator
x, y, and z are the operands

List of operators is given here.

Comments

Verilog HDL also have two types of commenting, similar to that of C programming language. // is used for single line commenting and '/*' and '*/' are used for commenting multiple lines which start with /* and end with */.
EX: // single line comment
/* Multiple line
commenting */
/* This is a // LEGAL comment */
/* This is an /* ILLEGAL */ comment */

Whitespace

- - \b - backspace
- - \t - tab space

- \n - new line

In verilog Whitespace is ignored except when it separates tokens. Whitespace is not ignored in strings. Whitesapces are generally used in writing test benches.

Strings

A string in verilog is same as that of C programming language. It is a sequence of characters enclosed in double quotes. String are treated as sequence of one byte ASCII values, hence they can be of one line only, they cannot be of multiple lines.
Ex: " This is a string "
" This is not treated as
string in verilog HDL "

Identifiers

Identifiers are user-defined words for variables, function names, module names, block names and instance names.Identifiers begin with a letter or underscore and can include any number of letters, digits and underscores. It is not legal to start identifiers with number or the dollar($) symbol in Verilog HDL. Identifiers in Verilog are case-sensitive.

Keywords

Keywords are special words reserved to define the language constructs. In verilog all keywords are in lowercase only. A list of all keywords in Verilog is given below:

always	event	output	strong1
and	for	parameter	supply0
assign	force	pmos	supply1
attribute	forever	posedge	table
begin	fork	primitive	task
buf	function	pull0	time
bufif0	highz0	pull1	tran
bufif1	highz1	pulldown	tranif0
case	if	pullup	tranif1
casex	ifnone	rcmos	tri
casez	initial	real	tri0
cmos	inout	realtime	tri1
deassign	input	reg	triand
default	integer	release	trior
defparam	join	repeat	trireg
disable	medium	rnmos	unsigned
edge	module	rpmos	vectored
else	large	rtran	wait
end	macromodule	rtranif0	wand
endattribute	nand	rtranif1	weak0

endcase	negedge	scalared	weak1
endfunction	nmos	signed	while
endmodule	nor	small	wire
endprimitive	not	specify	wor
endspecify	notif0	specparam	xnor
endtable	notif1	strength	xor
endtask	or	strong0	

Verilog keywords also includes compiler directives, system tasks, and functions. Most of the keywords will be explained in the later sections.

Number Specification

Sized Number Specification

Representation: *[size]'[base][number]*

- [size] is written only in decimal and specifies the number of bits.
- [base] could be 'd' or 'D' for decimal, 'h' or 'H' for hexadecimal, 'b' or 'B' for binary, and 'o' or 'O' for octal.
- [number] The number is specified as consecutive digits. Uppercase letters are legal for number specification (in case of hexadecimal numbers).

Ex: 4'b1111 : 4-bit binary number
16'h1A2F : 16-bit hexadecimal number
32'd1 : 32-bit decimal number
8'o3 : 8-bit octal number

Unsized Number Specification

By default numbers that are specified without a [base] specification are decimal numbers. Numbers that are written without a [size] specification have a default number of bits that is simulator and/or machine specific (generally 32).

Ex: 123 : This is a decimal number
'hc3 : This is a hexadecimal number
Number of bits depends on simulator/machine, generally 32.

x or z values

x - Unknown value.
z - High impedance value
An x or z sets four bits for a number in the hexadecimal base, three bits for a number in the octal base, and one bit for a number in the binary base.

Note: If the most significant bit of a number is 0, x, or z, the number is automatically extended to

fill the most significant bits, respectively, with 0, x, or z. This makes it easy to assign x or z to whole vector. If the most significant digit is `1`, then it is also zero extended.

Negative Numbers

Representation: *-[size]'[base][number]*

Ex: -8'd9 : 8-bit negative number stored as 2's complement of 8
-8'sd3 : Used for performing signed integer math
4'd-2 : Illegal

Underscore(_) and question(?) mark

An underscore, "**_**" is allowed to use anywhere in a number except in the beginning. It is used only to improve readability of numbers and are ignored by Verilog. A question mark "**?**" is the alternative for z w.r.t. numbers
Ex: 8'b1100_1101 : Underscore improves readability
4'b1??1 : same as 4'b1zz1

<< Previous **Home** **Next >>**

0 Comments

Labels: Verilog Tutorial

Basics: Number Specification

Sized Number Specification

Representation: *[size]'[base][number]*

- [size] is written only in decimal and specifies the number of bits.
- [base] could be 'd' or 'D' for decimal, 'h' or 'H' for hexadecimal, 'b' or 'B' for binary, and 'o' or 'O' for octal.
- [number] The number is specified as consecutive digits. Uppercase letters are legal for number specification (in case of hexadecimal numbers).

Ex: 4'b1111 : 4-bit binary number
16'h1A2F : 16-bit hexadecimal number
32'd1 : 32-bit decimal number
8'o3 : 8-bit octal number

Unsized Number Specification

By default numbers that are specified without a [base] specification are decimal numbers. Numbers that are written without a [size] specification have a default number of bits that is simulator and/or machine specific (generally 32).

Ex: 123 : This is a decimal number
'hc3 : This is a hexadecimal number
Number of bits depends on simulator/machine, generally 32.

x or z values

x - Unknown value.
z - High impedance value
An x or z sets four bits for a number in the hexadecimal base, three bits for a number in the octal base, and one bit for a number in the binary base.

Note: If the most significant bit of a number is 0, x, or z, the number is automatically extended to fill the most significant bits, respectively, with 0, x, or z. This makes it easy to assign x or z to whole vector. If the most significant digit is `1`, then it is also zero extended.

Negative Numbers

Representation: *-[size]'[base][number]*

Ex: -8'd9 : 8-bit negative number stored as 2's complement of 8
-8'sd3 : Used for performing signed integer math
4'd-2 : Illegal

Underscore(_) and question(?) mark

An underscore, "**_**" is allowed to use anywhere in a number except in the beginning. It is used only to improve readability of numbers and are ignored by Verilog. A question mark "**?**" is the alternative for z w.r.t. numbers
Ex: 8'b1100_1101 : Underscore improves readability
4'b1??1 : same as 4'b1zz1

0 Comments

Labels: Verilog Tutorial

Introduction to Verilog HDL

>> **Introduction**
>> **The VLSI Design Flow**
>> **Importance of HDLs**
>> **Verilog HDL**

>> **Why Verilog ?**
>> **Digital Design Methods**

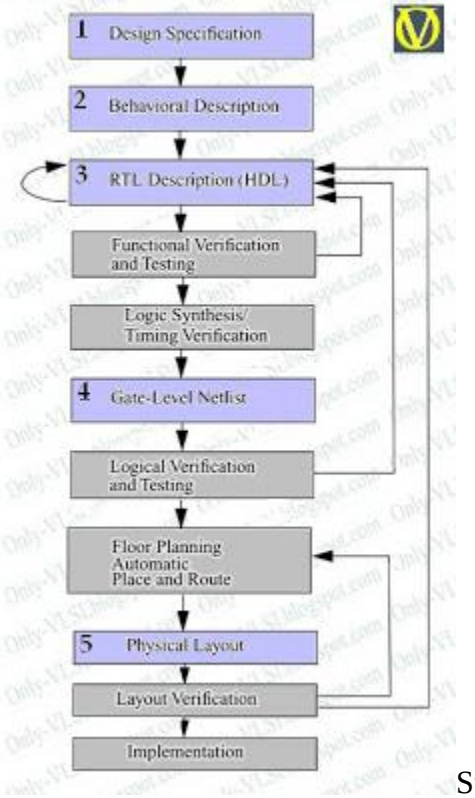
Introduction

With the advent of VLSI technology and increased usage of digital circuits, designers has to design single chips with millions of transistors. It became almost impossible to verify these circuits of high complexity on breadboard. Hence Computer-aided techniques became critical for verification and design of VLSI digital circuits.As designs got larger and more complex, logic simulation assumed an important role in the design process. Designers could iron out functional bugs in the architecture before the chip was designed further. All these factors which led to the evolution of Computer-Aided Digital Design, intern led to the emergence of Hardware Description Languages.

Verilog HDL and VHDL are the popular HDLs.Today, Verilog HDL is an accepted IEEE standard. In 1995, the original standard IEEE 1364-1995 was approved. IEEE 1364-2001 is the latest Verilog HDL standard that made significant improvements to the original standard.

The VLSI Design Flow

The VLSI IC circuits design flow is shown in the figure below. The various level of design are numbered and the gray coloured blocks show processes in the design flow.



Specifications comes first, they describe abstractly the functionality, interface, and the architecture of the digital IC circuit to be designed.

- Behavioral description is then created to analyze the design in terms of functionality, performance, compliance to given standards, and other specifications.
- RTL description is done using HDLs. This RTL description is simulated to test functionality. From here onwards we need the help of EDA tools.
- RTL description is then converted to a gate-level net list using logic synthesis tools. A gate-level netlist is a description of the circuit in terms of gates and connections between them, which are made in such a way that they meet the timing, power and area specifications.
- Finally a physical layout is made, which will be verified and then sent to fabrication.

Importance of HDLs

- RTL descriptions, independent of specific fabrication technology can be made an verified.
- functional verification of the design can be done early in the design cycle.
- Better representation of design due to simplicity of HDLs when compared to gate-level schematics.
- Modification and optimization of the design became easy with HDLs.
- Cuts down design cycle time significantly because the chance of a functional bug at a later stage in the design-flow is minimal.

Verilog HDL

Verilog HDL is one of the most used HDLs. It can be used to describe designs at four levels of abstraction:

1. Algorithmic level.
2. Register transfer level (RTL).
3. Gate level.
4. Switch level (the switches are MOS transistors inside gates).

Why Verilog ?

- Easy to learn and easy to use, due to its similarity in syntax to that of the C programming language.
- Different levels of abstraction can be mixed in the same design.
- Availability of Verilog HDL libraries for post-logic synthesis simulation.
- Most of the synthesis tools support Verilog HDL.
- The *Programming Language Interface (PLI)* is a powerful feature that allows the user to write custom C code to interact with the internal data structures of Verilog. Designers can customize a Verilog HDL simulator to their needs with the PLI.

Digital design methods

Digital design methods are of two types:

1. ***Top-down* design method** : In this design method we first define the top-level block and then we build necessary sub-blocks, which are required to build the top-level block. Then the sub-blocks are divided further into smaller-blocks, and so on. The bottom level blocks are called as leaf cells. By saying bottom level it means that the leaf cell cannot be divided further.
2. ***Bottom-up* design method** : In this design method we first find the bottom leaf cells, and then start building upper sub-blocks and building so on, we reach the top-level block of the design.

In general a combination of both types is used. These types of design methods helps the design architects, logics designers, and circuit designers. Design architects gives specifications to the logic designers, who follow one of the design methods or both. They identify the leaf cells. Circuit designers design those leaf cells, and they try to optimize leaf cells in terms of power, area, and speed. Hence all the design goes parallel and helps finishing the job faster.

Basics: Lexical Tokens

>> **Operators**
>> **Comments**
>> **Whitespace**
>> **Strings**
>> **Identifiers**
>> **Keywords**
>> **Number Specification**

Operators

There are three types of operators: unary, binary, and ternary, which have one, two, and three operands respectively.

Unary : Single operand, which precede the operand.
Ex: x = ~y
~ is a unary operator
y is the operand

binary : Comes between two operands.
Ex: x = y || z
|| is a binary operator
y and z are the operands

ternary : Ternary operators have two separate operators that separate three operands.
Ex: p = x ? y : z
? : is a ternary operator
x, y, and z are the operands

List of operators is given here.

Comments

Verilog HDL also have two types of commenting, similar to that of C programming language. // is used for single line commenting and '/*' and '*/' are used for commenting multiple lines which start with /* and end with */.
EX: // single line comment
/* Multiple line
commenting */
/* This is a // LEGAL comment */
/* This is an /* ILLEGAL */ comment */

Whitespace

- - \b - backspace
- - \t - tab space
- - \n - new line

In verilog Whitespace is ignored except when it separates tokens. Whitespace is not ignored in strings. Whitesapces are generally used in writing test benches.

Strings

A string in verilog is same as that of C programming language. It is a sequence of characters enclosed in double quotes. String are treated as sequence of one byte ASCII values, hence they can be of one line only, they cannot be of multiple lines.
Ex: " This is a string "
" This is not treated as
string in verilog HDL "

Identifiers

Identifiers are user-defined words for variables, function names, module names, block names and instance names.Identifiers begin with a letter or underscore and can include any number of letters, digits and underscores. It is not legal to start identifiers with number or the dollar($) symbol in Verilog HDL. Identifiers in Verilog are case-sensitive.

Keywords

Keywords are special words reserved to define the language constructs. In verilog all keywords are in lowercase only. A list of all keywords in Verilog is given below:

always	event	output	strong1
and	for	parameter	supply0
assign	force	pmos	supply1
attribute	forever	posedge	table
begin	fork	primitive	task
buf	function	pull0	time
bufif0	highz0	pull1	tran
bufif1	highz1	pulldown	tranif0
case	if	pullup	tranif1
casex	ifnone	rcmos	tri
casez	initial	real	tri0
cmos	inout	realtime	tri1
deassign	input	reg	triand
default	integer	release	trior
defparam	join	repeat	trireg
disable	medium	rnmos	unsigned
edge	module	rpmos	vectored
else	large	rtran	wait
end	macromodule	rtranif0	wand
endattribute	nand	rtranif1	weak0
endcase	negedge	scalared	weak1
endfunction	nmos	signed	while
endmodule	nor	small	wire
endprimitive	not	specify	wor
endspecify	notif0	specparam	xnor
endtable	notif1	strength	xor
endtask	or	strong0	

Verilog keywords also includes compiler directives, system tasks, and functions. Most of the keywords will be explained in the later sections.

Number Specification

Sized Number Specification

Representation: *[size]'[base][number]*

- [size] is written only in decimal and specifies the number of bits.
- [base] could be 'd' or 'D' for decimal, 'h' or 'H' for hexadecimal, 'b' or 'B' for binary, and 'o' or 'O' for octal.
- [number] The number is specified as consecutive digits. Uppercase letters are legal for number specification (in case of hexadecimal numbers).

Ex: 4'b1111 : 4-bit binary number
16'h1A2F : 16-bit hexadecimal number
32'd1 : 32-bit decimal number
8'o3 : 8-bit octal number

Unsized Number Specification

By default numbers that are specified without a [base] specification are decimal numbers. Numbers that are written without a [size] specification have a default number of bits that is simulator and/or machine specific (generally 32).

Ex: 123 : This is a decimal number
'hc3 : This is a hexadecimal number
Number of bits depends on simulator/machine, generally 32.

x or z values

x - Unknown value.
z - High impedance value
An x or z sets four bits for a number in the hexadecimal base, three bits for a number in the octal base, and one bit for a number in the binary base.

Note: If the most significant bit of a number is 0, x, or z, the number is automatically extended to fill the most significant bits, respectively, with 0, x, or z. This makes it easy to assign x or z to whole vector. If the most significant digit is `1`, then it is also zero extended.

Negative Numbers

Representation: *-[size]'[base][number]*

Ex: -8'd9 : 8-bit negative number stored as 2's complement of 8
-8'sd3 : Used for performing signed integer math
4'd-2 : Illegal

Underscore(_) and question(?) mark

An underscore, "**_**" is allowed to use anywhere in a number except in the beginning. It is used only to improve readability of numbers and are ignored by Verilog. A question mark "**?**" is the alternative for z w.r.t. numbers
Ex: 8'b1100_1101 : Underscore improves readability
4'b1??1 : same as 4'b1zz1

Basics: Data Types

>> **Value Set**
>> **Nets**

>> **Registers**
>> **Integers**
>> **Real Numbers**
>> **Parameters**
>> **Vectors**
>> **Arrays**
>> **Strings**
>> **Time Data Type**

Value Set

The Verilog HDL value set consists of four basic values:

- 0 - represents a logic zero, or a false condition.
- 1 - represents a logic one, or a true condition.
- x - represents an unknown logic value.
- z - represents a high-impedance state.

The values 0 and 1 are logical complements of one another. Almost all of the data types in the Verilog HDL store all four basic values.

Nets

Nets are used to make connections between hardware elements. Nets simply reflect the value at one end(head) to the other end(tail). It means the value they carry is continuously driven by the output of a hardware element to which they are connected to. Nets are generally declared using the keyword *wire*. The default value of net (wire) is z. If a net has no driver, then its value is **z**.

Registers

Registers are data storage elements. They hold the value until they are replaced by some other value. Register doesn't need a driver, they can be changed at anytime in a simulation. Registers are generally declared with the keyword *reg*. Its default value is x. Register data types should not be confused with hardware registers, these are simply variables.

Integers

Integer is a register data type of 32 bits. The only difference of declaring it as integer is that, it becomes a signed value. When you declare it as a 32 bit register (array) it is an unsigned value. It is declared using the keyword *integer*.

Real Numbers

Real number can be declared using the keyword *real*. They can be assigned values as follows:
real r_1;

```
r_1 = 1.234; // Decimal notation.
r_1 = 3e4; // Scientific notation.
```

Parameters

Parameters are the constants that can be declared using the keyword *parameter*. Parameters are in general used for customization of a design. Parameters are declared as follows:

```
parameter p_1 = 123; // p_1 is a constant with value 123.
```

Keyword *defparam* can be used to change a parameter value at module instantiation. Keyword *localparam* is usedd to declare local parameters, this is used when their value should not be changed.

Vectors

Vectors can be a net or reg data types. They are declared as [high:low] or [low:high], but the left number is always the MSB of the vector.

```
wire [7:0] v_1; // v_1[7] is the MSB.
reg [0:15] v_2; // v_2[15] is the MSB.
```

In the above examples: If it is written as v_1[5:2], it is the part of the entire vector which contains 4 bits in order: v_1[5], v_1[4], v_1[3], v_1[2]. Similarly v_2[0:7], means the first half part of the vecotr v_2.
Vector parts can also be specified in a different way:
vector_name[start_bit+:width] : part-select increments from start_bit. In above example: v_2[0:7] is same as v_2[0+:8]. vector_name[start_bit-:width] : part-select decrements from start_bit. In above example: v_1[5:2] is same as v_1[5-:4].

Arrays

Arrays of reg, integer, real, time, and vectors are allowed. Arrays are declared as follows:

```
reg a_1[0:7];
real a_3[15:0];
wire [0:3] a_4[7:0]; // Array of vector
integer a_5[0:3][6:0]; // Double dimensional array
```

Strings

Strings are register data types. For storing a character, we need a 8-bit register data type. So if you want to create string variable of length *n*. The string should be declared as register data type of length *n*8*.

```
reg [8*8-1:0] string_1; // string_1 is a string of length 8.
```

Time Data Type

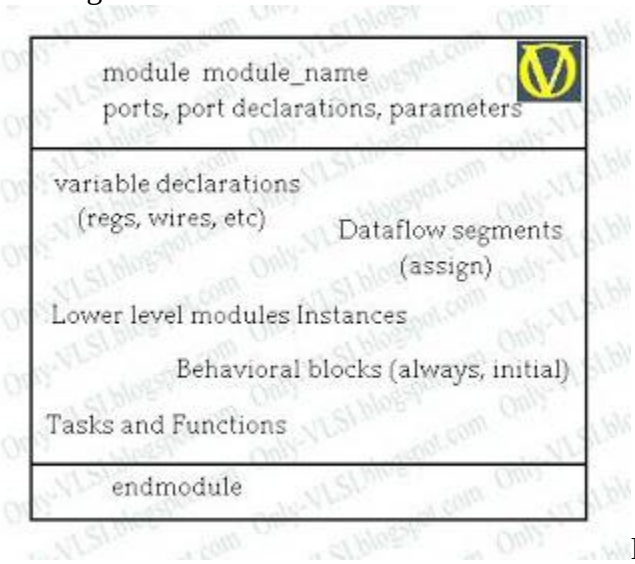
Time data type is declared using the keyword *time*. These are generally used to store simulation time. In general it is 64-bit long.

```
time t_1;
t_1 = $time; // assigns current simulation time to t_1.
```

There are some other data types, but are considered to be advanced data types, hence they are not discussed here.

Modules

A module is the basic building block in Verilog HDL. In general many elements are grouped to form a module, to provide a common functionality, which can be used at many places in the design. Port interface (using input and output ports) helps in providing the necessary functionality to the higher-level blocks. Thus any design modifications at lower level can be easily implemented without affecting the entire design code. The structure of a module is show in the figure below.



Keyword ***module*** is used to begin a module and it ends with the keyword ***endmodule***. The syntax is as follows:

module *module_name*

// internals
---
endmodule

Example: D Flip-flop implementation (Try to understand the module structure, ignore unknown constraints/statements).

```
module D_FlipFlop(q, d, clk, reset);
```

```
// Port declarations
output q;
reg q;
input d, clk, reset;

// Internal statements - Logic
always @(posedge reset or poseedge clk)
if (reset)
q < = 1'b0;
else
q < = d;

// endmodule statement
endmodule
```

Note:

- Multiple modules can be defined in a single design file with any order.
- See that the endmodule statement should not written as endmodule; (no ; is used).
- All components except module, module name, and endmodule are optional.
- The 5 internal components can come in any order.

Ports

Modules communicate with external world using ports. They provide interface to the modules. A module definition contains list of ports. All ports in the list of ports must be declared in the module, ports can be one the following types:

- Input port, declared using keyword **input**.
- Output port, declared using keyword **output**.
- Bidirectional port, declared using keyword **inout**.

All the ports declared are considered to be as wire by default. If a port is intended to be a wire, it is sufficient to declare it as *output*, *input*, or *inout*. If output port holds its value it should be declared as *reg* type. Ports of type *input* and *inout* cannot be declared as *reg* because *reg* variables hold values and input ports should not hold values but simply reflect the changes in the external signals they are connected to.

Port Connection Rules

- Inputs: Always of type *net*(*wire*). Externally, they can be connected to *reg* or *net* type variable.
- Outputs: Can be of *reg* or *net* type. Externally, they must be connected to a *net* type variable.

- Bidirectional ports (*inout*): Always of type *net*. Externally, they must be connected to a *net* type variable.

Note:

- It is possible to connect internal and external ports of different size. In general you will receive a warning message for width mismatch.
- There can be unconnected ports in module instances.

Ports can declared in a module in C-language style:

module module_1(input a, input b, output c);
--
// Internals
--
endmodule

If there is an instance of above module, in some other module. Port connections can be made in two types.

Connection by Ordered List:

```
module_1 instance_name_1 ( A, B, C);
```

Connecting ports by name:

```
module_1 instance_name_2 (.a(A), .c(C), .b(B));
```

In connecting port by name, order is ignored.

List of Operators

>> **Logical Operators**
>> **Relational Operators**
>> **Equality Operators**
>> **Arithmetic Operators**
>> **Bitwise Operators**
>> **Reduction Operators**
>> **Shift Operators**
>> **Conditional Operators**
>> **Replication Operators**
>> **Concatenation Operators**
>> **Operator Precedence**

Logical Operators

Symbol	Description	#Operators

!	Logical negation	One
||	Logical OR	Two
&&	Logical AND	Two

Relational Operators

Symbol	Description	#Operators
>	Greater than	Two
<	Less than	Two
>=	Greater than or equal to	Two
<=	Less than or equal to	Two

Equality Operators

Symbol	Description	#Operators
==	Equality	Two
!=	Inequality	Two
===	Case equality	Two
!==	Case inequality	Two

Arithmetic Operators

Symbol	Description	#Operators
+	Add	Two
-	Substract	Two
*	Multiply	Two
/	Divide	Two
**	Power	Two
%	Modulus	Two

Bitwise Operators

Symbol	Description	#Operators
~	Bitwise negation	One
&	Bitwise AND	Two
|	Bitwise OR	Two

^	Bitwise XOR	Two
^~ or ~^	Bitwise XNOR	Two

Reduction Operators

Symbol	Description	#Operators
&	Reduction AND	One
~&	Reduction NAND	One
|	Reduction OR	One
~|	Reduction NOR	One
^	Reduction XOR	One
^~ or ~^	Reduction XNOR	One

Shift Operators

Symbol	Description	#Operators
>>	Right shift	Two
<<	Left shift	Two
>>>	Arithmetic right shift	Two
<<<	Arithmetic left shift	Two

Conditional Operators

Symbol	Description	#Operators
?:	Conditional	Two

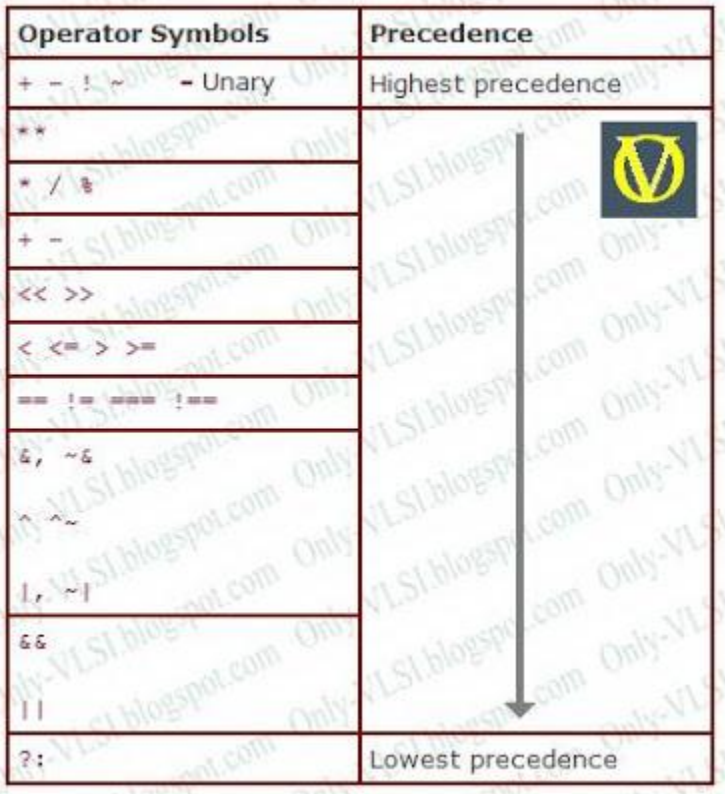
Replication Operators

Symbol	Description	#Operators
{ { } }	Replication	> One

Concatenation Operators

Symbol	Description	#Operators
{ }	Concatenation	> One

Operator Precedence



Gate-Level Modeling

>> **Introduction**
>> **Gate Primitives**
>> **Delays**
>> **Examples**

Introduction

In Verilog HDL a module can be defined using various levels of abstraction. There are four levels of abstraction in verilog. They are:

- Behavioral or algorithmic level: This is the highest level of abstraction. A module can be implemented in terms of the design algorithm. The designer no need to have any knowledge of hardware implementation.
- Data flow level: In this level the module is designed by specifying the data flow. Designer must how data flows between various registers of the design.
- Gate level: The module is implemented in terms of logic gates and interconnections between these gates. Designer should know the gate-level diagram of the design.
- Switch level: This is the lowest level of abstraction. The design is implemented using switches/transistors. Designer requires the knowledge of switch-level implementation details.

Gate-level modeling is virtually the lowest-level of abstraction, because the switch-level abstraction is rarely used. In general, gate-level modeling is used for implementing lowest level modules in a design like, full-adder, multiplexers, etc. Verilog HDL has gate primitives for all basic gates.

Gate Primitives

Gate primitives are predefined in Verilog, which are ready to use. They are instantiated like modules. There are two classes of gate primitives: Multiple input gate primitives and Single input gate primitives.
Multiple input gate primitives include and, nand, or, nor, xor, and xnor. These can have multiple inputs and a single output. They are instantiated as follows:

```
// Two input AND gate.
and and_1 (out, in0, in1);

// Three input NAND gate.
nand nand_1 (out, in0, in1, in2);

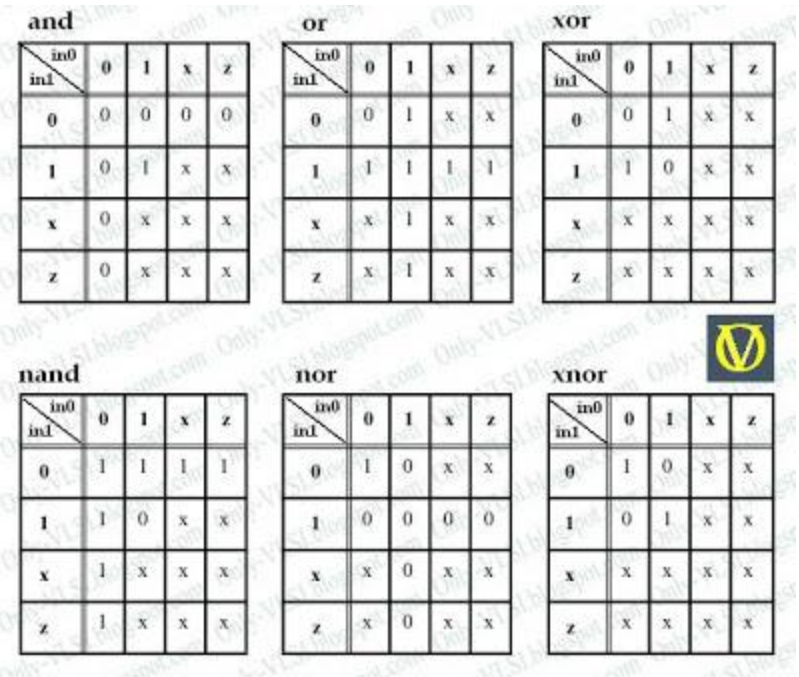
// Two input OR gate.
or or_1 (out, in0, in1);

// Four input NOR gate.
nor nor_1 (out, in0, in1, in2, in3);

// Five input XOR gate.
xor xor_1 (out, in0, in1, in2, in3, in4);

// Two input XNOR gate.
xnor and_1 (out, in0, in1);
```

Note that instance name is not mandatory for gate primitive instantiation. The truth tables of multiple input gate primitives are as follows:



and

in1 \ in0	0	1	x	z
0	0	0	0	0
1	0	1	x	x
x	0	x	x	x
z	0	x	x	x

or

in1 \ in0	0	1	x	z
0	0	1	x	x
1	1	1	1	1
x	x	1	x	x
z	x	1	x	x

xor

in1 \ in0	0	1	x	z
0	0	1	x	x
1	1	0	x	x
x	x	x	x	x
z	x	x	x	x

nand

in1 \ in0	0	1	x	z
0	1	1	1	1
1	1	0	x	x
x	1	x	x	x
z	1	x	x	x

nor

in1 \ in0	0	1	x	z
0	1	0	x	x
1	0	0	0	0
x	x	0	x	x
z	x	0	x	x

xnor

in1 \ in0	0	1	x	z
0	1	0	x	x
1	0	1	x	x
x	x	x	x	x
z	x	x	x	x

Single input gate primitives include not, buf, notif1, bufif1, notif0, and bufif0. These have a single input and one or more outputs. Gate primitives notif1, bufif1, notif0, and bufif0 have a control signal. The gates propagate if only control signal is asserted, else the output will be high impedance state (z). They are instantiated as follows:

```
// Inverting gate.
not not_1 (out, in);

// Two output buffer gate.
buf buf_1 (out0, out1, in);

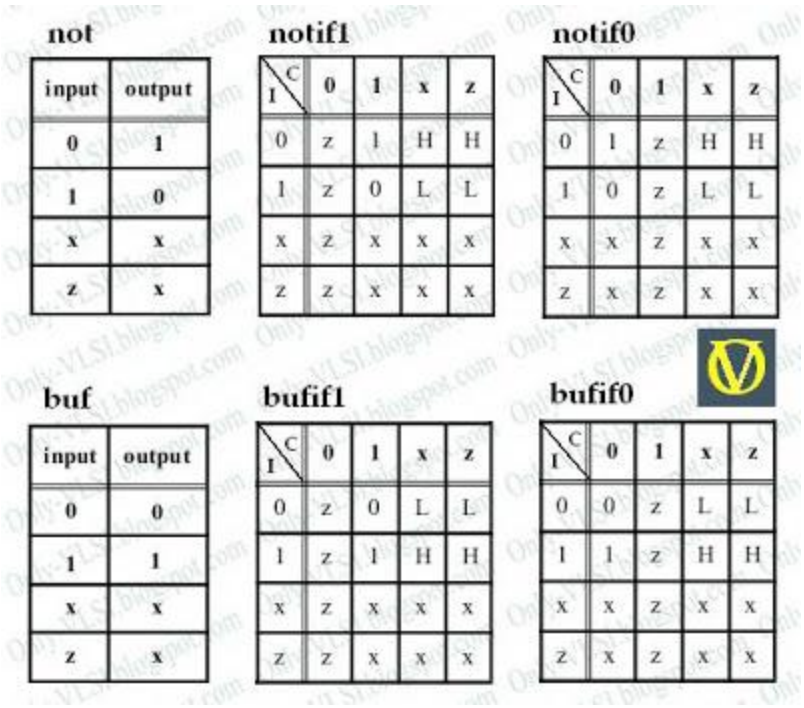
// Single output Inverting gate with active-high control signal.
notif1 notif1_1 (out, in, ctrl);

// Double output buffer gate with active-high control signal.
bufif1 bufif1_1 (out0, out1, in, ctrl);

// Single output Inverting gate with active-low control signal.
notif0 notif0_1 (out, in, ctrl);

// Single output buffer gate with active-low control signal.
bufif0 bufif1_0 (out, in, ctrl);
```

The truth tables are as follows:



not

input	output
0	1
1	0
x	x
z	x

notif1

I \ C	0	1	x	z
0	z	1	H	H
1	z	0	L	L
x	z	x	x	x
z	z	x	x	x

notif0

I \ C	0	1	x	z
0	1	z	H	H
1	0	z	L	L
x	x	z	x	x
z	x	z	x	x

buf

input	output
0	0
1	1
x	x
z	x

bufif1

I \ C	0	1	x	z
0	z	0	L	L
1	z	1	H	H
x	z	x	x	x
z	z	x	x	x

bufif0

I \ C	0	1	x	z
0	0	z	L	L
1	1	z	H	H
x	x	z	x	x
z	x	z	x	x

Array of Instances:

```
wire [3:0] out, in0, in1;
and and_array[3:0] (out, in0, in1);
```

The above statement is equivalent to following bunch of statements:

```
and and_array0 (out[0], in0[0], in1[0]);
and and_array1 (out[1], in0[1], in1[1]);
and and_array2 (out[2], in0[2], in1[2]);
and and_array3 (out[3], in0[3], in1[3]);
```

>> Examples

Gate Delays:

In Verilog, a designer can specify the gate delays in a gate primitive instance. This helps the designer to get a real time behavior of the logic circuit.

Rise delay: It is equal to the time taken by a gate output transition to 1, from another value 0, x, or z.

Fall delay: It is equal to the time taken by a gate output transition to 0, from another value 1, x, or z.

Turn-off delay: It is equal to the time taken by a gate output transition to high impedance state, from another value 1, x, or z.

- If the gate output changes to x, the minimum of the three delays is considered.
- If only one delay is specified, it is used for all delays.
- If two values are specified, they are considered as rise, and fall delays.
- If three values are specified, they are considered as rise, fall, and turn-off delays.
- The default value of all delays is zero.

```
and #(5) and_1 (out, in0, in1);
// All delay values are 5 time units.

nand #(3,4,5) nand_1 (out, in0, in1);
// rise delay = 3, fall delay = 4, and turn-off delay = 5.

or #(3,4) or_1 (out, in0, in1);
// rise delay = 3, fall delay = 4, and turn-off delay = min(3,4)
= 3.
```

There is another way of specifying delay times in verilog, Min:Typ:Max values for each delay. This helps designer to have a much better real time experience of design simulation, as in real time logic circuits the delays are not constant. The user can choose one of the delay values using +maxdelays, +typdelays, and +mindelays at run time. The typical value is the default value.

```
and #(4:5:6) and_1 (out, in0, in1);
// For all delay values: Min=4, Typ=5, Max=6.

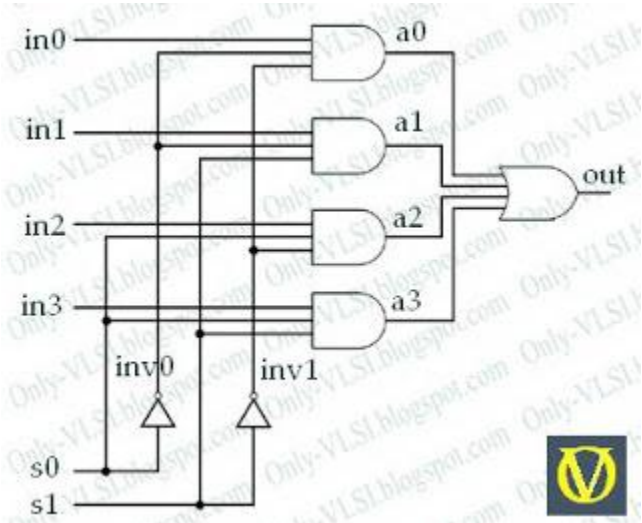
nand #(3:4:5,4:5:6,5:6:7) nand_1 (out, in0, in1);
// rise delay: Min=3, Typ=4, Max=5, fall delay: Min=4, Typ=5,
Max=6, turn-off delay: Min=5, Typ=6, Max=7.
```

In the above example, if the designer chooses typical values, then rise delay = 4, fall delay = 5, turn-off delay = 6.

Examples:

1. Gate level modeling of a 4x1 multiplexer.

The gate-level circuit diagram of 4x1 mux is shown below. It is used to write a module for 4x1 mux.




```
module 4x1_mux (out, in0, in1, in2, in3, s0, s1);

// port declarations
output out; // Output port.
input in0, in1, in2. in3; // Input ports.
input s0, s1; // Input ports: select lines.

// intermediate wires
wire inv0, inv1; // Inverter outputs.
wire a0, a1, a2, a3; // AND gates outputs.

// Inverters.
not not_0 (inv0, s0);
not not_1 (inv1, s1);

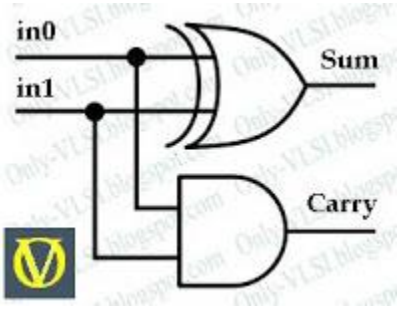
// 3-input AND gates.
and and_0 (a0, in0, inv0, inv1);
and and_1 (a1, in1, inv0, s1);
and and_2 (a2, in2, s0, inv1);
and and_3 (a3, in3, s0, s1);

// 4-input OR gate.
or or_0 (out, a0, a1, a2, a3);

endmodule
```

2. Implementation of a full adder using half adders.

Half adder:




```
module half_adder (sum, carry, in0, in1);

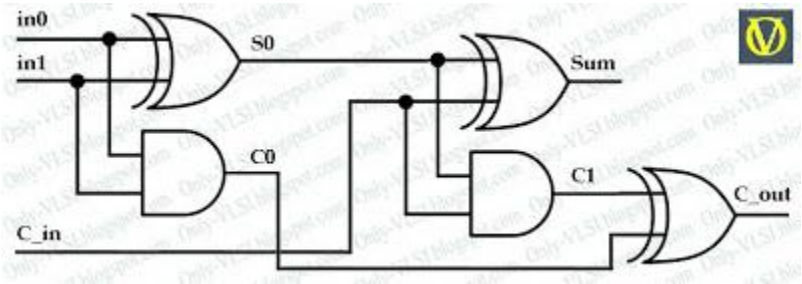
output sum, carry;
input in0, in1;

// 2-input XOR gate.
xor xor_1 (sum, in0, in1);

// 2-input AND gate.
and and_1 (carry, in0, in1);

endmodule
```

Full adder:




```
module full_adder (sum, c_out, ino, in1, c_in);

output sum, c_out;
input in0, in1, c_in;

wire s0, c0, c1;

// Half adder : port connecting by order.
half_adder ha_0 (s0, c0, in0, in1);

// Half adder : port connecting by name.
half_adder ha_1 (.sum(sum),
                 .in0(s0),
                 .in1(c_in),
                 .carry(c1));

// 2-input XOR gate, to get c_out.
xor xor_1 (c_out, c0, c1);

endmodule
```

Dataflow Modeling

>> **Introduction**
>> **The *assign* Statement**
>> **Delays**
>> **Examples**

Introduction

Dataflow modeling is a higher level of abstraction. The designer no need have any knowledge of logic circuit. He should be aware of data flow of the design. The gate level modeling becomes very complex for a VLSI circuit. Hence dataflow modeling became a very important way of implementing the design.
In dataflow modeling most of the design is implemented using continuous assignments, which are used to drive a value onto a net. The continuous assignments are made using the keyword *assign*.

The *assign* statement

The *assign* statement is used to make continuous assignment in the dataflow modeling. The *assign* statement usage is given below:

```
assign out = in0 + in1; // in0 + in1 is evaluated and then
assigned to out.
```

Note:

- The LHS of assign statement must always be a scalar or vector net or a concatenation. It cannot be a register.
- Continuous statements are always active statements.
- Registers or nets or function calls can come in the RHS of the assignment.
- The RHS expression is evaluated whenever one of its operands changes. Then the result is assigned to the LHS.
- Delays can be specified.

Examples:

```
assign out[3:0] = in0[3:0] & in1[3:0];

assign {o3, o2, o1, o0} = in0[3:0] | {in1[2:0],in2}; // Use of
concatenation.
```

Implicit Net Declaration:

```
wire in0, in1;
```

```
assign out = in0 ^ in1;
```

In the above example out is undeclared, but verilog makes an implicit net declaration for out.

Implicit Continuous Assignment:

```
wire out = in0 ^ in1;
```

The above line is the implicit continuous assignment. It is same as,

```
wire out;
assign out = in0 ^ in1;
```

Delays

There are three types of delays associated with dataflow modeling. They are: Normal/regular assignment delay, implicit continuous assignment delay and net declaration delay.

Normal/regular assignment delay:

```
assign #10 out = in0 | in1;
```

If there is any change in the operands in the RHS, then RHS expression will be evaluated after 10 units of time. Lets say that at time t, if there is change in one of the operands in the above example, then the expression is calculated at t+10 units of time. The value of RHS operands present at time t+10 is used to evaluate the expression.

Implicit continuous assignment delay:

```
wire #10 out = in0 ^ in1;
```

is same as

```
wire out;
assign 10 out = in0 ^ in1;
```

Net declaration delay:

```
wire #10 out;
assign out = in;
```

is same as

```
wire out;
assign #10 out = in;
```

Examples

1. Implementation of a 2x4 decoder.

```
module decoder_2x4 (out, in0, in1);

output out[0:3];
input in0, in1;

// Data flow modeling uses logic operators.
assign out[0:3] = { ~in0 & ~in1, in0 & ~in1,
                    ~in0 & in1, in0 & in1 };

endmodule
```

2. Implementation of a 4x1 multiplexer.

```
module mux_4x1 (out, in0, in1, in2, in3, s0, s1);

output out;
input in0, in1, in2, in3;
input s0, s1;

assign out = (~s0 & ~s1 & in0)|(s0 & ~s1 & in1)|
             (~s0 & s1 & in2)|(s0 & s1 & in0);

endmodule
```

3. Implementation of a 8x1 multiplexer using 4x1 multiplexers.

```
module mux_8x1 (out, in, sel);

output out;
input [7:0] in;
input [2:0] sel;

wire m1, m2;

// Instances of 4x1 multiplexers.
mux_4x1 mux_1 (m1, in[0], in[1], in[2],
               in[3], sel[0], sel[1]);
mux_4x1 mux_2 (m2, in[4], in[5], in[6],
               in[7], sel[0], sel[1]);

assign out = (~sel[2] & m1)|(sel[2] & m2);

endmodule
```

4. Implementation of a Full adder.

```
module full_adder (sum, c_out, in0, in1, c_in);

output sum, c_out;
input in0, in1, c_in;

assign { c_out, sum } = in0 + in1 + c_in;
```

```
endmodule
```

Behavioral Modeling

>> **Introduction**
>> **The initial Construct**
>> **The always Construct**
>> **Procedural Assignments**
>> **Block Statements**
>> **Conditional (if-else) Statement**
>> **Case Statement**
>> **Loop Statements**
>> **Examples**

Introduction

Behavioral modeling is the highest level of abstraction in the Verilog HDL. The other modeling techniques are relatively detailed. They require some knowledge of how hardware, or hardware signals work. The abstraction in this modeling is as simple as writing the logic in C language. This is a very powerful abstraction technique. All that designer needs is the algorithm of the design, which is the basic information for any design.

Most of the behavioral modeling is done using two important constructs: initial and always. All the other behavioral statements appear only inside these two structured procedure constructs.

The initial Construct

The statements which come under the *initial* construct constitute the initial block. The initial block is executed only once in the simulation, at time 0. If there is more than one initial block. Then all the initial blocks are executed concurrently. The initial construct is used as follows:

```
initial
begin
reset = 1'b0;
clk = 1'b1;
end

or

initial
clk = 1'b1;
```

In the first initial block there are more than one statements hence they are written between begin and end. If there is only one statement then there is no need to put begin and end.

The always Construct

The statements which come under the *always* construct constitute the always block. The always block starts at time 0, and keeps on executing all the simulation time. It works like a infinite loop. It is generally used to model a functionality that is continuously repeated.

```
always
#5 clk = ~clk;

initial
clk = 1'b0;
```

The above code generates a clock signal clk, with a time period of 10 units. The initial blocks initiates the clk value to 0 at time 0. Then after every 5 units of time it toggled, hence we get a time period of 10 units. This is the way in general used to generate a clock signal for use in test benches.

```
always @(posedge clk, negedge reset)
begin
a = b + c;
    d = 1'b1;
end
```

In the above example, the always block will be executed whenever there is a positive edge in the clk signal, or there is negative edge in the reset signal. This type of always is generally used in implement a FSM, which has a reset signal.

```
always @(b,c,d)
begin
    a = ( b + c )*d;
    e = b | c;
end
```

In the above example, whenever there is a change in b, c, or d the always block will be executed. Here the list b, c, and d is called the *sensitivity list*.

In the Verilog 2000, we can replace always @(b,c,d) with always @(*), it is equivalent to include all input signals, used in the always block. This is very useful when always blocks is used for implementing the combination logic.

Procedural Assignments

Procedural assignments are used for updating *reg*, *integer*, *time*, *real*, *realtime*, and *memory* data types. The variables will retain their values until updated by another procedural assignment. There is a significant difference between procedural assignments and continuous assignments. Continuous assignments drive nets and are evaluated and updated whenever an input operand changes value. Where as procedural assignments update the value of variables under the control of the procedural flow constructs that surround them.

The LHS of a procedural assignment could be:

- *reg*, *integer*, *real*, *realtime*, or *time* data type.

- Bit-select of a *reg*, *integer*, or *time* data type, rest of the bits are untouched.
- Part-select of a *reg*, *integer*, or *time* data type, rest of the bits are untouched.
- Memory word.
- Concatenation of any of the previous four forms can be specified.

When the RHS evaluates to fewer bits than the LHS, then if the right-hand side is signed, it will be sign-extended to the size of the left-hand side.

There are two types of procedural assignments: blocking and non-blocking assignments.

Blocking assignments: A blocking assignment statements are executed in the order they are specified in a sequential block. The execution of next statement begin only after the completion of the present blocking assignments. A blocking assignment will not block the execution of the next statement in a parallel block. The blocking assignments are made using the operator =.

```
initial
begin
    a = 1;
    b = #5 2;
    c = #2 3;
end
```

In the above example, a is assigned value 1 at time 0, and b is assigned value 2 at time 5, and c is assigned value 3 at time 7.

Non-blocking assignments: The nonblocking assignment allows assignment scheduling without blocking the procedural flow. The nonblocking assignment statement can be used whenever several variable assignments within the same time step can be made without regard to order or dependence upon each other. Non-blocking assignments are made using the operator <=.
Note: <= is same for less than or equal to operator, so whenever it appears in a expression it is considered to be comparison operator and not as non-blocking assignment.

```
initial
begin
    a <= 1;
    b <= #5 2;
    c <= #2 3;
end
```

In the above example, a is assigned value 1 at time 0, and b is assigned value 2 at time 5, and c is assigned value 3 at time 2 (because all the statements execution starts at time 0, as they are non-blocking assignments.

Block Statements

Block statements are used to group two or more statements together, so that they act as one statement. There are two types of blocks:

- Sequential block.

- Parallel block.

Sequential block: The sequential block is defined using the keywords *begin* and *end*. The procedural statements in sequential block will be executed sequentially in the given order. In sequential block delay values for each statement shall be treated relative to the simulation time of the execution of the previous statement. The control will pass out of the block after the execution of last statement.

Parallel block: The parallel block is defined using the keywords *fork* and *join*. The procedural statements in parallel block will be executed concurrently. In parallel block delay values for each statement are considered to be relative to the simulation time of entering the block. The delay control can be used to provide time-ordering for procedural assignments. The control shall pass out of the block after the execution of the last time-ordered statement.

Note that blocks can be nested. The sequential and parallel blocks can be mixed.

Block names: All the blocks can be named, by adding *: block_name* after the keyword *begin* or *fork*. The advantages of naming a block are:

- It allows to declare local variables, which can be accessed by using hierarchical name referencing.
- They can be disabled using the *disable* statement (*disable block_name;*).

Conditional (if-else) Statement

The condition (if-else) statement is used to make a decision whether a statement is executed or not. The keywords *if* and *else* are used to make conditional statement. The conditional statement can appear in the following forms.

```
if ( condition_1 )
    statement_1;

if ( condition_2 )
    statement_2;
else
    statement_3;

if ( condition_3 )
    statement_4;
else if ( condition_4 )
    statement_5;
else
    statement_6;

if ( condition_5 )
begin
    statement_7;
    statement_8;
end
```

```
else
begin
    statement_9;
    statement_10;
end
```

Conditional (if-else) statement usage is similar to that if-else statement of C programming language, except that parenthesis are replaced by *begin* and *end*.

Case Statement

The case statement is a multi-way decision statement that tests whether an expression matches one of the expressions and branches accordingly. Keywords *case* and *endcase* are used to make a case statement. The case statement syntax is as follows.

```
case (expression)
    case_item_1: statement_1;
    case_item_2: statement_2;
    case_item_3: statement_3;
    ...
    ...
    default: default_statement;
endcase
```

If there are multiple statements under a single match, then they are grouped using begin, and end keywords. The default item is optional.

Case statement with don't cares: casez and casex

casez treats high-impedance values (z) as don't cares. *casex* treats both high-impedance (z) and unknown (x) values as don't cares. Don't-care values (z values for *casez*, z and x values for *casex*) in any bit of either the case expression or the case items shall be treated as don't-care conditions during the comparison, and that bit position shall not be considered. The don't cares are represented using the ? mark.

Loop Statements

There are four types of looping statements in Verilog:

- forever
- repeat
- while
- for

Forever Loop

Forever loop is defined using the keyword forever, which Continuously executes a statement. It

terminates when the system task $finish is called. A forever loop can also be ended by using the disable statement.

```
initial
begin
    clk = 1'b0;
    forever #5 clk = ~clk;
end
```

In the above example, a clock signal with time period 10 units of time is obtained.

Repeat Loop

Repeat loop is defined using the keyword repeat. The repeat loop block continuously executes the block for a given number of times. The number of times the loop executes can be mention using a constant or an expression. The expression is calculated only once, before the start of loop and not during the execution of the loop. If the expression value turns out to be z or x, then it is treated as zero, and hence loop block is not executed at all.

```
initial
begin
    a = 10;
    b = 5;
    b <= #10 10;
    i = 0;
    repeat(a*b)
    begin
        $display("repeat in progress");
        #1 i = i + 1;
    end
end
```

In the above example the loop block is executed only 50 times, and not 100 times. It calculates (a*b) at the beginning, and uses that value only.

While Loop

The while loop is defined using the keyword while. The while loop contains an expression. The loop continues until the expression is true. It terminates when the expression is false. If the calculated value of expression is z or x, it is treated as a false. The value of expression is calculated each time before starting the loop. All the statements (if more than one) are mentioned in blocks which begins and ends with keyword begin and end keywords.

```
initial
begin
    a = 20;
    i = 0;
    while (i < a)
    begin
    $display("%d",i);
    i = i + 1;
    a = a - 1;
```

```
    end
end
```

In the above example the loop executes for 10 times. (observe that a is decrementing by one and i is incrementing by one, so loop terminated when both i and a become 10).

For Loop

The For loop is defined using the keyword for. The execution of for loop block is controlled by a three step process, as follows:

1. Executes an assignment, normally used to initialize a variable that controls the number of times the for block is executed.
2. Evaluates an expression, if the result is false or z or x, the for-loop shall terminate, and if it is true, the for-loop shall execute its block.
3. Executes an assignment normally used to modify the value of the loop-control variable and then repeats with second step.

Note that the first step is executed only once.

```
initial
begin
    a = 20;
    for (i = 0; i < a; i = i + 1, a = a - 1)
    $display("%d",i);
end
```

The above example produces the same result as the example used to illustrate the functionality of the while loop.

Examples

1. Implementation of a 4x1 multiplexer.

```
module 4x1_mux (out, in0, in1, in2, in3, s0, s1);

output out;

// out is declared as reg, as default is wire

reg out;

// out is declared as reg, because we will
// do a procedural assignment to it.

input in0, in1, in2, in3, s0, s1;

// always @(*) is equivalent to
// always @( in0, in1, in2, in3, s0, s1 )

always @(*)
```

```
begin
  case ({s1,s0})
      2'b00: out = in0;
      2'b01: out = in1;
      2'b10: out = in2;
      2'b11: out = in3;
      default: out = 1'bx;
  endcase
end

endmodule
```

2. Implementation of a full adder.

```
module full_adder (sum, c_out, in0, in1, c_in);

output sum, c_out;
reg sum, c_out

input in0, in1, c_in;

always @(*)
  {c_out, sum} = in0 + in1 + c_in;

endmodule
```

3. Implementation of a 8-bit binary counter.

```
module ( count, reset, clk );

output [7:0] count;
reg [7:0] count;

input reset, clk;

// consider reset as active low signal

always @( posedge clk, negedge reset)
begin
  if(reset == 1'b0)
      count <= 8'h00;
  else
      count <= count + 8'h01;
end

endmodule
```

Implementation of a 8-bit counter is a very good example, which explains the advantage of behavioral modeling. Just imagine how difficult it will be implementing a 8-bit counter using gate-level modeling.
In the above example the incrementation occurs on every positive edge of the clock. When count becomes 8'hFF, the next increment will make it 8'h00, hence there is no need of any modulus operator. Reset signal is active low.

Tasks and Functions

>> Introduction
>> Differences
>> Tasks
>> Functions
>> Examples

Introduction

Tasks and functions are introduced in the verilog, to provide the ability to execute common procedures from different places in a description. This helps the designer to break up large behavioral designs into smaller pieces. The designer has to abstract the similar pieces in the description and replace them either functions or tasks. This also improves the readability of the code, and hence easier to debug. Tasks and functions must be defined in a module and are local to the module. Tasks are used when:

- There are delay, timing, or event control constructs in the code.
- There is no input.
- There is zero output or more than one output argument.

Functions are used when:

- The code executes in zero simulation time.
- The code provides only one output(return value) and has at least one input.
- There are no delay, timing, or event control constructs.

Differences

Functions	**Tasks**
Can enable another function but not another task.	Can enable other tasks and functions.
Executes in 0 simulation time.	May execute in non-zero simulation time.
Must not contain any delay, event, or timing control statements.	May contain delay, event, or timing control statements.
Must have at least one input argument. They can have more than one input.	May have zero or more arguments of type input, output, or inout.
Functions always return a single value. They cannot have output or inout arguments.	Tasks do not return with a value, but can pass multiple values through output and inout arguments.

Tasks

There are two ways of defining a task. The first way shall begin with the keyword task, followed by the optional keyword automatic, followed by a name for the task, and ending with the keyword endtask. The keyword automatic declares an automatic task that is reentrant with all the task declarations allocated dynamically for each concurrent task entry. Task item declarations can specify the following:

- Input arguments.
- Output arguments.
- Inout arguments.
- All data types that can be declared in a procedural block

The second way shall begin with the keyword task, followed by a name for the task and a parenthesis which encloses task port list. The port list shall consist of zero or more comma separated ports. The task body shall follow and then the keyword endtask.

In both ways, the port declarations are same. Tasks without the optional keyword automatic are static tasks, with all declared items being statically allocated. These items shall be shared across all uses of the task executing concurrently. Task with the optional keyword automatic are automatic tasks. All items declared inside automatic tasks are allocated dynamically for each invocation. Automatic task items can not be accessed by hierarchical references. Automatic tasks can be invoked through use of their hierarchical name.

Functions

Functions are mainly used to return a value, which shall be used in an expression. The functions are declared using the keyword function, and definition ends with the keyword endfunction.

If a function is called concurrently from two locations, the results are non-deterministic because both calls operate on the same variable space. The keyword automatic declares a recursive function with all the function declarations allocated dynamically for each recursive call. Automatic function items can not be accessed by hierarchical references. Automatic functions can be invoked through the use of their hierarchical name.

When a function is declared, a register with function name is declared implicitly inside Verilog HDL. The output of a function is passed back by setting the value of that register appropriately.

Examples

1. Simple task example, where task is used to get the address tag and offset of a given address.

```
module example1_task;

input addr;
wire [31:0] addr;

wire [23:0] addr_tag;
wire [7:0] offset;
```

```
task get_tag_and_offset ( addr, tag, offset);

input addr;
output tag, offset;

begin
 tag = addr[31:8];
 offset = addr[7:0];
end
endtask

always @(addr)
begin
 get_tag_and_offset (addr, addr_tag, addr_offset);
end

// other internals of module

endmodule
```

2. Task example, which uses the global variables of a module. Here task is used to do temperature conversion.

```
module example2_global;

real t1;
real t2;

// task uses the global variables of the module

task t_convert;
begin
 t2 = (9/5)*(t1+32);
end
endtask

always @(t1)
begin
 t_convert();
end

endmodule
```